# 🇫🇷 Devops

## Docker

2026/2027

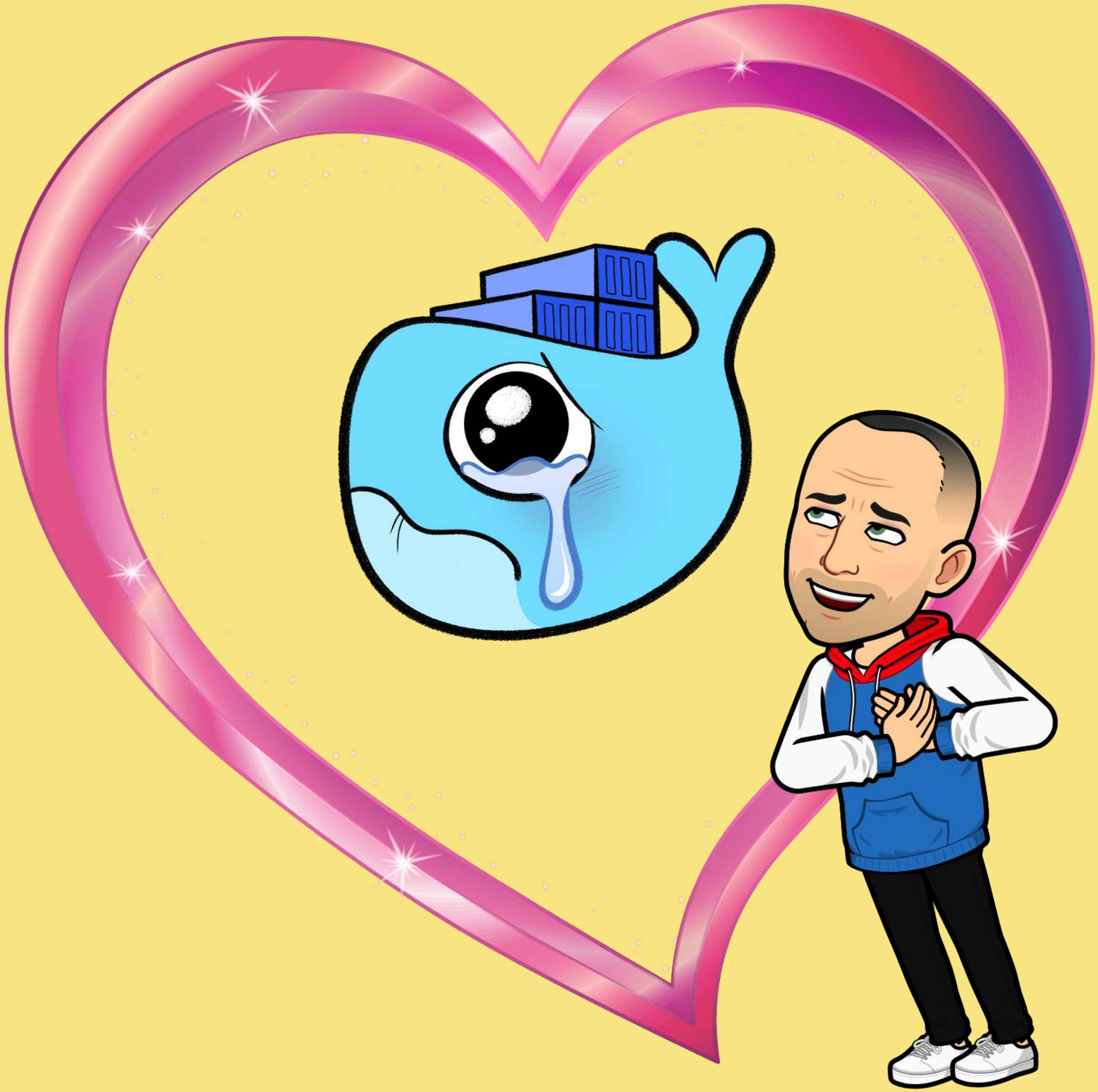
- Présentation disponible à l'adresse: https://gounthar.github.io/cours-devops-docker/docs/bonus-tp13-overflow
- Version PDF de la présentation : Cliquez ici
- Contenu sous licence Creative Commons Attribution 4.0 International License
- Code source de la présentation: https://github.com/gounthar/cours-devops-docker
- Mis à jour le : 26 octobre 2026

# Comment utiliser cette présentation ?

- Pour naviguer, utilisez les flèches en bas à droite (ou celles de votre clavier)
  - Gauche/Droite: changer de chapitre
  - Haut/Bas: naviguer dans un chapitre
- Pour avoir une vue globale : utiliser la touche "o" (pour "**Overview**")

# Bonjour !

```
poddingue@my-machine:~$
```



Bruno VERACHTEN

- Community Manager & Platform Evangelist chez Vates, l'éditeur de XCP-ng et Xen Orchestra 👨🏻‍🔧
- Me contacter :
  - ✉ gounthar@gmail.com
  - gounthar
  - https://bruno.verachten.fr/
  - Bruno Verachten
  - @poddingue

# Et vous ?

# A propos du cours

- Première itération d'une découverte Docker dans le cadre du DevOps
- Contenu entièrement libre et open-source
- Méchamment basé sur le travail de Damien Duportal et Amaury Willemant
    - N'hésitez pas ouvrir des Pull Request si vous voyez des améliorations ou problèmes: sur cette page (😉 wink wink)

# Calendrier

🚧 Work in progress… 🚧

- 22 septembre après-midi
- 29 septembre après-midi
- 06 octobre après-midi
- …

# 📊 Évaluation

- Pourquoi ? s’assurer que vous avez acquis un minimum de concepts
- Quoi ? Sans doute une note sur 20, basée sur une liste de critères exhaustifs
- Comment ? Un projet Gitlab à me rendre

# Plan

- Intro
- Base
- Containers
- Images
- Fichiers, nommage, inspect
- Volumes
- Réseaux
- Docker Compose
- Bonus

La suite: vers la droite ➡️

# Docker

## "La Base"

# Pourquoi ?

🤔 Quel est le problème ?

# Pourquoi commencer par un problème ?

🤔 Commençons plutôt par une définition:

*Docker c'est …*

# Pourquoi commencer par un problème ?

🤔 Définition quelque peu datée (2014):

> *Docker is …*

# Pourquoi commencer par un problème ?

## 🤔 Définition quelque peu datée (2014):

> *Docker is a toolset for Linux containers designed to 'build, ship and run' distributed applications.*

https://www.infoq.com/articles/docker-future/

# Linux containers ?

🤔 Nous voilà bien…

*C’est quoi un container?*

# Linux containers ?

## 🤔 Nous voilà bien…

*C'est quoi un container?*

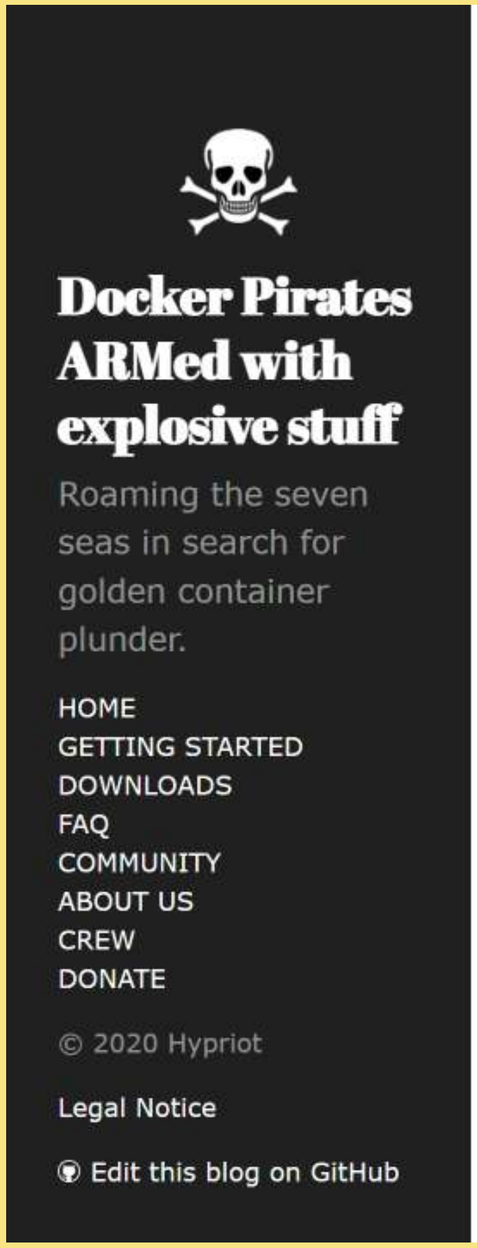
Vous voulez la version enfant de 5 ans? Je ne crois pas…

# Docker est vieux

Depuis mars 2013, soit 13 ans déjà…

# Docker est vieux

Mais moi aussi…




## Docker on Raspberry Pi Workshop in Brussels

Thu, Mar 17, 2016

A couple of weeks ago we have been invited by the fine folks over at the Docker User Group in Brussels to help conduct a workshop. And of course this workshop was about Docker. Still it was not your ordinary Docker workshop.

It was special because instead of being a workshop about Docker on big servers it was about Docker on really small ARM devices. The very same devices that power the upcoming IoT revolution.

Turns out it is really amazing what you can do with Docker on those tiny machines.

# Docker on arm

Aucune raison que ça soit réservé aux puissants! La révolution vaincra!

LinuxGizmos.com

All News | Boards | Chips | Devices | Software | LinuxDevices.com Archive | About | Contact | Subscribe

Follow LinuxGizmos:

* get email updates *

## Open source ResinOS adds Docker to ARM/Linux boards

Oct 17, 2016 — by Eric Brown 4,449 views

Resin.io has spun off the Yocto-based OS behind its Resin.io IoT framework as a ResinOS 2.0 distro for running Docker containers on Linux IoT devices.

Resin.io, the company behind the Linux/Javascript-based Resin.io IoT framework for deploying applications as Docker containers, began spinning off the Linux OS behind the framework as an open source project over a year ago. The open source ResinOS is now publicly available on its own in a stable 2.0.0-beta.1 version, letting other developers create their own Docker-based IoT networks. ResinOS can run on 20 different mostly ARM-based embedded Linux platforms including the Raspberry Pi, BeagleBone, and Odroid-C1, enabling secure rollouts of updated applications over a heterogeneous network.

**ResinOS 2.0 architecture**
(click image to enlarge)

When enterprise-level CIOs are asked to integrate embedded devices into their networks, their first question is usually "Can they run Docker?" The answer is probably not, especially if they run on ARM processors

Search LinuxGizmos: Search

Search LinuxGizmos.com + LinuxDevices Archive: ENHANCED BY Google Search

LinuxGizmos Sponsor ads:

(advertise here)

Top 10 trending posts...

» Libre Computer showcases low-cost SBC with PoE support
» ASRock Industrial's present 4x4 BOX 7040 Series mini PCs
» (Updated) The ZimaBlade is an upcoming low-cost x86 personal server
» GOWIN & Andes Technologies collaborate and reveal 22nm SoC FPGA
» Espressif Systems presents ESP32-S3-BOX-3 AIoT kit
» Milk-V Duo is a $9.00 RISC-V tiny embedded computer
» SATA HATs support up to four drives on Raspberry Pi 4 or Rock Pi 4
» Sipeed to launch RISC-V based Lichee Cluster 4A
» New SBC powered by Allwinner T507-H processor
» Orange Pi introduces a new Rockchip based Computer Module

LinuxGizmos Sponsor ads:

(advertise here)

Follow LinuxGizmos or subscribe to our posts:

# Docker on *

Linux everywhere… And then Docker to follow.

```
target "debian_jdk11" {
  dockerfile = "debian/Dockerfile"
  context = "."
  args = {
    JAVA_MAJOR_VERSION = "11"
    version = "${PARENT_IMAGE_VERSION}"
  }
  tags = [
    equal(ON_TAG, "true") ? "${REGISTRY}/${JENKINS_REPO}:${PARENT_IMAGE_VERSION}": "",
    equal(ON_TAG, "true") ? "${REGISTRY}/${JENKINS_REPO}:${PARENT_IMAGE_VERSION}-jdk11": "",
    "${REGISTRY}/${JENKINS_REPO}:jdk11",
    "${REGISTRY}/${JENKINS_REPO}:latest",
    "${REGISTRY}/${JENKINS_REPO}:latest-jdk11",
  ]
  platforms = ["linux/amd64", "linux/arm64", "linux/arm/v7", "linux/s390x", "linux/ppc64le"]
}
```

# On n’avait pas parlé d’un problème?

Intégrateur Développeur Testeuse

**DEV**

# On n’avait pas parlé d’un problème?

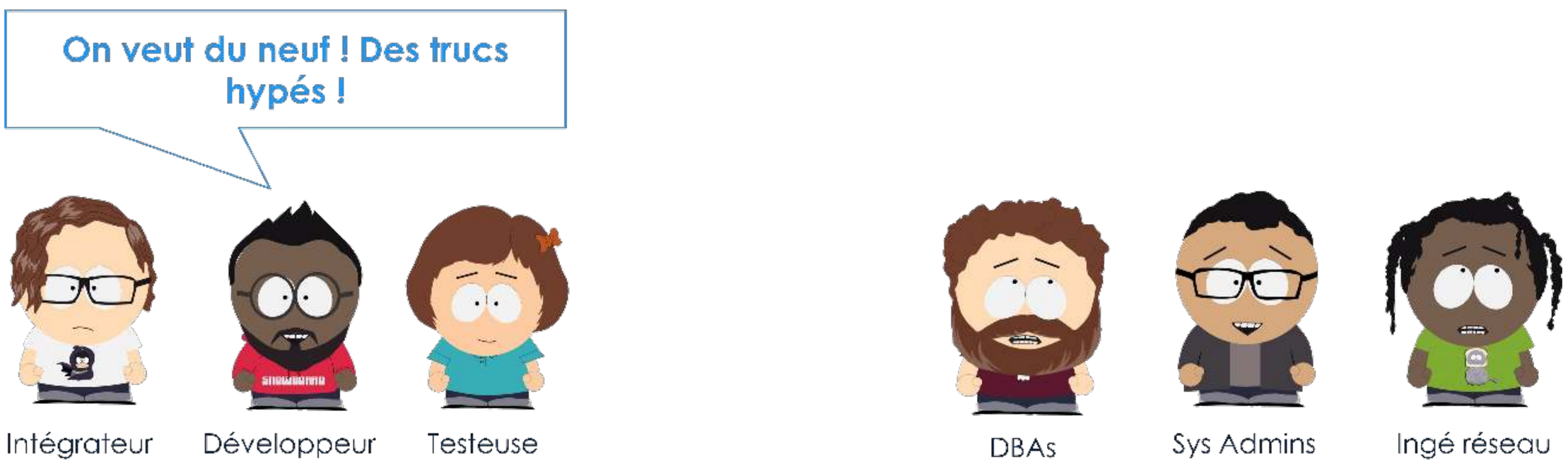
Intégrateur Développeur Testeuse

**DEV**

DBAs Sys Admins Ingé réseau

**OPS**

# On n’avait pas parlé d’un problème?



# On n’avait pas parlé d’un problème?

On veut du neuf ! Des trucs hypés !

Intégrateur

Développeur

Testeuse

**DEV**

On veut un truc stable !

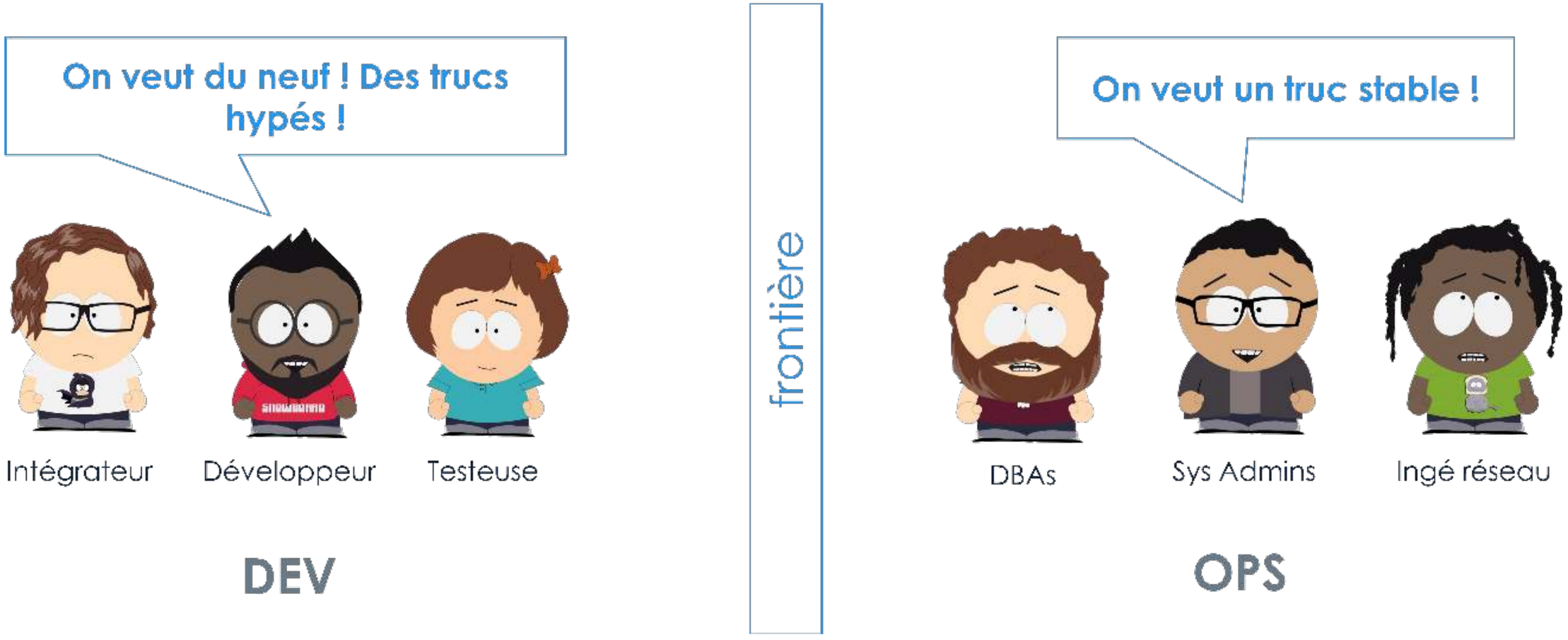
DBAs

Sys Admins

Ingé réseau

**OPS**

# On n’avait pas parlé d’un problème?



# On n’avait pas parlé d’un problème?

Histoire vraie.

Hey salut !

# On n’avait pas parlé d’un problème?

Histoire vraie.

Hey salut !

*cc*

# On n’avait pas parlé d’un problème?

Histoire vraie.

Hey salut !

*cc*

Tu peux mettre à jour les packages système steup ? j’ai un truc à tester.

# On n’avait pas parlé d’un problème?

Histoire vraie.

Hey salut !

*cc*

Tu peux mettre à jour les packages système steup ? j’ai un truc à tester.

*nan*

# On n’avait pas parlé d’un problème?

Histoire vraie.

`Hey salut !`

*cc*

`Tu peux mettre à jour les packages système steup ? j’ai un truc à tester.`

*nan*

`???`

# On n’avait pas parlé d’un problème?

Histoire vraie.

Hey salut !

*cc*

Tu peux mettre à jour les packages système steup ? j’ai un truc à tester.

*nan*

???

*pas standard dsl*

# On n’avait pas parlé d’un problème?

Histoire vraie.

```
Hey salut !
```

*cc*

```
  Tu peux mettre à jour les
packages système steup ? j’ai
      un truc à tester.
```

*nan*

```
???
```

*pas standard dsl*

# On n’avait pas parlé d’un problème?

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | Static Website | ? | ? | ? | ? | ? | ? | ? |
| | Web Frontend | ? | ? | ? | ? | ? | ? | ? |
| | Background Workers | ? | ? | ? | ? | ? | ? | ? |
| | User DB | ? | ? | ? | ? | ? | ? | ? |
| | Analytics DB | ? | ? | ? | ? | ? | ? | ? |
| | Queue | ? | ? | ? | ? | ? | ? | ? |
| | | Developm ent VM | QA Server | Single Prod Server | Production Cluster | Public cloud | Developer’s Laptop | Customer Servers |
| | | | |  | | | | |

*Source: https://blog.docker.com/2013/08/paas-present-and-future/*

Problème de temps **exponentiel**

# Déjà vu ?

L’IT n’est pas la seule industrie à résoudre des problèmes…

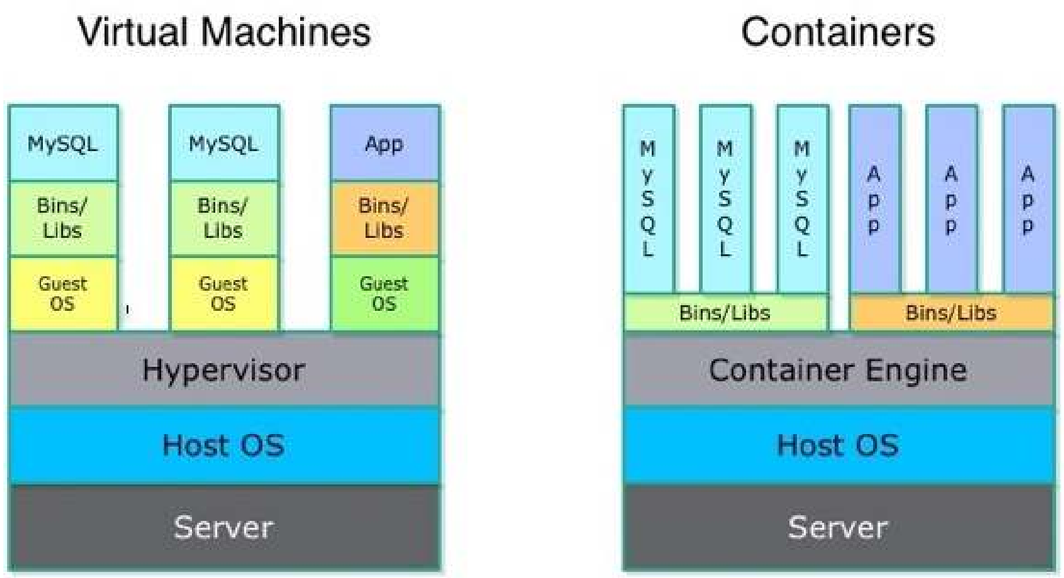
| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | ? | ? | ? | ? | ? | ? | ? |
| | ? | ? | ? | ? | ? | ? | ? |
| | ? | ? | ? | ? | ? | ? | ? |
| | ? | ? | ? | ? | ? | ? | ? |
| | ? | ? | ? | ? | ? | ? | ? |
| | ? | ? | ? | ? | ? | ? | ? |
| | | | | | | | |

# Solution: Le conteneur intermodal

"Separation of Concerns"

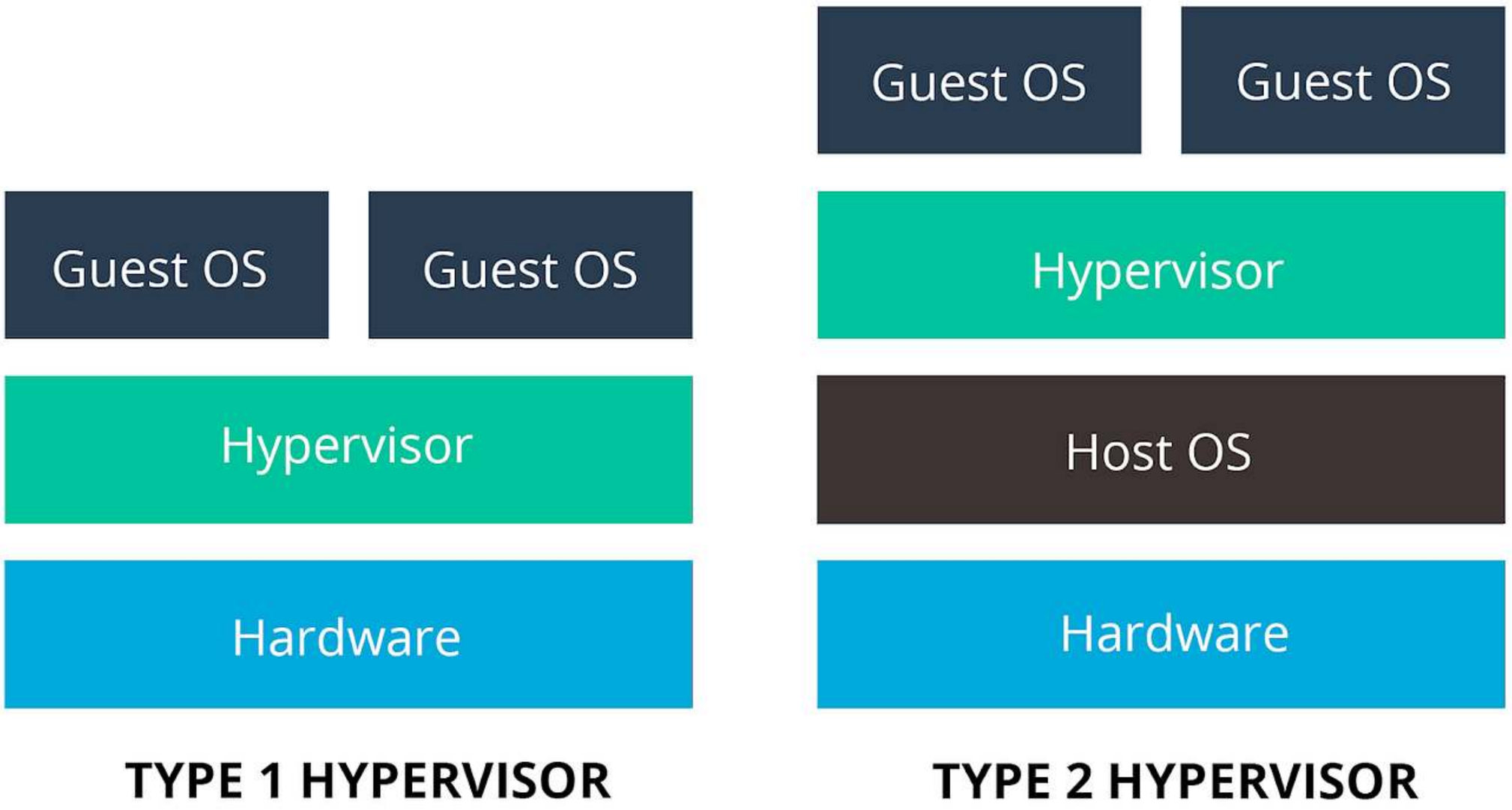
# Comment ça marche ?

"Virtualisation **Légère**"



# Comment ça marche ?

Virtualisation



# Comment ça marche ?

Virtualisation

# Un type 1 que vous pouvez installer

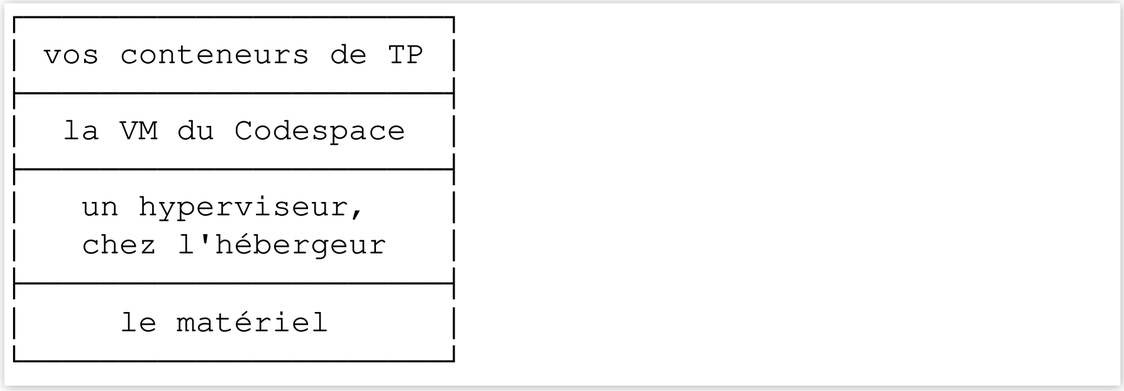
- Bâtie sur l'hyperviseur **Xen** : elle s'installe sur le matériel nu, donc bien du type 1
- Née d'un fork de **XenServer**, aujourd'hui projet en incubation du **Xen Project**, hébergé par la Linux Foundation
- L'ISO se télécharge et s'installe sans licence : un type 1 que vous pouvez vraiment essayer, sur une machine que vous lui **dédiez** — elle ne fera plus que ça

⚖️ Transparence : je travaille chez Vates, l'éditeur de XCP-ng et de Xen Orchestra. Je les montre parce qu'ils sont libres et ouvrables, pas parce qu'il faudrait les acheter.

# On ne pilote pas XCP-ng à la main

- Sur un hôte XCP-ng, ni bureau ni fenêtre : la machine ne montre qu'une console
- L'administration passe par un outil **séparé**, qui parle à l'hôte par le réseau
- **Xen Orchestra** est celui de XCP-ng : visualiser, gérer, sauvegarder et déléguer un parc, depuis un navigateur
- Sauvegardes complètes ou incrémentales, réplication, instantanés tournants — qui ne remplacent pas une sauvegarde
- Deux façons de l'obtenir : l'appliance clé en main, ou une installation depuis les sources

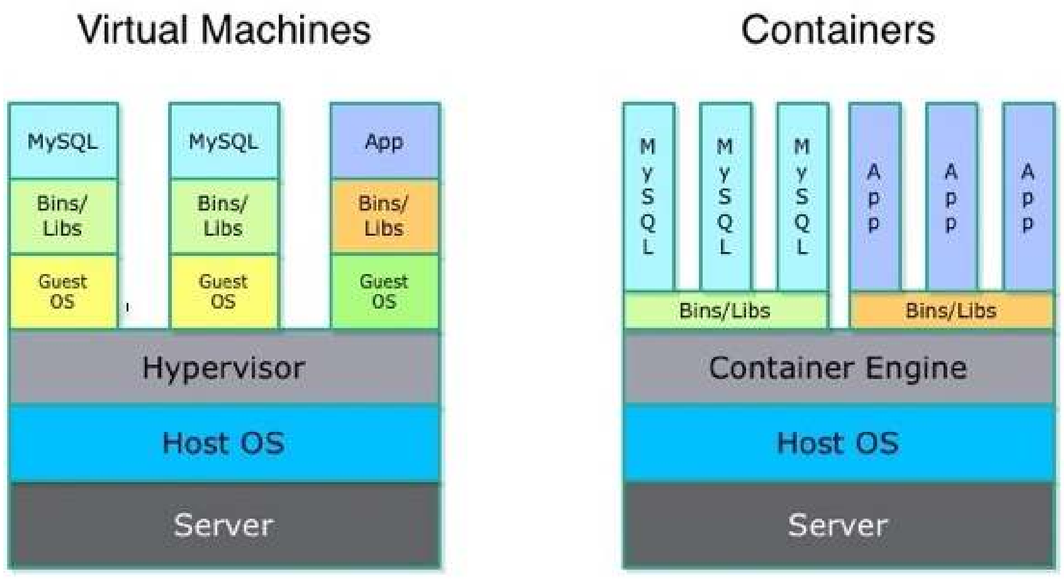
# VM et conteneur ne s'opposent pas




- Depuis le premier TP, vos conteneurs tournent **déjà** dans une machine virtuelle : GitHub héberge chaque Codespace sur une VM
- La question n'est donc jamais « VM ou conteneur ? », mais « qu'est-ce que je mets à quel étage ? »
- Dans les TP Docker, on travaille à l'étage du haut ; en préparant votre environnement, vous construirez aussi l'étage VM, sur XCP-ng

# Comment ça marche ?

"Virtualisation **Légère**"




- Légère, vraiment?

# Légère, vraiment!

http://web.archive.org/web/20200810061020/https://www.docker.com/blog/raspberry-pi-dockercon-challenge-winner/

Aside from the above issues (and there are more caveats in the documentation), my installation is a pretty faithful reproduction of ESXi on a server or home lab. However, ESXi itself uses about 1.3GB of RAM, leaving only around 6.5GB usable. This could support perhaps 4-5 small VMs, but would be much more useful for containers, which have less individual VM overhead.

# Légère, vraiment!

http://web.archive.org/web/20240304145820/https://www.architecting.it/blog/esxi-on-raspberry-pi/

## Proof of Concept

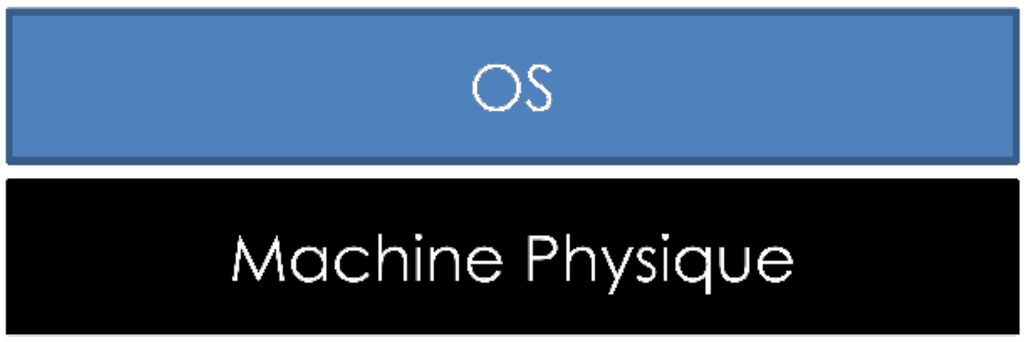
Remember that Flings are just proofs of concept and not always aimed at final production. VMware suggests a range of hardware options for ARM-based workloads and this is more likely where we will see the initial development of ESXi on ARM. The challenge for end users here is to determine whether the price/performance/power/cooling ratio works better on ARM than x86. Of course applications also need to be available, but it's not hard to find ARM-compiled versions of most open source software and operating systems.

# Conteneur != VM

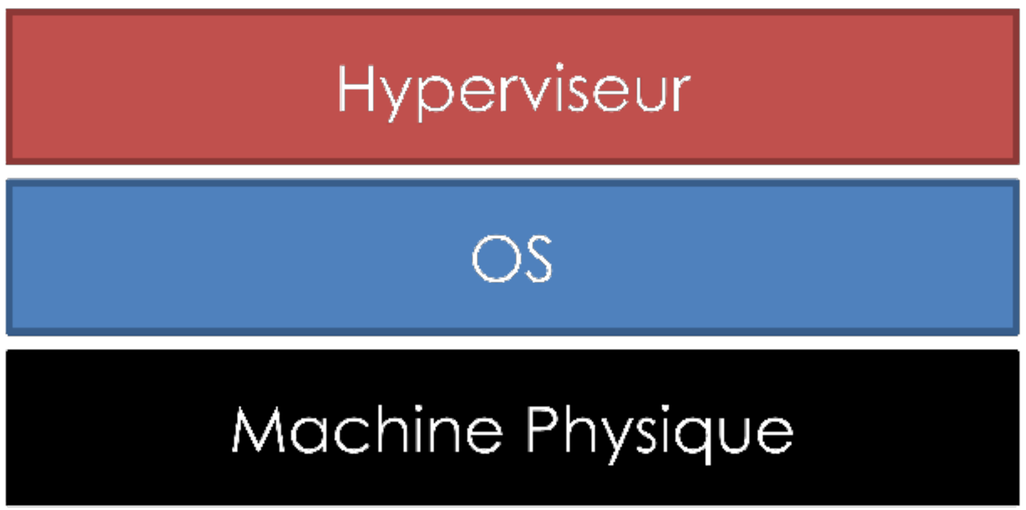
- Idée erronée mais citée trop fréquemment !

# Conteneur != VM

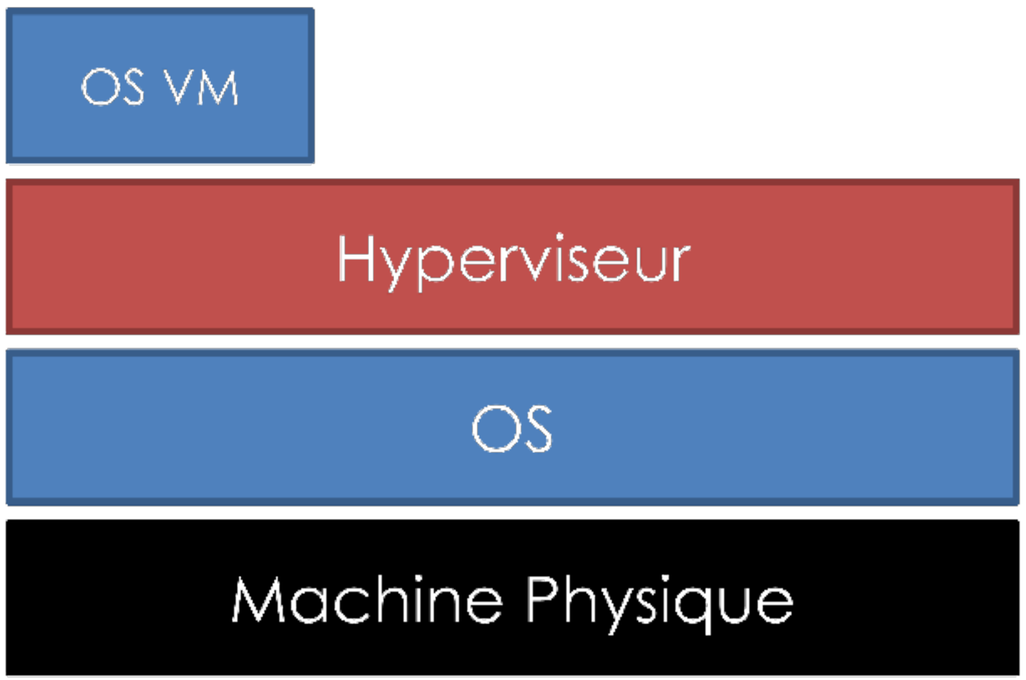
# Conteneur != VM



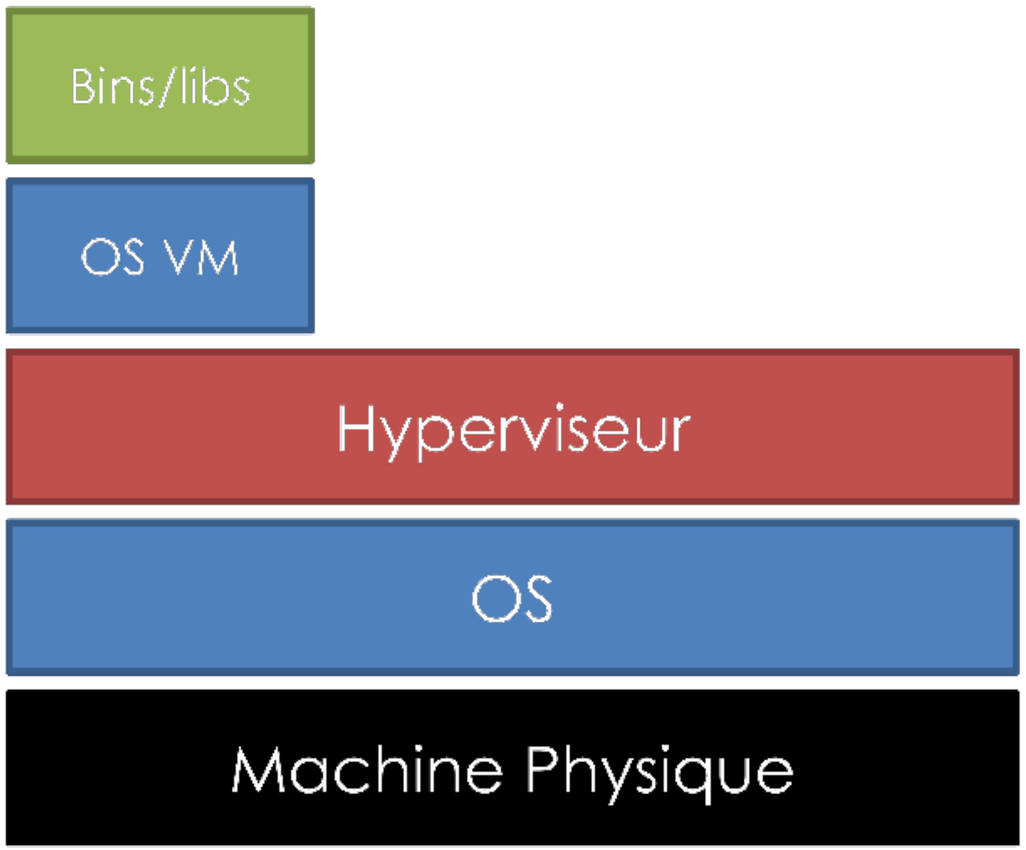
# Conteneur != VM



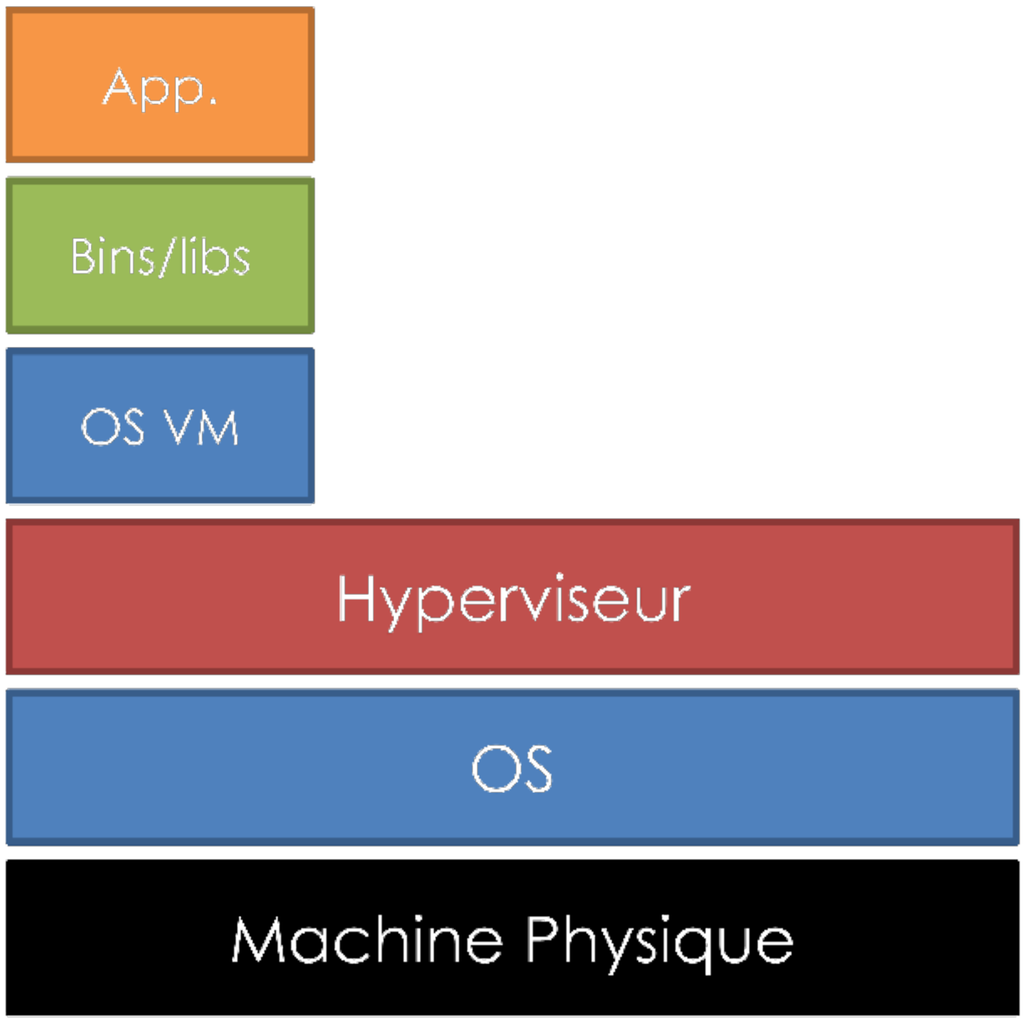
# Conteneur != VM



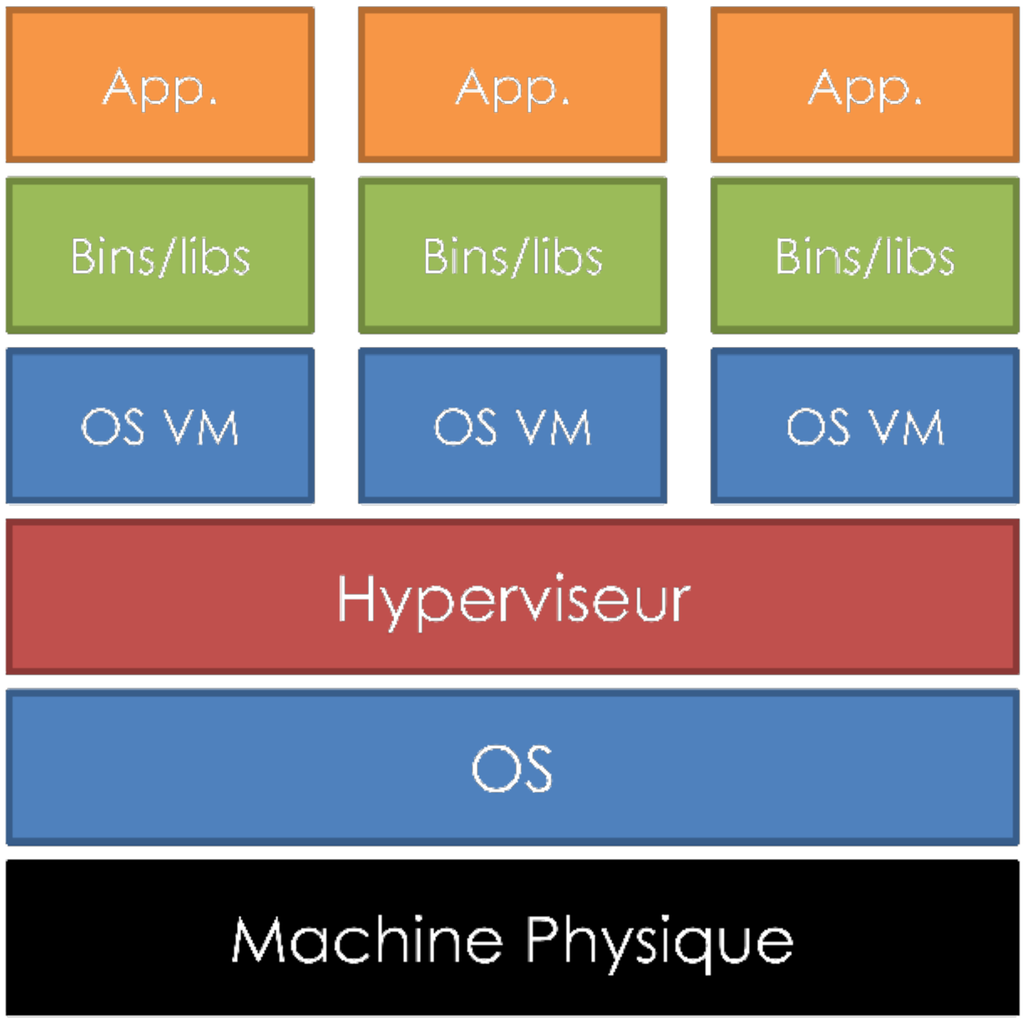
# Conteneur != VM



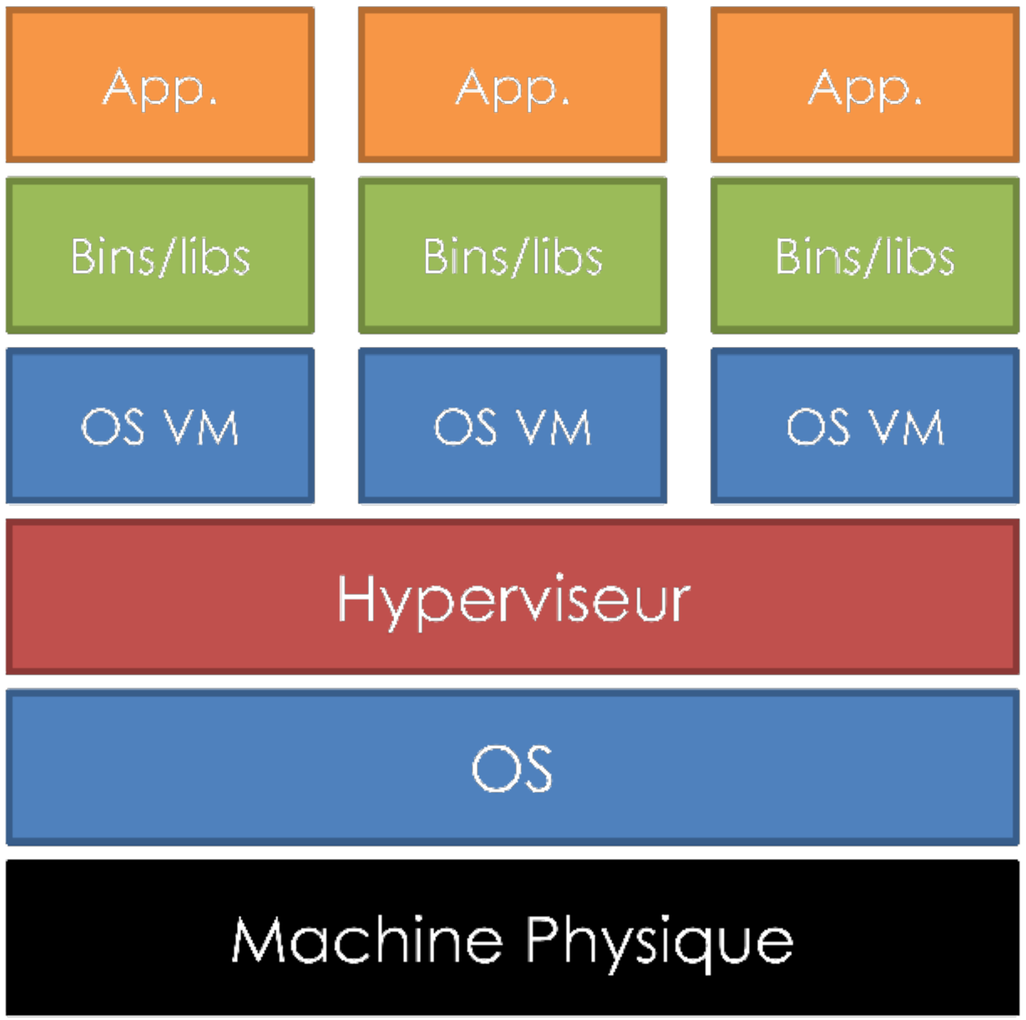
# Conteneur != VM

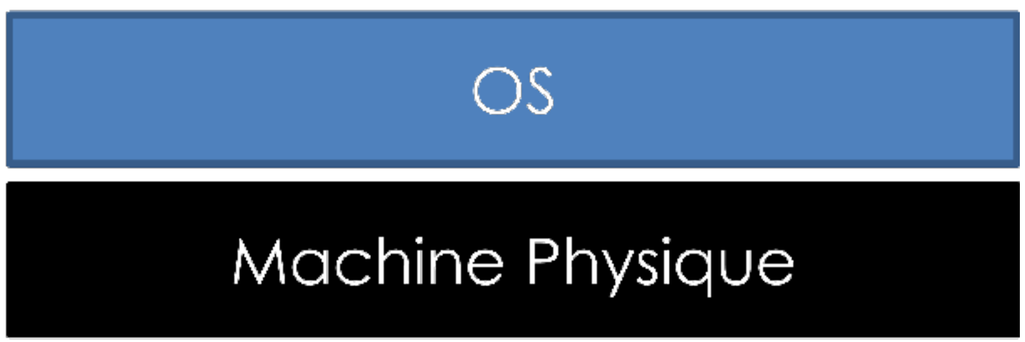


# Conteneur != VM

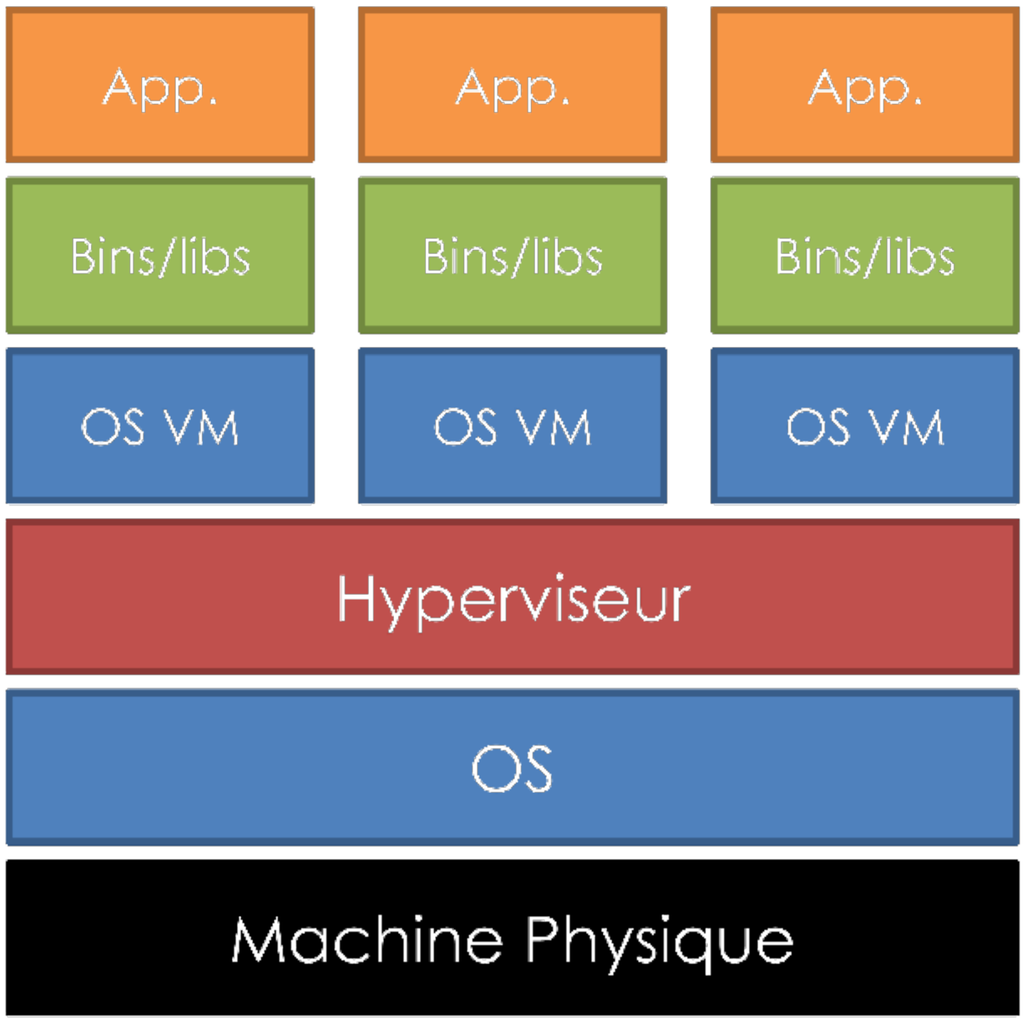


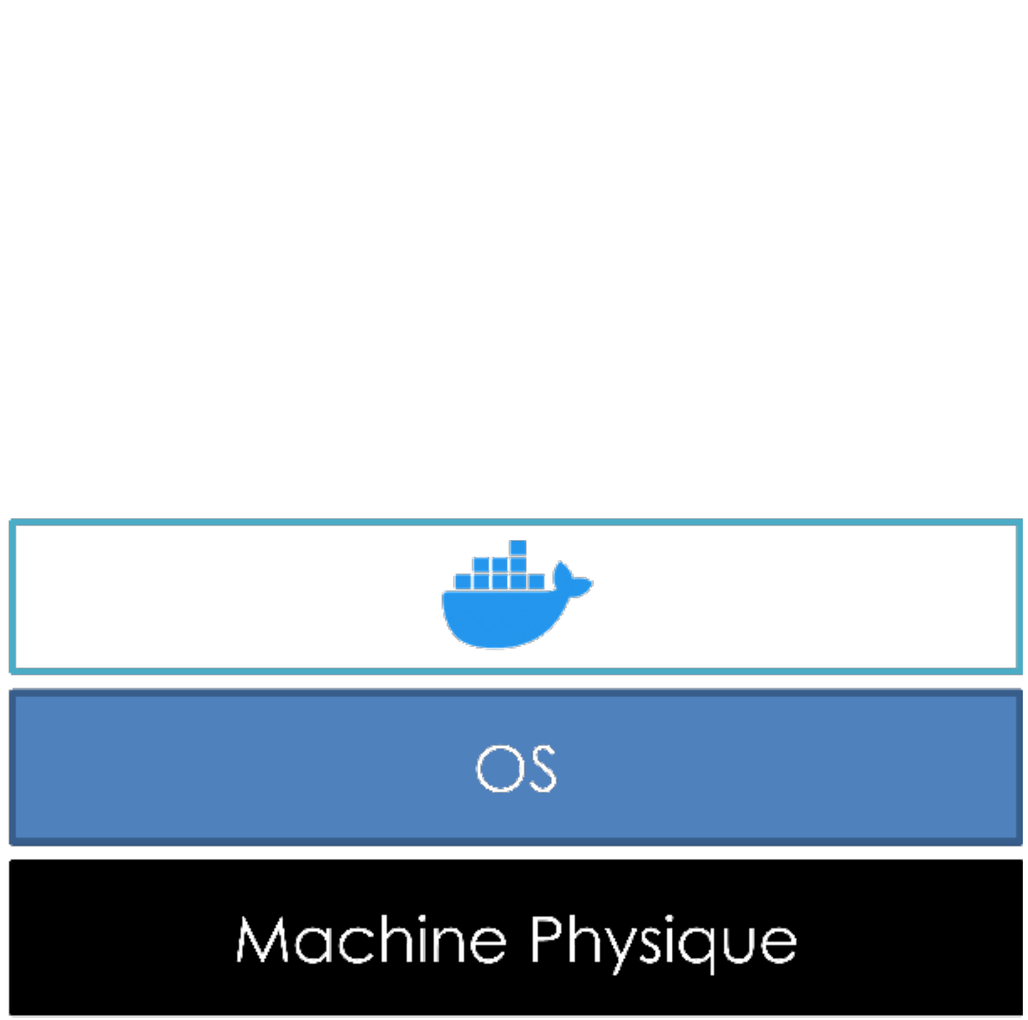
# Conteneur != VM



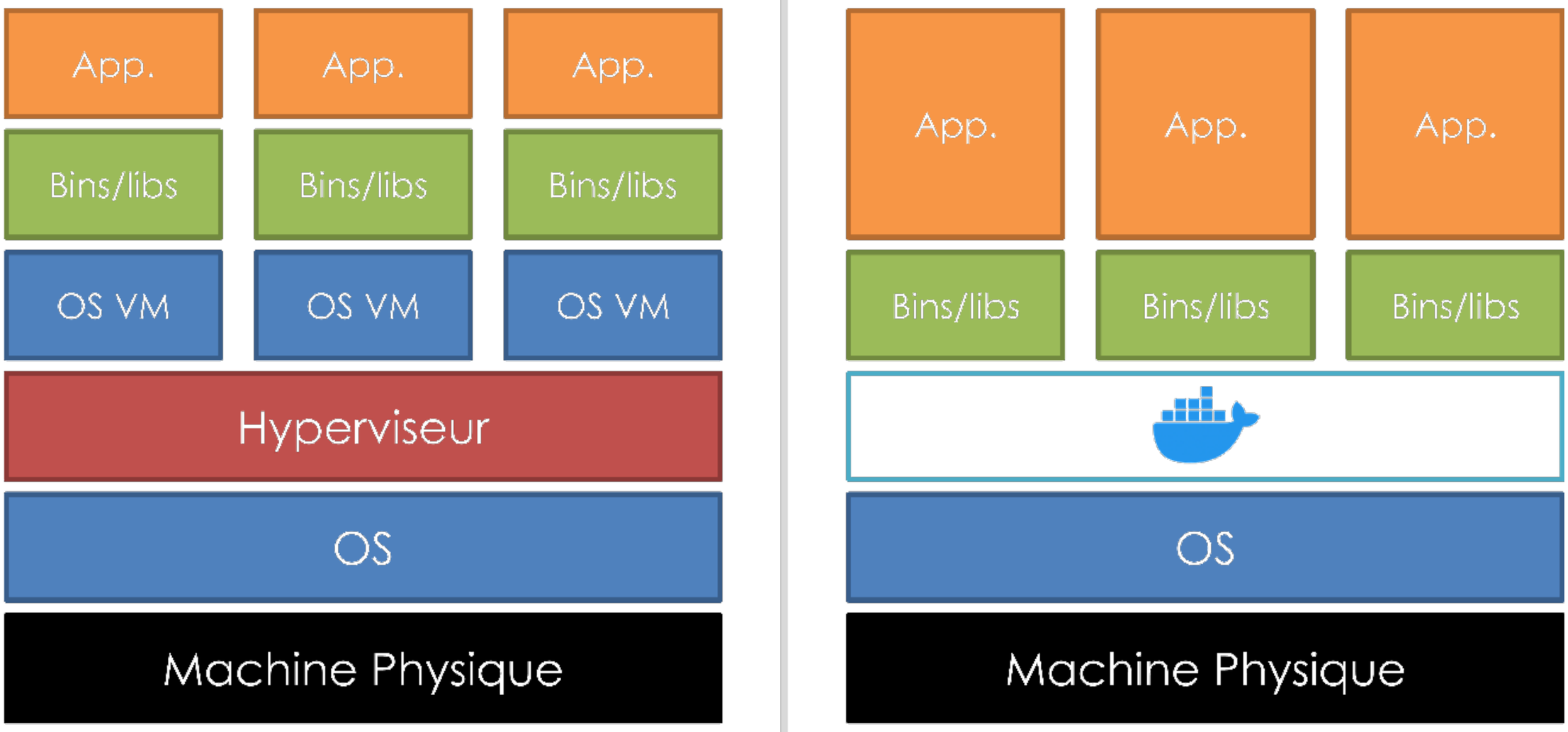



# Conteneur != VM



# Conteneur != VM



# Conteneur != VM

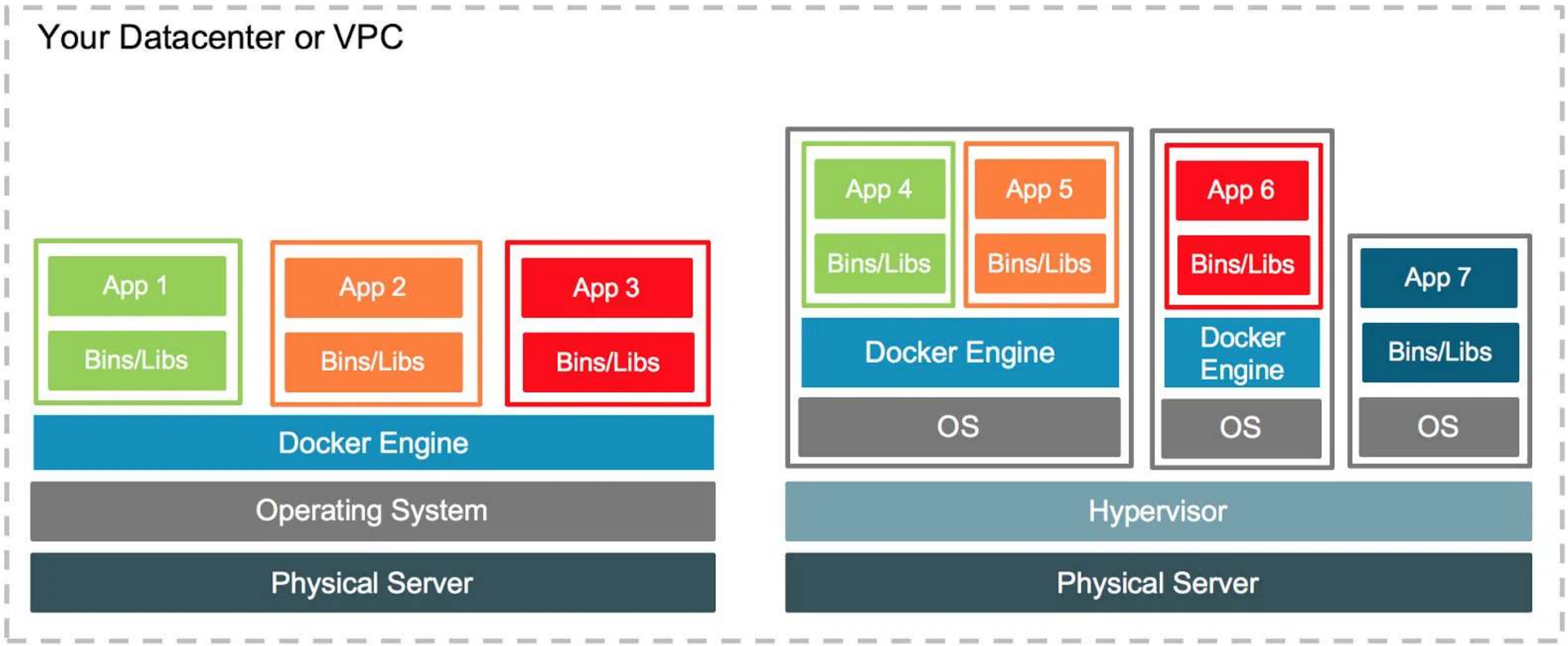
"Separation of concerns": 1 "tâche" par conteneur

VM

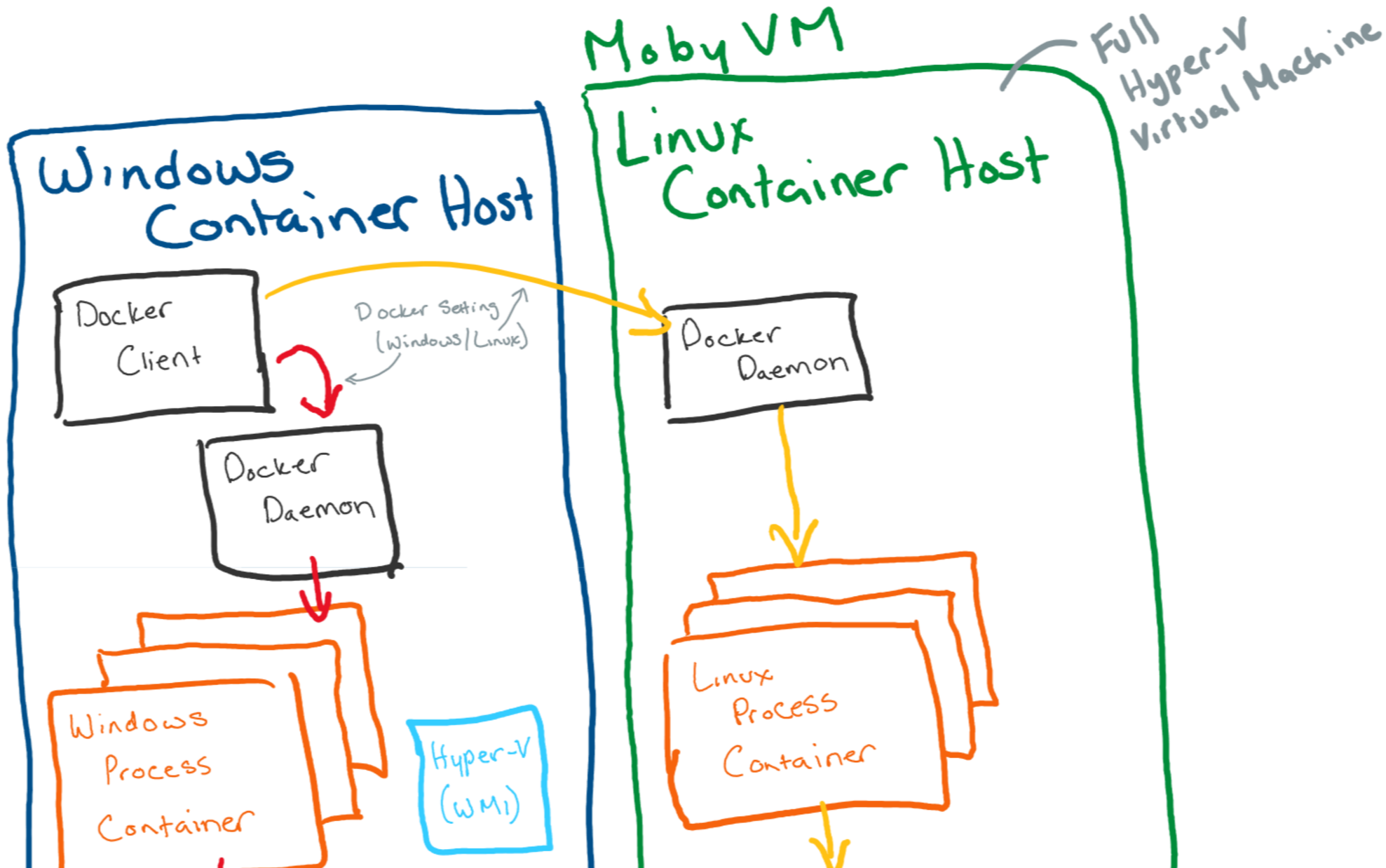
Containers

# VMs & Conteneurs

Non exclusifs mutuellement



# 🤡 Docker, c'est pas un peu une VM quand même?



# 🤡 Docker, c’est pas un peu une VM quand même?

- Docker sur Windows pour exécuter des conteneurs Linux :

- Docker sur Linux pour exécuter des conteneurs Linux :

# Cas d’usage

Le bac à sable

# Cas d’usage

Le bac à sable

- Un environnement tout propre tout neuf !

# Cas d’usage

## Le bac à sable

- Un environnement tout propre tout neuf !
- Le poste de travail n’est pas impacté

# Cas d’usage

## Le bac à sable

- Un environnement tout propre tout neuf !
- Le poste de travail n’est pas impacté
- La configuration reste également isolée sans effet sur le poste de travail

# Cas d’usage

La machine de dév

# Cas d’usage

La machine de dév

- Avoir un environnement reproductible pour rendre homogène le développement et les tests.

# Cas d’usage

## La machine de dév

# Cas d’usage

La machine de dév

# Cas d’usage

## Déploiement en production

# Cas d’usage

## Déploiement en production

- Avec un container, si ca fonctionne en local, ca fonctionne en prod !

# Cas d’usage

## Outillage jetable

# Cas d’usage

## Outillage jetable

- On instancie des applications sans les installer

# Cas d’usage

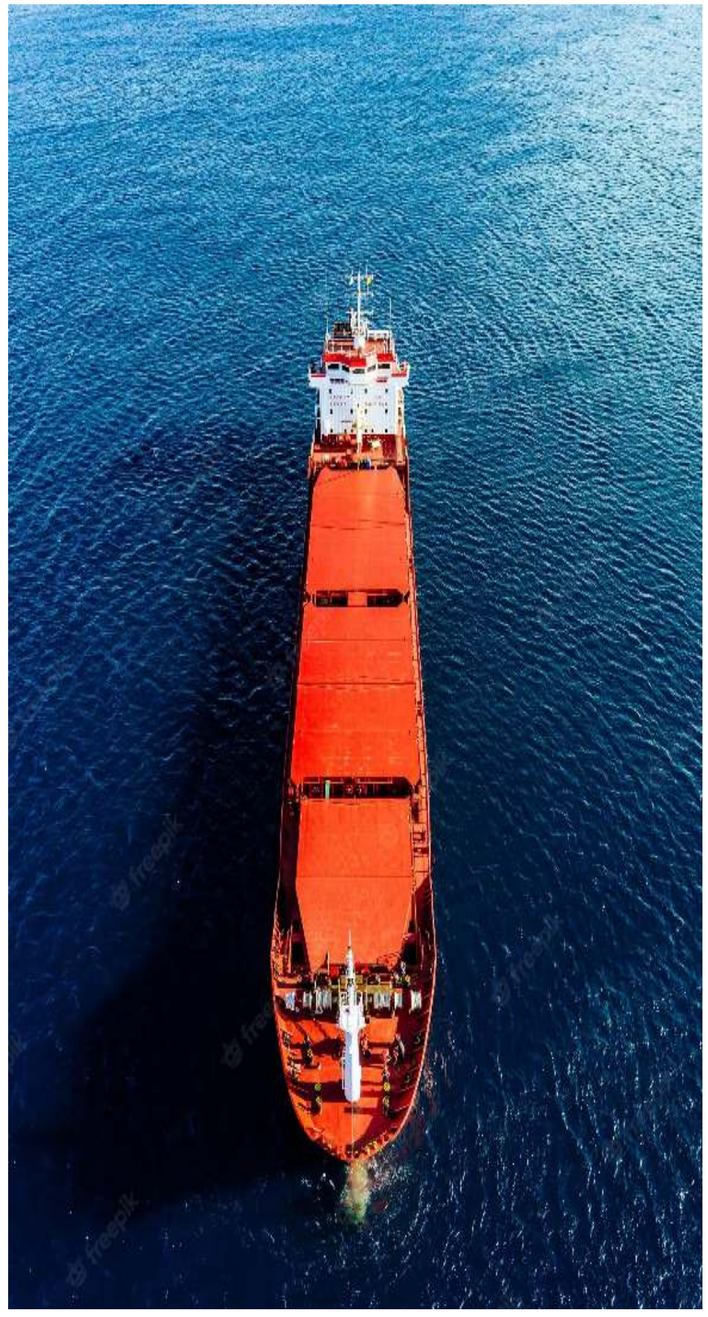
## Outillage jetable

- On instancie des applications sans les installer
- On crée le container, on l’utilise puis on le jette

# Beau boulot! 🤗

On a la base, remplissons là maintenant de containers!

La suite: vers la droite ➡️



# Préparer votre environnement de travail

# Particularités du réseau

# Utiliser Docker derrière un proxy ou un cache universitaire

# Pourquoi ? – Le Problème

Lorsque nous sommes sur le réseau de l'université, Docker a du mal à télécharger directement les images depuis Docker Hub à cause des restrictions réseau. Nous sortons tous avec la même adresse IP, et en peu de temps, elle sera blacklistée pour un moment, ce qui nous pénalisera tous. Pour résoudre ce problème, l'université met à disposition un serveur de cache local.

# 1. Concepts de base Docker

Avant toute configuration, introduisons les notions essentielles, que nous verrons en détail par la suite :

```
# Que sont les images Docker ?
docker image pull hello-world
docker container run hello-world

# Comment les images Docker sont nommées ?
[registry/][username/]image-name[:tag]

# Exemples :
library/ubuntu:22.04    # Image officielle (library est implicite)
ubuntu:22.04            # Identique à ci-dessus
user/mon-app:latest     # Image personnalisée
```

# 2. Le concept de cache de registre

Au lieu de récupérer les images directement depuis Docker Hub, nous utilisons le cache fourni par l'université.

# 2. Le concept de cache de registre

Au lieu de récupérer les images directement depuis Docker Hub, nous utilisons le cache fourni par l'université.

Avant (problème) :

```
docker image pull ubuntu:22.04    # Échec sur le réseau universitaire, ou succès temporaire avant d'être blacklistés
```

# 2. Le concept de cache de registre

Au lieu de récupérer les images directement depuis Docker Hub, nous utilisons le cache fourni par l'université.

Avant (problème) :

```
docker image pull ubuntu:22.04   # Échec sur le réseau universitaire, ou succès temporaire avant d'être blacklistés
```

Après (solution) :

```
docker image pull cache-ili.univ-artois.fr:80/proxy_cache/library/ubuntu:22.04
```

# 3. Approche simplifiée de configuration

Option A : utiliser un **registry mirror**.

# 3. Approche simplifiée de configuration

Option A : utiliser un **registry mirror**.

Modifier `/etc/docker/daemon.json` :

```
{
  "registry-mirrors": ["http://cache-ili.univ-artois.fr:80/proxy_cache/"],
  "insecure-registries": ["cache-ili.univ-artois.fr:80"]
}
```

# 3. Approche simplifiée de configuration

Option A : utiliser un **registry mirror**.

Modifier `/etc/docker/daemon.json` :

```
{
  "registry-mirrors": ["http://cache-ili.univ-artois.fr:80/proxy_cache/"],
  "insecure-registries": ["cache-ili.univ-artois.fr:80"]
}
```

Désormais, les commandes Docker standards fonctionnent :

```
docker image pull ubuntu:22.04   # Utilise automatiquement le cache !
docker container run gradle      # Fonctionne de manière transparente
```

# 4. Script étape par étape

## Étape 1 – Comprendre le problème

- Docker Hub nous voit tous avec la même adresse IP, qui mène au blacklist
- Nous devons utiliser le serveur de cache local.

# 4. Script étape par étape

## Étape 1 – Comprendre le problème

- Docker Hub nous voit tous avec la même adresse IP, qui mène au blacklist
- Nous devons utiliser le serveur de cache local.

## Étape 2 – Configurer Docker une seule fois

```
sudo nano /etc/docker/daemon.json
```

# 4. Script étape par étape

## Étape 1 – Comprendre le problème

- Docker Hub nous voit tous avec la même adresse IP, qui mène au blacklist
- Nous devons utiliser le serveur de cache local.

## Étape 2 – Configurer Docker une seule fois

```
sudo nano /etc/docker/daemon.json
```

Ajouter :

```
{
  "registry-mirrors": ["http://cache-ili.univ-artois.fr:80/proxy_cache/"],
  "insecure-registries": ["cache-ili.univ-artois.fr:80"]
}
```

# 4. Script étape par étape

## Étape 1 – Comprendre le problème

- Docker Hub nous voit tous avec la même adresse IP, qui mène au blacklist
- Nous devons utiliser le serveur de cache local.

## Étape 2 – Configurer Docker une seule fois

```
sudo nano /etc/docker/daemon.json
```

Ajouter :

```
{
  "registry-mirrors": ["http://cache-ili.univ-artois.fr:80/proxy_cache/"],
  "insecure-registries": ["cache-ili.univ-artois.fr:80"]
}
```

## Étape 3 – Redémarrer Docker

```
sudo systemctl restart docker
```

# 4. Script étape par étape

## Étape 3 – Redémarrer Docker

```
sudo systemctl restart docker
```

# Étape 4 – Utiliser Docker normalement

```
docker image pull ubuntu:22.04    # Utilise automatiquement le cache
docker container run python:3.9   # Fonctionne sans problème
```

# 4. Script étape par étape

## Étape 4 – Utiliser Docker normalement

```
docker image pull ubuntu:22.04    # Utilise automatiquement le cache
docker container run python:3.9   # Fonctionne sans problème
```

## Étape 5 – Explication visuelle

Votre client Docker → Cache universitaire → Docker Hub (si nécessaire) (cache-ili.univ-artois.fr)

**Avantages** : - Téléchargements plus rapides (images déjà en cache) - Fonctionne derrière le pare-feu universitaire - Pas besoin de changer vos commandes

# 4. Script étape par étape

## Étape 5 – Explication visuelle

Votre client Docker → Cache universitaire → Docker Hub (si nécessaire) (cache-ili.univ-artois.fr)

**Avantages** : - Téléchargements plus rapides (images déjà en cache) - Fonctionne derrière le pare-feu universitaire - Pas besoin de changer vos commandes

## Étape 6 – Problèmes courants

```
# Permission refusée ? Ajoutez votre utilisateur au groupe docker :
sudo usermod -aG docker $USER
newgrp docker    # Ou déconnectez-vous / reconnectez-vous

# Toujours un souci ? Testez le cache :
docker image pull cache-ili.univ-artois.fr:80/proxy_cache/library/hello-world
```

# 5. Proxy HTTP pour les conteneurs et les builds

Le cache de registre ne concerne que les **images** (`docker image pull`, `FROM`).

Ce qui se passe **dans** un conteneur ou dans un `RUN` (`apk add`, `pip install`, `curl`) sort sur le réseau et doit passer par le proxy HTTP de l'université : `cache-etu.univ-artois.fr:3128`.

# 5. Proxy HTTP pour les conteneurs et les builds

## Option 1 – `~/.docker/config.json` (Recommandé)

Créer ou modifier `~/.docker/config.json` :

```
{
  "proxies": {
    "default": {
      "httpProxy": "http://cache-etu.univ-artois.fr:3128",
      "httpsProxy": "http://cache-etu.univ-artois.fr:3128",
      "noProxy": "localhost,127.0.0.1"
    }
  }
}
```

# 5. Proxy HTTP pour les conteneurs et les builds

## Option 1 – `~/.docker/config.json` (Recommandé)

Le client Docker ajoute alors les variables de proxy :

- à chaque **nouveau** conteneur (`docker container run`) ;
- à chaque `docker image build`, comme arguments de build.

```
docker container run --rm alpine:3.24.2 env | grep -i proxy
# HTTP_PROXY=http://cache-etu.univ-artois.fr:3128
# HTTPS_PROXY=... NO_PROXY=... et les formes en minuscules
```

L'image construite, elle, ne garde rien : `docker image inspect` ne montre que `PATH` dans `Config.Env`. Aucun redémarrage n'est nécessaire, et le `Dockerfile` reste le même partout.

# 5. Proxy HTTP pour les conteneurs et les builds

## Option 1 – Deux limites

- **Les noms de conteneurs** : `noProxy` ne les couvre pas. Un `curl web2` entre deux conteneurs part vers le proxy. Utilisez `curl --noproxy '*' web2`.
- **Maven** ignore `HTTP_PROXY` et `HTTPS_PROXY`. Il lui faut un `settings.xml` (voir les travaux pratiques #7). `apk`, `wget`, `curl` et `git`, eux, suivent les variables.

# 5. Proxy HTTP pour les conteneurs et les builds

## Option 2 – Arguments de build, pour un build ponctuel

```
docker image build \
  --build-arg HTTP_PROXY=http://cache-etu.univ-artois.fr:3128 \
  --build-arg HTTPS_PROXY=http://cache-etu.univ-artois.fr:3128 .
```

Dockerfile correspondant, **sans** ligne `ARG` :

```
FROM alpine:3.24.2
RUN apk add --no-cache curl
```

`HTTP_PROXY`, `HTTPS_PROXY`, `NO_PROXY` (et leurs minuscules) sont des arguments prédéfinis : les `RUN` les voient sans déclaration.

# 5. Proxy HTTP pour les conteneurs et les builds

## Option 2 – Arguments de build : le piège de `ARG`

Si le Dockerfile déclare `ARG HTTPS_PROXY`, la valeur apparaît dans l'historique de l'image :

```
docker image history --no-trunc --format '{{.CreatedBy}}' mon-image | grep -i proxy
# RUN |1 HTTPS_PROXY=http://cache-etu.univ-artois.fr:3128 /bin/sh -c apk add --no-cache curl # buildkit
# ARG HTTPS_PROXY=http://cache-etu.univ-artois.fr:3128
```

Sans la ligne `ARG`, les arguments prédéfinis sont exclus de `docker image history`. Ne déclarez donc **pas** les variables de proxy dans le Dockerfile.

# 5. Proxy HTTP pour les conteneurs et les builds

## Option 3 – Proxy avec identifiants : secret de build

Un proxy qui demande un login contient le mot de passe dans son URL. Pour le passer à **un seul** `RUN` sans qu'il reste nulle part, BuildKit propose les secrets de build :

```
FROM alpine:3.24.2
RUN --mount=type=secret,id=proxy,env=HTTPS_PROXY \
    apk add --no-cache curl
```

```
read -rsp 'URL du proxy : ' PROXY_URL; echo    # saisie masquée
PROXY_URL="$PROXY_URL" docker image build --secret id=proxy,env=PROXY_URL .
unset PROXY_URL
```

Le mot de passe ne passe ni par l'historique du shell, ni par un `export` : seul ce `docker image build` le reçoit.

Le secret n'existe que pendant ce `RUN` : ni `docker image history`, ni `Config.Env` ne le contiennent.

# 5. Proxy HTTP pour les conteneurs et les builds

## Option 4 – `ENV` dans le Dockerfile : à éviter

```
FROM alpine:3.24.2
ENV HTTPS_PROXY=http://cache-etu.univ-artois.fr:3128
RUN apk add --no-cache curl
```

Ça fonctionne (`ENV` après `FROM` et avant le `RUN`), mais la valeur est **gravée dans l'image** :

- elle apparaît dans `docker image history` et dans `Config.Env` ;
- chaque conteneur lancé depuis cette image, partout, essaiera de passer par le proxy de l'université ;
- si l'URL contient un mot de passe, il est livré avec l'image.

À réserver au dépannage, sur une image que vous ne partagez pas.

# 5. Proxy HTTP pour les conteneurs et les builds

## Option 5 – `daemon.json` : le proxy du démon seulement

```
{
  "proxies": {
    "http-proxy": "http://cache-etu.univ-artois.fr:3128",
    "https-proxy": "http://cache-etu.univ-artois.fr:3128",
    "no-proxy": "localhost,127.0.0.1"
  }
}
```

Ce bloc de `/etc/docker/daemon.json` sert au **démon** : `pull` et `push` vers les registres, communication entre nœuds d'un swarm. Il ne s'applique **ni** aux processus des conteneurs, **ni** aux `RUN` d'un build : `apk add` échoue toujours. Docker Desktop l'ignore (le proxy se règle dans ses paramètres).

# 6. En résumé

| **Méthode** | **Conteneurs** | **`RUN` d'un build** | **Dans l'image** |
|---|---|---|---|
| `~/.docker/config.json` | oui | oui | non |
| `--build-arg`, sans `ARG` | non | oui | non |
| `--build-arg` + `ARG` | non | oui | historique |
| secret de build | non | un `RUN` | non |
| `ENV` | oui | oui | oui |
| `daemon.json` | non | non | non |

# 6. En résumé (suite)

- Pour la salle de TP : `~/.docker/config.json`.
- Pour un proxy avec mot de passe : un secret de build.
- Le cache de registre (`registry-mirrors`) reste utile, mais pour les images seulement.

# Environnement Cloud : GitHub Codespaces

# Outils Nécessaires 🛠️

- Un navigateur web récent (et décent)
- Un compte sur GitHub

# GitHub Codespaces

GitHub Codespaces : Environnement de développement dans le ☁️ "nuage"

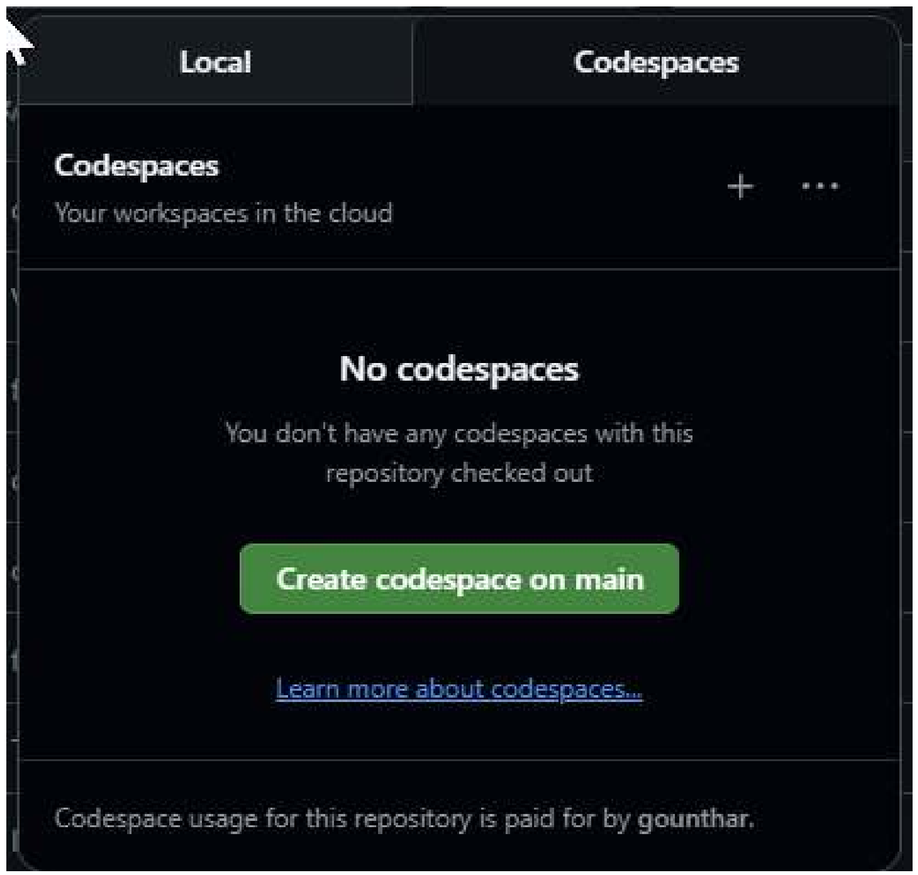
- **But:** reproductible et instantané
- Puissance de calcul sur un serveur distant
- Éditeur VSCode dans le navigateur
- Docker **déjà installé et configuré** 🐳

# Pourquoi Codespaces ? 🤔

# Prérequis 📋

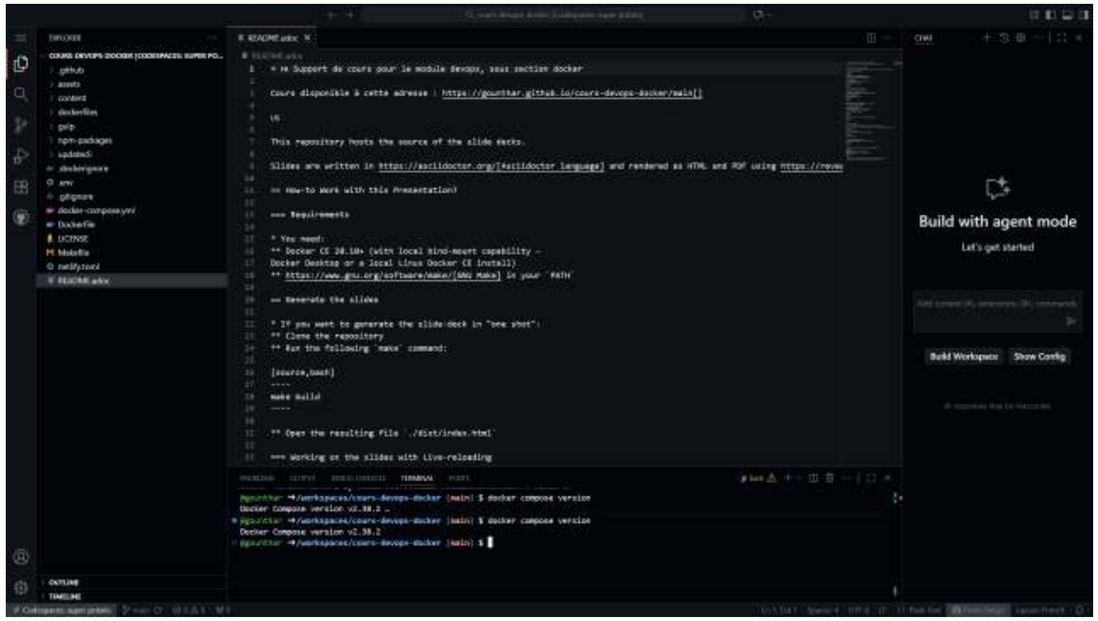
Vous avez déjà ce qu’il faut ! ✓

# Démarrer avec Codespaces 🚀




⚠️ Passez à la slide suivante pour voir l'interface

# Interface Codespaces 💻




**Gauche** : Explorateur de fichiers
→ Arborescence, Git

**Centre** : Éditeur de code
→ Coloration, autocomplétion

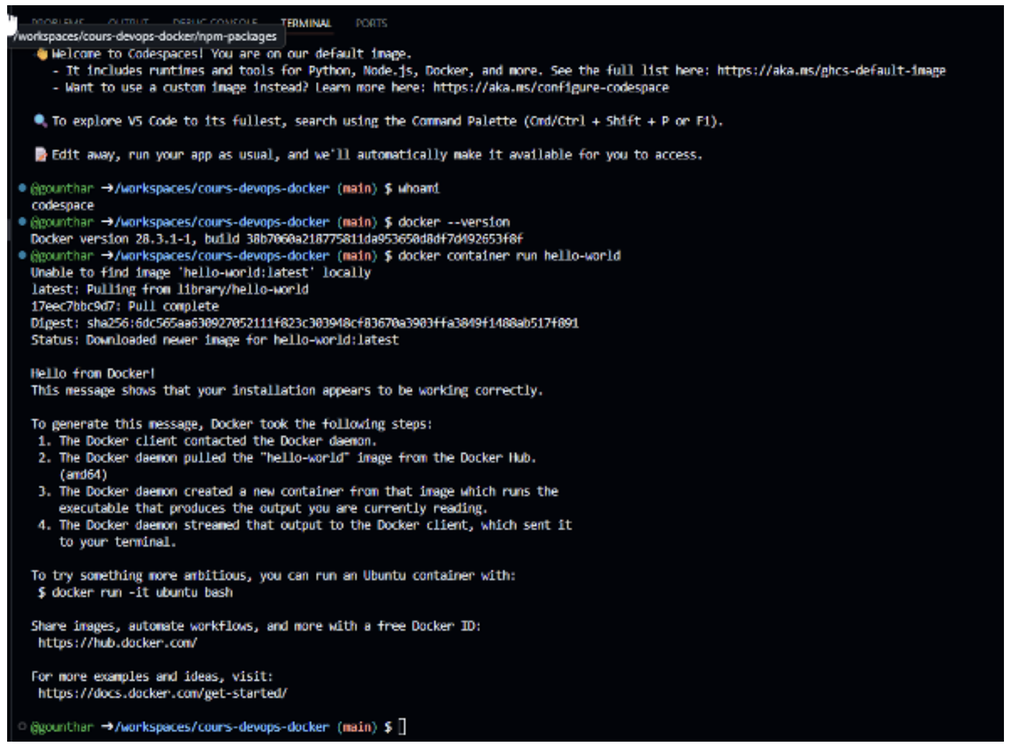
**Bas** : Terminal intégré
→ Bash/Zsh, Docker

# Terminal Codespaces

Le terminal est votre outil principal pour Docker !

```
# Vérifier l'utilisateur
whoami
# Résultat attendu : codespace

# Vérifier Docker
docker --version
# Résultat attendu : Docker version XX.XX.X

# Tester Docker
docker container run hello-world
```

⚠️ Passez à la slide suivante pour comprendre la configuration



# Configuration Codespaces (Optionnelle) ⚙️

Codespaces peut être personnalisé avec `.devcontainer/`

```
// .devcontainer/devcontainer.json
{
  "name": "Docker DevOps Course",
  "image": "mcr.microsoft.com/devcontainers/base:ubuntu",
  "features": {
    "ghcr.io/devcontainers/features/docker-in-docker:2": {}
  }
}
```

# Checkpoint 🎯

Vérifiez que tout fonctionne :

# Gestion de votre Codespace 🔧

Optimisez votre quota gratuit de 60h/mois :

💡 **Astuce** : 60h = 2h/semaine pendant 30 semaines (largement suffisant !)

# Avantages de Codespaces 🌟

**Pour vous :**

# Pour le cours :

# Limites et Alternatives ⚠️

**Limites de Codespaces :**

💡 Vous pouvez combiner les deux approches !

# Ressources et Aide 📚

**Documentation officielle :**

- GitHub Codespaces Docs
- VSCode Documentation
- Docker Documentation

**Support :**

- 🙋 Questions pendant le cours
- 💬 Forum/Discord du cours (si disponible)
- 🐛 GitHub Issues pour bugs

💡 N'hésitez jamais à poser des questions !

# Recap : GitHub Codespaces ✅

Ce que vous devez retenir :

➡️ Prêts pour commencer à utiliser Docker !

# Installation locale de Docker

# Installer Docker sur Ubuntu 26.04

## **Le piège :** `apt install docker.io` n'installe pas ce qu'il faut

⚠️ Ni `docker compose`, ni `docker buildx`, et Docker demande de retirer ce paquet

# Pourquoi pas `docker.io` ? 🤔

### Ce que donne `apt install docker.io`

```
docker compose version
# docker: unknown command: docker compose

docker buildx version
# docker: unknown command: docker buildx
```

Les deux plugins sont en `Suggests:`, donc apt ne les installe pas

### Et la doc Docker demande de le retirer

`docker.io` figure sur la liste officielle des paquets à désinstaller **avant** d'installer Docker, aux côtés de `podman-docker`.

➡️ On utilise donc le **dépôt apt officiel Docker**

# ⚠️ D’abord : êtes-vous derrière le proxy ? 🏫

Sur le réseau de l’université, `apt` et `curl` doivent passer par le proxy. Définir `http_proxy` dans son shell **ne suffit pas** : `sudo` efface les variables d’environnement.

```
sudo tee /etc/apt/apt.conf.d/99proxy <<'PROXY'
Acquire::http::Proxy "http://cache-etu.univ-artois.fr:3128/";
Acquire::https::Proxy "http://cache-etu.univ-artois.fr:3128/";
PROXY

export http_proxy=http://cache-etu.univ-artois.fr:3128/
export https_proxy=http://cache-etu.univ-artois.fr:3128/
```

🏠 Chez vous, rien de tout ça : passez directement à l’étape 1

# Étape 1 : retirer les paquets en conflit 🧹

```
sudo apt remove $(dpkg --get-selections \
  docker.io docker-compose docker-compose-v2 \
  docker-doc docker-buildx podman-docker \
  containerd runc | cut -f1)
```

✅ Sans effet si aucun n'est installé — la commande ne retire rien et se termine normalement

# Étape 2 : ajouter le dépôt officiel 🔑

```
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /tmp/docker.asc
sudo install -m 0644 /tmp/docker.asc /etc/apt/keyrings/docker.asc
```

🔑 La clé GPG sert à vérifier la signature des paquets téléchargés ⚠️ `curl` sans `sudo`, volontairement : lancé en root il perdrait votre proxy

# Étape 2 (suite) : déclarer la source 📝

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

# Étape 3 : installer et vérifier ✅

```
sudo apt install docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin

sudo usermod -aG docker $USER
```

⚠️ Logout/login requis pour que le groupe `docker` s'applique

# Ça marche ? 🎯

```
docker --version
docker compose version
docker buildx version
docker container run hello-world
```

✅ Cette fois `docker compose` et `docker buildx` répondent

# Et si Podman est installé ? 🤔

**Symptôme :** `docker` semble installé, mais répond `Cannot connect to the Docker daemon`

C'est le paquet `podman-docker` qui pose un `/usr/bin/docker` redirigeant vers Podman. Podman seul ne fournit aucune commande `docker` : on obtiendrait `command not found`.

```
# Qui répond, au juste ?
command -v docker
echo $DOCKER_HOST
docker context ls

# Retirer le wrapper suffit
sudo apt remove podman-docker
```

⚠️ Ubuntu **n'installe pas** Podman par défaut : ce cas ne concerne que qui l'a installé lui-même

# Et si le téléchargement bloque ?

**Symptôme :** `curl` ou `apt` restent muets, puis expirent. Pourtant `echo $http_proxy` affiche bien le proxy, et la même commande **sans** `sudo` passe.

```
# Le proxy est-il visible par root ?
sudo printenv http_proxy
# (aucune sortie)

# Trois sorties possibles
sudo -E curl ...
curl ... puis sudo install ...
/etc/apt/apt.conf.d/99proxy
```

`sudo` repart d'un environnement neuf : `sudoers(5)` appelle ça `env_reset`, actif par défaut. `http_proxy` n'est dans aucune liste conservée, donc le `curl` lancé en root sort en direct et se fait jeter par le réseau.

💡 Rien dans le message d'erreur ne nomme `sudo` : c'est ce qui rend la panne longue à diagnostiquer

# Environnement prêt ! ✅

Votre environnement Docker est maintenant configuré 🎉

- ✅ GitHub Codespaces : Docker déjà installé
- ✅ Ubuntu 26.04 : Docker installé par le dépôt officiel
- ✅ Prêt pour la suite !

# Prochaine étape 🚀

Avant de découvrir Docker, un détour essentiel…

# Pourquoi la CLI ? 💻

# Une VM à vous sur XCP-ng

# La virtualisation, depuis le début 🖥️

**Objectif des deux séances :** faire tourner Docker dans **votre** machine virtuelle, sur un hyperviseur XCP-ng

- Séance 1 : comprendre ce qu'est une VM, et ce qui la rend possible
- Séance 2 : créer la vôtre, y entrer en SSH, y installer Docker

Les types d'hyperviseur et « VM et conteneur ne s'opposent pas » ont été vus dans l'introduction : on part de là.

# Pourquoi virtualiser ? 🤔

# Une vieille idée

# Le x86 ne respectait pas la règle 🚫

```
anneau 3   applications
anneau 2   (inutilisé)
anneau 1   (inutilisé)
anneau 0   noyau de l'OS
```

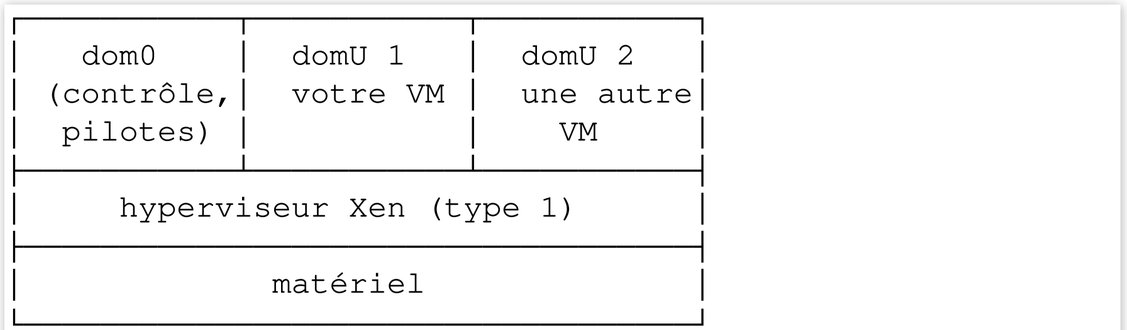
- L’OS invité veut l’anneau 0, mais l’hyperviseur l’occupe déjà
- **17 instructions** du Pentium sont sensibles sans être privilégiées : elles s’exécutent sans piège et **mentent** à l’invité
- Exemple : `POPF` ignore en silence le changement du drapeau d’interruption hors de l’anneau 0

# Trois façons de contourner le problème 🛠️

| Approche | Principe | Prix |
|---|---|---|
| **Traduction binaire** (VMware) | Réécrire à la volée le code noyau de l'invité | Complexité |
| **Paravirtualisation** (Xen, 2003) | Modifier l'OS invité : il appelle l'hyperviseur (*hypercall*) | Un noyau adapté |
| **Assistance matérielle** (VT-x, AMD-V) | Un mode « invité » dans le processeur, qui piège ce qu'il faut | Presque rien |

# Et la mémoire ?

# Xen, l'architecture 🏗️



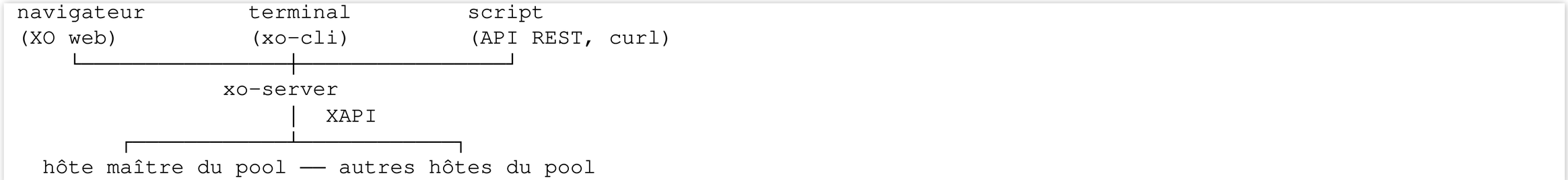

- L'hyperviseur Xen est **petit** : CPU, mémoire, interruptions
- Le **dom0** est une VM privilégiée : il porte les pilotes matériels et l'outillage
- Les **domU** sont les VM « normales », dont la vôtre

# PV, HVM, PVH : les modes d’une VM Xen 🔀

# L’agent invité 🕵️

- Un petit service **dans** la VM, qui renseigne l’hôte : OS, mémoire, **adresses IP**
- Sans lui, Xen Orchestra ne sait pas quelle IP a obtenue votre VM
- Sous Linux : `xe-guest-utilities` (service `xe-linux-distribution`), ou son successeur en Rust, `xen-guest-agent`

# Un pool, XAPI et Xen Orchestra 🧭




- Plusieurs hôtes XCP-ng forment un **pool**, piloté par son **maître**
- **XAPI** est l'outillage natif ; `xe` est sa ligne de commande sur le dom0
- **Xen Orchestra** parle XAPI, et expose une interface web, `xo-cli` et une **API REST**

# Ce qu’on donne à une VM : CPU et mémoire

# Ce qu'on donne à une VM : disque 💾

```
SR  (Storage Repository)   le stockage : disque local, NFS, iSCSI...
 └── VDI (Virtual Disk Image)   un disque virtuel, un fichier ou un volume
       └── VBD (Virtual Block Device)   le branchement du VDI dans la VM
```

- Votre VM voit `/dev/xvda`, sans savoir où il est vraiment
- SR **local** : rapide, mais la VM est liée à cet hôte
- SR **partagé** : la VM peut changer d'hôte sans s'arrêter

# Ce qu'on donne à une VM : réseau 🌐

```
VM ── VIF (carte réseau virtuelle)
        └── réseau XCP-ng (un bridge sur l'hôte, éventuellement un VLAN)
              └── PIF (carte réseau physique de l'hôte)
```

- Chaque VM a une ou plusieurs **VIF**
- Les VIF branchées sur le même réseau se voient, comme sur un switch
- Le **VLAN** sépare des réseaux sur le même câble : un réseau par étudiant, par exemple

# Templates et cloud-init 📦

- Un **template** est une VM modèle, jamais démarrée, qu'on clone
- **cloud-init** configure la VM **au premier démarrage** : nom, utilisateur, **clés SSH**, taille du disque
- Résultat : une VM prête en une minute, sans passer par un installateur

# Le cycle de vie d’une VM 🔄

# Démo : sous le capot, `xe` 🔧

```
# Sur le dom0 d'un hôte XCP-ng (démo enseignant uniquement)
xe vm-list power-state=running params=name-label,uuid
xe vm-param-list uuid=<UUID>
xe vbd-list vm-uuid=<UUID> params=device,vdi-uuid
```

- Ce que Xen Orchestra affiche vient d’ici : XAPI
- Vous n’aurez pas accès au dom0 : sur un pool partagé, c’est la clé de toutes les VM

# Déléguer : le self-service 🎟️

- L'administrateur crée un **resource set** : des templates, un stockage, un réseau, et des **quotas** (vCPU, mémoire, disque)
- Il le confie à un utilisateur ou à un groupe
- L'utilisateur crée **ses** VM, dans ce bac à sable, et pas ailleurs
- C'est exactement ce que vous aurez en séance 2

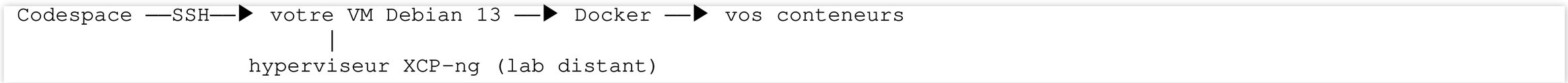
# Avant la prochaine séance ✅

1. Connectez-vous à Xen Orchestra avec le compte fourni
2. Repérez votre **resource set** : template, stockage, réseau
3. Notez vos **quotas**
4. Ne créez pas encore de VM

Prochaine séance : votre VM, SSH depuis le Codespace, puis Docker.

# 🎓 Docker dans votre VM XCP-ng

# 🎓 Travaux pratiques VM : Docker dans ma VM




Objectif : une VM à vous, créée par vous, dans laquelle `docker run` fonctionne.

# Étape 1 : une clé SSH, dans le Codespace 🔑

```
ssh-keygen -t ed25519 -C "tp-vm" -f ~/.ssh/tp-vm
cat ~/.ssh/tp-vm.pub
```

- Copiez la ligne affichée : c'est la **clé publique**, elle peut circuler
- `~/.ssh/tp-vm` sans `.pub` est la clé **privée** : elle ne quitte jamais le Codespace

# Étape 2 : créer sa VM dans Xen Orchestra

1. Bouton **New VM**
2. Votre **resource set**, puis le template **Debian 13** (les seuls proposés)
3. Nom : `tp-vm-<votre-login>`
4. Ressources : restez dans vos quotas
5. Partie cloud-init : cochez **SSH key**, collez votre clé publique
6. **Create**

Bouton **Create** grisé ? Vous demandez plus que ce qui reste dans votre quota.

# Étape 3 : premier contact, par la console 👀

- Ouvrez votre VM dans Xen Orchestra, onglet **Console**
- Vous voyez le démarrage, puis une invite de connexion
- Une fois la VM **Running**, l'onglet **General** affiche son **adresse IP**

Pas d'IP affichée ? C'est l'agent invité qui la remonte : attendez la fin du démarrage, puis prévenez l'enseignant.

# Étape 4 : entrer en SSH depuis le Codespace 🚪

```
# Depuis le terminal du Codespace
ssh -i ~/.ssh/tp-vm debian@<IP-de-votre-VM>
```

- `debian` est l'utilisateur créé par cloud-init dans les images Debian
- La première connexion demande de confirmer l'empreinte de la machine : `yes`

# Étape 5 : installer Docker 🐳

Même méthode que sur votre poste (dépôt officiel), avec l'URL **Debian** :

```
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Pas de proxy ici : la VM est sur le lab, pas sur le réseau de l'université.

# Étape 5 (suite) : déclarer le dépôt et installer 📝

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker $USER
```

Déconnectez-vous puis reconnectez-vous, pour que le groupe `docker` soit pris en compte.

# Étape 6 : ça marche ? ✅

```
docker run --rm hello-world
docker info --format '{{.OperatingSystem}} / {{.KernelVersion}}'
```

- Puis rejouez un TP déjà fait plus tôt, dans la VM cette fois

Quel noyau affiche `docker info` ? Celui de la VM, pas celui de l'hôte XCP-ng.

# Étape 7 : snapshot, bêtise, revert 🔁

1. Dans Xen Orchestra : onglet **Snapshots**, créez `docker-ok`
2. Dans la VM, faites une bêtise contrôlée :

```
sudo apt purge -y docker-ce
```

3. Vérifiez que `docker run --rm hello-world` échoue : le client est resté, le démon est parti
4. Revenez au snapshot `docker-ok` : la VM redémarre, reconnectez-vous en SSH
5. `docker run --rm hello-world` refonctionne

# Étape 8 : la même chose, en script 🤖

Créez un **jeton** dans Xen Orchestra (vos paramètres utilisateur), puis, depuis le Codespace :

```
XO=https://<adresse-de-xo>
read -rsp "Jeton XO : " TOKEN; echo
curl -b authenticationToken=$TOKEN \
  "$XO/rest/v0/vms?fields=name_label,power_state"
curl -X POST -b authenticationToken=$TOKEN \
  -H 'Content-Type: application/json' \
  -d '{"name_label": "depuis-curl"}' \
  "$XO/rest/v0/vms/<UUID>/actions/snapshot?sync=true"
unset TOKEN
```

C’est la même API que celle qu’utilisent Terraform, Packer ou Jenkins pour piloter des VM.

# 📤 Rendu du TP

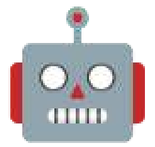
Un fichier texte qui contient :

1. La sortie de `docker info --format '{{.OperatingSystem}} / {{.KernelVersion}}'` **dans la VM**
2. La sortie de `docker run --rm hello-world` dans la VM
3. La sortie de l'appel `curl` qui liste vos VM, avec vos snapshots visibles dans XO (capture)
4. Trois lignes : à quel étage tournent le conteneur, Docker, la VM, l'hyperviseur ?

# ✅ Solution : Travaux pratiques VM

- Le conteneur tourne sur le **noyau de la VM** (Debian 13) : c'est ce que montre `docker info`
- Docker est un **processus** de la VM, comme un autre
- La VM est un **domU**, ordonnancé par l'hyperviseur Xen, à côté du dom0
- L'hyperviseur est posé sur le **matériel** du lab, à distance

# 🤖 Et quand une machine s’en charge : GitHub Actions

# La même image, ailleurs 

Votre Codespace tourne. Docker répond. Tout marche.

# GitHub Actions en une diapo

Un fichier dans `.github/workflows/`, et GitHub exécute vos commandes à chaque `push`.

```yaml
name: Bonjour
on:
  - push
jobs:
  dire_bonjour:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7 # Récupère le contenu du dépôt correspondant au commit du workflow en cours
```

# Le runner n’est pas votre Codespace ⚠️

**Votre Codespace**

`devcontainers/base:ubuntu`

Vos outils, votre version

**Le runner par défaut**

`ubuntu-latest`

Les outils de GitHub, leurs versions

# La même image, en CI ✅

```
    container:
      image: mcr.microsoft.com/devcontainers/base:ubuntu # La même image que le Codespace
```

# Trois pièges, mesurés 🔍

# 🎓 Exercice : la même image partout

- **But** : faire dire le `README` du dépôt par une vache, depuis la CI
- 👷 À vous d'écrire le workflow :
  - 💡 le job doit tourner dans la même image que votre Codespace
  - `cowsay` n'est pas installé dans l'image : il faut l'ajouter
  - 💡 pensez à récupérer le contenu du dépôt avant de le lire

# ✓ Solution

```yaml
name: Bonjour
on:
  - push
jobs:
  dire_bonjour:
    runs-on: ubuntu-latest
    container:
      image: mcr.microsoft.com/devcontainers/base:ubuntu # La même image que le Codespace
    steps:
      - uses: actions/checkout@v7 # Récupère le contenu du dépôt correspondant au commit du workflow en cours
      - run: | # Pas de `sudo` : dans un conteneur, le job tourne déjà en root
          apt-get update
          apt-get install -y cowsay
      - run: cat README.adoc | /usr/games/cowsay # `cowsay` s'installe dans /usr/games, absent du PATH
```

# Ce cours est construit comme ça

Les diapos que vous regardez sortent de `.github/workflows/build-workflow.yml`

# Récapitulatif 📌

# Ligne de commande

## Guide de survie

# 🤔 Problématique

- Communication Humain <→ Machine
- Base commune de TOUS les outils

# CLI

- 🇬🇧 CLI == "Command Line Interface"
- 🇫🇷 "Interface de Ligne de Commande"

# REPL

Pour les théoriciens et curieux :

- 🇬🇧 REPL == "Read–eval–print loop"

https://en.wikipedia.org/wiki/Read%E2%80%93eval%E2%80%93print_loop

# Anatomie d'une commande

```
ls --color=always -l /bin
```

- Séparateur : l'espace
- Premier élément (`ls`) : c'est la commande
- Les éléments commençant par un tiret − sont des "options" et/ou drapeaux ("flags")
  - "Option" == "Optionnel"
- Les autres éléments sont des arguments (`/bin`)
  - Nécessaire (par opposition)

# Manuel des commande

- Afficher le manuel de `<commande>` :

```
# Commande 'man' avec comme argument le nom de ladite commande
man <commande>
```

  - Navigation avec les flèches haut et bas
    - Tapez `/` puis une chaîne de texte pour chercher
      - Touche `n` pour sauter d'occurrence en occurrence
  - Touche `q` pour quitter

🎓 Essayez avec `ls`, chercher le mot `color`

- 💡 La majorité des commandes fournit également une option (`--help`), un flag (`-h`) ou un argument (`help`)
- Google c'est pratique aussi hein !

# Raccourcis

Dans un terminal Unix/Linux/WSL :

- `CTRL + C` : Annuler le process ou prompt en cours
- `CTRL + L` : Nettoyer le terminal
- `CTRL + A` : Positionner le curseur au début de la ligne
- `CTRL + E` : Positionner le curseur à la fin de la ligne
- `CTRL + R` : Rechercher dans l'historique de commandes

🎓 Essayez-les !

# Commandes de base 1/2

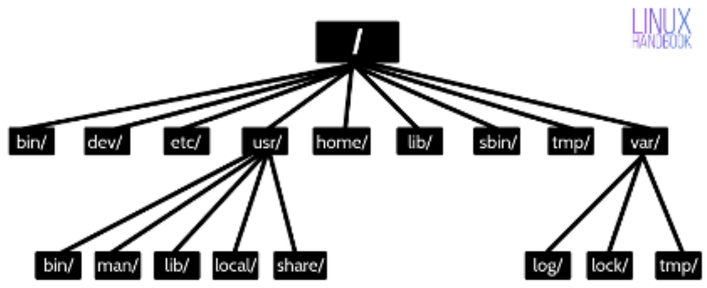
- `pwd` : Afficher le répertoire courant
  - 🎓 Option `-P` ?
- `ls` : Lister le contenu du répertoire courant
  - Options `-a` et `-l` ?
- `cd` : Changer de répertoire
  - Sans argument : que se passe t'il ?
- `cat` : Afficher le contenu d'un fichier
  - Essayez avec plusieurs arguments
- `mkdir` : créer un répertoire
  - 🎓 Option `-p` ?

# Commandes de base 2/2

- `echo` : Afficher un (des) message(s)
- `rm` : Supprimer un fichier ou dossier
- `touch` : Créer un fichier
- `grep` : Chercher un motif de texte

# Arborescence de fichiers 1/2

- Le système de fichier a une structure d'arbre
  - La racine du disque dur c'est `/` : 🎓 `ls -l /`
  - Le séparateur c'est également `/` : 🎓 `ls -l /usr/bin`
- Deux types de chemins :
  - Absolu (depuis la racine): Commence par `/` (Ex. `/usr/bin`)
  - Sinon c'est relatif (e.g. depuis le dossier courant) (Ex `./bin` ou `local/bin/`)




Source

# Arborescence de fichiers 2/2

- Le dossier "courant" c'est `.` : 🎓 `ls -l ./bin # Dans le dossier /usr`
- Le dossier "parent" c'est `..` : `ls -l ../ # Dans le dossier /usr`
- ~ (tilde) c'est un raccourci vers le dossier de l'utilisateur courant : `ls -l ~`
- Sensible à la casse (majuscules/minuscules) et aux espaces : 🎓

```
ls -l /bin
ls -l /Bin
mkdir ~/"Accent tué"
ls -d ~/Accent\ tué
```

# Un language (?)

- Variables interpolées avec le caractère "dollar" `$` :

```
echo $MA_VARIABLE
echo "$MA_VARIABLE"
echo ${MA_VARIABLE}

# Recommendation
echo "${MA_VARIABLE}"

MA_VARIABLE="Salut tout le monde"

echo "${MA_VARIABLE}"
```

- Sous commandes avec `$(<command>)` :

```
echo ">> Contenu de /tmp :\n$(ls /tmp)"
```

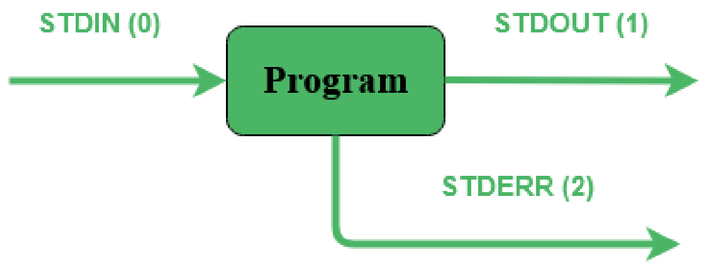
- Des `if`, des `for` et plein d'autres trucs (https://tldp.org/LDP/abs/html/)

# Codes de sortie

- Chaque exécution de commande renvoie un code de retour (🇬🇧 "exit code")
  - Nombre entier entre 0 et 255 (en POSIX)
- Code accessible dans la variable **éphémère** `$?` :

```
ls /tmp
echo $?

ls /do_not_exist
echo $?

# Une seconde fois. Que se passe-t'il ?
echo $?
```

# Entrée, sortie standard et d'erreur




```
ls -l /tmp
echo "Hello" > /tmp/hello.txt
ls -l /tmp
ls -l /tmp >/dev/null
ls -l /tmp 1>/dev/null

ls -l /do_not_exist
ls -l /do_not_exist 1>/dev/null
ls -l /do_not_exist 2>/dev/null

ls -l /tmp /do_not_exist
ls -l /tmp /do_not_exist 1>/dev/null 2>&1
```

# Pipelines

- Le caractère "pipe" `|` permet de chaîner des commandes
  - Le "stdout" de la première commande est branchée sur le "stdin" de la seconde
    - Exemple : Afficher les fichiers/dossiers contenant le lettre `d` dans le dossier `/bin` :

```
ls -l /bin

ls -l /bin | grep "d" --color=auto
```

# Exécution 1/2

- Les commandes sont des fichier binaires exécutables sur le système :

```bash
command -v cat # équivalent de "which cat"

ls -l "$(command -v cat)"
```

- La variable d'environnement `$PATH` liste les dossiers dans lesquels chercher les binaires
  - 💡 Utiliser cette variable quand une commande fraîchement installée n'est pas trouvée

# Exécution 2/2

- Exécution de scripts :
  - Soit appel direct avec l'interprétateur : `sh ~/monscript.txt`
  - Soit droit d'exécution avec un "shebang" (e.g. `#!/bin/bash`)

```
$ chmod +x ./monscript.sh

$ head -n1 ./monscript.sh
#!/bin/bash

$ ./monscript.sh
# Exécution
```

# Git

## Guide de survie

# 🤔 Problématique

- Support de communication
  - Humain → Machine
  - Humain <→ Humain
- Besoins de traçabilité, de définition explicite et de gestion de conflits
  - Collaboration requise pour chaque changement (revue, responsabilités)

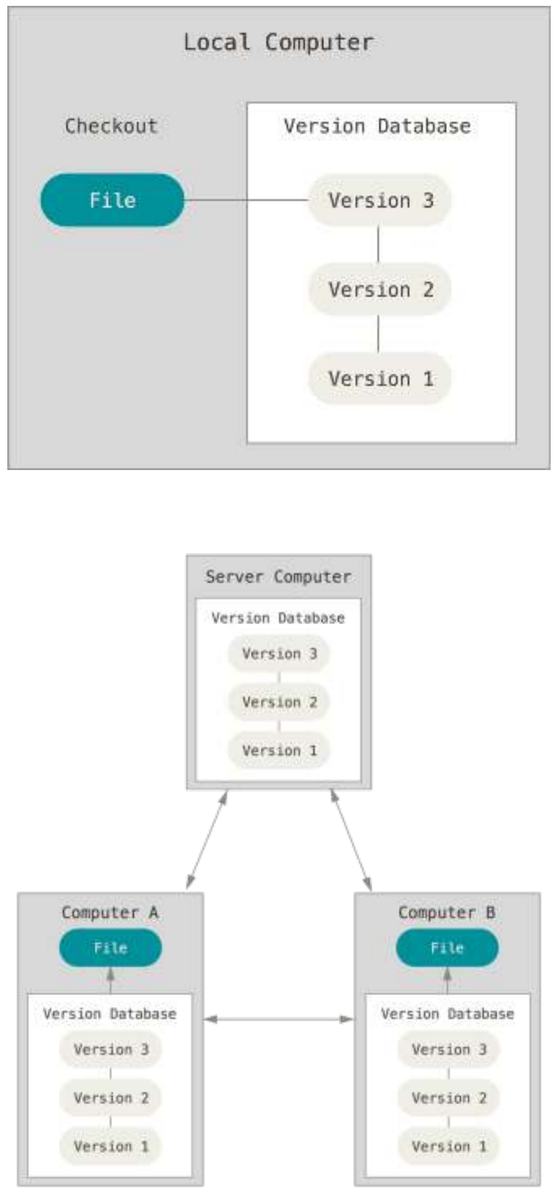
# Tracer le changement dans le code avec un **VCS** : 🇬🇧 Version Control System

également connu sous le nom de SCM (🇬🇧 Source Code Management)

# Pourquoi un VCS ?

- Pour conserver une trace de **tous** les changements dans un historique
  - "Source unique de vérité" (🇬🇧 *Single Source of Truth*)
- Pour **collaborer** efficacement sur un même référentiel

# Concepts des VCS




Source : https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

# Quel VCS utiliser ?

Nous allons utiliser **Git**

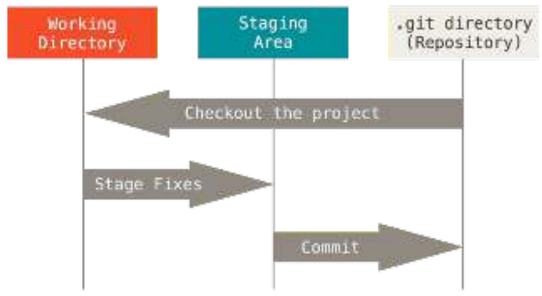
# Git

*Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.*

https://git-scm.com/

# Les 3 états avec Git

- L’historique ("Version Database") : dossier `.git`
- Dossier de votre projet ("Working Directory") - Commande
- La zone d’index ("Staging Area")




Source : https://git-scm.com/book/fr/v2/D%C3%A9marrage-rapide-Rudiments-de-Git#*les_trois*%C3%A9tats

# 🎓 Exercice avec Git - 1.1

- Dans le terminal de votre Codespace :
  - Créez un dossier vide nommé `projet-vcs-1` dans le répertoire `/workspaces`, puis positionnez-vous dans ce dossier

```
mkdir -p /workspaces/projet-vcs-1/
cd /workspaces/projet-vcs-1/
```

  - Est-ce qu'il y a un dossier `.git/` ?
  - Essayez la commande `git status` ?
- Initialisez le dépôt git avec `git init`
  - Est-ce qu'il y a un dossier `.git/` ?
  - Essayez la commande `git status` ?

# ✅ Solution de l’exercice avec Git - 1.1

```
mkdir -p /workspaces/projet-vcs-1/
cd /workspaces/projet-vcs-1/
ls -la # Pas de dossier .git
git status # Erreur "fatal: not a git repository"
git init ./
ls -la # On a un dossier .git
git status # Succès avec un message "On branch main No commits yet"
```

# 🎓 Exercice avec Git - 1.2

- Créez un fichier `README.md` dedans avec un titre et vos nom et prénoms
  - Essayez la commande `git status` ?
- Ajoutez le fichier à la zone d'indexation à l'aide de la commande `git add (...)`
  - Essayez la commande `git status` ?
- Créez un commit qui ajoute le fichier `README.md` avec un message, à l'aide de la commande `git commit -m <message>`
  - Essayez la commande `git status` ?

# ✅ Solution de l’exercice avec Git - 1.2

```
echo "# Read Me\n\nObi Wan" > ./README.md
git status # Message "Untracked file"

git add ./README.md
git status # Message "Changes to be committed"
git commit -m "Ajout du README au projet"
git status  # Message "nothing to commit, working tree clean"
```

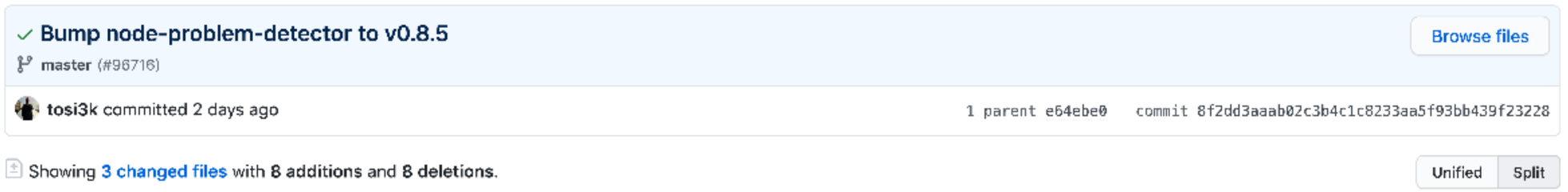
# Terminologie de Git - Diff et changeset

**diff:** un ensemble de lignes "changées" sur un fichier donné

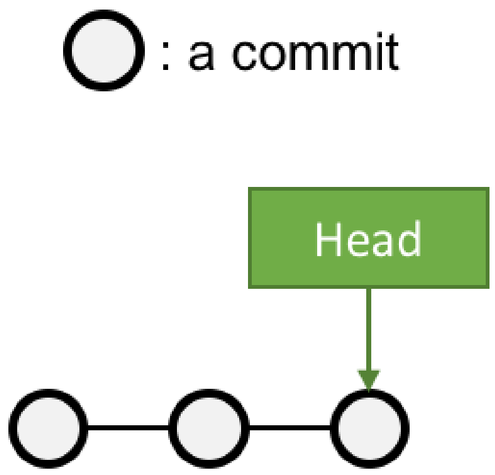
**changeset:** un ensemble de "diff" (donc peut couvrir plusieurs fichiers)

# Terminologie de Git - Commit

**commit:** un changeset qui possède un (commit) parent, associé à un message




*"HEAD"*: C’est le dernier commit dans l’historique



# 🎓 Exercice avec Git - 2

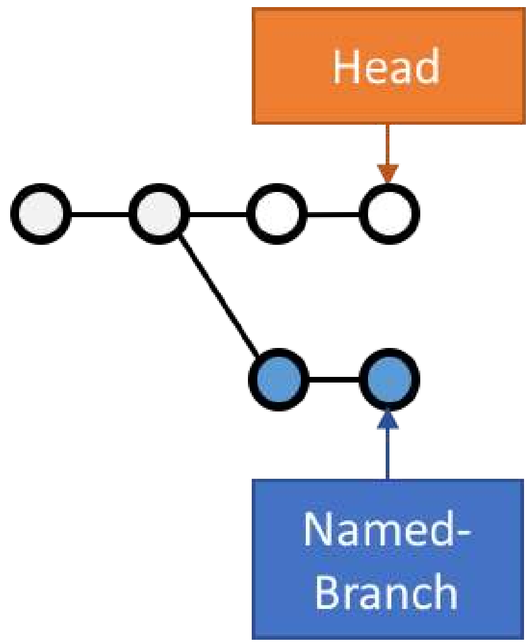
- Afficher la liste des commits
- Afficher le changeset associé à un commit
- Modifier du contenu dans `README.md` et afficher le diff
- Annulez ce changement sur `README.md`

# ✅ Solution de l’exercice avec Git - 2

```
git log

git show # Show the "HEAD" commit
echo "# Read Me\n\nObi Wan Kenobi" > ./README.md

git diff
git status

git checkout -- README.md
git status
```

# Terminologie de Git - Branche

- Abstraction d’une version "isolée" du code
- Concrètement, une **branche** est un alias pointant vers un "commit"



# 🎓 Exercice avec Git - 3

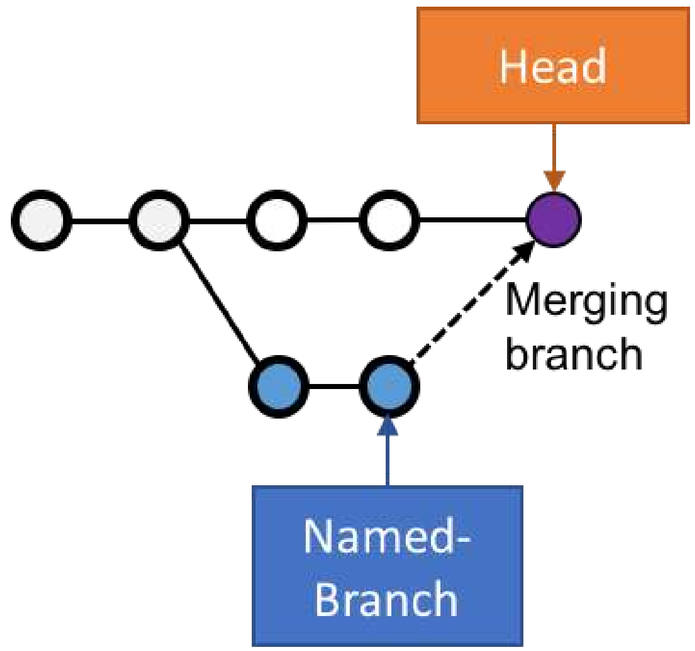
- Créer une branche nommée `feature/html`
- Ajouter un nouveau commit contenant un nouveau fichier `index.html` sur cette branche
- Afficher le graphe correspondant à cette branche avec `git log --graph`

# ✓ Solution de l’exercice avec Git - 3

```
git switch --create feature/html
# Ou alors: git branch feature/html && git switch feature/html
echo '<h1>Hello</h1>' > ./index.html
git add ./index.html && git commit --message="Ajout d'une page HTML par défaut" # -m / --message

git log
git log --graph

# 'git lg' n'existe pas par défaut : c'est un alias, à définir une fois pour toutes
git config --global alias.lg "log --graph --oneline --decorate --all"
git lg # cat ~/.gitconfig => l'alias apparaît maintenant dans la section [alias]
```

# Terminologie de Git - Merge

- On intègre une branche dans une autre en effectuant un **merge**
  - Un nouveau commit est créé, fruit de la combinaison de 2 autres commits



# 🎓 Exercice avec Git - 4

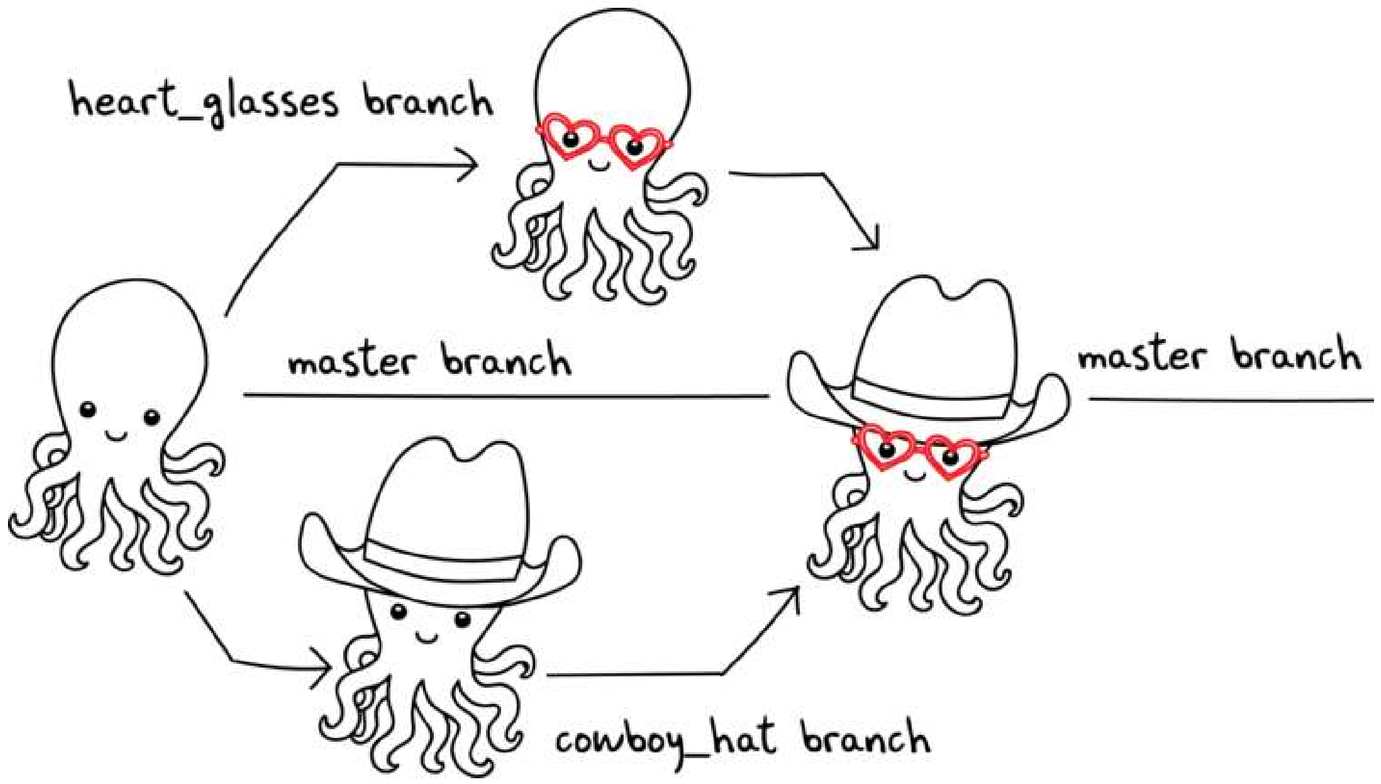
- Merger la branche `feature/html` dans la branche principale
  - ⚠️ Pensez à utiliser l'option `--no-ff`
- Afficher le graphe correspondant à cette branche avec `git log --graph`

# ✅ Solution de l’exercice avec Git - 4

```
git switch main
git merge --no-ff feature/html # Enregistrer puis fermer le fichier 'MERGE_MSG' qui a été ouvert
git log --graph

# git lg
```

# Feature Branch Flow

- **Une seule** branche **par** fonctionnalité



# Exemple d’usages de VCS

- "Infrastructure as Code" :
  - Besoins de traçabilité, de définition explicite et de gestion de conflits
  - Collaboration requise pour chaque changement (revue, responsabilités)
- Code Civil:
  - https://github.com/steeve/france.code-civil
  - https://github.com/steeve/france.code-civil/pull/40
  - https://github.com/steeve/france.code-civil/commit/b805ecf05a86162d149d3d182e04074ecf72c066

# Checkpoint 🎯

- `git` est un des (plus populaires) de système de contrôle de versions
- Cet outil vous permet:
  - D'avoir un historique auditable de votre code source
  - De collaborer efficacement sur le code source (conflit `git` == "PARLEZ-VOUS")

⇒ C'est une ligne de commande (trop?) complète qui nécessite de pratiquer

# Containers

# 🎓 Exercice : Votre premier conteneur

C’est à vous (ouf) !

- Allez dans votre Codespace
- Dans un terminal, tapez la commande suivante :

```
docker container run hello-world
# Equivalent de l'ancienne commande 'docker run'
```

# 🧭 Deux écritures pour la même commande

Depuis Docker 1.13 (2017), les commandes qui agissent sur un objet (conteneur, image, réseau, volume) s'écrivent aussi `docker OBJET VERBE`, qui dit sur quel type d'objet on agit.

| **Raccourci historique** | **Forme de gestion** |
|---|---|
| `docker run` | `docker container run` |
| `docker ps` | `docker container ls` |
| `docker images` | `docker image ls` |
| `docker rmi` | `docker image rm` |
| `docker pull` | `docker image pull` |

# 🧭 Laquelle utiliser ?

- Les deux formes marchent, et les raccourcis ne sont pas dépréciés
  - `docker container cp --help` liste d'ailleurs les deux sous `Aliases:`
- ⚠️ Le raccourci cache parfois l'objet : `docker pull` télécharge une **image**, donc la forme longue est `docker image pull`, pas `docker container pull`
- 🚫 Les commandes qui n'agissent sur aucun objet gardent leur forme unique : `docker login`, `docker version`, `docker search`
- 📏 **Dans ce cours, on écrit toujours la forme de gestion.** Vous rencontrerez les raccourcis partout ailleurs : sachez les lire.

# Anatomie

- Un service "Docker Engine" tourne en tâche de fond et publie une API REST
- La commande `docker container run ...` a lancé un client docker qui a envoyé une requête `POST` au service docker
- Le service a téléchargé une **Image** Docker depuis le registre **DockerHub**,
- Puis a exécuté un **conteneur** basé sur cette image

# Anatomie

$

$

# Anatomie

```
$ docker container run busybox echo hello world
```

```
$
```

# Anatomie

```
$ docker container run busybox echo hello world
hello world
```

```
$
```

# Anatomie

```
$ docker container run busybox echo hello world
hello world
```

```
$ docker container run alpine date
```

# Anatomie

```
$ docker container run busybox echo hello world
hello world
```

```
$ docker container run alpine date
Mon Oct 26 11:33:32 UTC 2026
```

# Anatomie

```
$ docker container run busybox echo hello world
hello world
```

Lancement d'un container

```
$ docker container run alpine date
Mon Oct 26 11:33:32 UTC 2026
```

# Anatomie

```
$ docker container run busybox echo hello world
hello world
```

Lancement d'un container

Image

```
$ docker container run alpine date
Mon Oct 26 11:33:32 UTC 2026
```

# Anatomie

```
$ docker container run busybox echo hello world
hello world
```

Lancement d'un container

Image

Commande lancée

```
$ docker container run alpine date
Mon Oct 26 11:33:32 UTC 2026
```

# Anatomie

# Anatomie

ON LANCE UNE COMMANDE DANS UN SHELL DANS UNE COMMANDE DANS UN SHELL
TOUT CA POUR AFFICHER LA DATE ???

# 🎓 Exercice : Où est mon conteneur ?

C’est à vous !

```
docker container ls --help
# ...
docker container ls
# ...
docker container ls --all
```

⇒ 🤔 comment comprenez vous les résultats des 2 dernières commandes ?

# ✅ Solution : Où est mon conteneur ?

Le conteneur est toujours présent dans le "Docker Engine" même en étant arrêté

```
CONTAINER ID   IMAGE         COMMAND    CREATED          STATUS                      PORTS     NAMES
109a9cdd3ec8   hello-world   "/hello"   33 seconds ago   Exited (0) 17 seconds ago             festive_faraday
```

- Un conteneur == une commande "conteneurisée"
  - cf. colonne "**COMMAND**"
- Quand la commande s’arrête : le conteneur s’arrête
  - cf. code de sortie dans la colonne "**STATUS**"

# Rappels d’anatomie

```
$ docker container run busybox echo hello world
```

# Rappels d'anatomie

```
$ docker container run busybox echo hello world
```

- Le moteur Docker crée un container à partir de l'image "busybox".

# 🫁 Rappels d’anatomie

```
$ docker container run busybox echo hello world
```

- Le moteur Docker crée un container à partir de l’image "busybox".
- Le moteur Docker démarre le container créé.

# Rappels d’anatomie

```
$ docker container run busybox echo hello world
```

- Le moteur Docker crée un container à partir de l’image "busybox".
- Le moteur Docker démarre le container créé.
- La commande "echo hello world" est exécutée à l’intérieur du container.

# 🫁 Rappels d'anatomie

```
$ docker container run busybox echo hello world
```

- Le moteur Docker crée un container à partir de l'image "busybox".
- Le moteur Docker démarre le container créé.
- La commande "echo hello world" est exécutée à l'intérieur du container.
- On obtient le résultat dans la sortie standard de la machine hôte.

# 🫁 Rappels d'anatomie

```
$ docker container run busybox echo hello world
```

- Le moteur Docker crée un container à partir de l'image "busybox".
- Le moteur Docker démarre le container créé.
- La commande "echo hello world" est exécutée à l'intérieur du container.
- On obtient le résultat dans la sortie standard de la machine hôte.
- Le container est stoppé.

# 🍽 Cas d’usage

## Outillage jetable

- Tester une version de Maven, de JDK, de NPM, …

# 🎓 Exercice : Cycle de vie d'un conteneur simple

- Lancez un nouveau conteneur nommé `bonjour`
  - 💡 `docker container run --help` ou Documentation en ligne
- Affichez les "logs" du conteneur (==traces d'exécution écrites sur le stdout + stderr de la commande conteneurisée)
  - 💡 `docker container logs --help` ou Documentation en ligne
- Lancez le conteneur avec la commande `docker container start`
  - Regardez le résultat dans les logs
- Supprimez le container avec la commande `docker container rm`

# ✅ Solution : Cycle de vie d’un conteneur simple

```
docker container run --name=bonjour hello-world
# Affiche le texte habituel

docker container logs bonjour
# Affiche le même texte : pratique si on a fermé le terminal

docker container start bonjour
# N'affiche pas le texte mais l'identifiant unique du conteneur 'bonjour'

docker container logs bonjour
# Le texte est affiché 2 fois !

docker container ls --all
# Le conteneur est présent
docker container rm bonjour
docker container ls --all
# Le conteneur n'est plus là : il a été supprimé ainsi que ses logs

docker container logs bonjour
# Error: No such container: bonjour
```

# 🤔 Que contient "hello-world" ?

- C’est une "image" de conteneur, c’est à dire un modèle (template) représentant une application auto-suffisante.
  - On peut voir ça comme un "paquetage" autonome
- C’est un système de fichier complet:
  - Il y a au moins une racine `/`
  - Ne contient que ce qui est censé être nécessaire (dépendances, librairies, binaires, etc.)

# Docker Hub

- https://hub.docker.com/ : C’est le registre d’images "par défaut"
  - Exemple : Image officielle de conteneur "Ubuntu"
- 🎓 Cherchez l’image `hello-world` pour en voir la page de documentation
  - 💡 pas besoin de créer de compte pour ça
- Il existe d’autre "registres" en fonction des besoins (GitHub GHCR, Google GCR, etc.)

# 🏓 Lancer un container interactif

```
docker container run  --interactive --tty alpine
```

# 🏓 Lancer un container interactif

```
docker container run  --interactive --tty alpine
/ $
```

# 🏓 Lancer un container interactif

```
docker container run  --interactive --tty alpine
/ $
```

- On lance un container à partir de l’image "alpine"

# 🏓 Lancer un container interactif

```
docker container run  --interactive --tty alpine
/ $
```

- On lance un container à partir de l'image "alpine"
- On lance un `sh` dans ce container

# 🏓 Lancer un container interactif

```
docker container run  --interactive --tty alpine
/ $
```

- On lance un container à partir de l’image "alpine"
- On lance un `sh` dans ce container
- On redirige l’entrée standard avec -i

# 🏓 Lancer un container interactif

```
docker container run  --interactive --tty alpine
/ $
```

- On lance un container à partir de l'image "alpine"
- On lance un `sh` dans ce container
- On redirige l'entrée standard avec -i
- On déclare un pseudo-terminal avec -t

# 🎓 Exercice : conteneur interactif

- Quelle distribution Linux est utilisée dans le terminal Codespaces ?
  - 💡 Regardez le fichier `/etc/os-release`
- Exécutez un conteneur interactif basé sur `alpine:3.24.2` (une distribution Linux ultra-légère) et regardez le contenu du fichier au même emplacement
  - `docker container run --help`
  - Demandez un `tty` à Docker
  - Activez le mode interactif
- Exécutez la même commande dans un conteneur basé sur la même image mais en **NON** interactif
  - 💡 Comment surcharger la commande par défaut ?

# ✓ Solution : conteneur interactif

```
$ cat /etc/os-release
# ... Ubuntu ....

$ docker container run --tty --interactive alpine:3.24.2
/ $ cat /etc/os-release
# ... Alpine ...
# Notez que le "prompt" du terminal est différent DANS le conteneur
/ $ exit
$ docker container ls --all

$ docker container run alpine:3.24.2 cat /etc/os-release
# ... Alpine ...
```

# 🏓 Utiliser un container interactif

Revenons dans notre container interactif de tout à l'heure…

```
/ $ curl google.fr
```

# 🏓 Utiliser un container interactif

```
/ $ curl google.fr
/bin/sh: curl: not found
```

# 🏓 Utiliser un container interactif

```
/ $ curl google.fr
/bin/sh: curl: not found
```

`cURL` n'est pas disponible par défaut sur Alpine. Il faut l'installer au préalable

# 🏓 Utiliser un container interactif

```
/ $ curl google.fr
/bin/sh: curl: not found
```

`cURL` n'est pas disponible par défaut sur Alpine. Il faut l'installer au préalable

```
/ $ apk update && apk add curl
```

# 🏓 Utiliser un container interactif

```
/ $ curl google.fr
/bin/sh: curl: not found
```

`cURL` n'est pas disponible par défaut sur Alpine. Il faut l'installer au préalable

```
/ $ apk update && apk add curl
v3.24.2-58-g29b9ec24b1b [https://dl-cdn.alpinelinux.org/alpine/v3.24/main]
v3.24.2-61-g887b5770c3e [https://dl-cdn.alpinelinux.org/alpine/v3.24/community]
OK: 28651 distinct packages available
(1/9) Installing brotli-libs (1.2.0-r1)
(2/9) Installing c-ares (1.34.8-r0)
(3/9) Installing libunistring (1.4.2-r0)
(4/9) Installing libidn2 (2.3.8-r0)
(5/9) Installing nghttp2-libs (1.69.0-r0)
(6/9) Installing libpsl (0.21.5-r3)
(7/9) Installing zstd-libs (1.5.7-r2)
(8/9) Installing libcurl (8.22.0-r0)
(9/9) Installing curl (8.22.0-r0)
Executing busybox-1.37.0-r31.trigger
OK: 13.1 MiB in 25 packages
```

# Utiliser un container interactif

```
/ $ curl google.fr
<HTML><HEAD><meta http-equiv="content-type" content="text/html;charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="http://www.google.fr/">here</A>.
</BODY></HTML>
```

C’est bon, on a `cURL`

# 🏓 Utiliser un container interactif

On peut quitter `sh` et revenir à la machine hôte !

```
/ $ exit
```

# 🏓 Utiliser un container interactif

On peut quitter `sh` et revenir à la machine hôte !

```
/ $ exit
```

Si on veut réutiliser `cURL` sur Alpine, c'est simple, on relance le shell, non? 🤔

# 🏓 Utiliser un container interactif

On peut quitter `sh` et revenir à la machine hôte !

```
/ $ exit
```

Si on veut réutiliser `cURL` sur Alpine, c'est simple, on relance le shell, non? 🤔

```
docker container run  --interactive --tty alpine
```

# 🏓 Utiliser un container interactif

On peut quitter `sh` et revenir à la machine hôte !

```
/ $ exit
```

Si on veut réutiliser `cURL` sur Alpine, c'est simple, on relance le shell, non? 🤔

```
docker container run  --interactive --tty alpine
```

On relance `cURL`:

```
/ $ curl google.fr
```

# 🏓 Utiliser un container interactif

On peut quitter `sh` et revenir à la machine hôte !

```
/ $ exit
```

Si on veut réutiliser `cURL` sur Alpine, c'est simple, on relance le shell, non? 🤔

```
docker container run  --interactive --tty alpine
```

On relance `cURL`:

```
/ $ curl google.fr
/bin/sh: curl: not found
```

# 🏓 Utiliser un container interactif

En fait, c'est logique !

```
docker container run
```

- Cette commande instancie un "nouveau container à chaque fois" !

# 🏓 Utiliser un container interactif

En fait, c'est logique !

```
docker container run
```

- Cette commande instancie un "nouveau container à chaque fois" !
- Chaque container est différent.

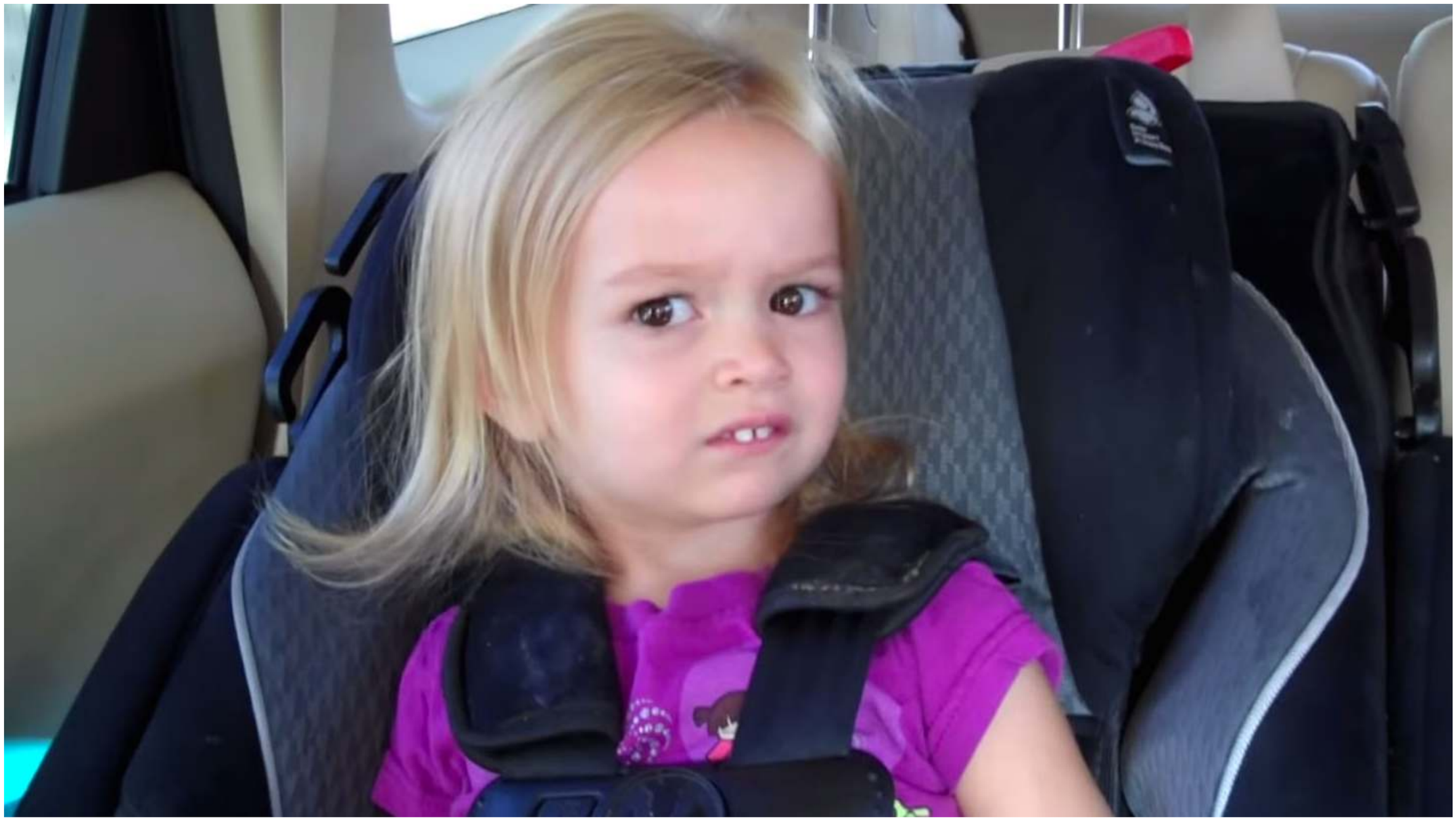
# 🏓 Utiliser un container interactif

En fait, c’est logique !

```
docker container run
```

- Cette commande instancie un "nouveau container à chaque fois" !
- Chaque container est différent.
- Aucun partage entre les containers à part le contenu de base de l’image.

# 🏓 Utiliser un container interactif



*"On m’a vendu un truc qui permet de lancer des tonnes de microservices… mais là, on télécharge nimps*

# ⛅ Conteneur en tâche de fond

Lançons un container bien particulier…

```
docker container run  --interactive --tty jpetazzo/clock
```

# ⛅ Conteneur en tâche de fond

Lançons un container bien particulier…

```
docker container run  --interactive --tty jpetazzo/clock
Mon Sep 25 10:33:51 UTC 2023
Mon Sep 25 10:33:52 UTC 2023
Mon Sep 25 10:33:53 UTC 2023
Mon Sep 25 10:33:54 UTC 2023
Mon Sep 25 10:33:55 UTC 2023
Mon Sep 25 10:33:56 UTC 2023
...
```

# ⛅ Conteneur en tâche de fond

Lançons un container bien particulier…

```
docker container run  --interactive --tty jpetazzo/clock
Mon Sep 25 10:33:51 UTC 2023
Mon Sep 25 10:33:52 UTC 2023
Mon Sep 25 10:33:53 UTC 2023
Mon Sep 25 10:33:54 UTC 2023
Mon Sep 25 10:33:55 UTC 2023
Mon Sep 25 10:33:56 UTC 2023
...
```

Ce container va tourner indéfiniment sauf si on le stoppe avec ⌨️ `Ctrl` + `C` …

# ⛅ Conteneur en tâche de fond

Lançons un container bien particulier…

```
docker container run  --interactive --tty jpetazzo/clock
Mon Sep 25 10:33:51 UTC 2023
Mon Sep 25 10:33:52 UTC 2023
Mon Sep 25 10:33:53 UTC 2023
Mon Sep 25 10:33:54 UTC 2023
Mon Sep 25 10:33:55 UTC 2023
Mon Sep 25 10:33:56 UTC 2023
...
```

Ce container va tourner indéfiniment sauf si on le stoppe avec ⌨️ `Ctrl` + `C` …

… mais ca va stopper le container !

# ⛅ Conteneur en tâche de fond

Lançons un container bien particulier…

```
docker container run  --interactive --tty jpetazzo/clock
Mon Sep 25 10:33:51 UTC 2023
Mon Sep 25 10:33:52 UTC 2023
Mon Sep 25 10:33:53 UTC 2023
Mon Sep 25 10:33:54 UTC 2023
Mon Sep 25 10:33:55 UTC 2023
Mon Sep 25 10:33:56 UTC 2023
...
```

Ce container va tourner indéfiniment sauf si on le stoppe avec ⌨️ `Ctrl` + `C` …

… mais ca va stopper le container !

Oui car quand on stoppe le processus principal d'un container, ce dernier n'a plus de raison d'exister et s'arrête naturellement.

# ⛅ Conteneur en tâche de fond

## La solution : le flag `--detach`

```
docker container run --detach jpetazzo/clock
```

# ⛅ Conteneur en tâche de fond

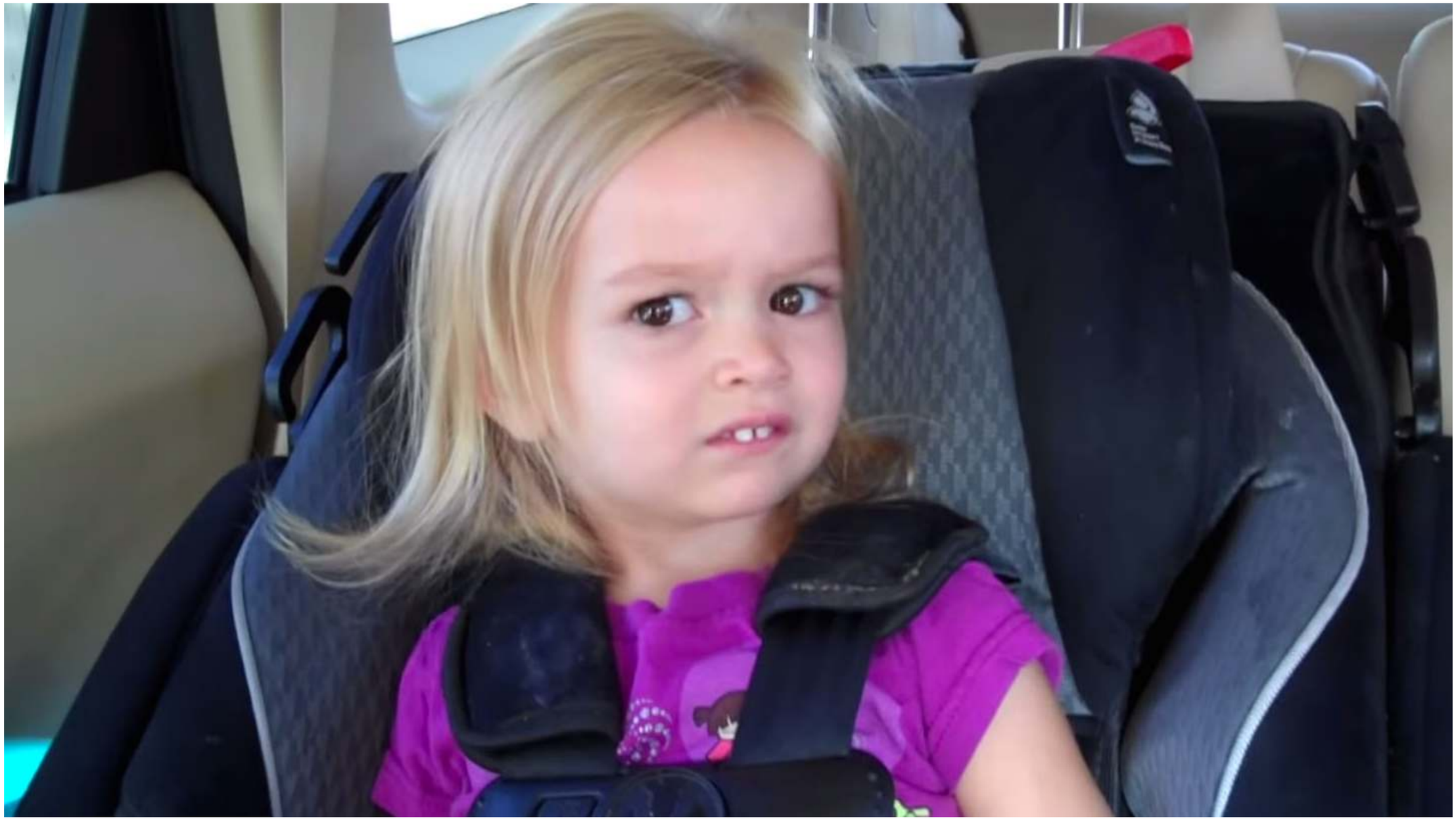
La solution : le flag `--detach`

```
docker container run --detach jpetazzo/clock
399f3b23bc0585991afa80dfee854cf0a953d782b99153b4e2cbc74ab6b07770
```

Le retour de cette commande correspond à l'identifiant unique du container.

Cette fois-ci, le container tourne, mais en arrière plan !

# ⛅ Conteneur en tâche de fond



*" Okay, maintenant il n’écrit la date nulle part… mais toutes les secondes… à l’aide ! "*

# ⛅ Conteneur en tâche de fond

Le processus principal de ce container écrit dans la sortie standard… du container !

Comment retrouver le contenu de la sortie standard du container ?

# ⛅ Conteneur en tâche de fond

Le processus principal de ce container écrit dans la sortie standard… du container !

## Comment retrouver le contenu de la sortie standard du container ?

```
docker container logs 399f3
```

# ⛅ Conteneur en tâche de fond

Le processus principal de ce container écrit dans la sortie standard… du container !

Comment retrouver le contenu de la sortie standard du container ?

```
docker container logs 399f3
Tue Sep 12 12:37:01 UTC 2023
Tue Sep 12 12:37:02 UTC 2023
Tue Sep 12 12:37:03 UTC 2023
Tue Sep 12 12:37:04 UTC 2023
Tue Sep 12 12:37:05 UTC 2023
Tue Sep 12 12:37:06 UTC 2023
Tue Sep 12 12:37:07 UTC 2023
Tue Sep 12 12:37:08 UTC 2023
Tue Sep 12 12:37:09 UTC 2023
Tue Sep 12 12:37:10 UTC 2023
Tue Sep 12 12:37:11 UTC 2023
```

# ⛅ Conteneur en tâche de fond

Le processus principal de ce container écrit dans la sortie standard… du container !

Comment retrouver le contenu de la sortie standard du container ?

```
docker container logs 399f3
Tue Sep 12 12:37:01 UTC 2023
Tue Sep 12 12:37:02 UTC 2023
Tue Sep 12 12:37:03 UTC 2023
Tue Sep 12 12:37:04 UTC 2023
Tue Sep 12 12:37:05 UTC 2023
Tue Sep 12 12:37:06 UTC 2023
Tue Sep 12 12:37:07 UTC 2023
Tue Sep 12 12:37:08 UTC 2023
Tue Sep 12 12:37:09 UTC 2023
Tue Sep 12 12:37:10 UTC 2023
Tue Sep 12 12:37:11 UTC 2023
```

Ouf ! On n’est pas obligé de saisir l’identifiant complet ! Il suffit de fournir le nombre suffisant de caractères pour que ce soit discriminant.

# 📖 Lister les containers

Comment savoir si j'ai des containers en cours d'exécution ?

💡 Commande vue un peu plus tôt…

# 📖 Lister les containers

Comment savoir si j'ai des containers en cours d'exécution ?

```
docker container ls
```

# 📖 Lister les containers

Comment savoir si j'ai des containers en cours d'exécution ?

```
docker container ls
CONTAINER ID   IMAGE                              COMMAND                  CREATED         STATUS       PORTS
02cbf2fb3721   cours-devops-docker-serve          "/sbin/tini -g gulp …"   3 hours ago     Up 3 hours   0.0.0.0:8000->8000/tc
ebfbe695b2ec   moby/buildkit:buildx-stable-1      "buildkitd"              2 months ago    Up 3 hours
```

# 📖 Lister les containers

Comment savoir si j'ai des containers en cours d'exécution ?

```
docker container ls
CONTAINER ID   IMAGE                                COMMAND                  CREATED         STATUS         PORTS
02cbf2fb3721   cours-devops-docker-serve            "/sbin/tini -g gulp …"   3 hours ago     Up 3 hours     0.0.0.0:8000->8000/tc
ebfbe695b2ec   moby/buildkit:buildx-stable-1        "buildkitd"              2 months ago    Up 3 hours
```

On obtient un tableau de tous les containers en cours d'exécution.

# 🛑 🏁 Stop / Start

Il est possible de stopper un container.

```
docker container stop 399f3
```

# 🛑 🏁 Stop / Start

Il est possible de stopper un container.

```
docker container stop 399f3
```

Pour redémarrer un container :

```
docker container start 399f3
```

# 📖 Lister tous les containers

Même les 💀.

# 📖 Lister tous les containers

Comment savoir si j'ai des containers stoppés ?

```
docker container ls --all
```

# 📖 Lister tous les containers

Comment savoir si j'ai des containers stoppés ?

```
docker container ls --all
CONTAINER ID   IMAGE                          COMMAND                  CREATED          STATUS                          PO
90725f661d4e   hello-world                    "/hello"                 13 seconds ago   Exited (0) 12 seconds ago
9d0a6586b9e1   busybox                        "echo hello world"       22 seconds ago   Exited (0) 21 seconds ago
368ed08a35e3   jpetazzo/clock                 "/bin/sh -c 'while d…"   6 minutes ago    Up 5 minutes
c036e57bbf05   jpetazzo/clock                 "/bin/sh -c 'while d…"   17 minutes ago   Exited (130) 17 minutes ago
02cbf2fb3721   cours-devops-docker-serve      "/sbin/tini -g gulp …"   3 hours ago      Up 3 hours                      0.
ebfbe695b2ec   moby/buildkit:buildx-stable-1  "buildkitd"              2 months ago     Up 4 hours
```

# 📖 Lister tous les containers

Comment savoir si j'ai des containers stoppés ?

```
 docker container ls --all
CONTAINER ID   IMAGE                          COMMAND                  CREATED          STATUS                          PO
90725f661d4e   hello-world                    "/hello"                 13 seconds ago   Exited (0) 12 seconds ago
9d0a6586b9e1   busybox                        "echo hello world"       22 seconds ago   Exited (0) 21 seconds ago
368ed08a35e3   jpetazzo/clock                 "/bin/sh -c 'while d…"   6 minutes ago    Up 5 minutes
c036e57bbf05   jpetazzo/clock                 "/bin/sh -c 'while d…"   17 minutes ago   Exited (130) 17 minutes ago
02cbf2fb3721   cours-devops-docker-serve      "/sbin/tini -g gulp …"   3 hours ago      Up 3 hours                      0.
ebfbe695b2ec   moby/buildkit:buildx-stable-1  "buildkitd"              2 months ago     Up 4 hours
```

Avec le flag `--all` , on obtient un tableau de tous les containers quel que soit leur état.

# Nettoyage

Tout container stoppé peut être supprimé.

# Nettoyage

Tout container stoppé peut être supprimé.

```
docker container rm 90725f661d4e
```

# Nettoyage

Tout container stoppé peut être supprimé.

```
docker container rm 90725f661d4e
90725f661d4e
```

# 🧽 Nettoyage

Tout container stoppé peut être supprimé.

```
docker container rm 90725f661d4e
90725f661d4e
```

Container "auto-nettoyant" 🗑️

# 🧽 Nettoyage

Tout container stoppé peut être supprimé.

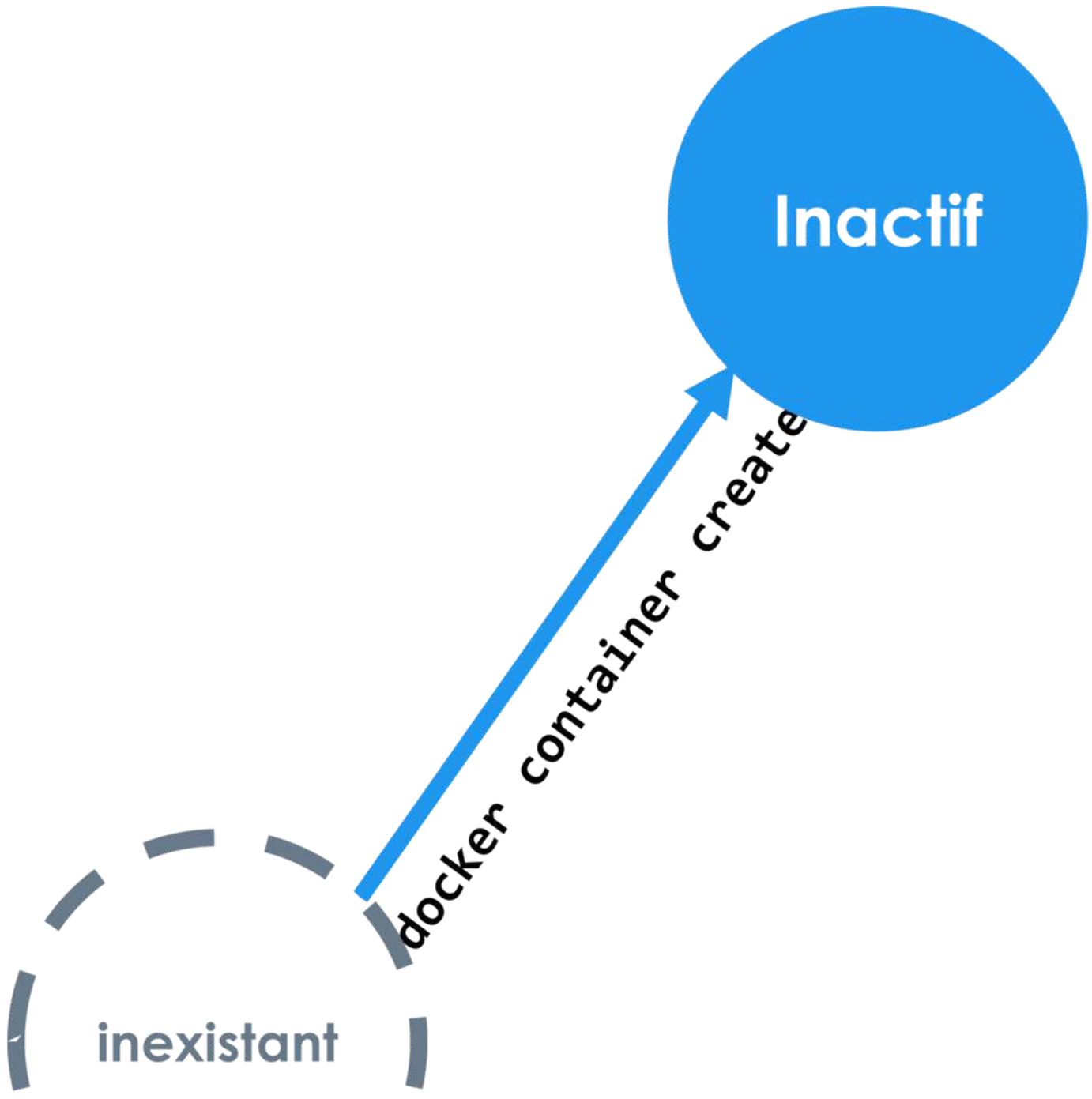
```
docker container rm 90725f661d4e
90725f661d4e
```

# Container "auto-nettoyant" 🗑️

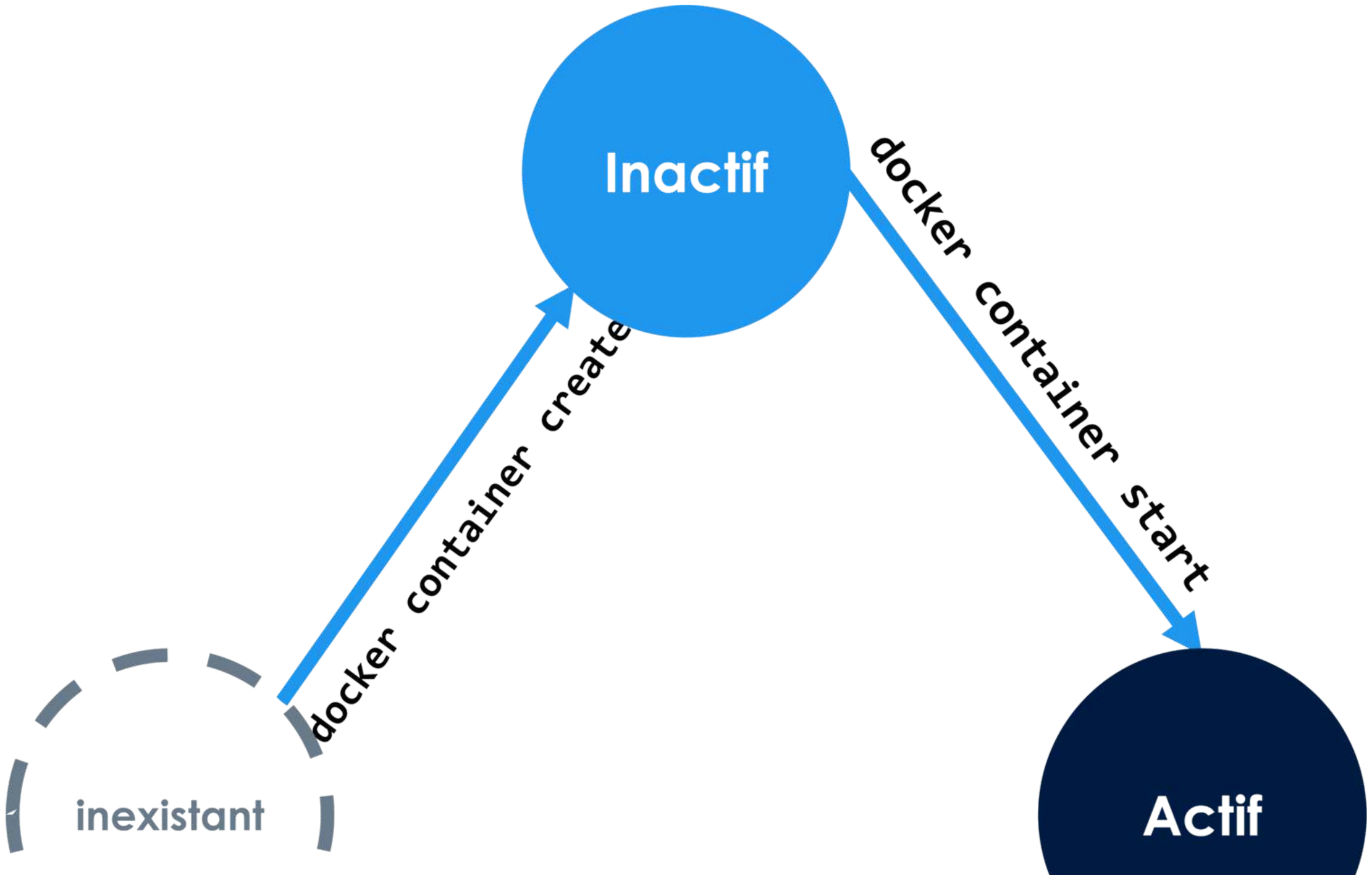
```
docker container run  --interactive --tty --rm jpetazzo/clock
```

Aussitôt stoppé, aussitôt supprimé !

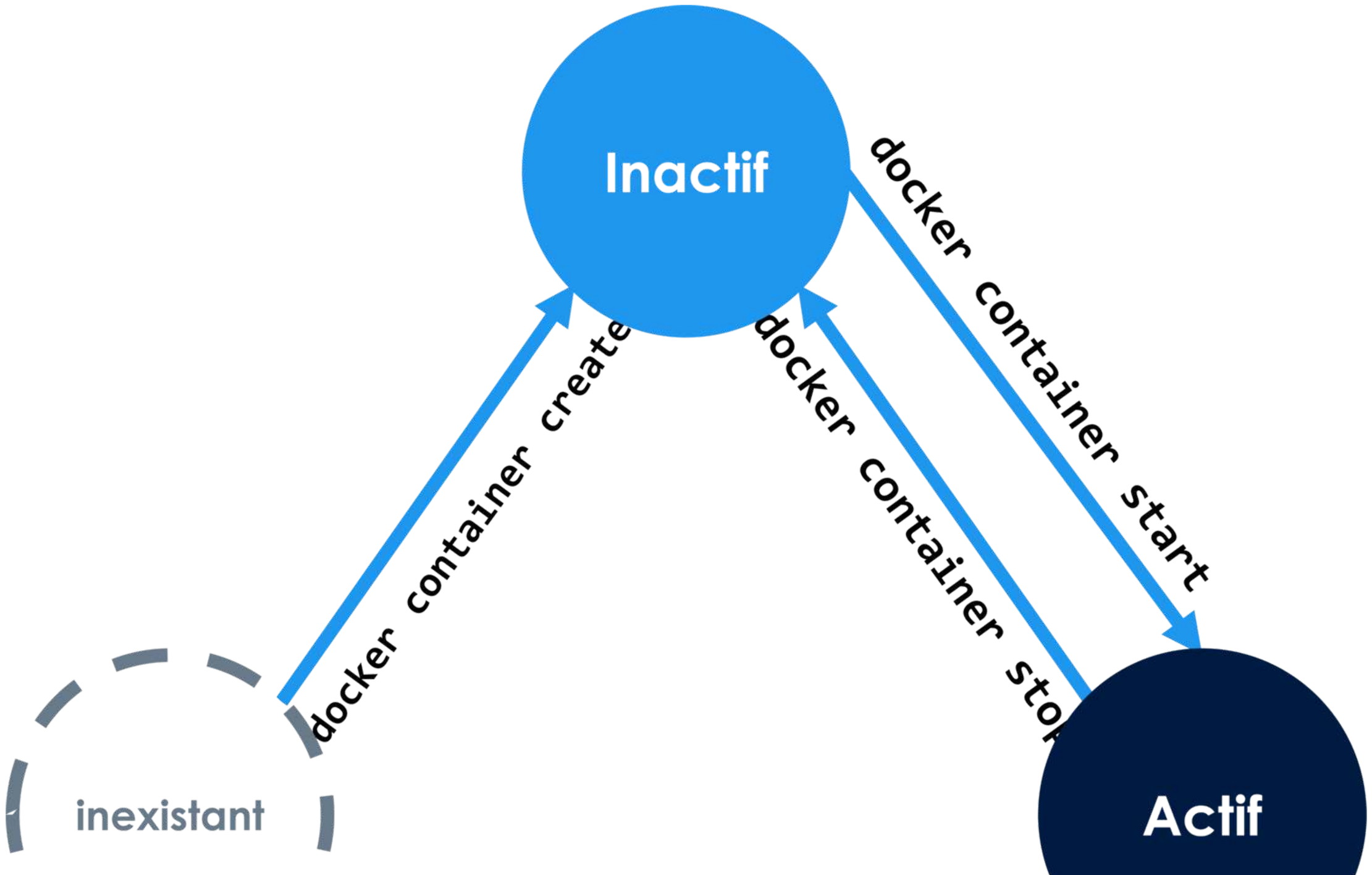
# ⏰ Rappel: cycle de vie d'un container



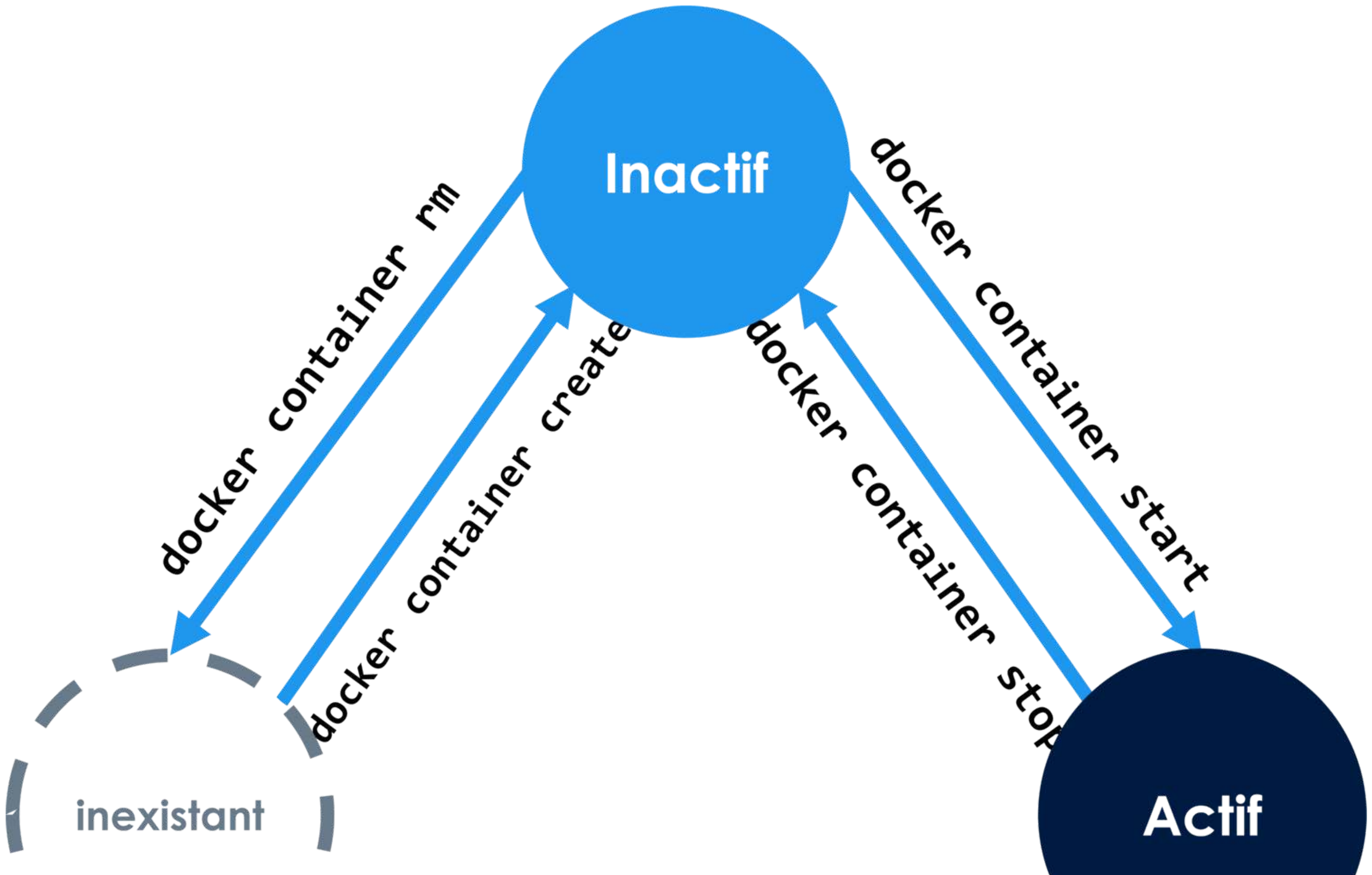
# ⏰ Rappel: cycle de vie d'un container



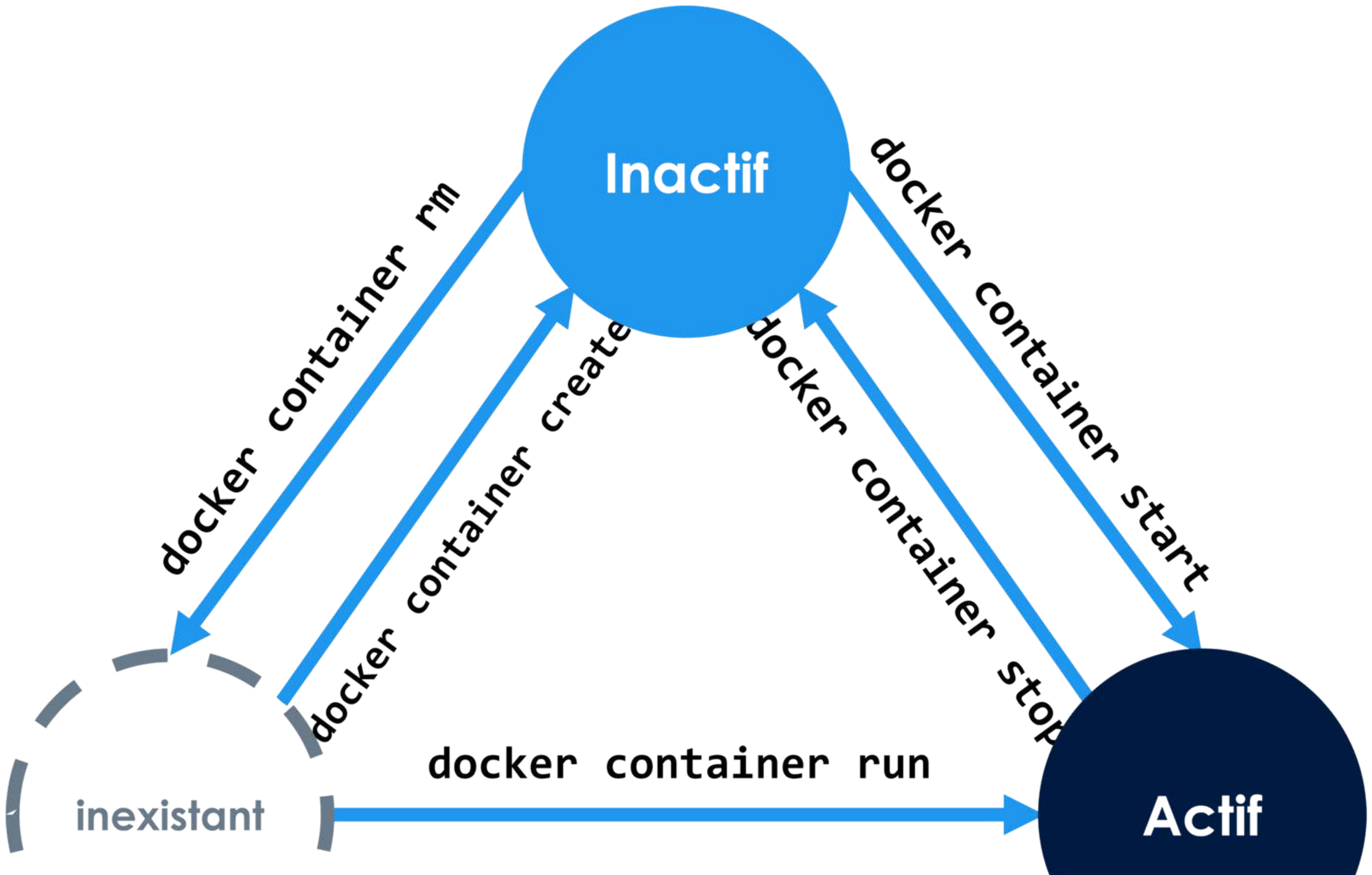
# ⏰ Rappel: cycle de vie d'un container



# ⏰ Rappel: cycle de vie d'un container



# ⏰ Rappel: cycle de vie d'un container



# 🔄 Reprendre le contrôle

Sur un container en arrière-plan

# 🔄 Reprendre le contrôle

Sur un container en arrière-plan

Il est possible d’interagir avec un container en arrière-plan en cours d’exécution.

# 🔄 Reprendre le contrôle

Sur un container en arrière-plan

Il est possible d’interagir avec un container en arrière-plan en cours d’exécution.

La commande suivante permet de lancer une commande à l’intérieur d’un container.

# 🔄 Reprendre le contrôle

Sur un container en arrière-plan

Il est possible d'interagir avec un container en arrière-plan en cours d'exécution.

La commande suivante permet de lancer une commande à l'intérieur d'un container.

```
docker container exec <containerID> echo "hello"
```

# 🔄 Reprendre le contrôle

Sur un container en arrière-plan

Il est possible d'interagir avec un container en arrière-plan en cours d'exécution.

La commande suivante permet de lancer une commande à l'intérieur d'un container.

```
docker container ls
```

# 🔄 Reprendre le contrôle

Sur un container en arrière-plan

Il est possible d'interagir avec un container en arrière-plan en cours d'exécution.

La commande suivante permet de lancer une commande à l'intérieur d'un container.

```
docker container ls
CONTAINER ID   IMAGE                              COMMAND                  CREATED         STATUS       PORTS
368ed08a35e3   jpetazzo/clock                     "/bin/sh -c 'while d…"   4 hours ago     Up 4 hours
02cbf2fb3721   cours-devops-docker-serve          "/sbin/tini -g gulp …"   7 hours ago     Up 7 hours   0.0.0.0:8000->8000/tc
ebfbe695b2ec   moby/buildkit:buildx-stable-1      "buildkitd"              2 months ago    Up 7 hours
```

# 🔄 Reprendre le contrôle

Sur un container en arrière-plan

Il est possible d'interagir avec un container en arrière-plan en cours d'exécution.

La commande suivante permet de lancer une commande à l'intérieur d'un container.

```
docker container ls
CONTAINER ID   IMAGE                                COMMAND                  CREATED         STATUS       PORTS
368ed08a35e3   jpetazzo/clock                       "/bin/sh -c 'while d…"   4 hours ago     Up 4 hours
02cbf2fb3721   cours-devops-docker-serve            "/sbin/tini -g gulp …"   7 hours ago     Up 7 hours   0.0.0.0:8000->8000/tc
ebfbe695b2ec   moby/buildkit:buildx-stable-1        "buildkitd"              2 months ago    Up 7 hours

docker container exec 368ed08a35e3 echo hello
hello
```

# 🔄 Reprendre le contrôle

Sur un container en arrière-plan

```
docker container exec --interactive --tty <containerID> bash
```

Ca fonctionne aussi en interactif !

# 🎓 Exercice : conteneur en tâche de fond

Étapes

- Lancer un container "daemon" `jpetazzo/clock`
- Utiliser l'équivalent de `tail -f` pour lire la sortie standard du container 💡
- Lancer un second container "daemon" 👹
- Stocker l'identifiant de ce container dans une variable du shell, en une seule commande et en jouant avec `docker container ls`
- Stopper le container avec cet identifiant
- Afficher les containers lancés 🏃📦
- Afficher les containers arrêtés 🛑📦

# ✅ Solution : conteneur en tâche de fond

```
# Lancer un container "daemon" `jpetazzo/clock`
docker container run --detach --name first-clock jpetazzo/clock

# Utiliser l'équivalent de `tail -f` pour lire la sortie standard du container 💡
docker container logs -f first-clock

# * Lancer un second container "daemon" 👹
docker container run --detach --name second-clock jpetazzo/clock

# Stocker l'identifiant de ce container dans une variable du shell, en une seule commande et en jouant avec `docker contai
# --filter "name=second-clock" filters the list of containers to include only those with the name "second-clock."
container_id=$(docker container ls -q --filter "name=second-clock")

# Stopper le container avec cet identifiant
docker container stop "$container_id"

# Afficher les containers lancés  🏃📦
docker container ls

# Afficher les containers arrêtés 🛑📦
docker container ls --filter "status=exited"
```

# 🎓 Exercice : conteneur en tâche de fond

- Relancer un des containers arrêtés.
- Exécuter un `ps -ef` dans ce container
- Quel est le PID du process principal ?
- Vérifier que le container "tourne" toujours
- Supprimer l’image (tip : `docker image rm`)
- Supprimer les containers
- Supprimer l’image (pour de vrai cette fois)

# ✓ Solution : conteneur en tâche de fond (1/2)

```
# Relancer un des containers arrêtés.
docker container start second-clock

# Exécuter un `ps -ef` dans ce container
docker container exec second-clock ps -ef
PID   USER     TIME  COMMAND
    1 root      0:00 /bin/sh -c while date; do sleep 1; done
   56 root      0:00 sleep 1
   57 root      0:00 ps -ef

# Quel est le PID du process principal ?
# 1

# Vérifier que le container "tourne" toujours
docker container ls
# (sortie abrégée : autres conteneurs et colonnes CREATED, PORTS retirés)
CONTAINER ID   IMAGE             COMMAND                  STATUS              NAMES
edea0c46c6d8   jpetazzo/clock    "/bin/sh -c 'while d…"   Up About a minute   second-clock
```

# ✅ Solution : conteneur en tâche de fond (2/2)

```
# Supprimer l'image (tip : `docker image rm`)
docker image rm jpetazzo/clock
Error response from daemon: conflict: unable to delete jpetazzo/clock:latest (must be forced)
 - container edea0c46c6d8 is using its referenced image dc06bbc3744f

# Supprimer les containers (first-clock utilise aussi l'image)
docker container stop first-clock second-clock
first-clock
second-clock

docker container rm first-clock second-clock
first-clock
second-clock

# Supprimer l'image (pour de vrai cette fois)
docker image rm jpetazzo/clock
Untagged: jpetazzo/clock:latest
Deleted: sha256:dc06bbc3744f7200404bff0bbb2516925e7adea115e07de9da8b36bf15fe3dd3
```

# Et `docker image rm -f` ?

Le message d'erreur suggère de forcer. Voici ce que fait vraiment `-f` quand un conteneur utilise encore l'image :

```
docker image rm -f jpetazzo/clock
Untagged: jpetazzo/clock:latest
```

- Seule l'**étiquette** est retirée, que le conteneur soit arrêté ou en cours d'exécution (il continue de tourner).
- `docker container ls --all` affiche ensuite l'ID de l'image à la place de son nom.
- Les couches restent sur le disque : l'espace n'est libéré qu'après suppression des conteneurs qui les utilisent.

# 🎓 Exercice : conteneur en tâche de fond

- Exécutez un conteneur, basé sur l'image `nginx` en tâche de fond ("Background"), nommé `webserver-1`
  - 💡 On parle de processus "détaché" (ou bien "démonisé") 👹
  - ⚠️ Pensez bien à `docker container ls`
- Regardez le contenu du fichier `/etc/os-release` dans ce conteneur
  - `docker container exec`
- Essayez d'arrêter, démarrer puis redémarrer le conteneur
  - ⚠️ Pensez bien à `docker container ls` à chaque fois
  - 💡 `stop`, `start`, `restart`

# ✓ Solution : conteneur en tâche de fond

```
docker container run --detach --name=webserver-1 nginx
# <ID du conteneur>

docker container ls
docker container ls --all

docker container exec webserver-1 cat /etc/os-release
# ... Debian ...

docker container stop webserver-1
docker container ls
docker container ls --all

docker container start webserver-1
docker container ls
docker container ls --all

docker container start webserver-1
docker container ls
```

# Checkpoint 🎯

Vous savez désormais:

- Maîtriser le cycle de vie des containers
- Interagir avec les containers existants

# Checkpoint 🎯

- Docker essaye de résoudre le problème de l’empaquetage le plus "portable" possible
  - On n’en a pas encore vu les effets, ça arrive !
- Vous avez vu qu’un containeur permet d’exécuter une commande dans un environnement "préparé"
  - Catalogue d’images Docker par défaut : Le Docker Hub

- Vous avez vu qu’on peut exécuter des conteneurs selon 3 modes :
  - "One shot"
  - Interactif
  - En tâche de fond

  ⇒ 🤔 Mais comment ces images sont-elles fabriquées ? Quelle confiance leur accorder ?

# Docker Images

# 🤔 Pourquoi des images ?

- Un **conteneur** est toujours exécuté depuis une **image**.
- Une **image de conteneur** (ou "Image Docker") est un modèle ("template") d'application auto-suffisant.

⇒ Permet de fournir un livrable portable (ou presque).

# 🤔 C’est quoi une image ?

C’est une collection de fichiers et de metadonnées.

# 🤔 C’est quoi une image ?

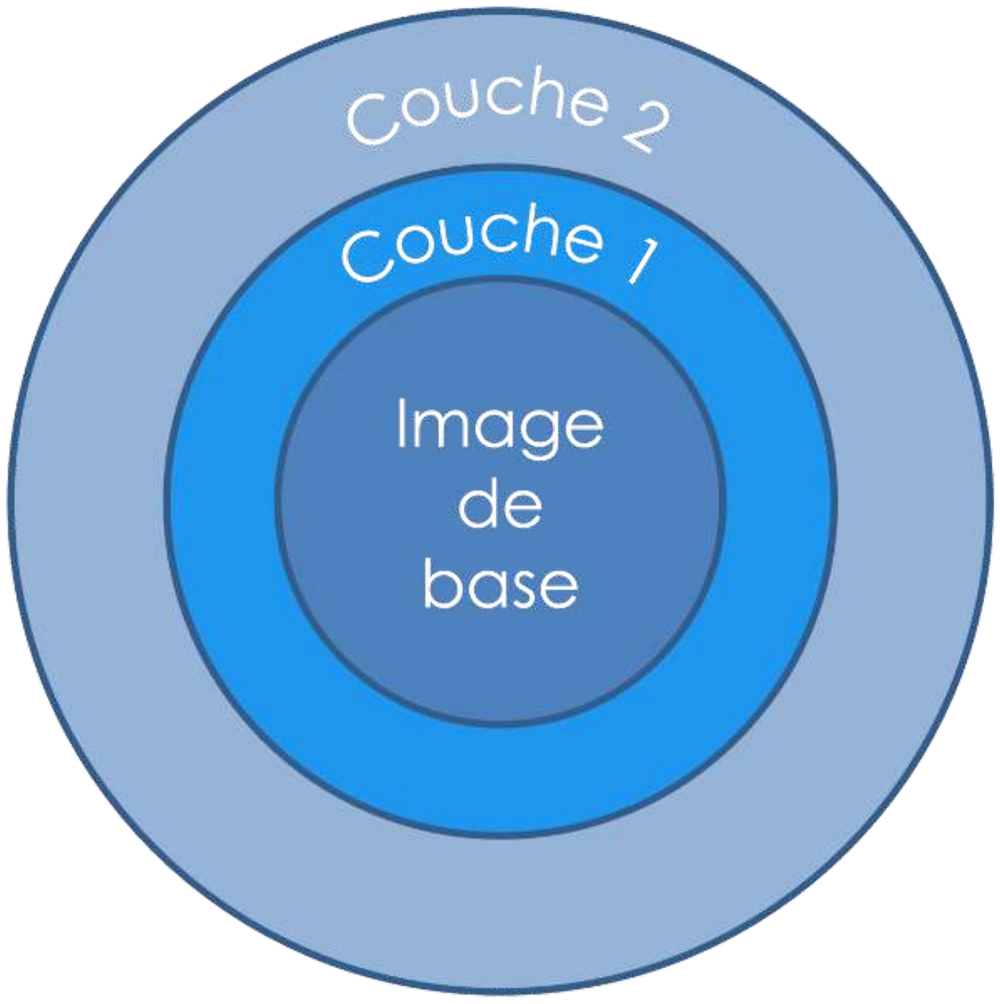
C’est une collection de fichiers et de metadonnées.

C’est une suite de couches superposées.

# 🤔 C’est quoi une image ?

C’est une collection de fichiers et de metadonnées.

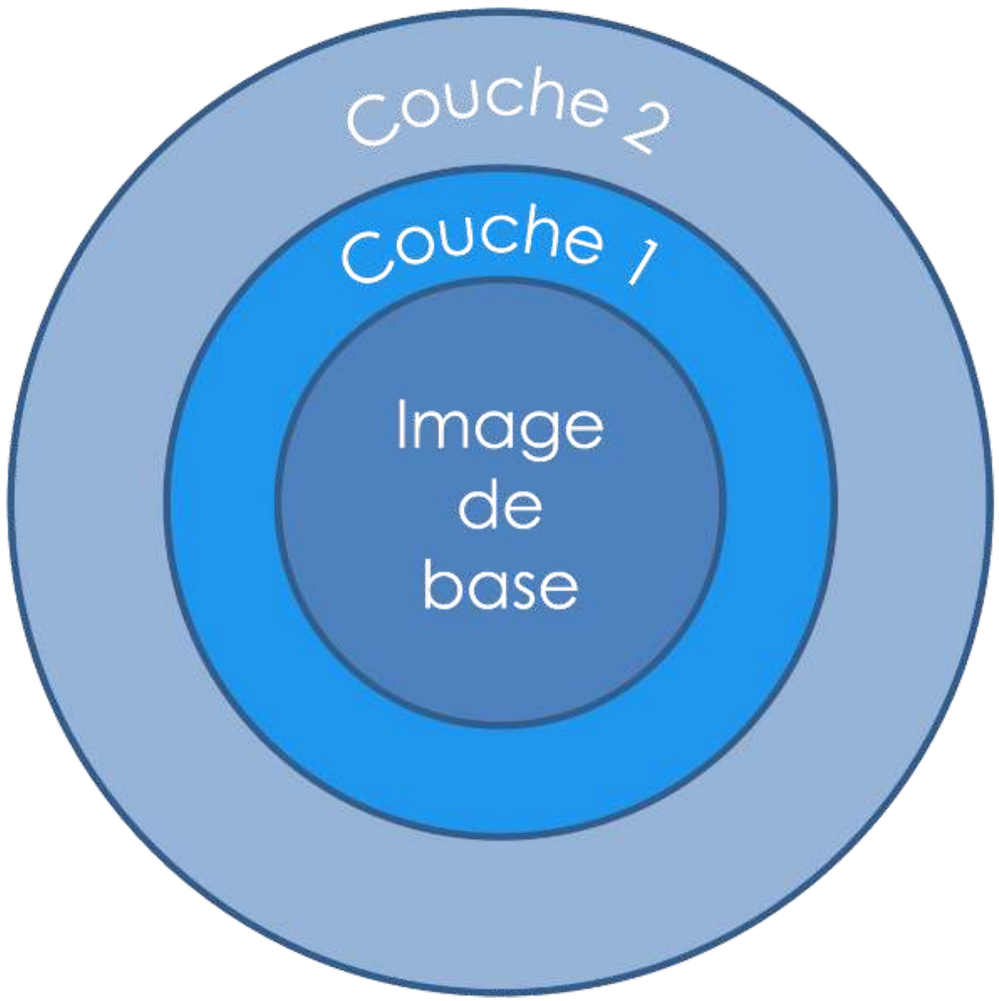
C’est une suite de couches superposées.

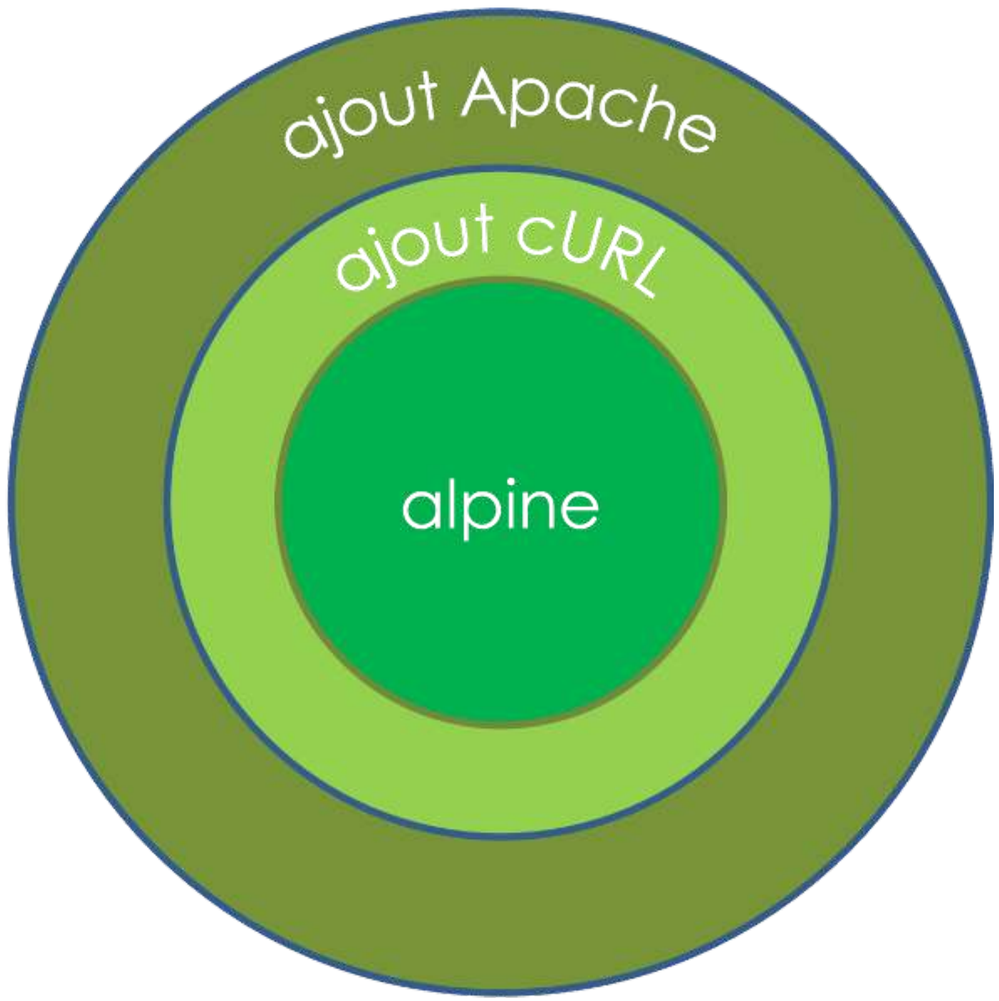


# 🤔 C’est quoi une image ?

C’est une collection de fichiers et de metadonnées.

C’est une suite de couches superposées.



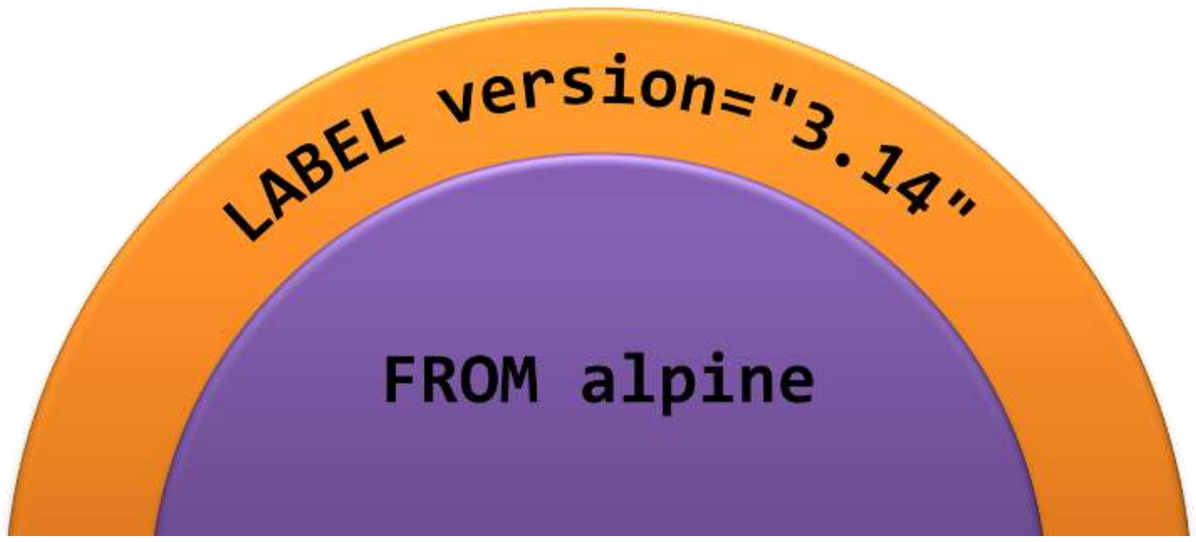



# Détail des couches

Exemple d’une image Apache HTTPd "custom"

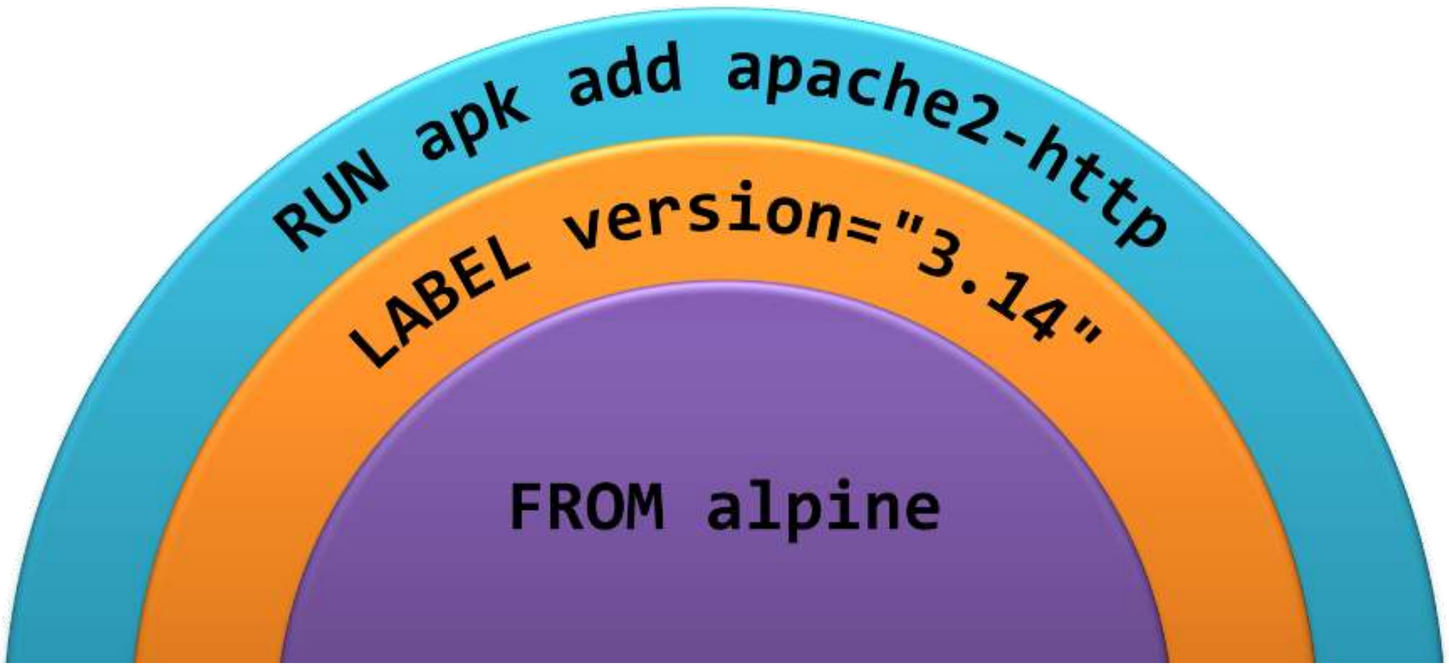
# Détail des couches

Exemple d’une image Apache HTTPd "custom"



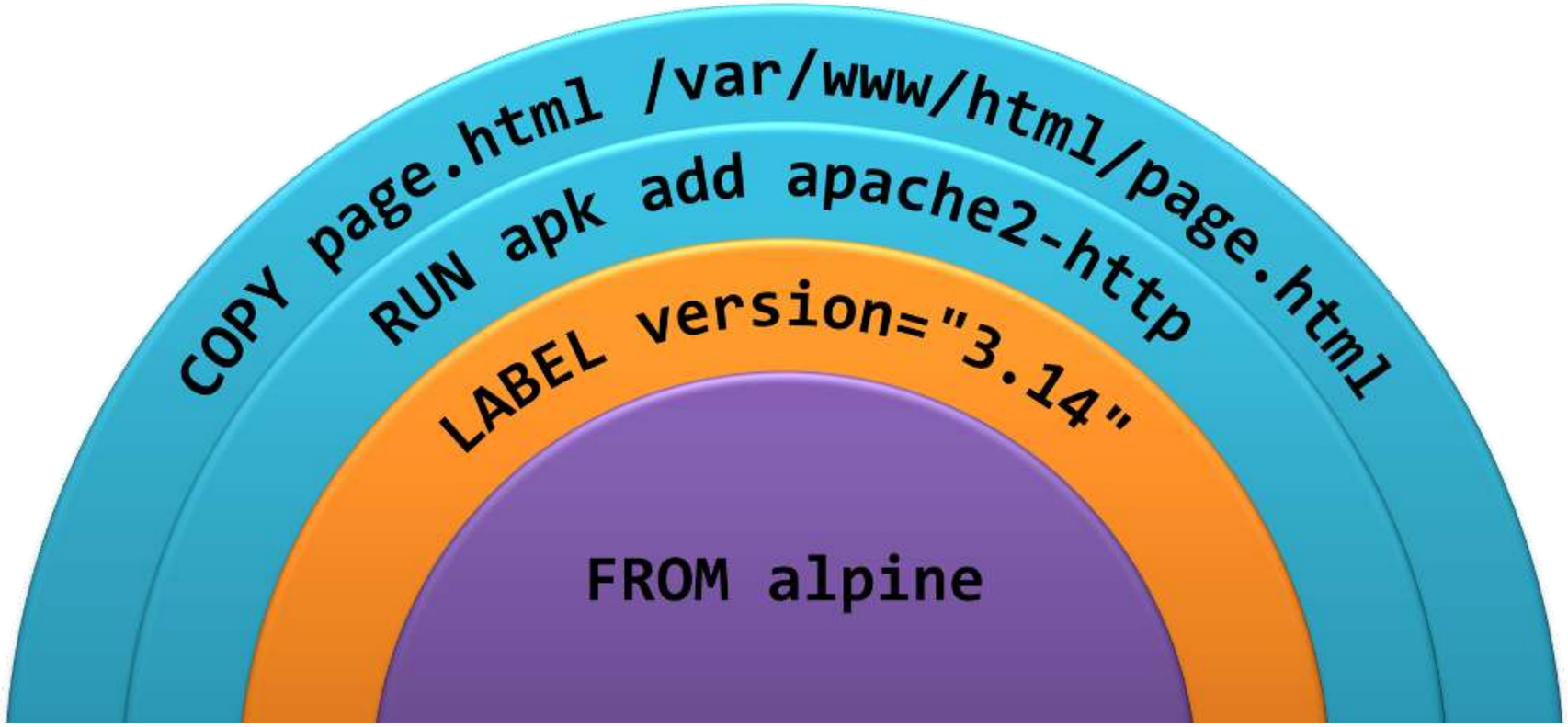
# Détail des couches

Exemple d’une image Apache HTTPd "custom"



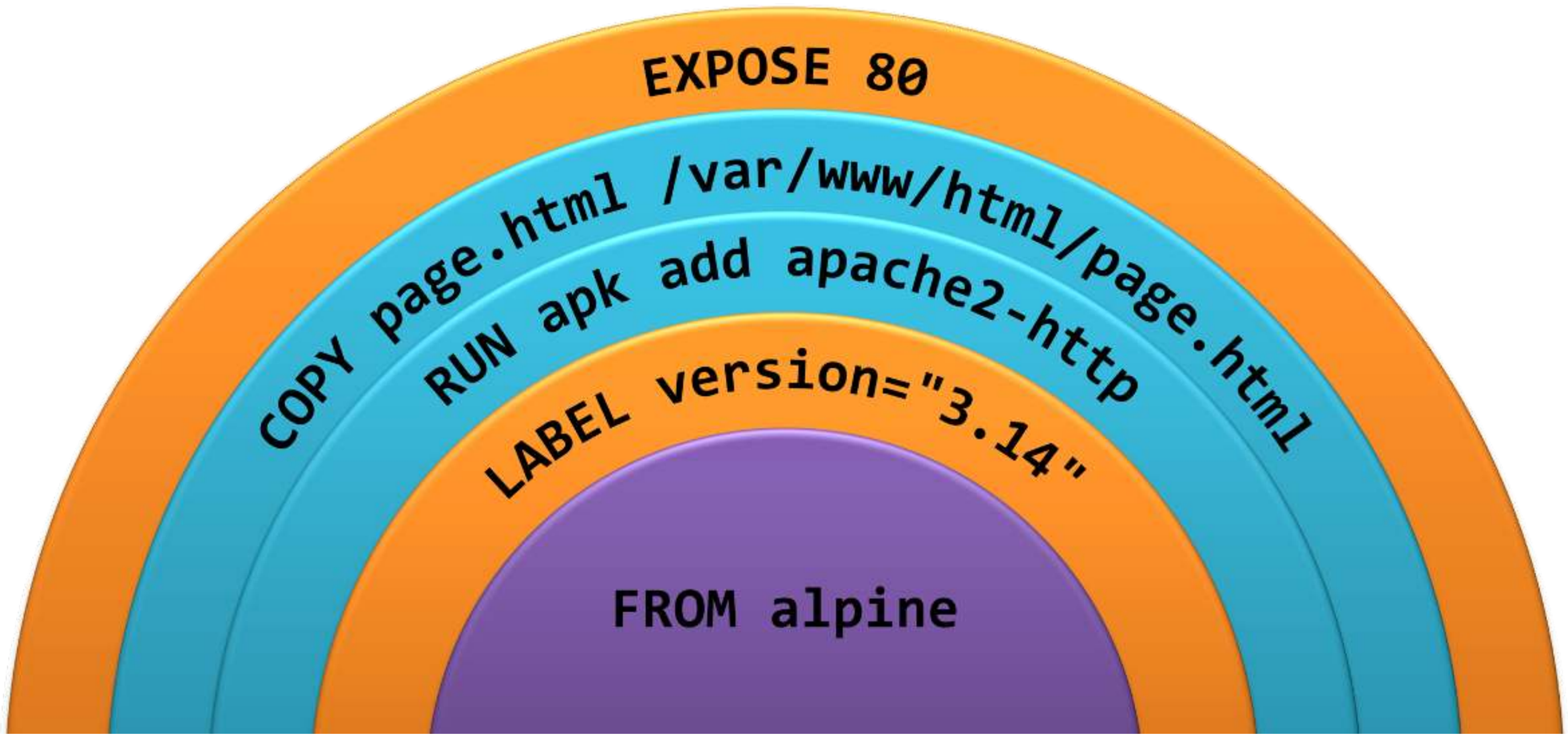
# Détail des couches

Exemple d’une image Apache HTTPd "custom"



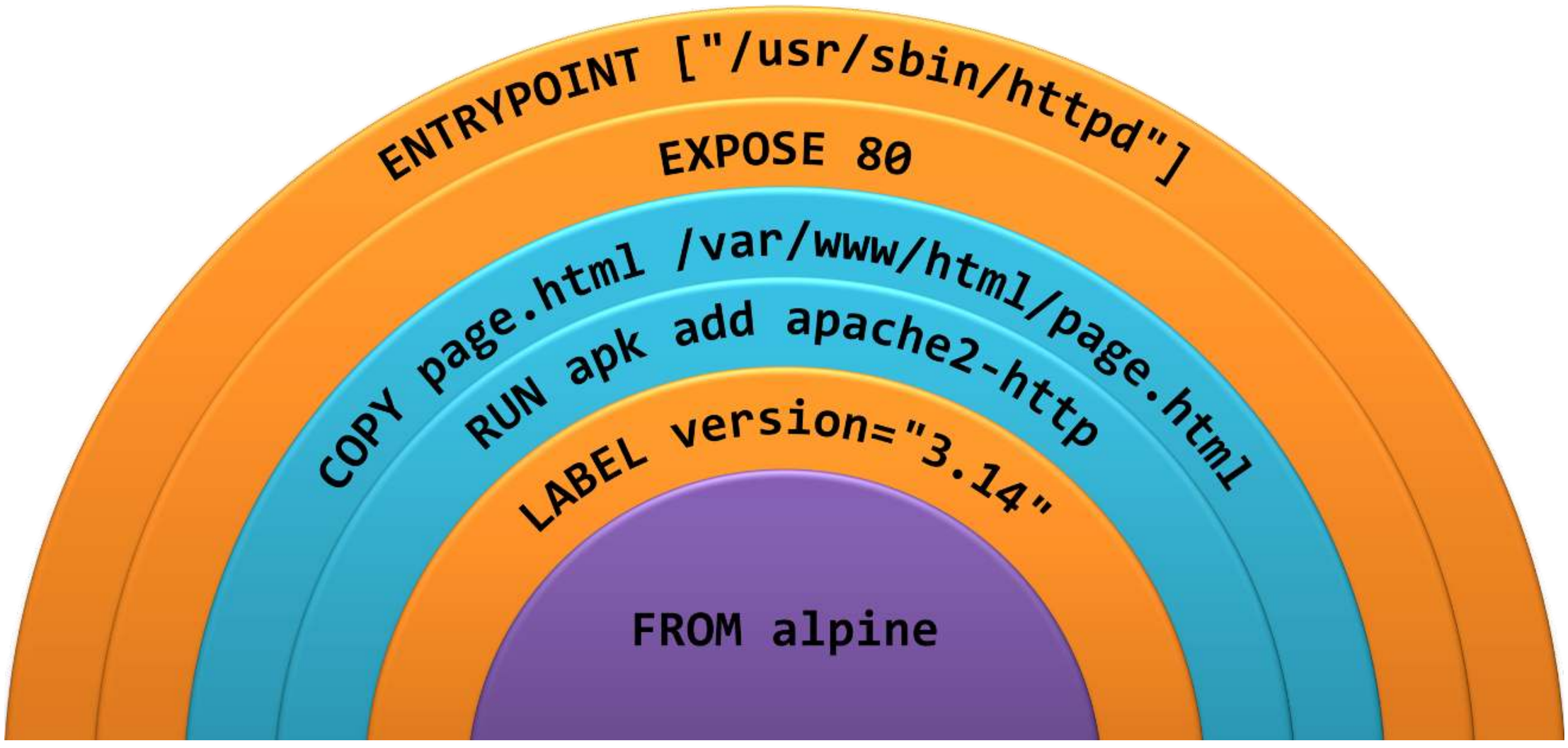
# Détail des couches

Exemple d’une image Apache HTTPd "custom"



# Détail des couches

Exemple d’une image Apache HTTPd "custom"

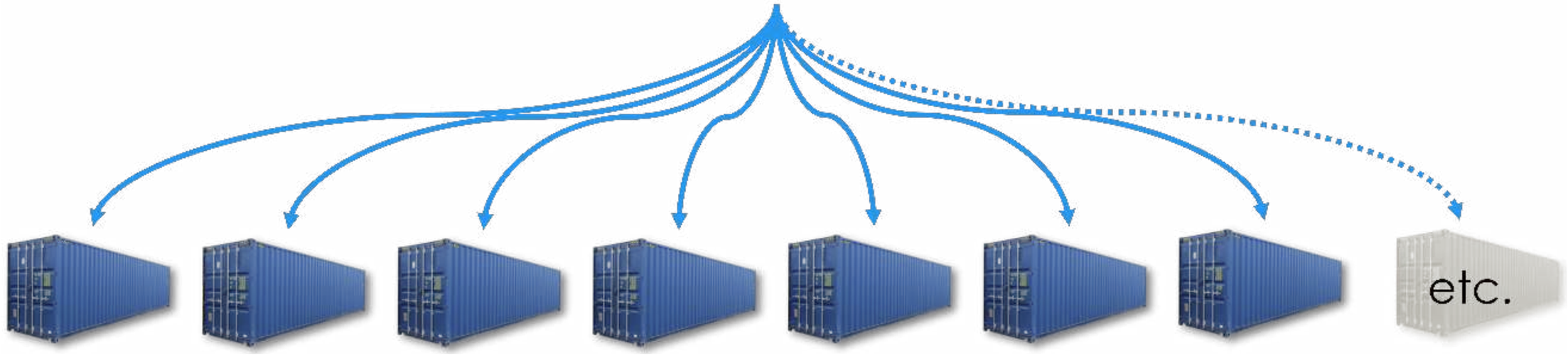


# Dicton du jour

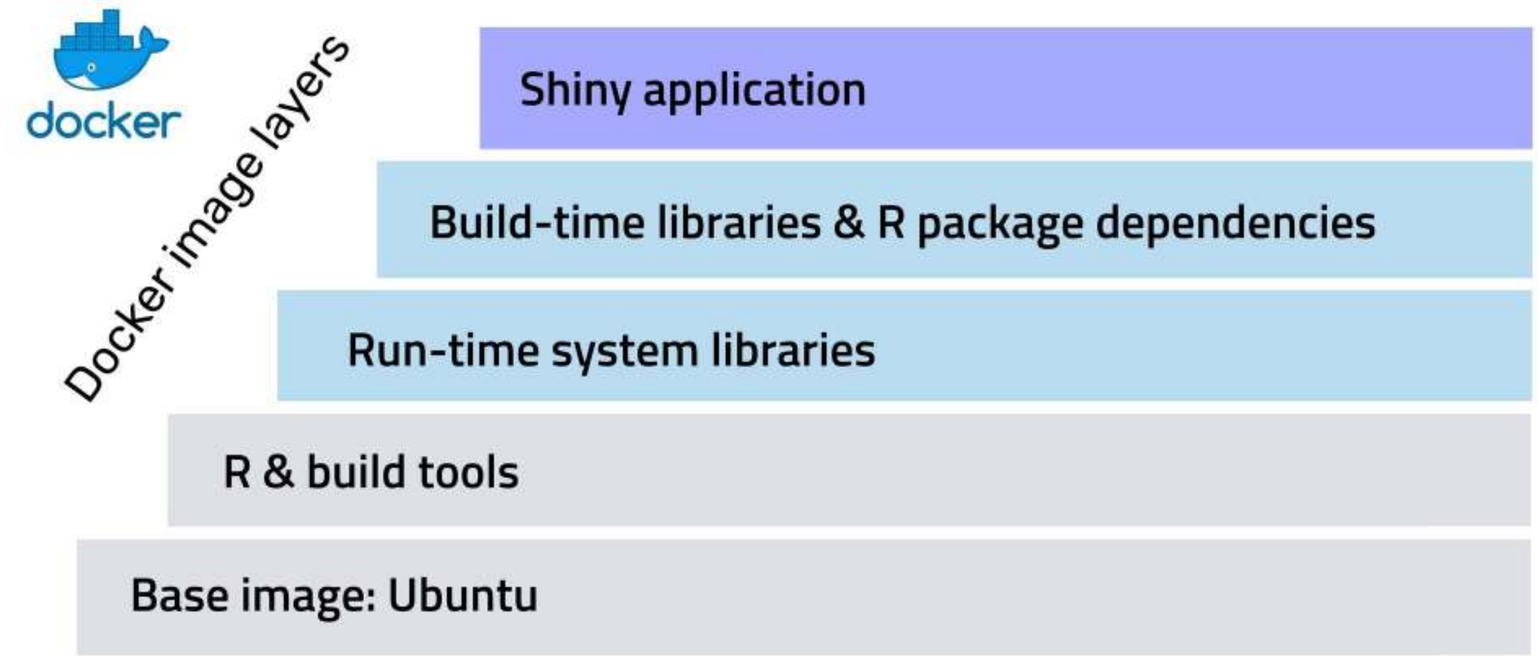
***"L'image est à la classe ce que le container est à l'objet"***

Amaury W.

14 septembre 2023

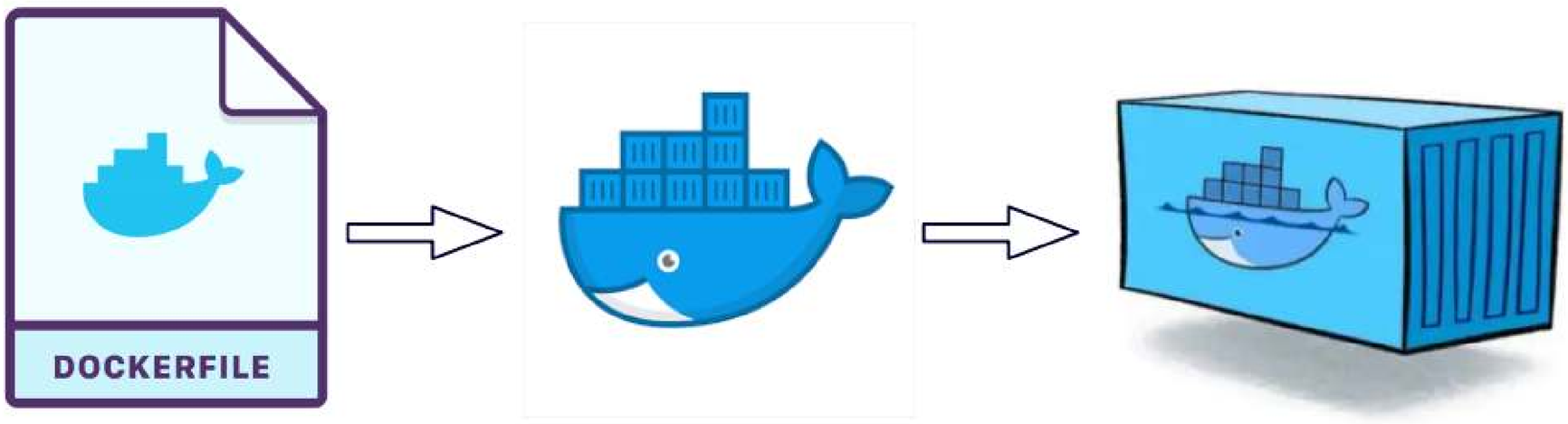


# 🤔 Application Auto-Suffisante ?




© Analythium

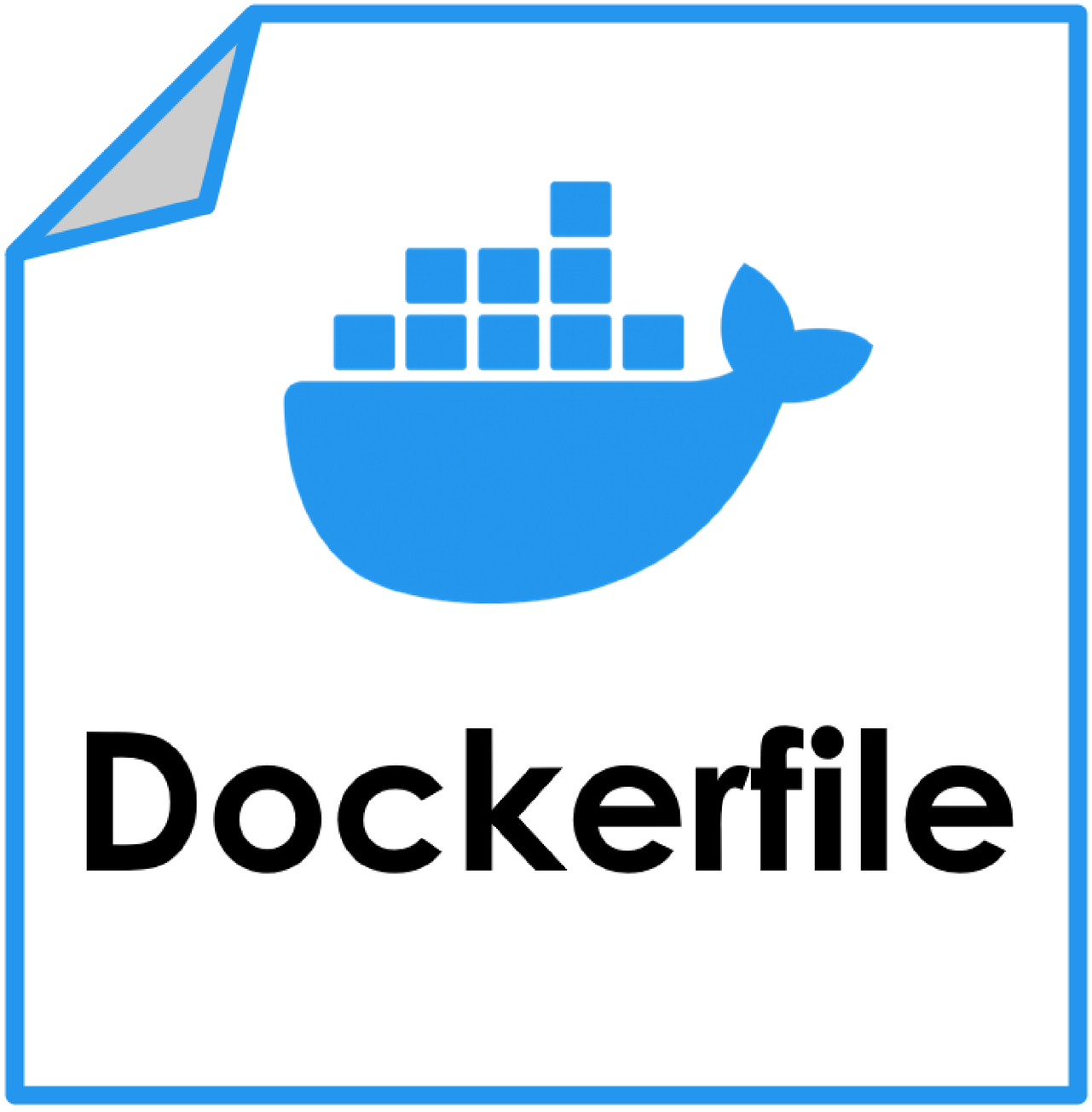
# C’est quoi le principe ?



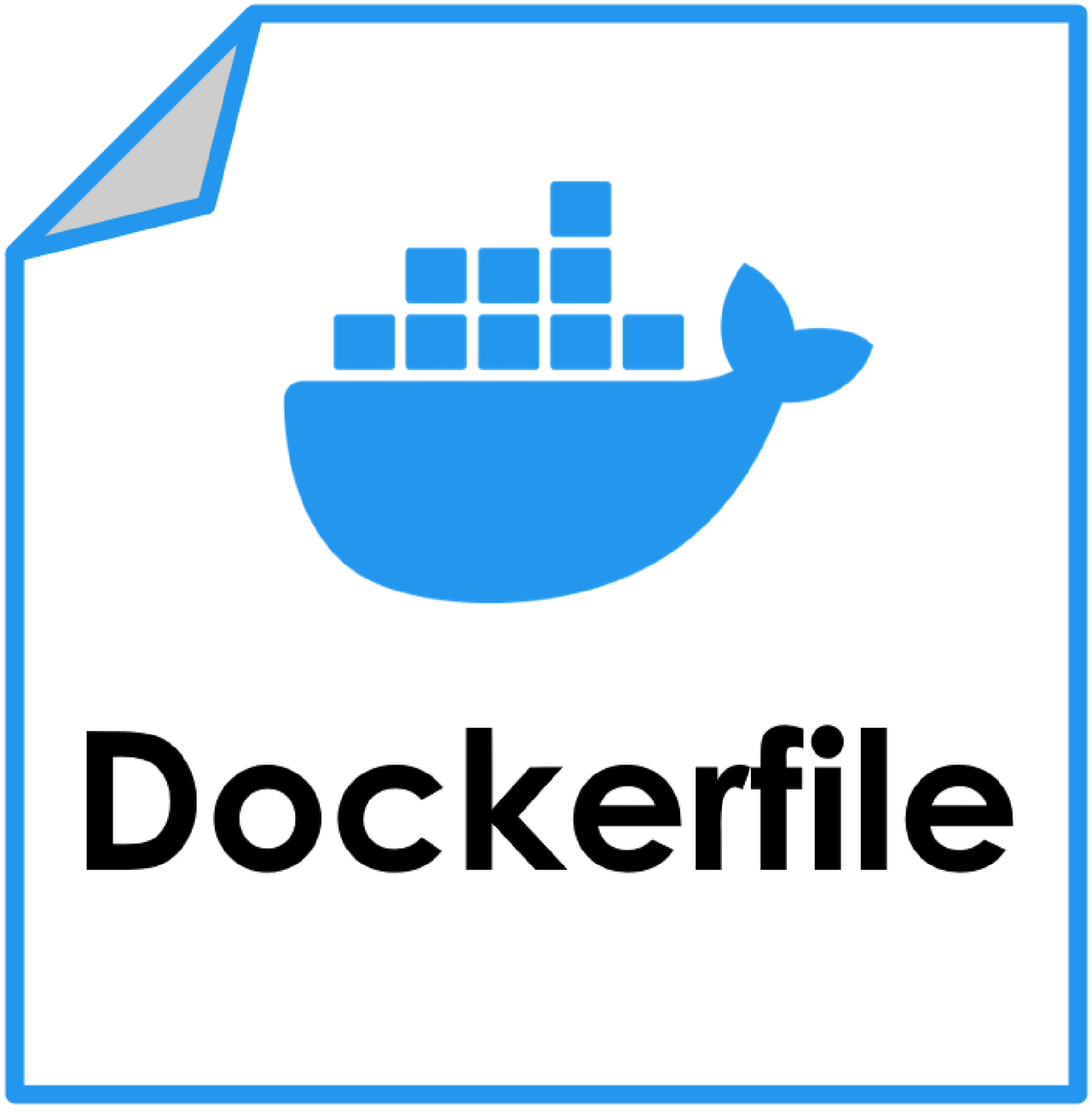

**Docker file** **Docker Image** **Docker Container**

# 🐋 📖 🍳 Le livre de recettes

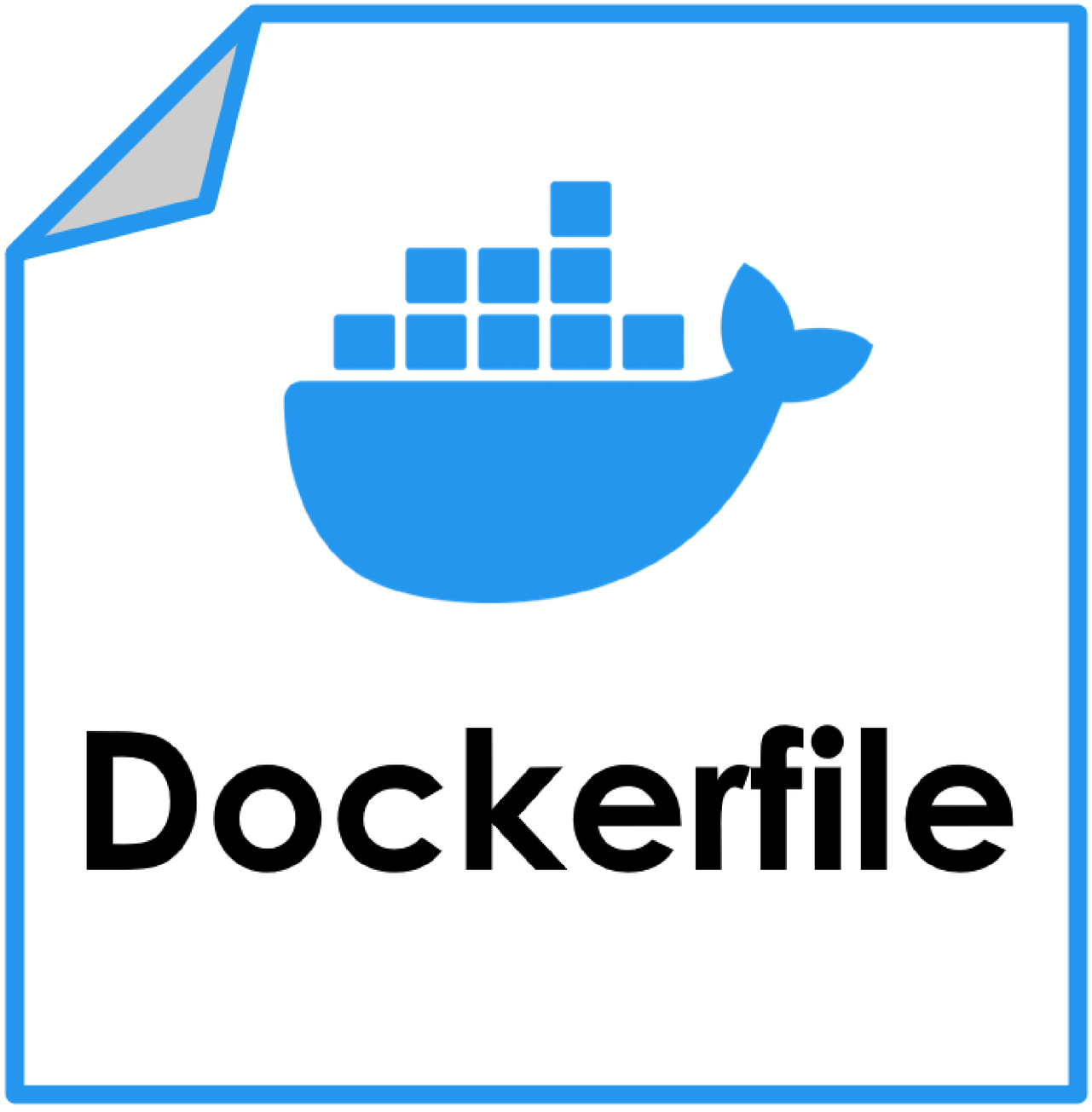
# 🐋 📖 🍳 Le livre de recettes



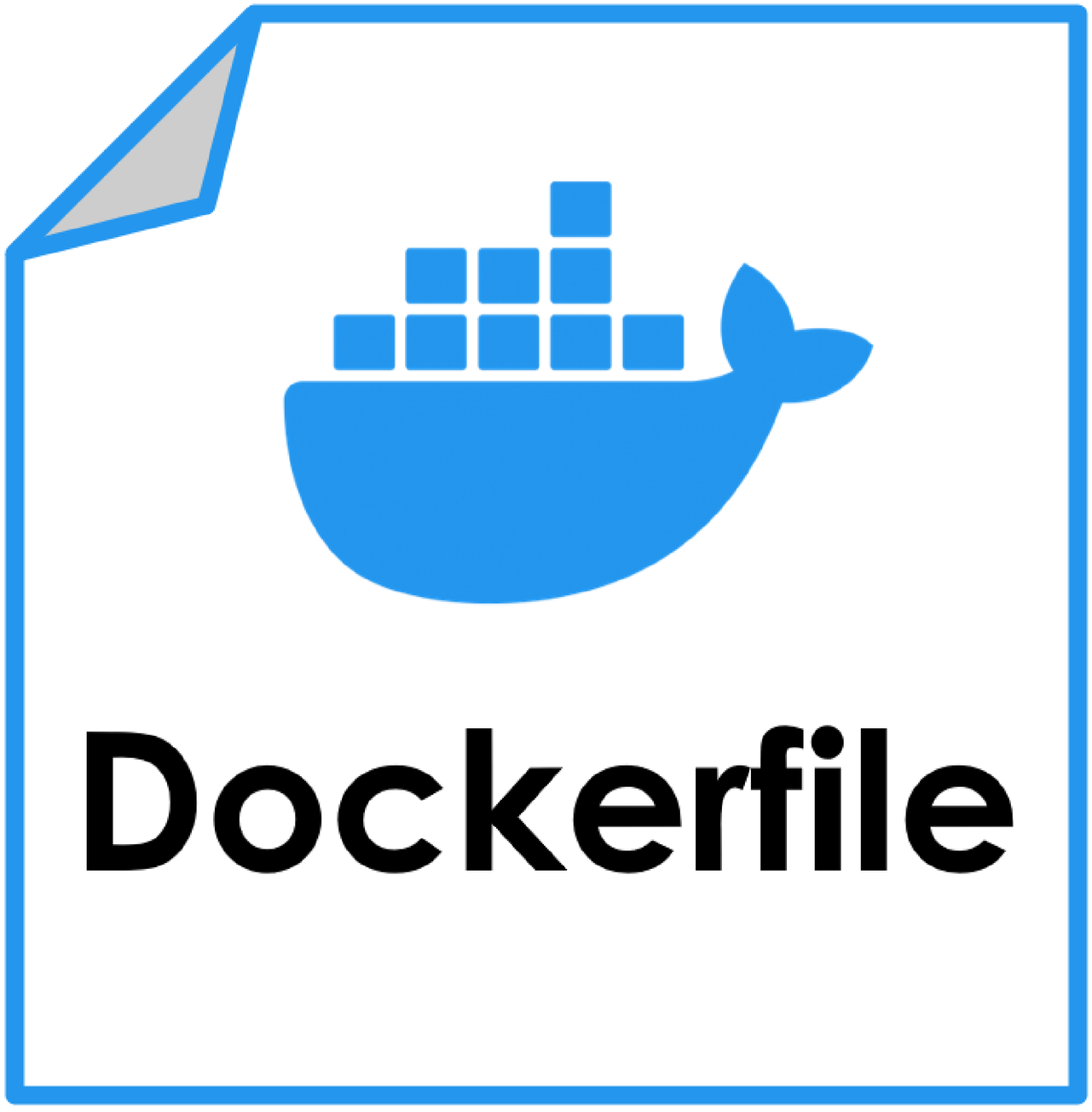
# 🐋 📖 🍳 Le livre de recettes



# 🐋 📖 🍳 Le livre de recettes



# 🐋 📖 🍳 Le livre de recettes



# 🤔 Pourquoi fabriquer sa propre image ?

❗ Problème :

```
cat /etc/os-release
# ...
git --version
# ...

# Même version de Linux que dans Codespaces
docker container run --rm ubuntu:24.04 git --version
# docker: Error response from daemon: failed to create shim task: OCI runtime create failed: runc create failed: unable t

# En interactif ?
docker container run --rm --tty --interactive ubuntu:24.04 git --version
```

# 🎓 Fabriquer sa première image

- **But :** fabriquer une image Docker qui contient `git`
- Dans votre Codespace, créez un dossier nommé `docker-git/`
- Dans ce dossier, créer un fichier `Dockerfile` avec le contenu ci-dessous :

```
FROM alpine:3.24.2

RUN apk add --no-cache git
```

- Fabriquez votre image avec la commande `docker image build --tag=docker-git chemin/vers/docker-git/`
- Testez l'image fraîchement fabriquée
  - 💡 `docker image ls`

# ✓ Fabriquer sa première image

```
cat <<EOF >Dockerfile
FROM alpine:3.24.2

RUN apk add --no-cache git
EOF

docker image build --tag=docker-git ./

docker image ls | grep docker-git

# Doit fonctionner
docker container run --rm docker-git:latest git --version
```

# 🐋 📖 🍳 Fabriquer son image

Un peu de cuisine…

# 🐋 📖 🍳 Fabriquer son image

## 💪🏁 DÉFI

Créer une image alpine avec un JRE installé.

Dockerfile

```
FROM alpine:3.24.2

LABEL maintainer="Tony Stark"

RUN apk add --no-cache openjdk17-jre-headless
```

Dockerfile

```
FROM alpine:3.24.2
LABEL maintainer="Tony Stark"

RUN apk add --no-cache openjdk17-jre-headless
```

couche de base

Dockerfile

```
FROM alpine:3.24.2
LABEL maintainer="Tony Stark"

RUN apk add --no-cache openjdk17-jre-headless
```

couche de base

metadata

Dockerfile

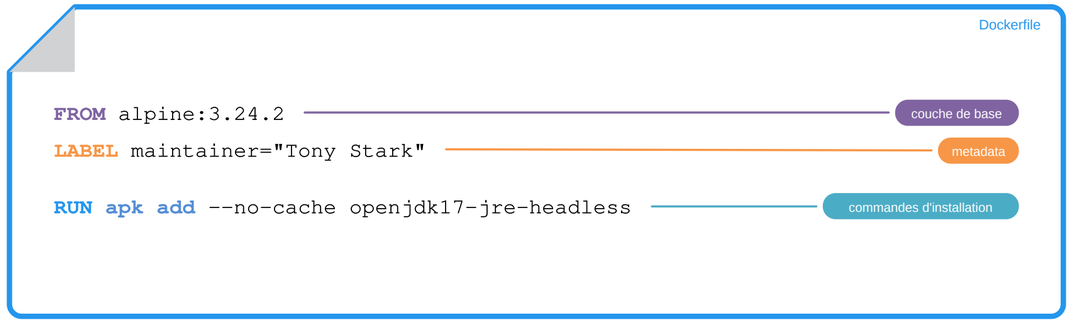
```
FROM alpine:3.24.2
LABEL maintainer="Tony Stark"

RUN apk add --no-cache openjdk17-jre-headless
```

couche de base

metadata

commandes d'installation

# 🐋 📖 🍳 Fabriquer son image




```
FROM alpine:3.24.2
LABEL maintainer="Tony Stark"

RUN apk add --no-cache openjdk17-jre-headless
```

# 🐋 📖 🍳 Cuistot, au boulot!

```
docker image build -t myjava:1.42 .
```

# 🐋 📖 🍳 Cuistot, au boulot!

```
docker image build -t myjava:1.42 .
```

nom de l'image à créer

# 🐋 📖 🍳 Cuistot, au boulot!

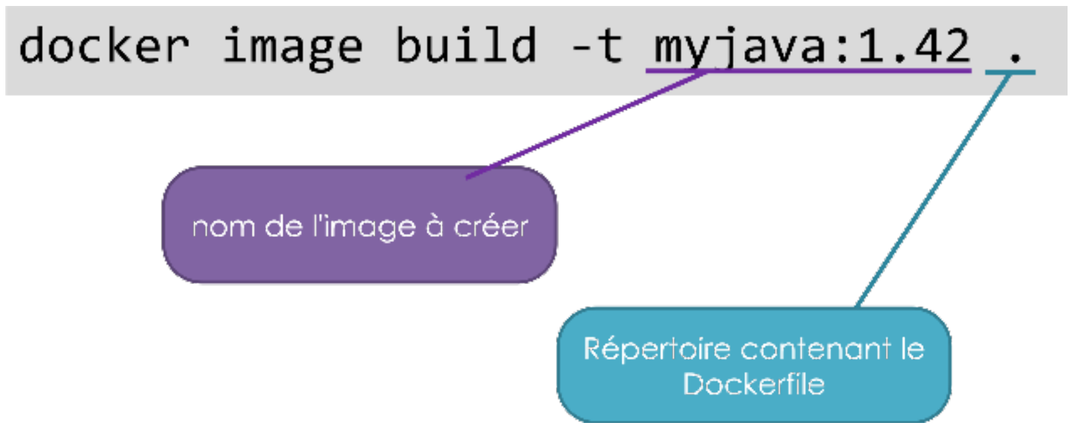
```
docker image build -t myjava:1.42 .
```

nom de l'image à créer

Répertoire contenant le Dockerfile

# 🐳 📖 🍳 Cuistot, au boulot!

```
docker image build -t myjava:1.42 .
```




```
docker image build -t myjava:1.42 .
```

# 🪜 Étapes de construction

```
Sending build context to Docker daemon  9.728k
Step 1/4 : FROM alpine:3.18
 ---> 7e6257c9f8d8
Step 2/4 : LABEL maintainer="Tony Stark"
 ---> Running in 8dfc06b5f18b
Removing intermediate container 8dfc06b5f18b
 ---> e7584e63d1fe
Step 3/4 : RUN apk update
 ---> Running in 01c16b79b7e6
Loaded plugins: fastestmirror, ovl
Determining fastest mirrors
...
```

# 🪜 Étapes de construction

```
Sending build context to Docker daemon  9.728kB
Step 1/4 : FROM alpine:3.18
 ---> 7e6257c9f8d8
Step 2/4 : LABEL maintainer="Tony Stark"
 ---> Using cache
 ---> e7584e63d1fe
Step 3/4 : RUN apk update
 ---> Running in 876b9c10017a
Loaded plugins: fastestmirror, ovl
Determining fastest mirrors
...
```

# 🪜 Étapes de construction

```
Sending build context to Docker daemon  9.728kB
Step 1/4 : FROM alpine:3.18
 ---> 7e6257c9f8d8
Step 2/4 : LABEL maintainer="Tony Stark"
 ---> Using cache
 ---> e7584e63d1fe
Step 3/4 : RUN apk update
 ---> Running in 876b9c10017a
Loaded plugins: fastestmirror, ovl
Determining fastest mirrors
...
```

On gagne du temps grâce aux builds précédents !

# Et après ?

L’image est dispo !

# Registre local, mais encore?

On les trouve où, ces images ?

# Les registres Docker

Ce sont des plates-formes qui hébergent les images.

Il est possible de créer ses propres registres (ex : registre privé d'entreprise)

DockerCon 2023: Our annual developer event is back — online & in person. Learn more.



# node

Docker Official Image · 1B+ · 10K+

Node.js is a JavaScript-based platform for server-side and networking applications.

```
docker pull node
```

Overview Tags

## Quick reference

- **Maintained by**:
  The Node.js Docker Team
- **Where to get help**:
  the Docker Community Slack, Server Fault, Unix & Linux, or Stack Overflow

## Supported tags and respective `Dockerfile` links

- `20-alpine3.17`, `20.7-alpine3.17`, `20.7.0-alpine3.17`, `alpine3.17`, `current-alpine3.17`
- `20-alpine`, `20-alpine3.18`, `20.7-alpine`, `20.7-alpine3.18`, `20.7.0-alpine`, `20.7.0-alpine3.18`, `alpine`, `alpine3.18`, `current-alpine`, `current-alpine3.18`
- `20`, `20-bookworm`, `20.7`, `20.7-bookworm`, `20.7.0`, `20.7.0-bookworm`, `bookworm`, `current`, `current-bookworm`, `latest`
- `20-bookworm-slim`, `20-slim`, `20.7-bookworm-slim`, `20.7-slim`, `20.7.0-bookworm-slim`, `20.7.0-slim`, `bookworm-slim`, `current-bookworm-slim`, `current-slim`, `slim`
- `20-bullseye`, `20.7-bullseye`, `20.7.0-bullseye`, `bullseye`, `current-bullseye`
- `20-bullseye-slim`, `20.7-bullseye-slim`, `20.7.0-bullseye-slim`, `bullseye-slim`, `current-`

### Recent Tags

lts-hydrogen lts-buster lts-bu
latest hydrogen-buster hydro
hydrogen-bookworm hydrogen

### About Official Images

Docker Official Images are a
open source and drop-in solu

#### Why Official Images?

These images have clear doc
best practices, and are desig
common use cases.

# 🖼️ Les images

Avant d’instancier un container, il faut récupérer l’image en local.

# Un téléchargement par couches

# 🖼️🏷️ Conventions de nommage des images

**REGISTRE**

Le registre sur lequel on souhaite récupérer l'image.

Optionnel quand on récupère depuis le Docker Hub.

# 🖼️🏷️ Conventions de nommage des images

# 🖼️🏷️ Conventions de nommage des images

# 🖼️🏷️ Conventions de nommage des images

# reg.mycompany.com/prj/myapp:12-4

| reg.mycompany.com | prj | myapp | 12-4 |
|---|---|---|---|

# `reg.mycompany.com/prj/myapp:12-4`

# `foo/bar:baz`

# reg.mycompany.com/prj/myapp:12-4

| reg.mycompany.com | prj | myapp | 12-4 |
|---|---|---|---|

# foo/bar:baz

| Docker Hub | foo | bar | baz |
|---|---|---|---|

# alpine

| Docker Hub | Image Officielle | alpine | latest |
|---|---|---|---|

# `reg.mycompany.com/prj/myapp:12-4`

| `reg.mycompany.com` | prj | myapp | 12-4 |
|---|---|---|---|

# `foo/bar:baz`

| Docker Hub | foo | bar | baz |
|---|---|---|---|

# `alpine`

| Docker Hub | Image Officielle | alpine | latest |
|---|---|---|---|

# `hello-world:42`

| Registre local | | hello-world | 42 |
|---|---|---|---|

# 📷🏷️ Conventions de nommage des images

# jenkins/jenkins

Sponsored OSS ☆

By Jenkins • Updated a day ago

The leading open source automation server

Image

Pulls 1B+

Overview **Tags**

Sort by Newest jdk21 ×

TAG

jdk21

Last pushed a day ago by jenkinsinfraadmin

`docker pull jenkins/jenkins:jdk…`

| DIGEST | OS/ARCH | COMPRESSED SIZE |
|---|---|---|
| 9b23ced2b7e4 | linux/amd64 | 282.07 MB |
| 7579a7d3caeb | linux/arm/v7 | 390.1 MB |
| b52835c4edf7 | linux/arm64 | 280.53 MB |

+2 more...

TAG

latest-jdk21-preview

Last pushed a day ago by jenkinsinfraadmin

`docker pull jenkins/jenkins:lat…`

| DIGEST | OS/ARCH | COMPRESSED SIZE |
|---|---|---|
| 9b23ced2b7e4 | linux/amd64 | 282.07 MB |
| 7579a7d3caeb | linux/arm/v7 | 390.1 MB |
| b52835c4edf7 | linux/arm64 | 280.53 MB |

+2 more...

TAG

2.424-jdk21

Last pushed a day ago by jenkinsinfraadmin

`docker pull jenkins/jenkins:2.4…`

| DIGEST | OS/ARCH | COMPRESSED SIZE |
|---|---|---|
| 9b23ced2b7e4 | linux/amd64 | 282.07 MB |
| 7579a7d3caeb | linux/arm/v7 | 390.1 MB |
| b52835c4edf7 | linux/arm64 | 280.53 MB |

+2 more...

# 📷🏷️ Conventions de nommage des images

Une même image peut avoir plusieurs noms et tags !

Le tag "latest" est régulièrement réaffecté sur les registres distants.

**Plusieurs noms pour une signature**

**Tags disponibles pour MySQL**

- `8.1.0`, `8.1`, `8`, `innovation`, `latest`, `8.1.0-oracle`, `8.1-oracle`, `8-oracle`, `innovation-oracle`, `oracle`
- `8.0.34`, `8.0`, `8.0.34-oracle`, `8.0-oracle`
- `8.0.34-debian`, `8.0-debian`
- `5.7.43`, `5.7`, `5`, `5.7.43-oracle`, `5.7-oracle`, `5-oracle`

# 📷🏷️ Conventions de nommage des images

Pour résumer…

# 📷🏷️ Conventions de nommage : Exemples

- `ubuntu:22.04` ⇒ `registry.docker.com/library/ubuntu:22.04`
- `dduportal/docker-asciidoctor` ⇒ `registry.docker.com/dduportal/docker-asciidoctor:latest`
- `ghcr.io/dduportal/docker-asciidoctor:1.3.2@sha256:xxxx`

# 🎓 Utilisons les tags

- Rappel : ⚠️ Friends don't let friends use `latest` 👫
- Il est temps de "taguer" votre première image !

```
docker image tag docker-git:latest docker-git:1.0.0
```

- Testez le fonctionnement avec le nouveau tag
- Comparez les 2 images dans la sortie de `docker image ls`

# ✓ Utilisons les tags

```
docker image tag docker-git:latest docker-git:1.0.0

# 2 lignes
docker image ls | grep docker-git
# 1 ligne
docker image ls | grep docker-git | grep latest
# 1 ligne
docker image ls | grep docker-git | grep '1.0.0'

# Doit fonctionner
docker container run --rm docker-git:1.0.0 git --version
```

# 🎓 Mettre à jour votre image (1.1.0)

- Mettez à jour votre image en version `1.1.0` avec les changements suivants :
  - Ajoutez un `LABEL` dont la clef est `description` (et la valeur de votre choix)
  - Configurez `git` pour utiliser une branche `main` par défaut au lieu de `master` (commande `git config --global init.defaultBranch main`)

- Indices :
  - 💡 Commande `docker image inspect <image name>`
  - Commande `git config --get init.defaultBranch` (dans le conteneur)
  - Ajoutez des lignes **à la fin** du `Dockerfile`
  - 💡 Documentation de référence des `Dockerfile`

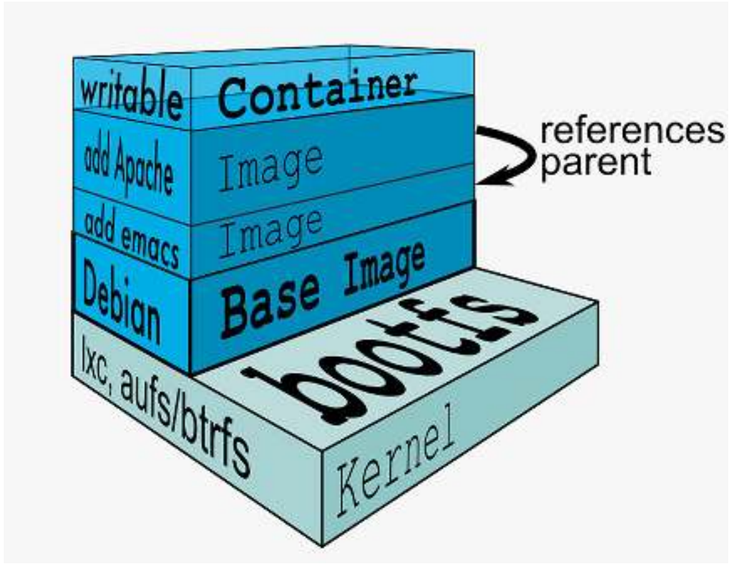
# ✅ Mettre à jour votre image (1.1.0)

```
cat ./Dockerfile
FROM alpine:3.24.1
RUN apk add --no-cache git
LABEL description="Une image contenant git préconfiguré"
RUN git config --global init.defaultBranch main

docker image build -t docker-git:1.1.0 ./docker-git/
# Sending build context to Docker daemon  2.048kB
# Step 1/4 : FROM alpine:3.24.1
#  ---> e40cf56b4be3
# Step 2/4 : RUN apk add --no-cache git
#  ---> Using cache
#  ---> 926b8d87f128
# Step 3/4 : LABEL description="Une image contenant git préconfiguré"
#  ---> Running in 0695fc62ecc8
# Removing intermediate container 0695fc62ecc8
#  ---> 68c7d4fb8c88
# Step 4/4 : RUN git config --global init.defaultBranch main
#  ---> Running in 7fb54ecf4070
# Removing intermediate container 7fb54ecf4070
#  ---> 2858ff394edb
Successfully built 2858ff394edb
Successfully tagged docker-git:1.1.0

docker container run --rm docker-git:1.0.0 git config --get init.defaultBranch
docker container run --rm docker-git:1.1.0 git config --get init.defaultBranch
# main
```

# Cache d'images & Layers

```
Step 2/4 : RUN apk add --no-cache git
  ---> Using cache
```

🤔 En fait, Docker n'a PAS exécuté cette commande la seconde fois ⇒ ça va beaucoup plus vite !




🎓 Essayez de voir les layers avec `dive`. Il n'est pas installé dans Codespaces, mais s'utilise en conteneur sans rien installer :

```
docker container run --rm --tty --interactive \
  --volume /var/run/docker.sock:/var/run/docker.sock \
  wagoodman/dive:latest <image>:<tag>
```

# 🎓 Cache d’images & Layers

- **But :** manipuler le cache d’images
- Commencez par vérifier que le cache est utilisé : relancez la dernière commande `docker image build` (plusieurs fois s’il le faut)
- Invalidez le cache en ajoutant le paquet APK `make` à installer en même temps que `git`
  - ⚠️ Tag `1.2.0`
- Vérifiez que le cache est bien présent de nouveau

# ✓ Cache d’images & Layers

```
# Build one time
docker image build -t docker-git:1.1.0 ./docker-git/
# Second time is fully cached
docker image build -t docker-git:1.1.0 ./docker-git/

cat Dockerfile
# FROM alpine:3.24.1
# RUN apk add --no-cache git make
# LABEL description="Une image contenant git préconfiguré"
# RUN git config --global init.defaultBranch main

# Build one time
docker image build -t docker-git:1.2.0 ./docker-git/
# Second time is fully cached
docker image build -t docker-git:1.2.0 ./docker-git/

## Vérification
# Renvoie une erreur
docker container run --rm docker-git:1.1.0 make --version
# Doit fonctionner
docker container run --rm docker-git:1.2.0 make --version
```

# Comportement par défaut des containers

Deux instructions permettent de définir la commande à lancer au démarrage du container.

# Comportement par défaut des containers

Deux instructions permettent de définir la commande à lancer au démarrage du container.

**CMD**

**ENTRYPOINT**

# CMD vs ENTRYPOINT

Cas d’usage : énoncé du besoin.

# CMD vs ENTRYPOINT

Cas d’usage. On va s’outiller!

# CMD vs ENTRYPOINT

Cas d’usage.

# CMD vs ENTRYPOINT

Cas d'usage.

```
$ docker container run -it my-curl:1.0 sh
/ $
```

# CMD vs ENTRYPOINT

Cas d’usage: un cran plus loin.

L’image actuelle, c’est bien mais pas hyper flexible !

# CMD vs ENTRYPOINT

Cas d'usage, un cran plus loin

Paramétrable, et hop!

# CMD vs ENTRYPOINT

Cas d'usage, un cran plus loin

Paramétrable, et hop!

```
FROM alpine:3.24
LABEL maintainer="John Doe"
RUN apk add --no-cache curl
# Si vous utilisez le proxy de l'Université
ENTRYPOINT ["curl", "-L", "--connect-timeout", "60", \
  "-x", "http://cache-etu.univ-artois.fr:3128"]
# Si vous utilisez votre propre proxy docker
# (sous Linux, lancer avec --add-host=host.docker.internal
# ENTRYPOINT ["curl", "-L", "--connect-timeout", "60", \
#   "-x", "http://host.docker.internal:3128"]
```

```
docker image build -t my-curl:2.0 .
```

```
docker container run --rm \
  my-curl:2.0 http://google.com
  % Total    % Received [...]
100  84301    0  84301    [...]
<!doctype html><html [...]
[...]
[...]</script></body></html>
```

# ENTRYPOINT + CMD

Une URL par défaut, toujours remplaçable.

```dockerfile
FROM alpine:3.24
LABEL maintainer="John Doe"
RUN apk add --no-cache curl
# Si vous utilisez le proxy de l'Université
ENTRYPOINT ["curl", "-L", "--connect-timeout", "60", \
  "-x", "http://cache-etu.univ-artois.fr:3128"]
# Si vous utilisez votre propre proxy docker
# (sous Linux, lancer avec
#  --add-host=host.docker.internal:host-gateway)
# ENTRYPOINT ["curl", "-L", "--connect-timeout", "60", \
#   "-x", "http://host.docker.internal:3128"]
CMD ["http://example.com"]
```

# Forme exec ou forme shell ?

```
# Forme exec (JSON)
ENTRYPOINT ["curl", "-L", ...]

# Forme shell
ENTRYPOINT curl -L ...
```

# Checkpoint 🎯

- Une image Docker fournit un environnement de système de fichier auto-suffisant (application, dépendances, binaries, etc.) comme modèle de base d'un conteneur
- Les images Docker ont une convention de nommage permettant d'identifier les images très précisément
- On peut spécifier une recette de fabrication d'image à l'aide d'un `Dockerfile` et de la commande `docker image build`

⇒ 🤔 et si on utilisait Docker pour nous aider dans l'intégration continue ?

# 📁 Nos fichiers dans les images

# 📂 Le répertoire de travail

# WORKDIR

```
FROM alpine:3.24
WORKDIR /repertoire/travail
RUN echo hello > ./world.txt
```

# 📂 WORKDIR

```
FROM alpine:3.24
WORKDIR /repertoire/travail
RUN echo hello > ./world.txt
```

```
$ docker image build --tag mon-img .
...
$ docker container run mon-img cat
/repertoire/travail/world.txt
hello
$ docker container run mon-img pwd
/repertoire/travail
$ docker container run --workdir /home mon-img pwd
/home
```

# 📂 WORKDIR

```
FROM alpine:3.24
WORKDIR /repertoire/travail
RUN echo hello > ./world.txt
```

```
$ docker image build --tag mon-img .
...
$ docker container run mon-img cat
/repertoire/travail/world.txt
hello
$ docker container run mon-img pwd
/repertoire/travail
$ docker container run --workdir /home mon-img pwd
/home
```

override du WORKDIR

# 📂 WORKDIR

```
FROM alpine:3.24
WORKDIR /repertoire/travail
RUN echo hello > ./world.txt
```

```
docker image build --tag mon-img .
...
```

```
$ docker container run mon-img cat /repertoire/travail/world.txt
hello
```

```
$ docker container run mon-img pwd
/repertoire/travail
```

```
$ docker container run --workdir /home mon-img pwd
/home
```

# 📁 Embarquer des fichiers

Cas d’usage.

# 📁 Embarquer des fichiers

Copie de fichiers.

# 📁 Embarquer des fichiers

Le mot clé `ADD`.

```
FROM image
ADD https://mon-nexus/foo/bar/1.0/package.tar /uncompressed/
```

# 📁 Embarquer des fichiers

## Le mot clé `ADD`

source : https://docs.docker.com/build/building/best-practices/#add-or-copy

Because image size matters, using `ADD` to fetch packages from remote URLs is strongly discouraged; you should use `curl` or `wget` instead. That way you can delete the files you no longer need after they've been extracted and you don't have to add another layer in your image. For example, you should avoid doing things like:

```
ADD https://example.com/big.tar.xz /usr/src/things/
RUN tar -xJf /usr/src/things/big.tar.xz -C /usr/src/things
RUN make -C /usr/src/things all
```

And instead, do something like:

```
RUN mkdir -p /usr/src/things \
    && curl -SL https://example.com/big.tar.xz \
    | tar -xJC /usr/src/things \
    && make -C /usr/src/things all
```

For other items (files, directories) that do not require `ADD` 's tar auto-extraction capability, you should always use `COPY` .

# 📁 Embarquer des fichiers

## Le mot clé `ADD`.

source : https://docs.docker.com/build/building/best-practices/#add-or-copy

Because image size matters, using `ADD` to fetch packages from remote URLs is strongly discouraged; you should use `curl` or `wget` instead. That way you can delete the files you no longer need after they've been extracted and you don't have to add another layer in your image. For example, you should avoid doing things like:

```
ADD https://example.com/big.tar.xz /usr/src/things/
RUN tar -xJf /usr/src/things/big.tar.xz -C /usr/src/things
RUN make -C /usr/src/things all
```

And instead, do something like:

```
RUN mkdir -p /usr/src/things \
    && curl -SL https://example.com/big.tar.xz \
    | tar -xJC /usr/src/things \
    && make -C /usr/src/things all
```

# 📁 Embarquer des fichiers

## Oui, mais pas la terre entière !

```
docker image build --tag myjava:1.42 ./
Sending build context to Docker daemon  172.8MB
```

# 📁 Embarquer des fichiers

Oui, mais pas la terre entière !

```
$ docker image build --tag myjava:1.42 ./
Sending build context to Docker daemon  172.8MB
```

Répertoire contenant le Dockerfile.
NB : tous les fichiers présents dans ce répertoire seront envoyés au Docker daemon pour construire vos images !

# 📁 Embarquer des fichiers

Oui, mais pas la terre entière !

```
$ docker image build --tag myjava:1.42 ./
Sending build context to Docker daemon  172.8MB
```

Répertoire contenant le Dockerfile.
NB : tous les fichiers présents dans ce répertoire seront envoyés au Docker daemon pour construire vos images !

?!!!

# 📁 Embarquer des fichiers

Oui, mais pas la terre entière !

```
$ docker image build --tag myjava:1.42 ./
Sending build context to Docker daemon  172.8MB
```

Répertoire contenant le Dockerfile.
NB : tous les fichiers présents dans ce répertoire seront envoyés au Docker daemon pour construire vos images !

?!!!

Si un gros fichier traîne dans le dossier alors qu'il n'est même pas utilisé ni référencé dans le Dockerfile, il sera tout de même envoyé au Docker daemon.

# 📁 Embarquer des fichiers

Oui, mais pas la terre entière !

# 🙈 .dockerignore

```
# ignore les dossiers .git et .cache
.git
.cache
```

| Commande | Usage |
|---|---|
| **FROM** | Pour spécifier l'image à partir de laquelle on construit la nouvelle image |
| **LABEL** | Pour décrire l'image à partir de clé/valeur |
| **RUN** | Pour exécuter une action au moment du build |
| **ENV** | Pour insérer une variable d'environnement dans tous les futurs containers de cette image |
| **ENTRYPOINT** | Pour définir une commande à lancer au démarrage du container |
| **CMD** | Pour définir une commande à lancer au démarrage du container ou pour compléter un ENTRYPOINT existant |
| **COPY** | Pour insérer des fichiers dans l'image |

# 📝 Renommage des images

```
$ docker tag 0e5574283393 fedora/httpd:version-1.0
```

# 📝 Renommage des images

```
$ docker tag 0e5574283393 fedora/httpd:version-1.0
```

ID existant

# 📝 Renommage des images

```
$ docker tag 0e5574283393 fedora/httpd:version-1.0
```

ID existant

tag à ajouter

# 📝 Renommage des images

```
$ docker tag 0e5574283393 fedora/httpd:version-1.0
```

```
$ docker tag httpd fedora/httpd:version-1.0
```

# 🖼️ LES IMAGES

Le bilan

Vous savez désormais:

- Rédiger un Dockerfile
- Nommer vos images
- Créer vos outils

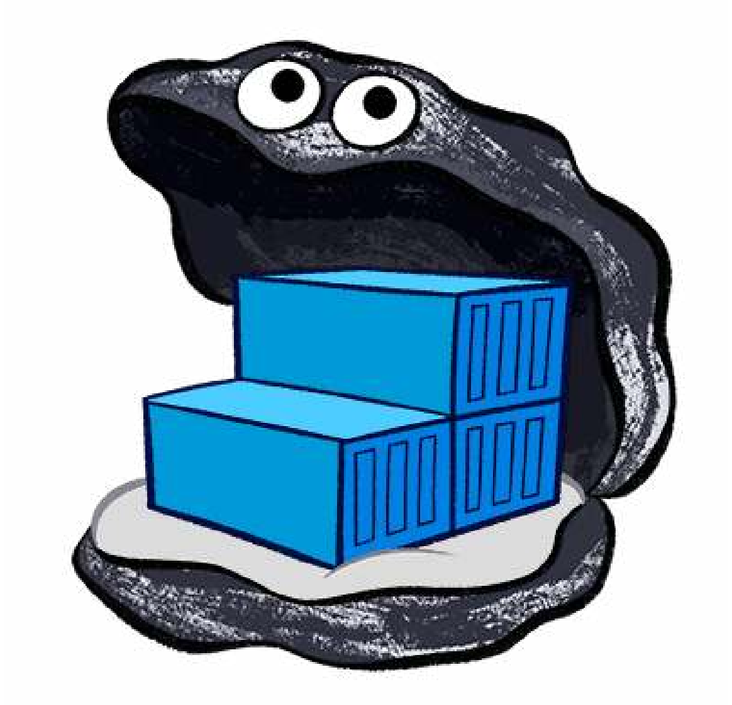
# 🖼️ LES IMAGES

Le bilan

Vous savez désormais:

- Rédiger un Dockerfile
- Nommer vos images
- Créer vos outils

# 🗂️ Les registries



# Golden Rule

**La construction d’une image doit être automatisée.**

# 🤖 = 🔒 ? Automatique == sécurisé

Le fait de déléguer la construction des images permet d'ajouter toute une chaîne de traitement, de contrôles des images pour s'assurer qu'elle respecte les règles RSSI.

# Exemple de mise en place

**Sources**

**TEST**

**PROD**

# Exemple de mise en place

Sources

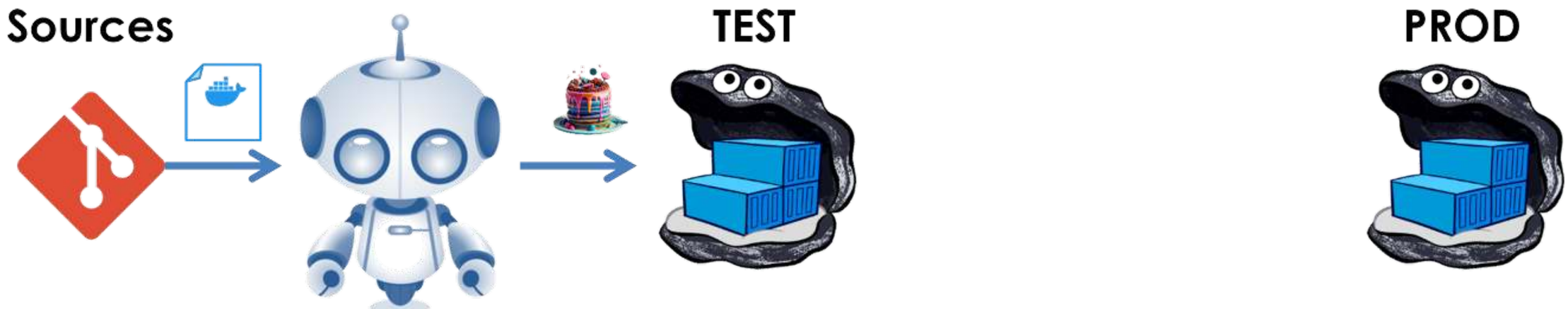
TEST

PROD

**1**

Un automate
récupère les sources
de l'image, la construit
et la pousse sur une
registry de test

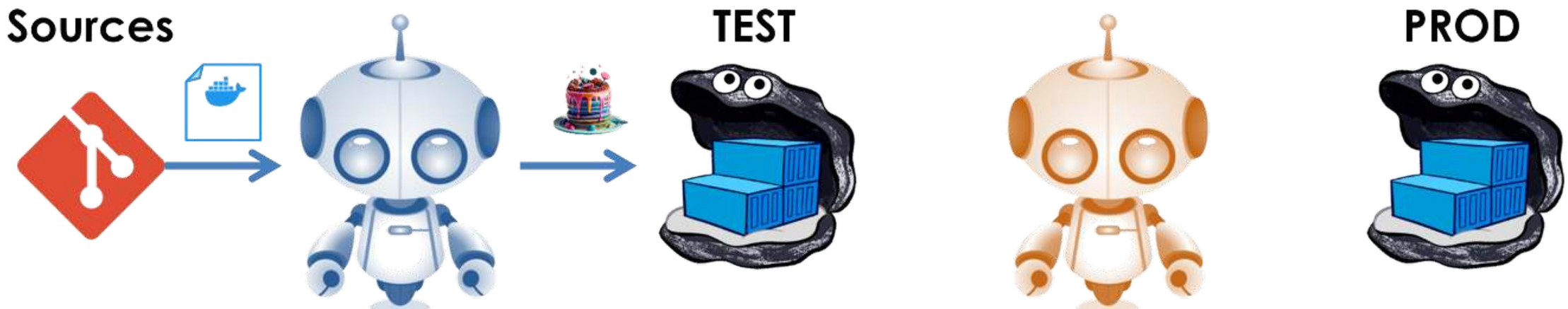
# Exemple de mise en place




1

Un automate
récupère les sources
de l'image, la construit
et la pousse sur une
registry de test

# Exemple de mise en place



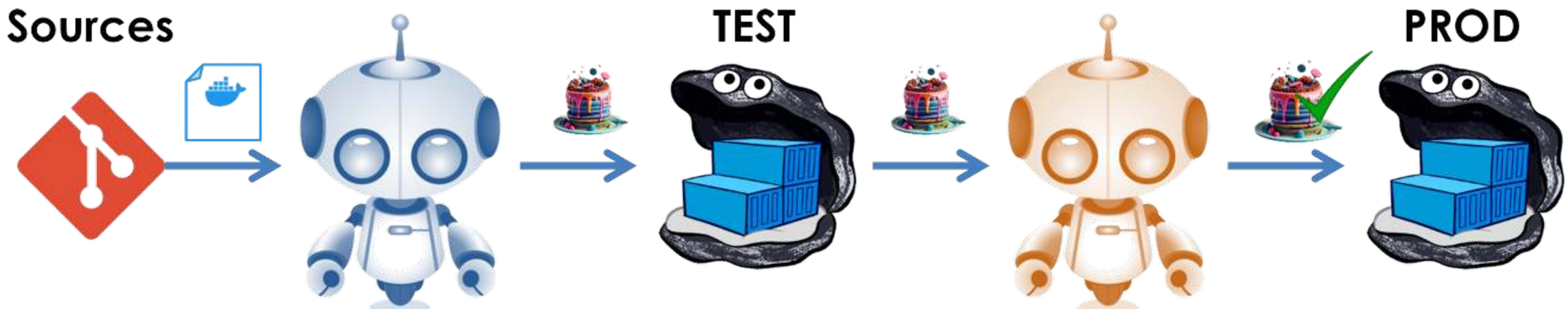

**1**

Un automate récupère les sources de l'image, la construit et la pousse sur une registry de test

**2**

Un automate récupère l'image sur la registry de test, effectue les contrôles et si elle est valide, la pousse sur la registry de prod

# Exemple de mise en place



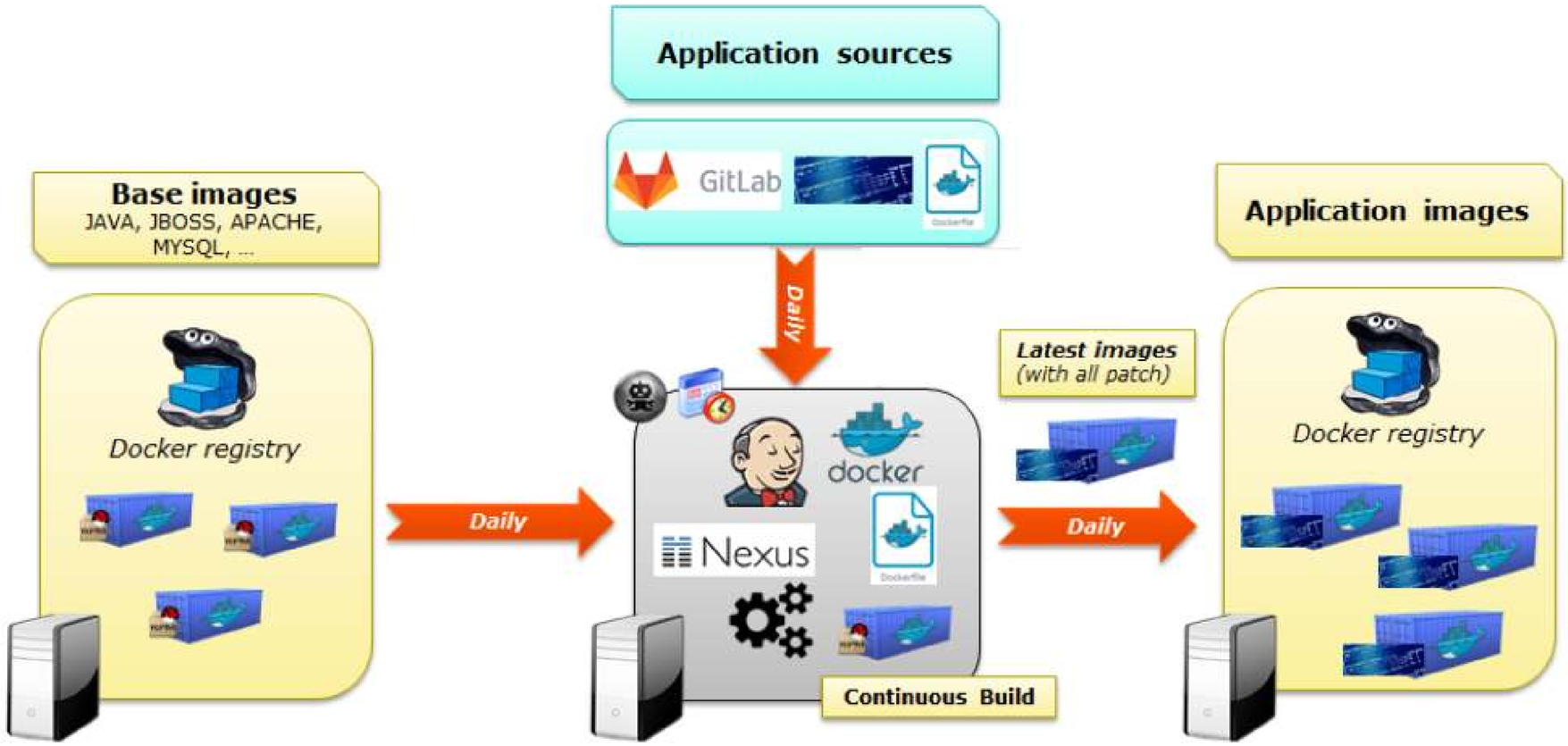

**1**

Un automate récupère les sources de l'image, la construit et la pousse sur une registry de test

**2**

Un automate récupère l'image sur la registry de test, effectue les contrôles et si elle est valide, la pousse sur la registry de prod

# ➡️ Exemple : continuous build



# 🎓 Travaux pratiques #5

https://gitlab.univ-artois.fr/bruno.verachten/devops-docker-tp05

# ✓ Solution Travaux pratiques #5

```
diff --git a/src/Dockerfile b/src/Dockerfile
index c92c67c..77919a5 100644
--- a/src/Dockerfile
+++ b/src/Dockerfile
@@ -11,2 +11,6 @@ EXPOSE 4321

+RUN adduser --disabled-password --gecos "" --home /home/www www
+USER www
+WORKDIR /home/www
+
 ENTRYPOINT ["/bin/start.sh"]
```

# ✓ Solution Travaux pratiques #5

```
diff --git a/src/start.sh b/src/start.sh
index 44006d6..a6ed60f 100755
--- a/src/start.sh
+++ b/src/start.sh
@@ -7 +7 @@ set -eux
-touch /home/www/started.time
+touch /tmp/started.time
@@ -15 +15 @@ fi
-date > /home/www/started.time
+date > /tmp/started.time
```

# ✅ Solution Travaux pratiques #5

```yaml
stages: ["validator.sh", "docker scout"]

variables:
  PROXY: "http://cache-etu.univ-artois.fr:3128"
  IMAGE: "devops-docker-tp05:latest"

validator-job:
  image: cache-ili.univ-artois.fr/proxy_cache/library/docker
  stage: "validator.sh"
  before_script:
    - export HTTP_PROXY="$PROXY"
    - export HTTPS_PROXY="$PROXY"
    - apk add -U bash
  script:
    - errors=$(./validator.sh)
    - echo "$errors"
    - exit $([ -z "$errors" ] && echo 0 || echo 1)
  tags: [docker2]

scout-job:
  stage: "docker scout"
  script:
    - cd ./src
    - docker build . -t "$IMAGE"
    - docker scout cves --format only-packages --only-vuln-packages "$IMAGE"
    - docker rmi "$IMAGE"
  tags: [myshell]
```

# ✅ Solution Travaux pratiques #5

```yaml
stages:
  - run_script
  - docker_scan

run_script:
  stage: run_script
  script:
    - chmod +x validator.sh src/start.sh && ./validator.sh

docker_scan:
  stage: docker_scan
  script:
    - IMG=$(echo img$$)
    - docker image build --tag $IMG ./src > /dev/null
    - echo "Will scan $IMG"
    - echo $DOCKER_TOKEN | docker login -u $DOCKER_USERNAME --password-stdin
    - docker scout cves --format only-packages --only-vuln-packages $IMG # Runs docker scout to scan the image
    - docker scout recommendations $IMG # View base image update recommendations
```

# 🔍 Inspection et nommage des containers

# Nommage des containers

```
$ docker container logs 47d6
Tue Oct 2400:39:52 UTC 2023
Tue Oct 2400:39:53 UTC 2023
...
```

# Nommage des containers

```
$ docker container ls
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS
NAMES
793906a4c2d2        alpine              "/bin/sh"           3 hours ago         Up 3 hours
festive_poitras
```

# Nommage des containers

```
$ docker container ls
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS
NAMES
793906a4c2d2        alpine              "/bin/sh"           3 hours ago         Up 3 hours
festive_poitras
```

Une première alternative !

# Nommage des containers

```
$ docker container ls
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS
NAMES
793906a4c2d2        alpine              "/bin/sh"           3 hours ago         Up 3 hours
festive_poitras
```

Une première alternative !

Par défaut, Docker génère un nom en combinant une humeur et un inventeur.

# Nommage des containers

## Humour de g33k

```go
// GetRandomName generates a random name from the list of adjectives and surnames in this package
// formatted as "adjective_surname". For example 'focused_turing'. If retry is non-zero, a random
// integer between 0 and 10 will be added to the end of the name, e.g `focused_turing3`
func GetRandomName(retry int) string {
begin:
        name := fmt.Sprintf("%s_%s", left[rand.Intn(len(left))], right[rand.Intn(len(right))])
        if name == "boring_wozniak" /* Steve Wozniak is not boring */ {
                goto begin
        }

        if retry > 0 {
                name = fmt.Sprintf("%s%d", name, rand.Intn(10))
        }
        return name
}
```

source : https://github.com/docker/engine/blob/master/pkg/namesgenerator/names-generator.go

# Nommage des containers

```
docker container run --detach --name my-web httpd
```

# 🦸 Renommage des containers

Attention, spoiler pour les n00bs de DC Comics

# 🦸 Renommage des containers

```
$ docker container rename clark_kent superman
```

attention, spoiler pour les n00bs de DC Comics

ancien nom

# 🦸 Renommage des containers

```
$ docker container rename clark_kent superman
```

attention, spoiler pour les n00bs de DC Comics

ancien nom

nouveau nom

# 🔍 Inspection des containers

```
docker container inspect my-web
[
    {
        "Id": "0ac8cdf8d447c6d316a04bd1a7f74cd2677eea3478f11f0be5241c1bb2d4c7da",
        "Created": "2023-10-24T19:40:11.847889117Z",
        "Path": "httpd-foreground",
        "Args": [],
        "State": {
            "Status": "running",
            "Running": true,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 3275,
            "ExitCode": 0,
            "Error": "",
            "StartedAt": "2023-10-24T19:40:12.363841649Z",
            "FinishedAt": "0001-01-01T00:00:00Z"
        },
...
]
```

# Comment exploiter le JSON ?

JQ est le meilleur outil pour parser du JSON dans le shell

# 🎓 Travaux pratiques #6

Étapes:

# ✅ Solution Travaux Pratiques #6

On démarre un container nginx avec la commande

```
docker container run --detach --name my-web nginx
```

# ✅ Solution Travaux Pratiques #6

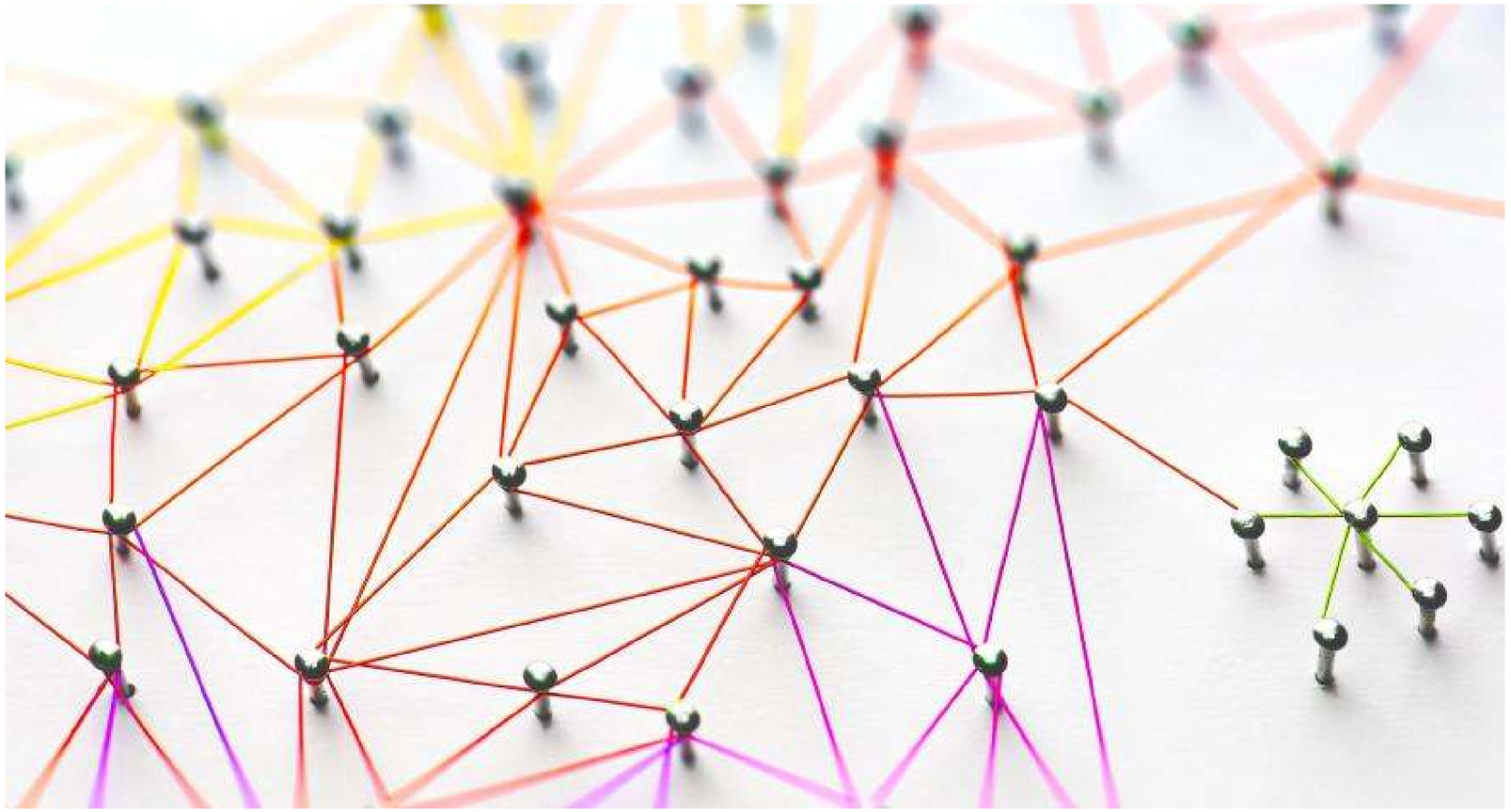
```
[
  "nginx",
  "-g",
  "daemon off;"
]
```

Qu'on pourrait nettoyer pour avoir `nginx -g daemon off;` avec:

# ✓ Solution Travaux Pratiques #6

```
# awk : découpe la colonne COMMAND, entre les en-têtes
# COMMAND et CREATED
docker container ls --filter "name=my-web" --no-trunc \
  | awk 'NR==1 {cmd=index($0, "COMMAND"); created=index($0, "CREATED")}
         NR>1  {print substr($0, cmd, created-cmd)}'
```

# Trouver son container en réseau



# Le Container en réseau

```
docker container run --detach nginx
bd12c4d7110d17ce80...
```

```
docker container ls
CONTAINER ID    IMAGE   COMMAND                  CREATED    STATUS    PORTS
bd12c4d71       nginx   "/docker-entrypoint.…"   35 s. ago  Up 33 s.  80/tcp
```

# 🔍 Que nous dit docker container inspect ?


```
"Networks": {
                "bridge": {
                    "IPAMConfig": null,
                    "Links": null,
                    "Aliases": null,
                    "NetworkID": "c0fcec43e3e8…eecd6558ac0870a468a3",
                    "EndpointID": "0ec8fb237a10e9227359b4…db23edc32",
                    "Gateway": "172.17.0.1",
                    "IPAddress": "172.17.0.2",
                    "IPPrefixLen": 16,
                    "IPv6Gateway": "",
                    "GlobalIPv6Address": "",
                    "GlobalIPv6PrefixLen": 0,
                    "MacAddress": "02:42:ac:11:00:02",
                    "DriverOpts": null
                }
            }
```

# 🔍 Que nous dit docker container inspect ?


```
"Networks": {
                "bridge": {
                    "IPAMConfig": null,
                    "Links": null,
                    "Aliases": null,
                    "NetworkID": "c0fcec43e3e8…eecd6558ac0870a468a3",
                    "EndpointID": "0ec8fb237a10e9227359b4…db23edc32",
                    "Gateway": "172.17.0.1",
                    "IPAddress": "172.17.0.2",
                    "IPPrefixLen": 16,
                    "IPv6Gateway": "",
                    "GlobalIPv6Address": "",
                    "GlobalIPv6PrefixLen": 0,
                    "MacAddress": "02:42:ac:11:00:02",
                    "DriverOpts": null
            }
        }
```

# 🔍 Que nous dit docker container inspect ?

```
"Networks": {
                "bridge": {
                    "IPAMConfig": null,
                    "Links": null,
                    "Aliases": null,
                    "NetworkID": "c0fcec43e3e8…eecd6558ac0870a468a3",
                    "EndpointID": "0ec8fb237a10e9227359b4…db23edc32",
                    "Gateway": "172.17.0.1",
                    "IPAddress": "172.17.0.2",
                    "IPPrefixLen": 16,
                    "IPv6Gateway": "",
                    "GlobalIPv6Address": "",
                    "GlobalIPv6PrefixLen": 0,
                    "MacAddress": "02:42:ac:11:00:02",
                    "DriverOpts": null
                }
            }
```

# 🔍 Que nous dit docker container inspect ?

L’ancienne forme, encore dans beaucoup de tutoriels, échoue avec Docker 29 :

```
docker container inspect --format "{{.NetworkSettings.IPAddress}}" <ContainerID>
template parsing error: [...] map has no entry for key "IPAddress"
```

L’adresse n’est plus que sous `.NetworkSettings.Networks.<réseau>`, d’où le `range`.

# Et voilà !

```
curl -I --noproxy '*' http://172.17.0.2:80
HTTP/1.1 200 OK
....
Server: nginx/1.25 ...
```

Il est maintenant facile d’interagir avec notre serveur web !

# Mapping de Port

```
$ docker container run --detach   -p 8000:80
nginx
```

le port de la machine hôte

# Mapping de Port

```
$ docker container run --detach   -p 8000:80 nginx
```

le port de la machine hôte

le port du container

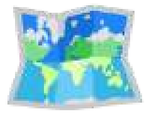
# Mapping de Port

```
$ docker container run --detach   -p 8000:80 nginx
```

le port de la machine hôte

le port du container

```
$ curl -I --noproxy '*' http://172.17.0.2:80
it works !
$ curl -I --noproxy '*' http://localhost:8000
it works !
```



# Mapping de Port

```
curl -I --noproxy '*' http://172.17.0.2:80
HTTP/1.1 200 OK
Server: nginx/1.25.2
Date: Wed, 18 Oct 2023 11:54:49 GMT
Content-Type: text/html
Content-Length: 284237
Last-Modified: Tue, 10 Oct 2023 12:12:13 GMT
Connection: keep-alive
ETag: "65253f9d-4564d"
Accept-Ranges: bytes
```

# 🎓 Travaux pratiques #7

Étapes:

# ✓ Solution travaux pratiques #7

```
<html><head></head><body><header>
<title>http://info.cern.ch</title>
</header>

<h1>http://info.cern.ch - home of the first website</h1>
<p>From here you can:</p>
<ul>
<li><a href="http://info.cern.ch/hypertext/WWW/TheProject.html">Browse the first website</a></li>
<li><a href="http://line-mode.cern.ch/www/hypertext/WWW/TheProject.html">Browse the first website using the line-mode brow
<li><a href="http://home.web.cern.ch/topics/birth-web">Learn about the birth of the web</a></li>
<li><a href="http://home.web.cern.ch/about">Learn about CERN, the physics laboratory where the web was born</a></li>
</ul>
</body></html>
```

# ✓ Solution travaux pratiques #7

```
wget -O sample.war \
  https://github.com/gounthar/tp07-sample/releases/download/v2026.10.19-091012/sample-2026.10.19-091012.war
# wget sort en 0 même sur une page HTML : vérifier l'archive
file sample.war
sample.war: Zip archive data, made by v2.0 UNIX, ...
```

# ✓ Solution travaux pratiques #7

```
git clone https://github.com/spring-guides/gs-spring-boot.git
```

C’est un début…

# ✅ Solution travaux pratiques #7

Derrière le proxy de l'université, `mvn package` échoue : Maven ignore `HTTP_PROXY` et `HTTPS_PROXY`. Il lit son proxy dans `~/.m2/settings.xml`.

```xml
<!-- Proxy de l'université pour Maven, qui ignore HTTP_PROXY et HTTPS_PROXY -->
<settings>
  <proxies>
    <proxy>
      <id>univ</id>
      <active>true</active>
      <protocol>http</protocol>
      <host>cache-etu.univ-artois.fr</host>
      <port>3128</port>
    </proxy>
  </proxies>
</settings>
```

```
mkdir -p ~/.m2 && cp settings.xml ~/.m2/settings.xml
```

Dans un `Dockerfile`, on le monte comme secret de build, le temps du seul `RUN mvn package` : il ne reste pas dans l'image. Sans `--secret` (hors de l'université), le montage est simplement absent.

# ✓ Solution travaux pratiques #7

Assemblons tout ça dans un `Dockerfile`.

```
# Use a Maven image for building the application
FROM maven:3.9-eclipse-temurin-25-alpine
WORKDIR /app

# Clone the Spring Boot application repository
RUN apk add git && git clone https://github.com/spring-guides/gs-spring-boot.git

# Change directory to the application's complete directory
WORKDIR /app/gs-spring-boot/complete

# Build the application using Maven; settings.xml (proxy) is mounted as a build secret, if provided
RUN --mount=type=secret,id=maven-settings,target=/root/.m2/settings.xml mvn package

WORKDIR /app/gs-spring-boot/complete/target

# Command to run the Spring Boot application
CMD ["java", "-jar", "spring-boot-complete-0.0.1-SNAPSHOT.jar"]
```

# ✅ Solution travaux pratiques #7

Et logiquement…

# ⏳ Attends un peu…

*Ça fonctionne, mais c'est comme si je laissais la grue dans la chambre une fois que j'avais fini de construire ma maison !*

# ✅ Solution travaux pratiques #7

On essaye un `Dockerfile` moins BTP?

# Une première solution

Avec un autre exemple…Une image pour construire l’appli:

# Une première solution

Avec un autre exemple

```
#!/bin/sh
echo Building alexellis2/href-counter:build

docker image build --build-arg https_proxy=$https_proxy --build-arg http_proxy=$http_proxy \
    -t alexellis2/href-counter:build . -f Dockerfile.build
docker container create --name extract alexellis2/href-counter:build
docker container cp extract:/go/src/github.com/alexellis/href-counter/app ./app
docker container rm -f extract

echo Building alexellis2/href-counter:latest
docker image build --no-cache -t alexellis2/href-counter:latest .
rm ./app
```

# Une première solution

Avec un autre exemple

build.sh

```
#!/bin/sh
echo Building alexellis2/href-counter:build

docker image build --build-arg https_proxy=$https_proxy --build-arg
http_proxy=$http_proxy \
    -t alexellis2/href-counter:build . -f Dockerfile.build


docker container create --name extract alexellis2/href-counter:build
docker container cp extract:/go/src/github.com/alexellis/href-counter/app ./app
docker container rm -f extract


echo Building alexellis2/href-counter:latest

docker image build --no-cache -t alexellis2/href-counter:latest .
rm ./app
```

on crée l'image de construction

# Une première solution

Avec un autre exemple

build.sh

```
#!/bin/sh
echo Building alexellis2/href-counter:build
```

on crée l'image de construction

```
docker image build --build-arg https_proxy=$https_proxy --build-arg
http_proxy=$http_proxy \
    -t alexellis2/href-counter:build . -f Dockerfile.build
```

on fait un container pour récupérer le build

```
docker container create --name extract alexellis2/href-counter:build
docker container cp extract:/go/src/github.com/alexellis/href-counter/app ./app
docker container rm -f extract
```

```
echo Building alexellis2/href-counter:latest

docker image build --no-cache -t alexellis2/href-counter:latest .
rm ./app
```

# Une première solution

Avec un autre exemple

build.sh

```
#!/bin/sh
echo Building alexellis2/href-counter:build
```

on crée l'image de construction

```
docker image build --build-arg https_proxy=$https_proxy --build-arg
http_proxy=$http_proxy \
    -t alexellis2/href-counter:build . -f Dockerfile.build
```

on fait un container pour récupérer le build

```
docker container create --name extract alexellis2/href-counter:build
docker container cp extract:/go/src/github.com/alexellis/href-counter/app ./app
docker container rm -f extract
```

```
echo Building alexellis2/href-counter:latest
```

on crée l'image d'exécution

```
docker image build --no-cache -t alexellis2/href-counter:latest .
rm ./app
```

# Une première solution

Avec un autre exemple

build.sh

```
#!/bin/sh
echo Building alexellis2/href-counter:build
```

on crée l'image de construction

```
docker image build --build-arg https_proxy=$https_proxy --build-arg
http_proxy=$http_proxy \
    -t alexellis2/href-counter:build . -f Dockerfile.build
```

on fait un container pour récupérer le build

```
docker container create --name extract alexellis2/href-counter:build
docker container cp extract:/go/src/github.com/alexellis/href-counter/app ./app
docker container rm -f extract
```

```
echo Building alexellis2/href-counter:latest
```

on crée l'image d'exécution

```
docker image build --no-cache -t alexellis2/href-counter:latest .
rm ./app
```

# Multistage build

```dockerfile
FROM golang:1.27

WORKDIR /go/src/github.com/alexellis/href-counter/

COPY go.mod .
COPY go.sum .
COPY app.go     .

RUN CGO_ENABLED=0 GOOS=linux go build  -ldflags "-s -w" -o app .

FROM alpine:3.24.2
RUN apk --no-cache add ca-certificates

WORKDIR /root/

COPY --from=0 /go/src/github.com/alexellis/href-counter/app    .

CMD ["./app"]
```

# Multistage build

```dockerfile
FROM golang:1.27

WORKDIR /go/src/github.com/alexellis/href-counter/

COPY go.mod .
COPY go.sum .
COPY app.go     .

RUN CGO_ENABLED=0 GOOS=linux go build  -ldflags "-s -w" -o app .

FROM alpine:3.24.2
RUN apk --no-cache add ca-certificates

WORKDIR /root/

COPY --from=0 /go/src/github.com/alexellis/href-counter/app    .

CMD ["./app"]
```

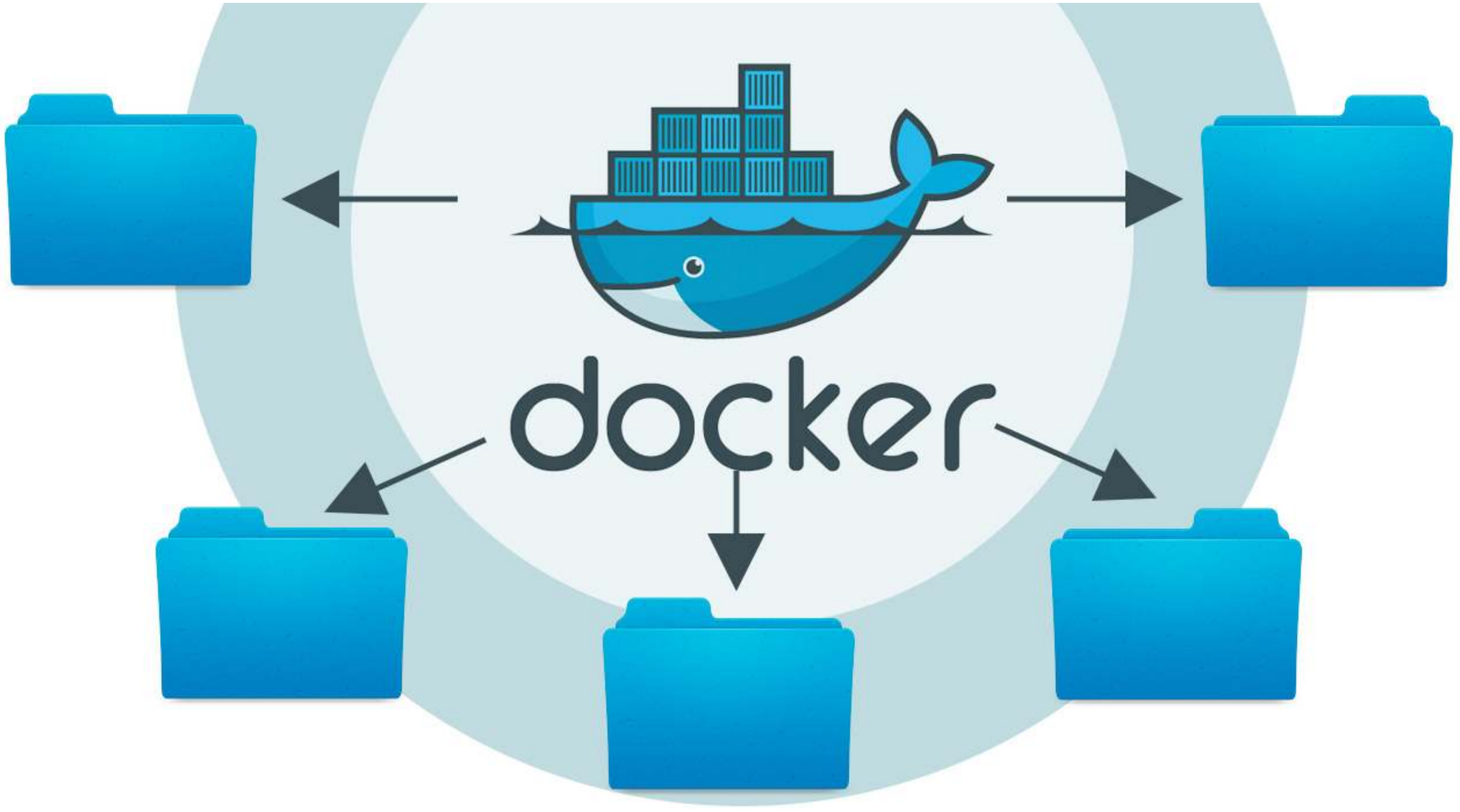
On copie les fichiers depuis le premier stage

# 📦 LES CONTAINERS

Le bilan : achievement unlocked

Vous savez désormais:

# Les volumes



# 3 types de gestion de données

Mapping de FS

Volumes

FS en RAM

# 3 types de gestion de données

Mapping de FS

Partage d'un répertoire de votre système hôte avec un conteneur Docker.

# 3 types de gestion de données

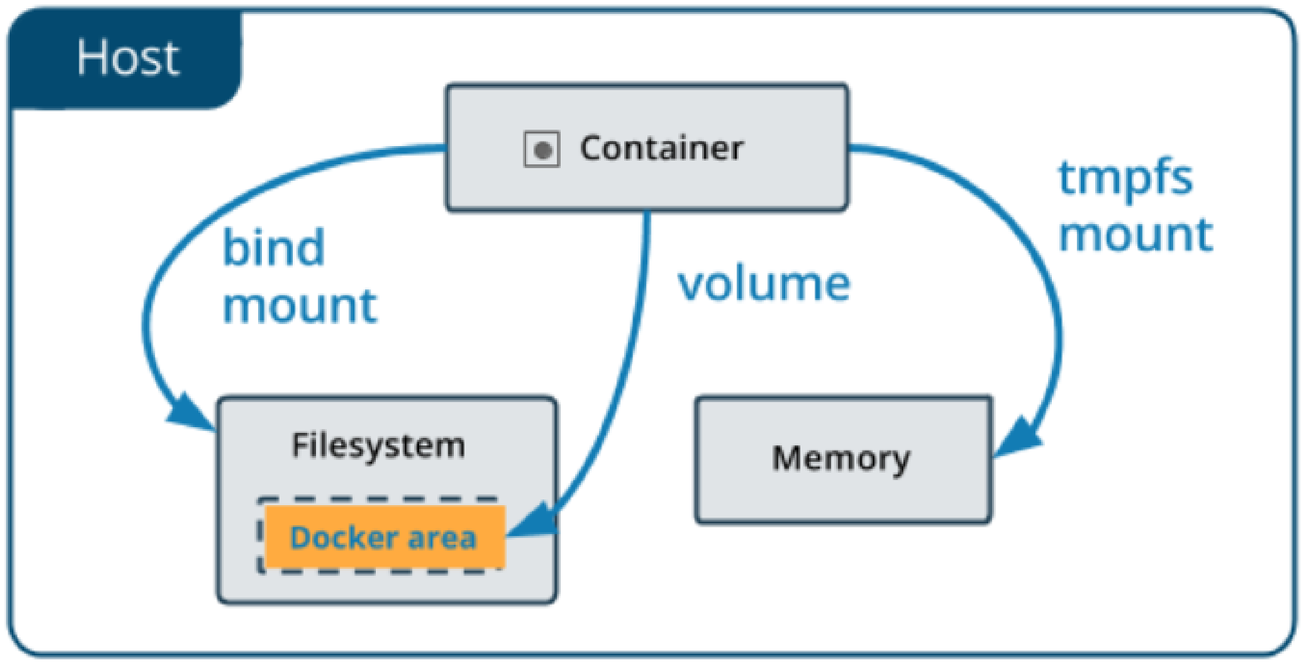
Volumes

Stockage persistant ⇒ données en dehors du système de fichiers du conteneur.

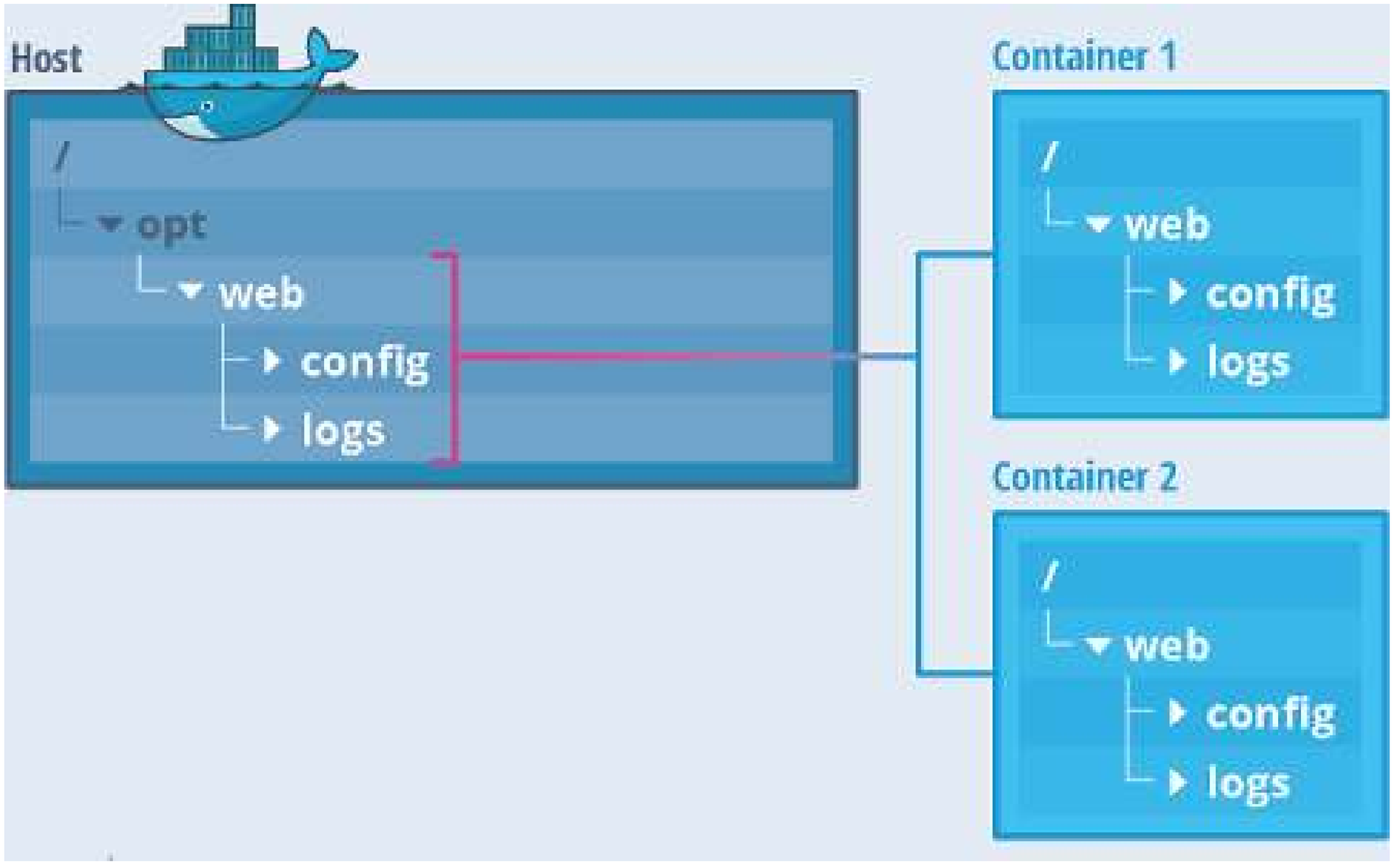
# 3 types de gestion de données

Système de fichiers temporaire stocké en RAM.

# 3 types de gestion de données



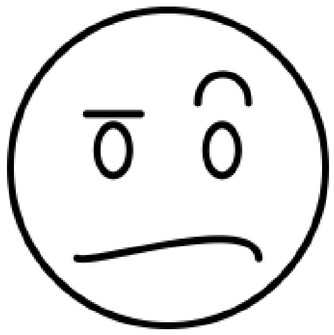
# Mapping de FileSystem

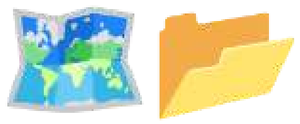


# Mapping de FileSystem

***"Créer une image avec les sources de mon application web juste pour distribuer des ressources statiques via Apache ! Merci Docker !"***

Un étudiant excédé



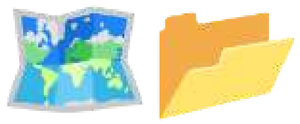


# Mapping de FileSystem

Héberger du contenu statique, d'après la page de l'image :

```
docker container run --name some-nginx \
  -v /some/content:/usr/share/nginx/html:ro \
  -d \
  nginx:1.31.8-alpine3.24
```

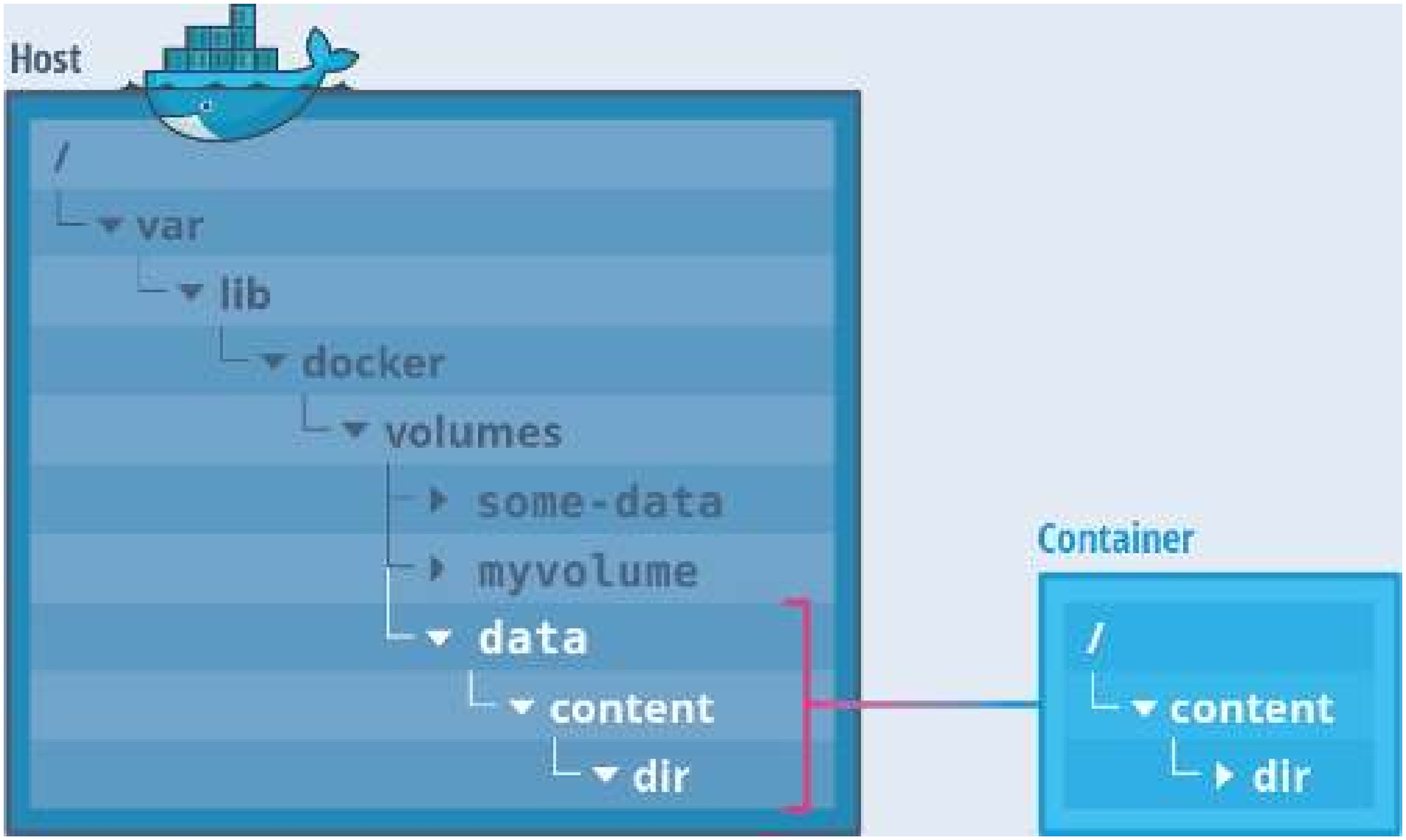
Source : https://hub.docker.com/_/nginx



# Mapping de FileSystem

```
-v /some/content:/usr/share/nginx/html:ro    # un dossier
-v ~/.bash_history:/.bash_history            # un fichier
```

# Les volumes


Host
/
var
lib
docker
volumes
some-data
myvolume
data
content
dir
Container
/
content
dir

# 📦📁 Les volumes

Il est possible de créer des "espaces mémoire" que l’on peut mettre à disposition des containers.

# 📦📁 Les volumes

Lister tous les volumes

```
docker volume ls
```

# 📦 📁 🕶️ Les volumes "anonymes"

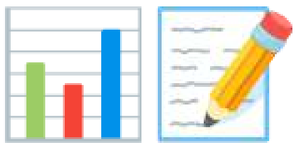
```dockerfile
FROM alpine:3.24
WORKDIR /repertoire/travail
VOLUME ["/data"]
RUN echo hello > ./world.txt
```

# 💾🧠 FileSystem en RAM

```
docker container run \
  -d \
  --read-only \
  --tmpfs /usr/local/apache2/logs \
  --tmpfs /tmp \
  httpd:2.4.69-alpine3.24
```

# 📊📝 Bilan de compétences

- Création d’images
- Création des containers
  - cycle de vie
  - nommage
  - débug
  - réseau
  - volumes

#  Bilan de compétences

- Création d'images ✅
- Création des containers ✅
  - cycle de vie ✅
  - nommage ✅
  - débug ✅
  - réseau ✅
  - volumes ✅

# 🎓 Travaux pratiques #8

Lancer un container `nginx` et lui faire distribuer une page web externe au container.

# ✅ Solution travaux pratiques #8

En utilisant un montage de répertoire (Bind Mount) :

# ✅ Solution travaux pratiques #8

En utilisant un volume nommé (Named Volume) :

# ✓ Solution travaux pratiques #8

En utilisant un volume nommé (Named Volume) :

Sans montage de l'hôte : `docker container cp` copie vers un container, même jamais démarré.

```
docker container create --name copie \
  -v mon-nginx-html:/cible busybox
# "/." : le contenu du répertoire, comme pour cp
docker container cp /chemin/vers/la/page/web/sur/lhote/. copie:/cible
docker container rm copie
```

# ✅ Solution travaux pratiques #8

En utilisant un volume nommé (Named Volume) :

Maintenant, lancez le conteneur Nginx et montez le volume nommé :

```
docker container run -d -p 8000:80 --name mon-nginx \
  -v mon-nginx-html:/usr/share/nginx/html \
  nginx:1.31.8-alpine3.24
```

# 🎓 Travaux pratiques #9

Étapes:

# ✅ Solution travaux pratiques #9

Créer un volume nommé "data" :

# ✅ Solution travaux pratiques #9

Dans le second conteneur (démarré à l'étape précédente), lisez les données du volume (le fichier texte) :

# 🎓 Travaux pratiques #10

Écrire un Dockerfile qui:

# ✅ Solution travaux pratiques #10

Tout d’abord, créez un répertoire pour votre projet et placez-vous dedans.

# ✅ Solution travaux pratiques #10

```
docker image build -t mon-image .
```

Assurez-vous que "mon-image" est le nom que vous souhaitez donner à votre image Docker. Après avoir créé l’image, vous pouvez exécuter un conteneur basé sur cette image :

# ✅ Solution travaux pratiques #10

```
cat *
```

```
Contenu du fichier 1
Contenu du fichier 2
Nouveau contenu du fichier 3
```

# ✓ Solution TP #10 : selon le constructeur

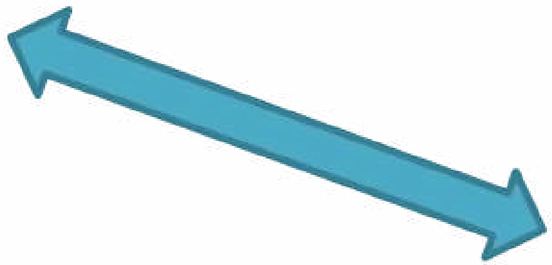
Le même Dockerfile, construit avec l'ancien constructeur (déprécié) :

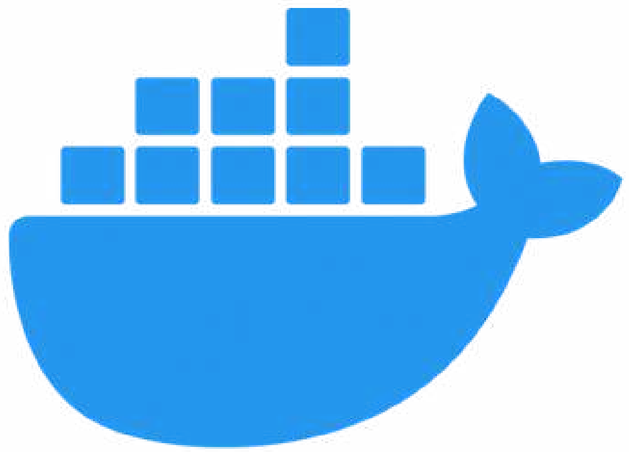
```
DOCKER_BUILDKIT=0 docker image build -t mon-image-legacy .
docker container run --rm mon-image-legacy ls /mon-dossier
```

```
fichier1.txt
fichier2.txt
```

# Réseaux et Docker

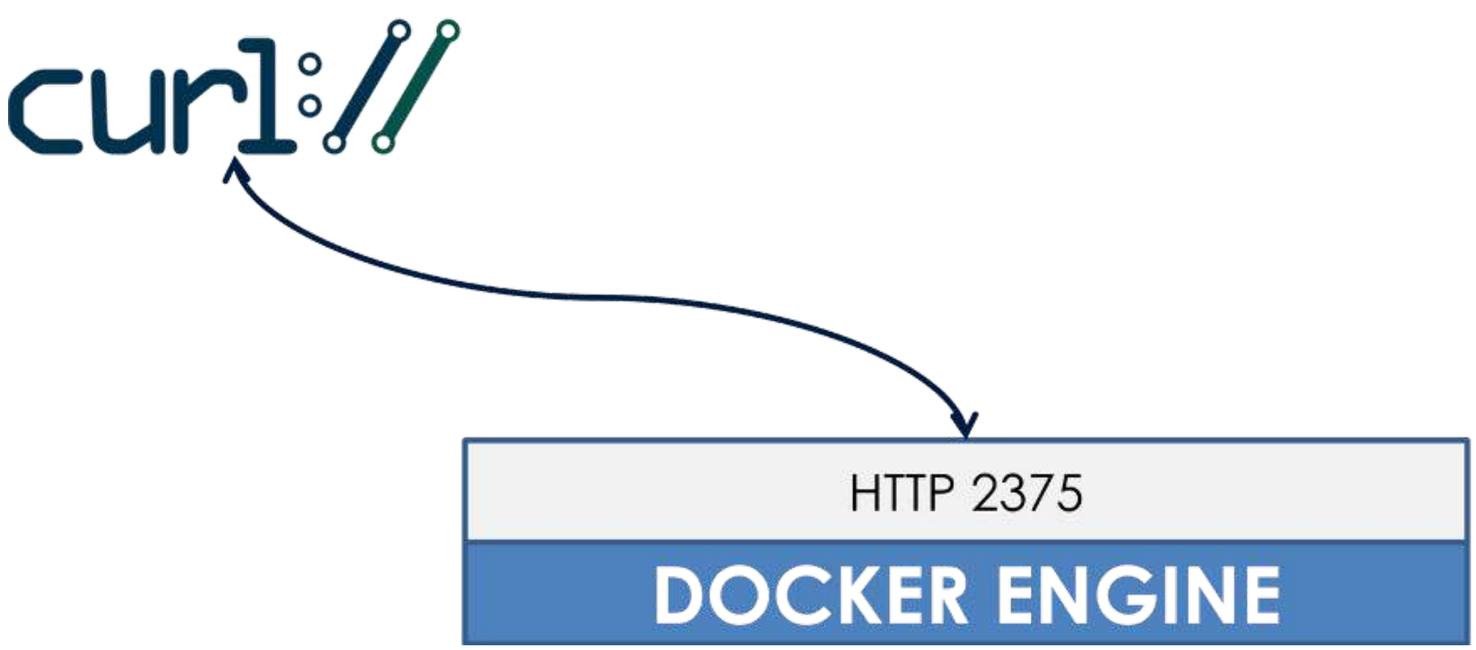
Interagir avec le Docker ENGINE



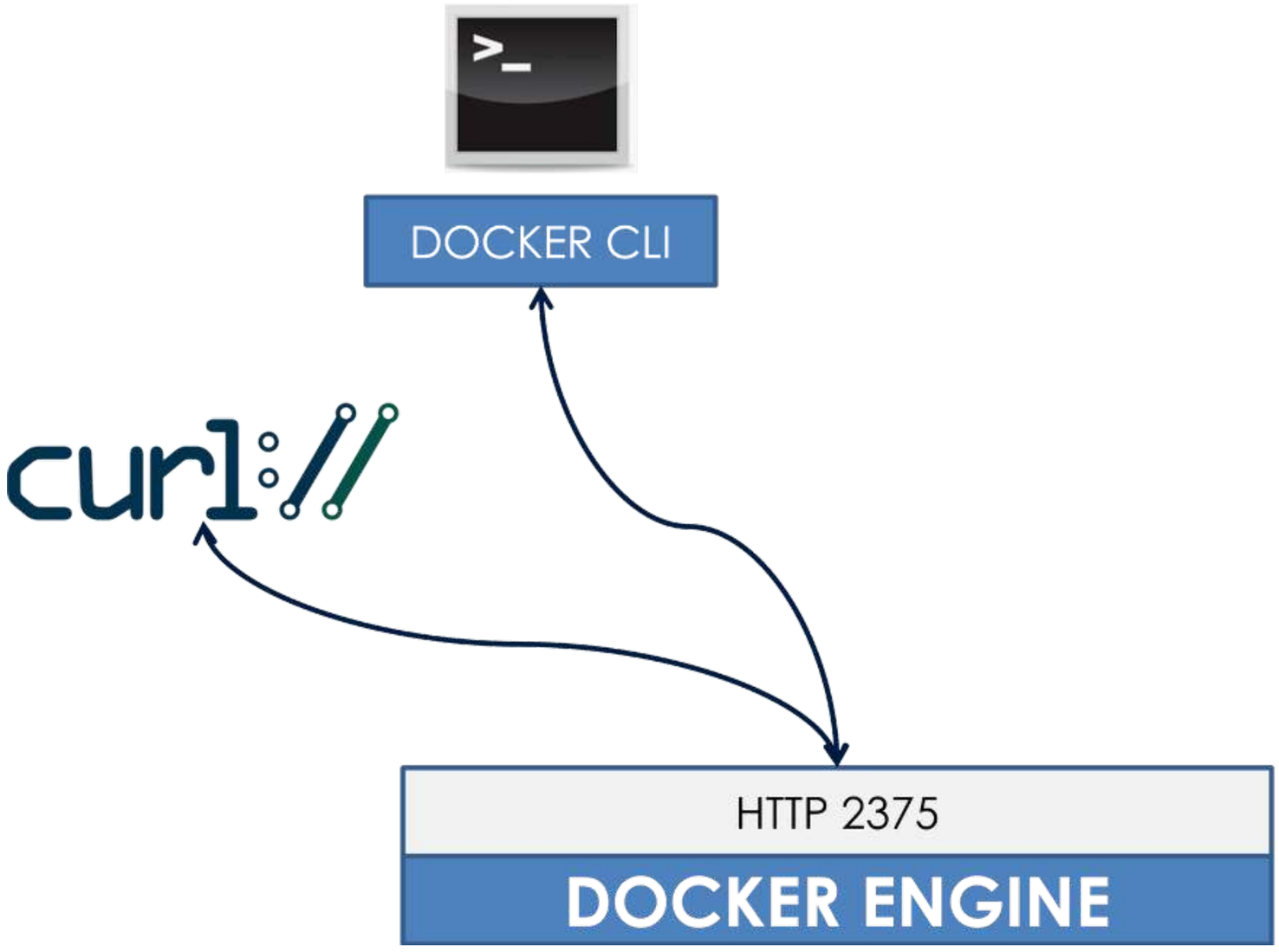


# 🤝 Interagir avec le Docker Engine

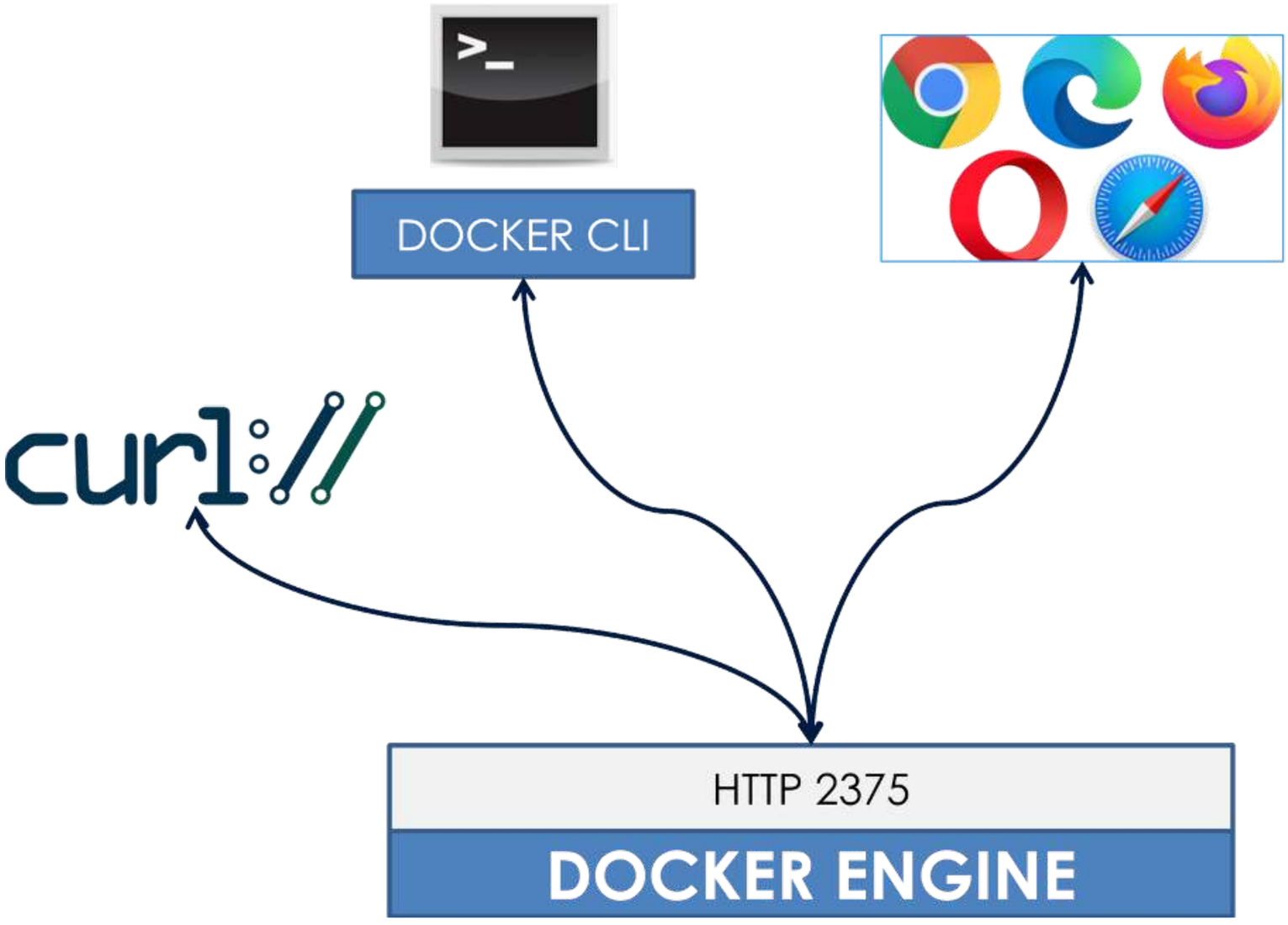
# 🤝 Interagir avec le Docker Engine



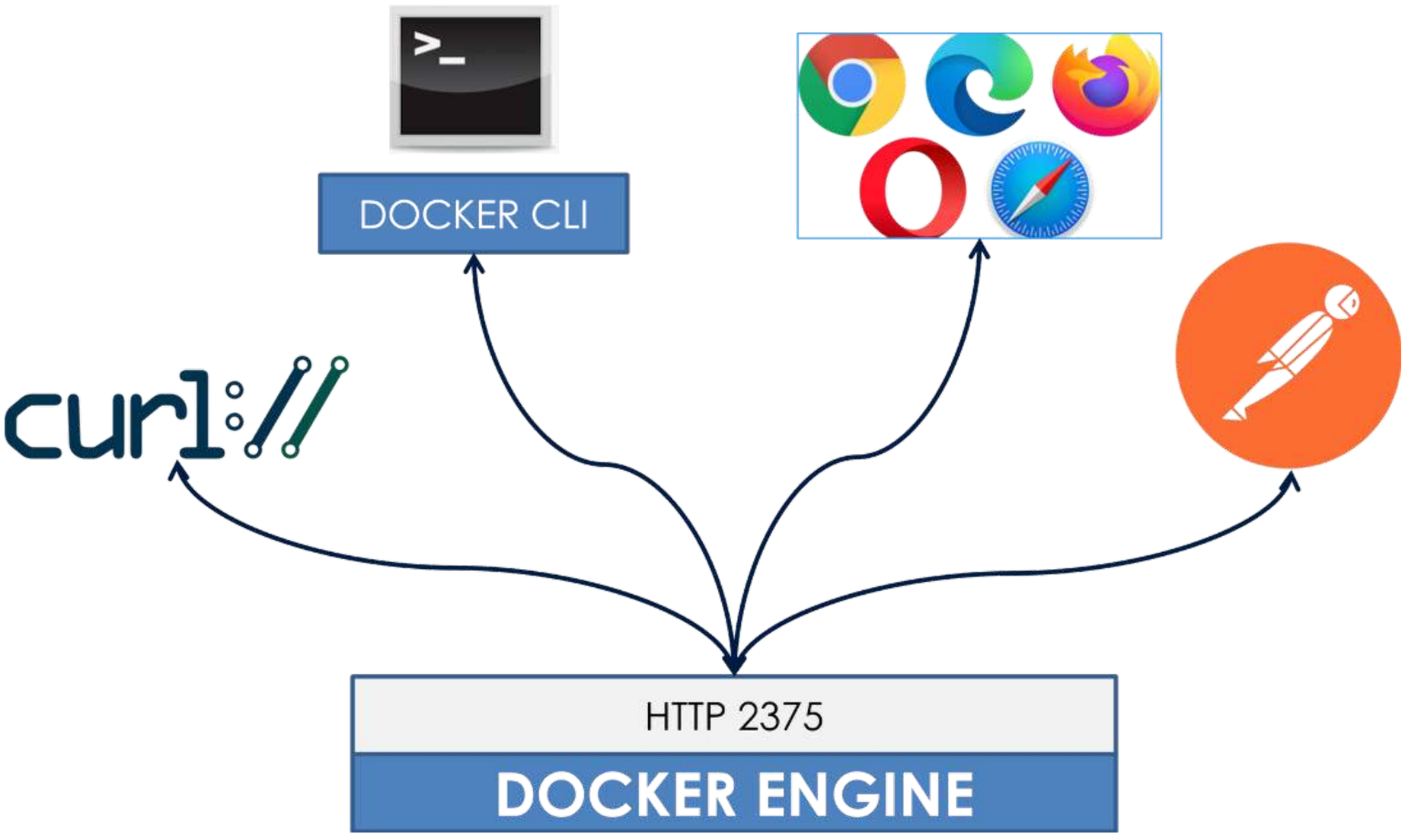
# 🤝 Interagir avec le Docker Engine



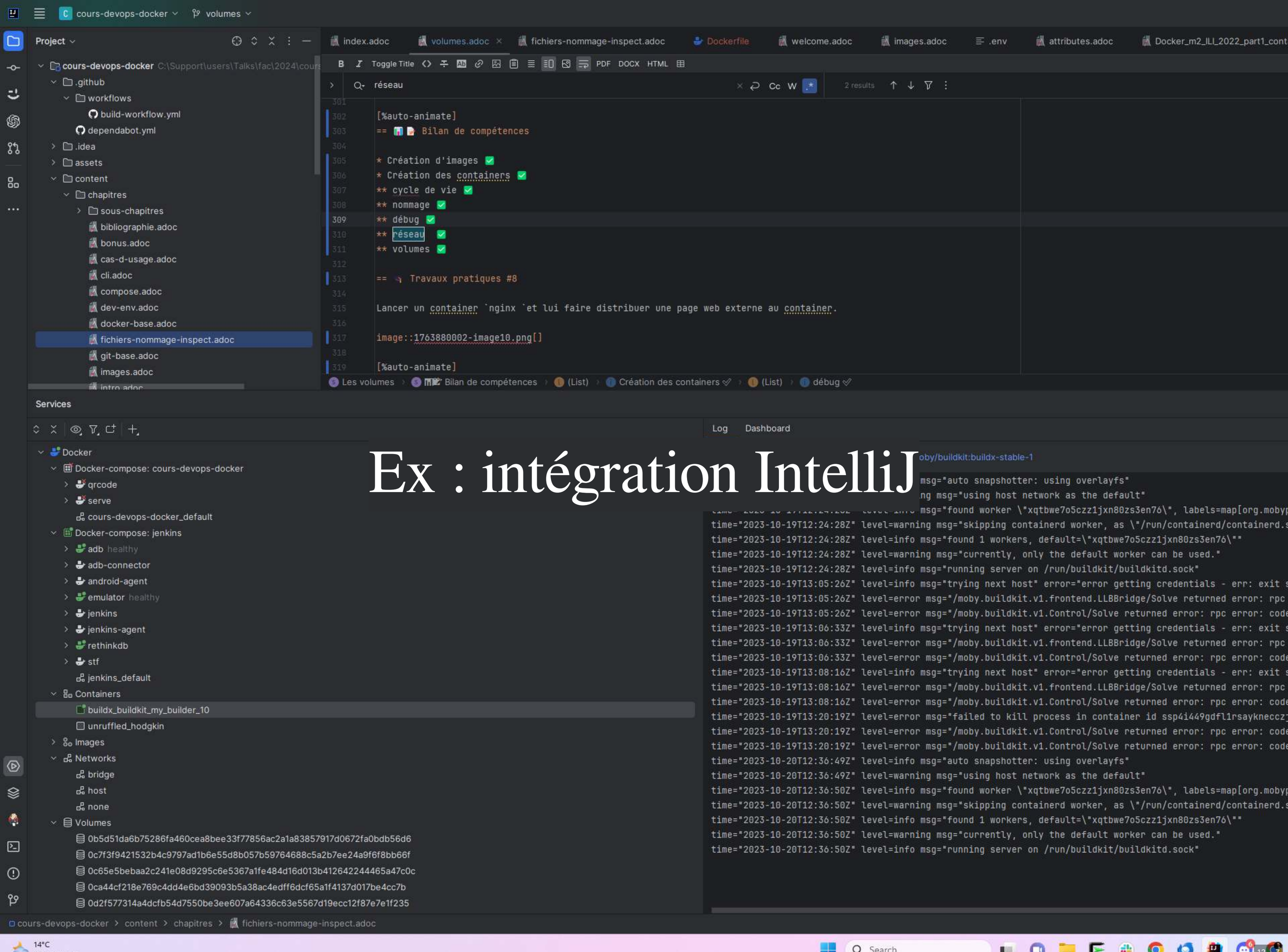
# 🤝 Interagir avec le Docker Engine



# 🤝 Interagir avec le Docker Engine



# Ex : intégration IntelliJ



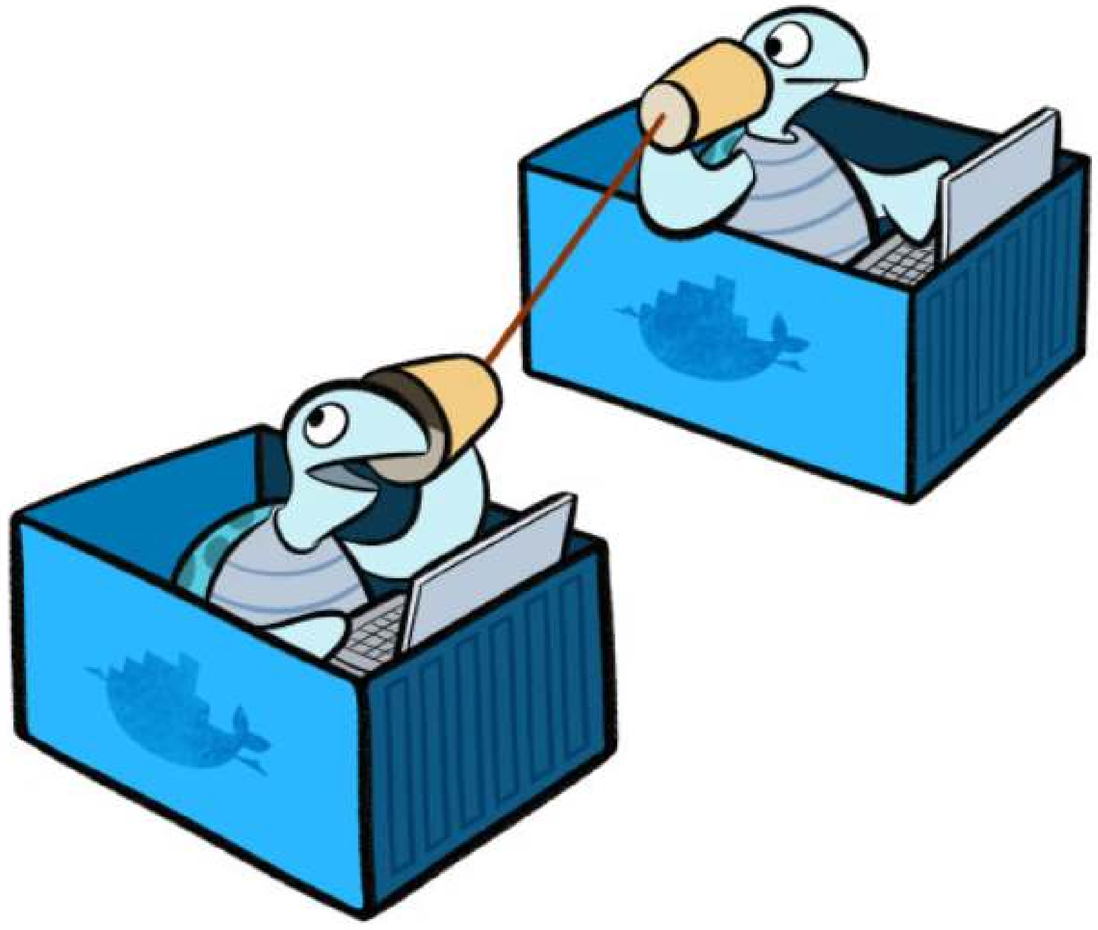
# 🤝 Interagir avec le Docker Engine

Sauf que…

# 🤝 Interagir avec le Docker Engine

Aujourd'hui :

# Les réseaux de containers



# Mapping de Port : Rappel

```
docker container run -d -p 8000:80 nginx
```

# 🗺️🔀 Mapping de Port : Rappel

```
$ docker container run -d -p 8000:80 nginx
```

le port de la machine hôte

# Mapping de Port : Rappel

```
$ docker container run -d -p 8000:80 nginx
```

le port de la machine hôte

le port du container

# Les réseaux

```
docker network ls
```

# Les réseaux

```
docker network ls
```

```
NETWORK ID     NAME                                                      DRIVER    SCOPE
11b7af7b16e4   bridge                                                    bridge    local
1112832d205b   cours-devops-docker_default                               bridge    local
7ab15cc28199   docker_volumes-backup-extension-desktop-extension_default  bridge    local
34caf4674478   host                                                      host      local
2a179c7be3b3   none                                                      null      local
0c577a792771   portainer_portainer-docker-extension-desktop-extension_default   bridge    local
```

| Bridge | Host | Null |
|---|---|---|
| Mode par défaut. C'est un sous réseau dans lequel les containers viennent s'attacher. | Le container sera attaché au réseau de la machine hôte. | Le container ne sera attaché à aucun réseau. |

# 🔍🌐 Les réseaux : inspection

```
docker network inspect bridge
```

# Les réseaux : création

```
docker network create mon-reseau
```

# 🐳🔗 Attacher un container à un réseau particulier

```
docker container run -d --net=mon-reseau --name=app img
```

# MicroDNS

```
docker network create mynet
```

# 🖥️🔍 MicroDNS

On peut appeler le container voisin par son nom !

```
$ docker network create mynet

$ docker container run -d --name web --net mynet nginx

$ docker container run --net mynet alpine ping web
PING nginx (172.18.0.2) 56(84) bytes of data.
64 bytes from web.mynet (172.18.0.2): icmp_seq=1 ttl=64 time=0.060 ms
64 bytes from web.mynet (172.18.0.2): icmp_seq=2 ttl=64 time=0.136 ms
64 bytes from web.mynet (172.18.0.2): icmp_seq=3 ttl=64 time=0.137 ms
64 bytes from web.mynet (172.18.0.2): icmp_seq=4 ttl=64 time=0.124 ms
64 bytes from web.mynet (172.18.0.2): icmp_seq=5 ttl=64 time=0.108 ms
64 bytes from web.mynet (172.18.0.2): icmp_seq=6 ttl=64 time=0.056 ms
64 bytes from web.mynet (172.18.0.2): icmp_seq=7 ttl=64 time=0.052 ms
```

# 🚨 Piège NGINX : le problème

NGINX, contrairement à d'autres outils (curl, wget, applications), résout les noms écrits en dur (bloc `upstream`, `proxy_pass http://nom:port`) **une seule fois, au chargement de la configuration**.

# 🚨 Piège NGINX : configuration défaillante

```
events {
    worker_connections 1024;
}

http {
    # ❌ AUCUNE directive resolver !

    upstream backend {
        server app-server:8080;  # ❌ ÉCHEC au démarrage !
    }

    server {
        listen 80;
        location / {
            proxy_pass http://backend;
        }
    }
}
```

**Résultat** : `nginx: [emerg] host not found in upstream "app-server:8080"`

Ajouter `resolver 127.0.0.11;` à cette configuration ne change **rien** : même message, même arrêt.

# 🚨 Piège NGINX : `resolver` + variable

```
events {
    worker_connections 1024;
}

http {
    # DNS interne Docker pour résolution dynamique
    resolver 127.0.0.11 valid=30s ipv6=off;

    server {
        listen 80;
        location / {
            # ✅ Utiliser une variable pour forcer la résolution dynamique
            set $backend_host "app-server:8080";
            proxy_pass http://$backend_host;
        }
    }
}
```

# 🚨 Piège NGINX : résolution à l'exécution

❗ `resolver` n'est lu que par la résolution **à l'exécution**. Un nom écrit en dur est résolu au chargement, sans lui.

**Deux façons de passer à la résolution à l'exécution** :

1. **Variable** (diapositive précédente) : `set $var "nom-service:port"` puis `proxy_pass http://$var`
2. **`upstream` avec `resolve`** (NGINX 1.27.3 et plus) : garde le bloc `upstream` et ses options (plusieurs serveurs, poids, secours, connexions persistantes)

# 🚨 Piège NGINX : `resolve`

```
http {
    resolver 127.0.0.11 valid=30s ipv6=off;

    upstream backend {
        zone backend 64k;               # obligatoire avec resolve
        server app-server:8080 resolve;
    }
    # server { ... proxy_pass http://backend; ... }
}
```

# 🚨 Piège NGINX : quand faut-il `resolver` ?

| Situation | `resolver` ? | Raison |
|---|---|---|
| `upstream` ou `proxy_pass` avec nom écrit en dur | ❌ SANS EFFET | Résolu au chargement, `resolver` ignoré |
| `proxy_pass` avec variable (`$backend_host`) | ✅ OBLIGATOIRE | Résolution à l'exécution |
| `upstream` avec `server … resolve` | ✅ OBLIGATOIRE | Résolution à l'exécution (+ `zone`) |
| `upstream` avec IP fixe | ❌ NON | Pas de résolution DNS |

# 🚨 Piège NGINX : paramètres de `resolver`

```
resolver 127.0.0.11 valid=30s ipv6=off;
#        ^^^^^^^^^^ ^^^^^^^^^ ^^^^^^^^
#        Adresse DNS  Cache TTL Désactiver IPv6
```

> ℹ️ Le réseau `bridge` par défaut n'a pas ce DNS : créer un réseau avec `docker network create`, ou laisser Compose le faire.

# 🚨 Piège NGINX : exercice (1/2)

Créez un projet avec :

1. **NGINX** (proxy) vers `webapp`

2. **webapp** (serveur web simple, port 80 dans le conteneur, 8080 sur l'hôte)

```yaml
services:
  webapp:
    image: nginx:1.31.8-alpine3.24
    ports:
      - "8080:80"

  proxy:
    image: nginx:1.31.8-alpine3.24
    ports:
      - "8000:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
```

# 🚨 Piège NGINX : exercice (2/2)

**Test 1** : bloc `upstream` vers `webapp:80`, sans `resolver`. Démarrer le proxy seul (`docker compose up proxy`) → Observer l'échec NGINX

**Test 1 bis** : ajouter `resolver 127.0.0.11;` au test 1 → Même échec, même message

**Test 2** : `resolver 127.0.0.11` et `proxy_pass` par variable (diapositive « `resolver` + variable »). Le proxy démarre seul, puis répond dès que `webapp` tourne → Succès !

# 🚨 Piège NGINX : en résumé

💡 **Règle d'or** : un nom écrit en dur est résolu **au chargement**. Pour résoudre à l'exécution : **variable** ou **`resolve`**, et alors **`resolver 127.0.0.11`** obligatoire.

⚠️ Avec un nom écrit en dur, `resolver` ou pas :

- ❌ Échec au démarrage si le service amont n'est pas encore lancé
- ❌ IP obsolète après redémarrage du conteneur amont
- ❌ Erreur : `host not found in upstream`

# 📶🔌 Se connecter à un réseau

```
docker network connect mynet myapp
```

# 🎓 Travaux pratiques #11

Étapes:

# ✅ Solution travaux pratiques #11

## Étape 1 : Lister et inspecter les réseaux existants

```
docker network ls
docker network inspect bridge
```

# ✅ Solution travaux pratiques #11

## Étape 3 : Vérifier le réseau auquel "web1" est attaché

```
network_id=$(docker container inspect \
  -f '{{.NetworkSettings.Networks.bridge.NetworkID}}' web1)
# docker network ls n'affiche que 12 caractères de l'identifiant
shortened_network_id=$(echo $network_id | cut -c 1-12)
docker network ls --format "table {{.ID}}\t{{.Name}}" \
  | awk -v network_id="$shortened_network_id" '$1 == network_id {print $2}'
```

```
bridge
```

# ✅ Solution travaux pratiques #11

Étape 5 : Lancer un container "nginx" nommé "web2" et l’attacher au réseau créé

```
docker container run --name web2 -d --network mon-reseau nginx
```

Étape 6 : Vérifier l’attachement du container "web2" au réseau

# ✅ Solution travaux pratiques #11

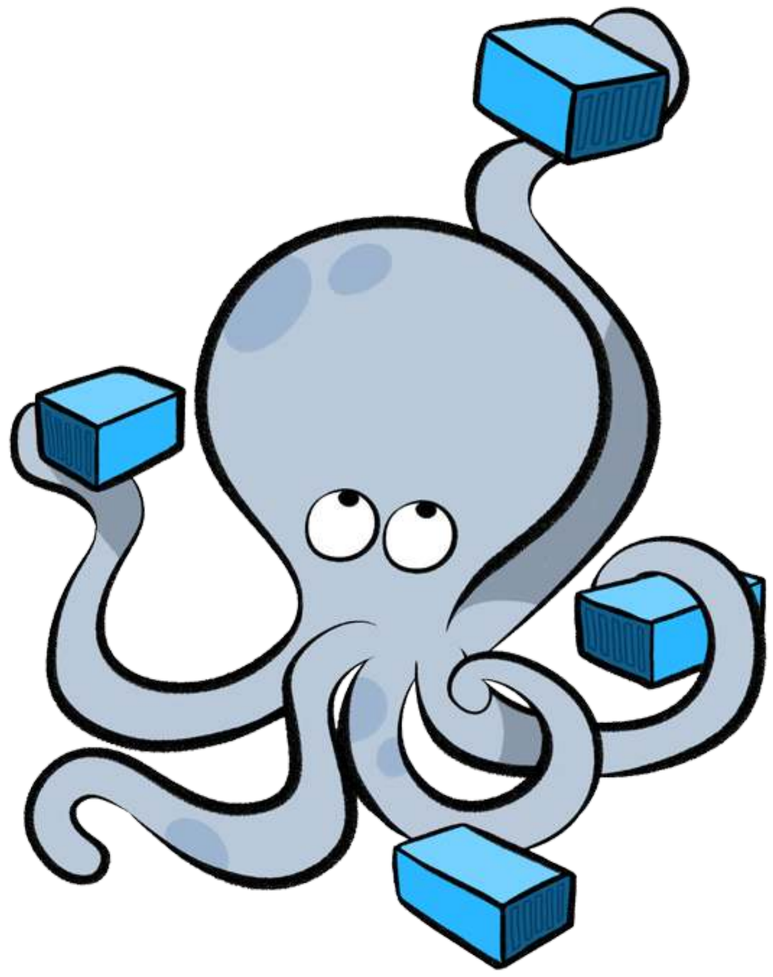
Étape 9 : Corriger pour permettre la communication entre "web1" et "web2"

# 🚨 Piège : localhost vs nom de service dans Docker

## La Confusion Fatale

| Terme | Hors Docker (machine locale) | Dans Docker (environnement conteneurisé) |
|---|---|---|
| `localhost` | Votre ordinateur | **Le conteneur actuel uniquement** |
| `127.0.0.1` | Votre ordinateur | **Le conteneur actuel uniquement** |
| `nom_service` | *(n'existe pas)* | **Un autre conteneur du même réseau** |
| `nom_service:port` | *(n'existe pas)* | **Communication inter-conteneurs** |

# 🐋🐙 Docker compose



# 🌐🏡 Un cas de la vraie vie

Une application riche repose souvent sur plusieurs éléments techniques à coordonner ensemble.

(Apache, Tomcat, Node, MongoDB, ElasticSearch, Logstash, etc.)

Comment automatiser ces déploiements ?

# 📜💻 On pourrait tout scripter…

```
#!/bin/sh
docker network create my-net
docker container run -d --net=my-net -p 3306:3306 mysql
docker container run -d --net=my-net -v /docs:/docs --name col1 httpd:2.4.69-trixie
docker container run -d --net=my-net -v /docs:/docs --name col2 httpd:2.4.69-trixie
```

# 📜💻 Tout scripter

# docker compose.yml

```
services:
```

# docker compose.yml

```
services:
  apache:



middle:


db:
```

# 🐳🐙 docker compose.yml

```
services:
  apache:
    image: "my-httpd:2.4"



middle:


db:
```

# 🐋🐙 docker compose.yml

```
services:
  apache:
    image: "my-httpd:2.4"
    ports:
      - "80:80"


middle:


db:
```

# docker compose.yml

```
services:
  apache:
    image: "my-httpd:2.4"
    ports:
      - "80:80"
    environment:
      - MIDDLE_HOST_1=middle


middle:



db:
```

# 🐋🐙 docker compose.yml

```
services:
  apache:
    image: "my-httpd:2.4"
    ports:
      - "80:80"
    environment:
      - MIDDLE_HOST_1=middle
    volumes:
      - ./etc/httpd/workers.properties:/etc/httpd/conf/workers.properties
middle:



db:
```

# docker compose.yml

```
services:
  apache:
    image: "my-httpd:2.4"
    ports:
      - "80:80"
    environment:
      - MIDDLE_HOST_1=middle
    volumes:
      - ./etc/httpd/workers.properties:/etc/httpd/conf/workers.properties
middle:
  build: .


db:
```

# 🐋🐙 docker compose.yml

```yaml
services:
  apache:
    image: "my-httpd:2.4"
    ports:
      - "80:80"
    environment:
      - MIDDLE_HOST_1=middle
    volumes:
      - ./etc/httpd/workers.properties:/etc/httpd/conf/workers.properties
middle:
  build: .
  environment:
    - DB_HOST=db
db:
```

# docker compose.yml

```yaml
services:
  apache:
    image: "my-httpd:2.4"
    ports:
      - "80:80"
    environment:
      - MIDDLE_HOST_1=middle
    volumes:
      - ./etc/httpd/workers.properties:/etc/httpd/conf/workers.properties
middle:
  build: .
  environment:
    - DB_HOST=db
db:
  image: "mysql:5.6"
```

# docker compose.yml

```yaml
services:
  apache:
    image: "my-httpd:2.4"
    ports:
      - "80:80"
    environment:
      - MIDDLE_HOST_1=middle
    volumes:
      - ./etc/httpd/workers.properties:/etc/httpd/conf/workers.properties
middle:
  build: .
  environment:
    - DB_HOST=db
db:
  image: "mysql:5.6"
  ports:
    - "3306:3306"
```

# 🤔➡️ Et après ?

```
docker compose up -d
docker compose build
docker compose logs
docker compose stop
docker compose restart
docker compose down
```

# 🏁 🔢 Ordre de démarrage

L’ordre de démarrage fait référence à la séquence dans laquelle les services définis dans un fichier Docker Compose sont lancés.

# 🏁🔢 Ordre de démarrage

Pourquoi est-ce important ?

# 🏁🔢 Ordre de démarrage

Comment Docker Compose gère-t-il l'ordre de démarrage ?

# 🏁🔢 Ordre de démarrage

Comment gérer les dépendances entre services ?

# 🏁🔢 Ordre de démarrage

Exemple Voici un exemple de fichier Docker Compose qui utilise `depends_on` et `healthcheck` pour gérer l'ordre de démarrage des services :

# 🏁🔢 Ordre de démarrage🤔 Conditions.

⚠️ Rappel: La directive `depends_on` dans un fichier Docker Compose est utilisée pour indiquer qu'un service dépend d'un autre service. Cela signifie que le service dépendant ne sera pas démarré tant que les services dont il dépend n'auront pas été démarrés.

# 🏁🔢 Ordre de démarrage?🤔 Autres conditions.

En plus de la condition service_healthy, il existe d'autres conditions que vous pouvez utiliser avec `depends_on`:

# 🏁🔢 Ordre de démarrage?🤔 Autre condition.

Voici un exemple de comment vous pourriez utiliser `service_started` dans un fichier Docker Compose :

# 🏁🔢 Ordre de démarrage & 🩺✅ healthcheck

> ⚠️ Rappel `healthcheck` est une instruction dans un `Dockerfile` qui permet de vérifier l'état de santé d'un service. Il peut être utilisé pour déterminer si un service est prêt à être utilisé ou non.

# 🏁🔢 Ordre de démarrage & 🩺✅ healthcheck

> ⚠️ Rappel `depends_on` est une directive dans un fichier Docker Compose qui indique qu'un service dépend d'un autre service. Il peut être utilisé pour contrôler l'ordre de démarrage des services.

# 🏁🔢 🩺✅ Comment healthcheck et depends_on travaillent ensemble ?

En combinant `healthcheck` et `depends_on`, vous pouvez contrôler l’ordre de démarrage des services en fonction de leur état de santé. Par exemple, vous pouvez vous assurer qu’un service de base de données est prêt à être utilisé avant de démarrer une application web qui en dépend.

# 🏁🔢 Ordre de démarrage 📊📝 Pour résumer

L’ordre de démarrage fait référence à la séquence dans laquelle les services définis dans un fichier Docker Compose sont lancés. Sans dépendance déclarée (`depends_on`, mais aussi `links`, `volumes_from` ou `network_mode: "service:..."`), les services démarrent en parallèle, dans un ordre qui change d’un lancement à l’autre.

# 🏁🔢 Ordre de démarrage 📊📝 Pour résumer

- `depends_on` : Cette directive peut être utilisée pour indiquer qu'un service dépend d'un autre service. Elle peut être utilisée avec deux conditions : `service_started` et `service_healthy`.

- `service_started` : Cette condition signifie que le service dépendant ne sera pas démarré tant que le service dont il dépend n'aura pas été démarré. Cependant, cela ne garantit pas que le service dont il dépend est prêt à être utilisé.

- `service_healthy` : Cette condition est utilisée avec la directive healthcheck dans un Dockerfile pour vérifier l'état de santé d'un service. Si la commande healthcheck réussit, Docker considère le service comme sain. Sinon, il est considéré comme malsain.

# 🏁🔢 Ordre de démarrage 📊📝 Pour résumer

Exemple Voici un exemple de fichier Docker Compose qui utilise depends_on et healthcheck pour gérer l’ordre de démarrage des services :

# 🏁🔢 Étude de cas 📚🔍

Un exemple de fichier docker-compose.yml utilisé dans Jenkins:

https://raw.githubusercontent.com/jenkins-docs/quickstart-tutorials/refs/heads/main/docker-compose.yaml

```
sidekick_service:
    # Configuration for the sidekick service
    image: ${DOCKERHUB_USERNAME}/jenkinsci-tutorials:sidekick_
    stdin_open: true
    tty: true
    entrypoint: sh -c "/usr/local/bin/keygen.sh /ssh-dir"  # Runs the keygen.sh script and specifies the output directory
    volumes:
      - agent-ssh-dir:/ssh-dir  # Mounts the agent-ssh-dir volume to the /ssh-dir path inside the container
    healthcheck:
      test: [ "CMD-SHELL", "[ -f /ssh-dir/conductor_ok ] || exit 1" ]  # Checks if the conductor_ok file exists in the /ss
      interval: 5s
      timeout: 10s
      retries: 5
```

# 🏁🔢 Étude de cas 📚🔍

```yaml
sidekick_service:
    # Configuration for the sidekick service
    image: ${DOCKERHUB_USERNAME}/jenkinsci-tutorials:sidekick_
    stdin_open: true
    tty: true
    entrypoint: sh -c "/usr/local/bin/keygen.sh /ssh-dir"  # Runs the keygen.sh script and specifies the output directory
    volumes:
      - agent-ssh-dir:/ssh-dir  # Mounts the agent-ssh-dir volume to the /ssh-dir path inside the container
    healthcheck:
      test: [ "CMD-SHELL", "[ -f /ssh-dir/conductor_ok ] || exit 1" ]  # Checks if the conductor_ok file exists in the /ss
      interval: 5s
      timeout: 10s
      retries: 5
```

- La directive `entrypoint` est utilisée pour spécifier la commande qui sera exécutée lorsque le conteneur démarre.
- Dans ce cas, elle exécute une commande shell qui exécute un script nommé `keygen.sh` situé à `/usr/local/bin/keygen.sh`.
- Le script reçoit un argument `/ssh-dir`, qui est le répertoire où le script effectuera ses opérations.

# 🏁🔢 Étude de cas 📚🔍

```yaml
sidekick_service:
    # Configuration for the sidekick service
    image: ${DOCKERHUB_USERNAME}/jenkinsci-tutorials:sidekick_
    stdin_open: true
    tty: true
    entrypoint: sh -c "/usr/local/bin/keygen.sh /ssh-dir"  # Runs the keygen.sh script and specifies the output directory
    volumes:
      - agent-ssh-dir:/ssh-dir  # Mounts the agent-ssh-dir volume to the /ssh-dir path inside the container
    healthcheck:
      test: [ "CMD-SHELL", "[ -f /ssh-dir/conductor_ok ] || exit 1" ]  # Checks if the conductor_ok file exists in the /ss
      interval: 5s
      timeout: 10s
      retries: 5
```

# 🏁🔢 Étude de cas 📚🔍

```
sidekick_service:
    # Configuration for the sidekick service
    image: ${DOCKERHUB_USERNAME}/jenkinsci-tutorials:sidekick_
    stdin_open: true
    tty: true
    entrypoint: sh -c "/usr/local/bin/keygen.sh /ssh-dir"  # Runs the keygen.sh script and specifies the output directory
    volumes:
      - agent-ssh-dir:/ssh-dir  # Mounts the agent-ssh-dir volume to the /ssh-dir path inside the container
    healthcheck:
      test: [ "CMD-SHELL", "[ -f /ssh-dir/conductor_ok ] || exit 1" ]  # Checks if the conductor_ok file exists in the /ss
      interval: 5s
      timeout: 10s
      retries: 5
```

# ⚠️ Piège : Outils manquants dans les images de base

## Le Problème

❗ Si la commande utilisée dans un health check **n’existe pas dans l’image**, le health check **échouera** !

## Exemples d’Images et Outils Disponibles

| Image de Base | Outils Inclus | Outils ABSENTS |
|---|---|---|
| `debian:12` | sh, bash, ps, grep | ✖ wget, curl, netcat (installables via apt) |

# 🏁🔢 Étude de cas 📚🔍

```
services:
  sidekick_service: [...]

  jenkins_controller: [...]
    depends_on:
      sidekick_service:
        condition: service_completed_successfully  # Depends on the successful
    healthcheck:
      test: [ "CMD-SHELL", "[ -f /ssh-dir/conductor_ok ] || exit 1" ]  # Check
      interval: 5s
      timeout: 10s
      retries: 5

  default_agent: [...]
    depends_on:
      sidekick_service:
        condition: service_completed_successfully  # Depends on the successful
      jenkins_controller:
        condition: service_started
    healthcheck:
      test: [ "CMD-SHELL", "[ -f /home/jenkins/.ssh/authorized_keys ] || exit
      interval: 5s
      timeout: 10s
      retries: 5
    volumes:
      - agent-ssh-dir:/home/jenkins/.ssh:ro  # Mounts the agent-ssh-dir volume
```

- Service jenkins_controller

# 🏁🔢 Étude de cas 📚🔍

```yaml
services:
  sidekick_service: [...]

  jenkins_controller: [...]
    depends_on:
      sidekick_service:
        condition: service_completed_successfully  # Depends on the successful
    healthcheck:
      test: [ "CMD-SHELL", "[ -f /ssh-dir/conductor_ok ] || exit 1" ]  # Check
      interval: 5s
      timeout: 10s
      retries: 5

  default_agent: [...]
    depends_on:
      sidekick_service:
        condition: service_completed_successfully  # Depends on the successful
      jenkins_controller:
        condition: service_started
    healthcheck:
      test: [ "CMD-SHELL", "[ -f /home/jenkins/.ssh/authorized_keys ] || exit
      interval: 5s
      timeout: 10s
      retries: 5
    volumes:
      - agent-ssh-dir:/home/jenkins/.ssh:ro  # Mounts the agent-ssh-dir volume
```

Un `healthcheck` est également défini pour ce service.

# 🏁🔢 Étude de cas 📚🔍

```yaml
services:
  sidekick_service: [...]

  jenkins_controller: [...]
    depends_on:
      sidekick_service:
        condition: service_completed_successfully  # Depends on the successful
    healthcheck:
      test: [ "CMD-SHELL", "[ -f /ssh-dir/conductor_ok ] || exit 1" ]  # Check
      interval: 5s
      timeout: 10s
      retries: 5

  default_agent: [...]
    depends_on:
      sidekick_service:
        condition: service_completed_successfully  # Depends on the successful
      jenkins_controller:
        condition: service_started
    healthcheck:
      test: [ "CMD-SHELL", "[ -f /home/jenkins/.ssh/authorized_keys ] || exit
      interval: 5s
      timeout: 10s
      retries: 5
    volumes:
      - agent-ssh-dir:/home/jenkins/.ssh:ro  # Mounts the agent-ssh-dir volume
```

Service `default_agent`

# 🏁🔢 Étude de cas 📚🔍

```yaml
services:
  sidekick_service: [...]

  jenkins_controller: [...]
    depends_on:
      sidekick_service:
        condition: service_completed_successfully  # Depends on the successful
    healthcheck:
      test: [ "CMD-SHELL", "[ -f /ssh-dir/conductor_ok ] || exit 1" ]  # Check
      interval: 5s
      timeout: 10s
      retries: 5

  default_agent: [...]
    depends_on:
      sidekick_service:
        condition: service_completed_successfully  # Depends on the successful
      jenkins_controller:
        condition: service_started
    healthcheck:
      test: [ "CMD-SHELL", "[ -f /home/jenkins/.ssh/authorized_keys ] || exit
      interval: 5s
      timeout: 10s
      retries: 5
    volumes:
      - agent-ssh-dir:/home/jenkins/.ssh:ro  # Mounts the agent-ssh-dir volume
```

Un `healthcheck` est également défini pour ce service.

# 🏁🔢 Étude de cas 📚🔍

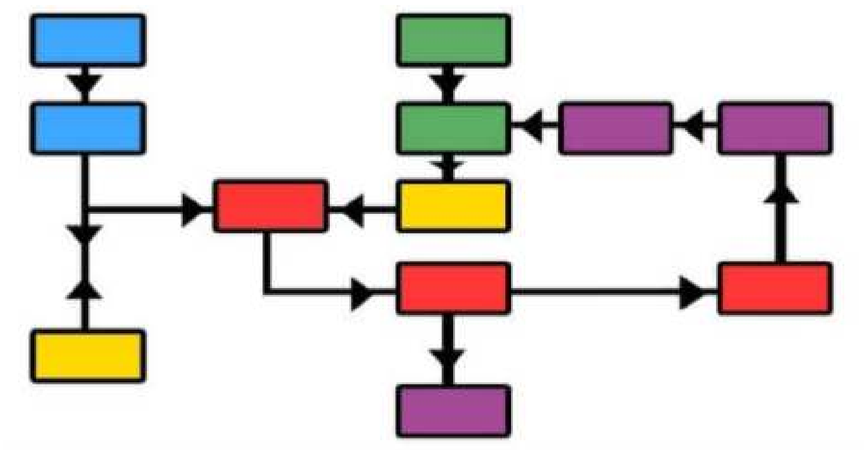
```yaml
services:
  sidekick_service: [...]

  jenkins_controller: [...]
    depends_on:
      sidekick_service:
        condition: service_completed_successfully  # Depends on the successful
    healthcheck:
      test: [ "CMD-SHELL", "[ -f /ssh-dir/conductor_ok ] || exit 1" ]  # Check
      interval: 5s
      timeout: 10s
      retries: 5

default_agent: [...]
  depends_on:
    sidekick_service:
      condition: service_completed_successfully  # Depends on the successful
    jenkins_controller:
      condition: service_started
  healthcheck:
    test: [ "CMD-SHELL", "[ -f /home/jenkins/.ssh/authorized_keys ] || exit
    interval: 5s
    timeout: 10s
    retries: 5
  volumes:
    - agent-ssh-dir:/home/jenkins/.ssh:ro  # Mounts the agent-ssh-dir volume
```

Enfin, un volume nommé `agent-ssh-dir` est monté sur le chemin `/home/jenkins/.ssh` à l'intérieur du conteneur en lecture seule.

# 🏁🔢 Étude de cas 📚🔍

À étudier au calme chez vous:

https://raw.githubusercontent.com/gounthar/MyFirstAndroidAppBuiltByJenkins/stf/jenkins/docker-compose.yml



# Transformer son application en docker compose

Deux approches différentes et complémentaires :

# 🎓 Travaux pratiques #12

https://gitlab.univ-artois.fr/bruno.verachten/devops-docker-tp12.git

Un `httpd` en reverse-proxy devant une application Java dans Tomcat. Écrivez un `docker-compose.yml` qui :

# 🎓 Travaux pratiques #12

Quelques indices :

# ✅ Solution Travaux pratiques #12

```yaml
services:
  proxy:
    build: ./proxy
    ports:
      - "8080:80"
    environment:
      backend_destination: "http://tomcat:8080/dockertraining/"
    depends_on:
      tomcat:
        # Wait until the application answers, not only until the container has started
        condition: service_healthy
tomcat:
  build: ./tomcat
  volumes:
    - ./app/dockertraining.war:/usr/local/tomcat/webapps/dockertraining.war:ro
  healthcheck:
    # -f: fail on an HTTP error, so a 404 before the WAR is deployed counts as "not ready"
    test: ["CMD", "curl", "-fs", "-o", "/dev/null", "http://localhost:8080/dockertraining/"]
    interval: 2s
    timeout: 3s
    retries: 30
    start_period: 5s
```

# ✅ Solution Travaux pratiques #12

```
# The official Tomcat 11 image ships neither curl nor wget.
# This image adds curl so that Compose can check the application is really deployed.
FROM tomcat:11.0.27-jdk25-temurin

RUN apt-get update \
 && apt-get install -y --no-install-recommends curl \
 && rm -rf /var/lib/apt/lists/*
```

# 🎓 Travaux pratiques #12-BIS 🔨🔥

Ça vous dirait d'avoir votre propre forge Gitlab ?

# 🎓 Travaux pratiques #12-BIS

# 🎓 Travaux pratiques #12-BIS 🔨🔥

Pas assez détaillé? 😨

# 🎓 Travaux pratiques #12-BIS 🔨🔥

- Sous `gitlab:`, ajoutez les détails de votre service.
  - Par exemple, `image: 'gitlab/gitlab-ce:19.5.1-ce.0'` pour spécifier l'image Docker à utiliser pour ce service.
  - Continuez à ajouter des détails pour le service gitlab, comme `restart: always` pour toujours redémarrer le conteneur s'il s'arrête, et `hostname: 'localhost'` pour définir le nom d'hôte du serveur GitLab.
- Définissez l'URL externe de votre serveur GitLab en ajoutant `environment:` et `GITLAB_OMNIBUS_CONFIG: |` sur de nouvelles lignes,
- puis `external_url 'http://localhost'` et `gitlab_rails['gitlab_shell_ssh_port'] = 2222` sur les lignes suivantes.
- Exposez les ports nécessaires en ajoutant `ports:` sur une nouvelle ligne,
- puis `- '80:80'`, `- '443:443'` et `- '2222:22'` sur les lignes suivantes.
- Pourquoi `2222` ? Le port `22` de l'hôte est souvent déjà pris par son propre serveur SSH, et Docker refuse alors de démarrer le conteneur (`address already in use`).

# 🎓 Travaux pratiques #12-BIS 🔨🔥

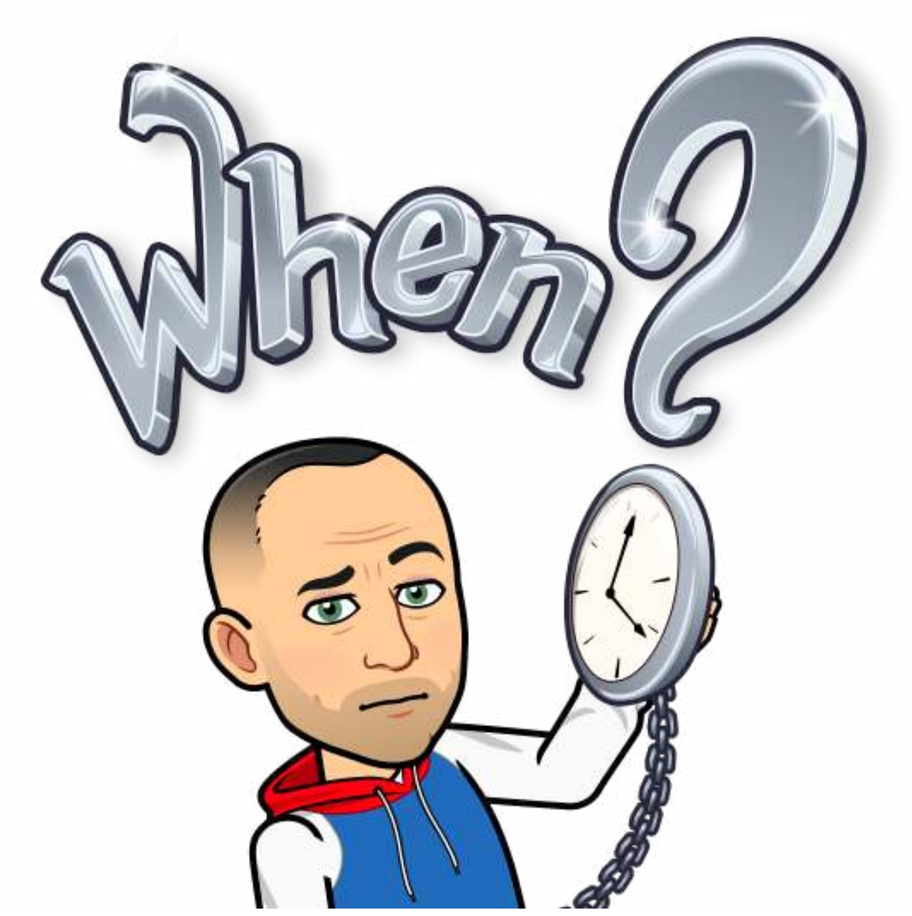
# 🎓 Travaux pratiques #12-BIS 🔨🔥

# 🎓 Travaux pratiques #12-BIS 🔨🔥

# ✅ Solution Travaux pratiques #12-BIS 🔨🔥

```yaml
# This is a Docker Compose file for setting up a GitLab server and two GitLab runners.

services:
  # The GitLab server service
  gitlab:
    # The Docker image to use for the GitLab server
    image: 'gitlab/gitlab-ce:19.5.1-ce.0'
    # Always restart the container if it stops
    restart: always
    # The hostname for the GitLab server
    hostname: 'localhost'
    # Environment variables for the GitLab server
    environment:
      # Configuration for the GitLab Omnibus package
      GITLAB_OMNIBUS_CONFIG: |
        # The external URL for the GitLab server
        external_url 'http://localhost'
        # SSH is published on host port 2222 (see ports below)
        gitlab_rails['gitlab_shell_ssh_port'] = 2222
    # The ports to expose from the GitLab server
    ports:
      - '80:80'   # HTTP
      - '443:443' # HTTPS
      - '2222:22' # SSH (host port 22 is often taken by the host's own sshd)
    # The volumes to mount for the GitLab server
    volumes:
      - 'gitlab_config:/etc/gitlab'       # Configuration files
      - 'gitlab_logs:/var/log/gitlab'     # Log files
      - 'gitlab_data:/var/opt/gitlab'     # Data files
```

# 🔨🔥 J’ai ma forge! Bon, et maintenant? 😐



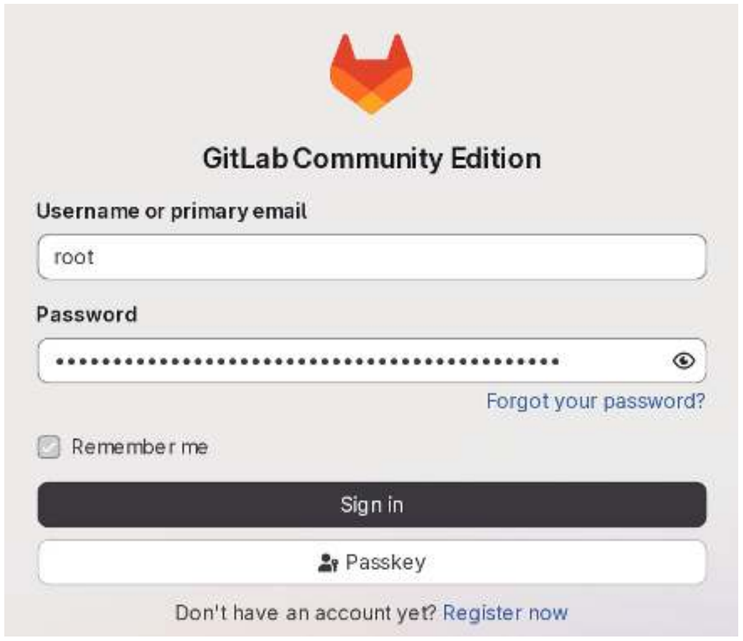
2,000 YEARS

🔨🔥 J’ai ma forge! Bon, et maintenant? 😐

Il va falloir se logguer, lier les runners au serveur, créer un projet, etc…

LATER

# 🔨🔥 J’ai ma forge! Bon, et maintenant? 😐




Pour trouver le mot de passe, il va falloir le demander gentiment à G

```
docker compose -f tp12-bis.yml exec -it gitlab grep 'Password:' /etc/gitlab/initial_ro
Password: qaPxXU+RqioolV3bAljvs2VYnyYa4jO/UYcis/UXLAk=
```

# 🔨🔥 J’ai ma forge! Bon, et maintenant? 😐

Pour utiliser le runner GitLab dans GitLab, vous devez le configurer.

# 🔨🔥 J’ai ma forge! Bon, et maintenant? 😐

GitLab propose :

```
gitlab-runner register --url http://localhost --token glrt-...
```

# 🔨🔥 J’ai ma forge! Bon, et maintenant? 😐

```
docker compose -f tp12-bis.yml exec gitlab-runner-1 \
  gitlab-runner register --non-interactive \
    --url http://gitlab --clone-url http://gitlab \
    --token glrt-... --name runner-1 \
    --executor docker --docker-image alpine:latest \
    --docker-network-mode <dossier>_default
```

# Runners

Create i

All 2 | Instance 2 | Group 0 | Project 0

Search or filter results...

Cre

Online 2 | Offline 0 | Stale 0

| Status | Runner configuration | Owner |
| --- | --- | --- |
| Online Idle | #2 (rj3JKAcT) Instance<br>2 Last contact: 1 minute ago 172.23.0.3<br>Created by **Administrator** 9 minutes ago | Administrator |
| Online Idle | #1 (bl16wVzQ) Instance<br>3 Last contact: 1 minute ago 172.23.0.4<br>Created by **Administrator** 13 minutes ago | Administrator |

# 👤 Profils dans Docker Compose 🐙

On a vu déjà l'instruction `depends_on` dans un fichier Docker Compose.

# 👤 Profils dans Docker Compose 🐙

Par exemple :

# 👤 Profils dans Docker Compose 🐙

# 👤 Profils dans Docker Compose 🐙

# 👤 Profils dans Docker Compose 🐙

```yaml
services:
  service_dev:
    image: mon_image_dev
    profiles:
      - dev
  service_prod:
    image: mon_image_prod
    profiles:
      - prod
```

# ⚠️ Piège : les services sans profil démarrent toujours

```yaml
services:
  db:
    image: mon_image_db
  service_dev:
    image: mon_image_dev
    profiles:
      - dev
```

```
docker compose config --services
# db
docker compose --profile dev config --services
# db
# service_dev
```

# Profils ou fichiers override ?

```
# compose.yml et compose.override.yml sont fusionnés automatiquement
docker compose up -d
# en production, on choisit les fichiers explicitement
docker compose -f compose.yml -f compose.prod.yml up -d
# voir le résultat de la fusion, sans rien lancer
docker compose -f compose.yml -f compose.prod.yml config
```

🔨🔥 Une autre forge, ça vous dit? 😐

# 🔨🔥 Une autre forge, ça vous dit? 😐

Indigeste? 😨

# 🔨🔥 Une autre forge, ça vous dit? 😐

Maven

```
FROM jenkins/ssh-agent:9.1.0-jdk21 as ssh-agent

# ca-certificates because curl will need it later on for the maven installation
RUN apt-get update && apt-get install -y --no-install-recommends ca-certificates curl && apt-get clean && \
  rm -rf /var/lib/apt/lists/*

# Now time to install maven
ARG MAVEN_VERSION=3.9.2

# Set SHELL flags for RUN commands to allow -e and pipefail
# Rationale:https://github.com/hadolint/hadolint/wiki/DL4006
SHELL ["/bin/bash", "-eo", "pipefail", "-c"]

# Add a checksum for the maven binary
RUN curl -sS -L -O --output-dir /tmp/ --create-dirs  https://archive.apache.org/dist/maven/maven-3/${MAVEN_VERSION}/binar
    && printf "%s" "$(sha512sum /tmp/apache-maven-${MAVEN_VERSION}-bin.tar.gz)" | sha512sum -c - \
    && curl -sS -L -O --output-dir /tmp/ --create-dirs  https://downloads.apache.org/maven/maven-3/${MAVEN_VERSION}/binar
    && printf "%s /tmp/apache-maven-${MAVEN_VERSION}-bin.tar.gz" "$(cat /tmp/apache-maven-${MAVEN_VERSION}-bin.tar.gz.sha
    && tar xzf "/tmp/apache-maven-${MAVEN_VERSION}-bin.tar.gz" -C /opt/ \
    && rm "/tmp/apache-maven-${MAVEN_VERSION}-bin.tar.gz" \
    && ln -s /opt/apache-maven-${MAVEN_VERSION} /opt/maven \
    && ln -s /opt/maven/bin/mvn /usr/bin/mvn \
    && mkdir -p /etc/profile.d \
    && echo "export JAVA_HOME=$JAVA_HOME \n \
             export M2_HOME=/opt/maven \n \
             export PATH=${M2_HOME}/bin:${PATH}" > /etc/profile.d/maven.sh
ENV M2_HOME="/opt/maven"
ENV PATH="${M2_HOME}/bin/:${PATH}"
RUN echo "PATH=${PATH}" >> /etc/environment && chown -R jenkins:jenkins "${JENKINS_AGENT_HOME}"
```

# 🔨🔥 Une autre forge, ça vous dit? 😐

Python

```
  # Le service python est un agent Jenkins avec Python installé.
  python:
    build: dockerfiles/python/.  # Le Dockerfile pour construire l'image du service Python.
    container_name: desktop-jenkins_agent-1  # Le nom du conteneur.
    profiles:
      - python  # Les profils à appliquer au service. Cela permet de personnaliser le comportement du service en fonction
    depends_on:  # Les services dont ce service dépend.
      sidekick_service:
        condition: service_completed_successfully  # Le sidekick_service doit se terminer avec succès avant que ce servic
      jenkins_controller:
        condition: service_started  # Le service jenkins_controller doit démarrer avant que ce service ne démarre.
    healthcheck:  # La commande de vérification de santé pour le service.
      test: [ "CMD-SHELL", "[ -f /home/jenkins/.ssh/authorized_keys ] || exit 1" ]  # Vérifie si le fichier authorized_keys
      interval: 5s  # Le temps entre les vérifications de santé.
      timeout: 10s  # Le temps à attendre avant de considérer que la vérification a échoué.
      retries: 5  # Le nombre d'échecs consécutifs nécessaires pour considérer un service comme malsain.
    volumes:
      - agent-ssh-dir:/home/jenkins/.ssh:ro  # Monte le volume agent-ssh-dir au chemin /home/jenkins/.ssh à l'intérieur d
```

# 🔨🔥 Une autre forge, ça vous dit? 😐

```
# Use the base image
FROM jenkins/ssh-agent:8.6.0-jdk17 AS ssh-agent

# Install necessary Dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    binutils ca-certificates curl git python3 python3-pip python3-setuptools python3-venv python3-wheel python3-dev wget

# Create an alias for python3 as python
RUN ln -s /usr/bin/python3 /usr/bin/python

# Install required python packages in a virtual environment:
# Debian trixie refuses a system-wide pip install (PEP 668, externally-managed-environment)
RUN python3 -m venv /opt/venv
ENV PATH="/opt/venv/bin:${PATH}"
RUN pip install docker-py feedparser nosexcover prometheus_client pycobertura pylint pytest pytest-cov requests setuptool

# Add the Jenkins agent user to the environment
RUN echo "PATH=${PATH}" >> /etc/environment

# Set ownership for Jenkins agent home directory
RUN chown -R jenkins:jenkins "${JENKINS_AGENT_HOME}"
```

# 🔨🔥 Une autre forge, ça vous dit? 😐

À vous maintenant!

```
git clone https://github.com/jenkins-docs/quickstart-tutorials.git
```

Search (CTRL+K)

All

| S | W | Name ↓ | Last Success | Last Failure | Last Duration |
|---|---|---|---|---|---|
| ✓ | ☀ | (simple) demo job | 13 sec #2 | N/A | 9.2 sec |

Icon: S M L

Icon legend  Atom feed for all  Atom feed for failures  A

# Autre étude de cas 📚🔍

Et pourquoi pas s’attaquer au docker compose qui a généré ce cours ?

# Autre étude de cas 📚🔍

Et pourquoi pas s'attaquer au docker compose qui a généré ce cours ?

```yaml
# Ceci est un fichier Docker Compose pour un projet qui comprend plusieurs services.

# 'x-slides-base' est une ancre YAML qui définit une configuration commune pour plusieurs services.
x-slides-base: &slides-base
  # La configuration de construction pour l'image Docker.
  build:
    # Le contexte de construction est le répertoire courant.
    context: ./
    args:
      # Active la fonction de mise en cache en ligne de Docker BuildKit.
      BUILDKIT_INLINE_CACHE: 1
# Variables d'environnement pour le conteneur Docker.
environment:
  - PRESENTATION_URL=${PRESENTATION_URL}
  - REPOSITORY_URL=${REPOSITORY_URL}
  # Le fichier racine à construire : SOURCE_FILE=index-examen.adoc make build
  - SOURCE_FILE=${SOURCE_FILE}
# L'ID utilisateur qui exécute les commandes à l'intérieur du conteneur.
user: ${CURRENT_UID}
# Un montage tmpfs pour des E/S plus rapides ; 1777 car le conteneur n'est pas root.
tmpfs:
  - ${BUILD_DIR}:mode=1777
# Volumes pour le conteneur Docker.
volumes:
  - ./content:/app/content
  - ./assets:/app/assets
  - ${DIST_DIR}:/app/dist
  - ./gulp/gulpfile.js:/app/gulpfile.js
  - ./gulp/tasks:/app/tasks
  - ./npm-packages:/app/npm-packages
```

# Autre étude de cas 📚🔍

Et pourquoi pas s’attaquer au docker compose qui a généré ce cours ?

# ➡️🔄 Les ancres dans `docker compose` ⚓

Docker Compose permet de définir et de gérer plusieurs services Docker dans un seul fichier YAML.

# ➡️🔄 Les ancres dans `docker compose` ⚓

Les ancres peuvent être référencées ailleurs dans le fichier YAML en utilisant l'opérateur `*`.

# Autre étude de cas 📚🔍

Le fameux qrcode…

# Autre étude de cas 📚🔍

Le service qui construit les slides à partir de `asciidoc` avec `reveal.js`.

# Autre étude de cas 📚🔍

Le service qui construit les slides à partir de `asciidoc` avec `reveal.js`.

# Autre étude de cas 📚🔍

Le service qui construit le pdf à partir des slides.

# Pourquoi une image construite, et pas l'image publiée ? 📄

L'image publiée `ghcr.io/astefanutti/decktape` ne suffit plus telle quelle.

# Autre étude de cas 📚🔍

Pour rappel, l’ancre contenait :

# ➡️🔄 Build quoi? 🛠️📦

# ➡️🔄 Environnement 🌍🌿

```
# Variables d'environnement pour le conteneur Docker.
environment:
  - PRESENTATION_URL=${PRESENTATION_URL}
  - REPOSITORY_URL=${REPOSITORY_URL}
  # Le fichier racine à construire : SOURCE_FILE=index-examen.adoc make build
  - SOURCE_FILE=${SOURCE_FILE}
```

Trois variables d'environnement sont définies pour le conteneur Docker : `PRESENTATION_URL`, `REPOSITORY_URL` et `SOURCE_FILE`.

`SOURCE_FILE` choisit le fichier racine : `SOURCE_FILE=index-examen.adoc make build` construit le sujet d'examen.

# ➡️🔄 Environnement 🌍🌿

```
# Variables d'environnement pour le conteneur Docker.
environment:
  - PRESENTATION_URL=${PRESENTATION_URL}
  - REPOSITORY_URL=${REPOSITORY_URL}
  # Le fichier racine à construire : SOURCE_FILE=index-examen.adoc make build
  - SOURCE_FILE=${SOURCE_FILE}
```

Lorsque Docker Compose rencontre cette syntaxe, il cherche la valeur de la variable d'environnement dans plusieurs endroits, en suivant un ordre spécifique :

# ➡️🔄 Environnement 🌍🌿

```
# Variables d'environnement pour le conteneur Docker.
environment:
  - PRESENTATION_URL=${PRESENTATION_URL}
  - REPOSITORY_URL=${REPOSITORY_URL}
  # Le fichier racine à construire : SOURCE_FILE=index-examen.adoc make build
  - SOURCE_FILE=${SOURCE_FILE}
```

- Si la variable n'est définie ni dans le shell ni dans le fichier `.env`, Docker Compose utilise la valeur par défaut spécifiée dans le fichier `docker-compose.yml` (s'il y en a une).
- Dans notre cas, aucune valeur par défaut n'est spécifiée. Docker Compose n'échoue pas pour autant : il émet un avertissement, substitue une **chaîne vide**, et continue.
- `The "PRESENTATION_URL" variable is not set. Defaulting to a blank string.` — code de sortie 0.

# Autre étude de cas 📚🔍

```
# L'ID utilisateur qui exécute les commandes à l'intérieur du conteneur.
user: ${CURRENT_UID}
```

# Rendre une variable vraiment obligatoire

Permissif :

```
user: ${CURRENT_UID}
```

Obligatoire :

```
user: ${CURRENT_UID:?CURRENT_UID est obligatoire}
```

# Autre étude de cas 📚🔍

```
# Ce Dockerfile met en place un environnement Node.js avec des outils et dépendances supplémentaires.
# Il est basé sur l'image Docker officielle de Node.js (variante Alpine).

FROM node:26-alpine

# curl (transferts par URL), git, tini (un init minuscule) et unzip. Pas de commentaire après un \ : il casse la ligne.
# hadolint ignore=DL3018
RUN apk add --no-cache \
  curl \
  git \
  tini \
  unzip

# Installer les dépendances NPM globalement (dernières versions)
# hadolint ignore=DL3016,DL3059
RUN npm install --global npm npm-check-updates # Mettre à jour npm à la dernière version et installer npm-check-updates

# Copier les dépendances NPM de l'application dans l'image Docker
COPY ./npm-packages /app/npm-packages
# Créer des liens symboliques pour package.json et package-lock.json à la racine de /app
# Cela permet d'exécuter les opérations npm sans erreur ENOENT
RUN ln -s /app/npm-packages/package.json /app/package.json \
  && ln -s /app/npm-packages/package-lock.json /app/package-lock.json

# Définir le répertoire de travail dans l'image Docker à /app
WORKDIR /app

# Télécharger et installer une version spécifique de FontAwesome
ARG FONTAWESOME_VERSION=6.4.0
RUN curl --silent --show-error --location --output /tmp/fontawesome.zip \
```

# Autre étude de cas 📚🔍

```
RUN npm install --global npm npm-check-updates # Mettre à jour npm à la dernière version et installer npm-check-updates

# Copier les dépendances NPM de l'application dans l'image Docker
COPY ./npm-packages /app/npm-packages
# Créer des liens symboliques pour package.json et package-lock.json à la racine de /app
# Cela permet d'exécuter les opérations npm sans erreur ENOENT
RUN ln -s /app/npm-packages/package.json /app/package.json \
  && ln -s /app/npm-packages/package-lock.json /app/package-lock.json
```

# Autre étude de cas 📚🔍

```
ARG FONTAWESOME_VERSION=6.4.0
RUN curl --silent --show-error --location --output /tmp/fontawesome.zip \
    "https://use.fontawesome.com/releases/v${FONTAWESOME_VERSION}/fontawesome-free-${FONTAWESOME_VERSION}-web.zip" \
  && unzip -q /tmp/fontawesome.zip -d /tmp \
  && mv /tmp/"fontawesome-free-${FONTAWESOME_VERSION}-web" /app/fontawesome \
  && rm -rf /tmp/font*
```

# Autre étude de cas 📚🔍

```
# Installer les dépendances NPM en utilisant le package-lock.json
# Si l'installation échoue, revenir à une installation npm régulière
RUN { npm install-clean && npx update-browserslist-db@latest; } || npm install

# Lier la commande gulp pour qu'elle soit disponible dans le PATH
# hadolint ignore=DL3059
RUN npm link gulp
```

# Autre étude de cas 📚🔍

```
# Copier les tâches gulp et la configuration dans l'image Docker
COPY ./gulp/tasks /app/tasks
COPY ./gulp/gulpfile.js /app/gulpfile.js

# Définir un volume pour le répertoire /app
VOLUME ["/app"]

# Exposer le port 8000 pour HTTP
EXPOSE 8000
```

# Autre étude de cas 📚🔍

```
# Utiliser tini comme point d'entrée, et exécuter gulp par défaut
ENTRYPOINT ["/sbin/tini","-g","gulp"]
CMD ["default"]
```

# Tini dans Docker

# 🤔 Pourquoi le PID 1 est-il spécial ?

```
docker container run --rm alpine ps
# PID   USER     TIME  COMMAND
#     1 root      0:00 ps
```

# 🤔 Qu’est-ce que Tini ?

Tini est un `init` minuscule mais valide pour les conteneurs. Il est conçu pour être le système init le plus simple possible.

# 🛠️ Que fait Tini ?

Tini fait deux choses :

1. Il lance votre processus comme son enfant direct (pas comme petit-enfant, comme le ferait un shell).
2. Il relaie ensuite les signaux à cet enfant, et récolte les processus orphelins qui lui reviennent.

# 🎯 Pourquoi Tini est-il utile dans Docker ?

1. Docker exécute un seul processus dans un conteneur par défaut. Quand un processus meurt, ses enfants orphelins sont rattachés au PID 1 ; si personne ne les récolte, ils restent à l'état de zombies.
2. Tini, en PID 1, récolte ces orphelins et transmet les signaux : `docker container stop` arrête alors proprement l'application.

# 🐋 Tini dans notre Dockerfile

Dans notre Dockerfile, nous utilisons Tini comme point d'entrée :

```
ENTRYPOINT ["/sbin/tini","-g","gulp"]
```

1. Cela signifie que Tini est le premier processus qui est lancé dans notre conteneur.
2. Il lancera ensuite `gulp` en tant que processus enfant.
3. Avec `-g`, tout signal envoyé au conteneur est relayé par Tini à tout le **groupe** de `gulp` (gulp et ses sous-processus).
4. Les processus orphelins qui finissent rattachés au PID 1 sont récoltés par Tini.

# Tini sans toucher au Dockerfile

```yaml
services:
  app:
    image: mon_image
    init: true
```

# 🐋 Docker peut travailler main dans la main avec d'autres outils… 👫🤝

```
# Ce Makefile est utilisé pour gérer le projet Docker Compose.

# Définir les valeurs par défaut pour les variables DIST_DIR et REPOSITORY_URL.
DIST_DIR ?= $(CURDIR)/dist
REPOSITORY_URL ?= file://$(CURDIR)
export REPOSITORY_URL DIST_DIR

# Les conteneurs de construction écrivent dans dist/ à travers un montage bind :
# ils doivent donc tourner sous l'utilisateur qui lance make. Laissés en root, ils
# y déposent des fichiers que `clean` ne peut plus supprimer depuis l'hôte, et
# `make all` meurt sur sa première cible. Calculé ici plutôt qu'écrit en dur : cet
# identifiant n'est pas le même sur un poste, dans un Codespace ou sur un agent
# d'intégration continue.
CURRENT_UID ?= $(shell id -u):$(shell id -g)
export CURRENT_UID

# Activer Docker BuildKit pour une construction plus rapide et la mise en cache des images.
DOCKER_BUILDKIT ?= 1
COMPOSE_DOCKER_CLI_BUILD ?= 1
export DOCKER_BUILDKIT COMPOSE_DOCKER_CLI_BUILD

# Définition des commandes shell réutilisables pour Docker Compose.

# compose_cmd est une fonction qui exécute la commande 'docker compose' avec le fichier docker-compose.yml du répertoire c
# Elle prend un argument $(1) qui représente les options supplémentaires à passer à la commande 'docker compose'.
# $(CURDIR) est une variable d'environnement dans le Makefile qui représente le répertoire courant dans lequel
# le Makefile est exécuté. C'est une fonctionnalité intégrée de GNU Make. Elle est souvent utilisée pour référencer
# des fichiers ou des répertoires relatifs au répertoire courant.
compose_cmd = docker compose --file=$(CURDIR)/docker-compose.yml $(1)
```

# 🐋 Docker peut travailler main dans la main avec d’autres outils… 👫🤝

```
# Les conteneurs de construction écrivent dans dist/ à travers un montage bind :
# ils doivent donc tourner sous l'utilisateur qui lance make. Laissés en root, ils
# y déposent des fichiers que `clean` ne peut plus supprimer depuis l'hôte, et
# `make all` meurt sur sa première cible. Calculé ici plutôt qu'écrit en dur : cet
# identifiant n'est pas le même sur un poste, dans un Codespace ou sur un agent
# d'intégration continue.
CURRENT_UID ?= $(shell id -u):$(shell id -g)
export CURRENT_UID
```

# 🐋 Docker peut travailler main dans la main avec d'autres outils… 👫🤝

```
# Activer Docker BuildKit pour une construction plus rapide et la mise en cache des images.
DOCKER_BUILDKIT ?= 1
COMPOSE_DOCKER_CLI_BUILD ?= 1
export DOCKER_BUILDKIT COMPOSE_DOCKER_CLI_BUILD
```

Cette configuration de docker compose concerne Docker BuildKit, une fonctionnalité de Docker qui améliore les performances de construction des images Docker.

- La ligne `DOCKER_BUILDKIT ?= 1` vérifie si la variable d'environnement `DOCKER_BUILDKIT` est déjà définie.
  - Si ce n'est pas le cas, elle lui attribue la valeur `1`, ce qui active Docker BuildKit.
- De même, `COMPOSE_DOCKER_CLI_BUILD ?= 1` vérifie si la variable d'environnement `COMPOSE_DOCKER_CLI_BUILD` est déjà définie.
  - Si ce n'est pas le cas, elle lui attribue la valeur `1`.
  - Elle servait à l'ancien `docker-compose` (v1, en Python), qui ne passait pas par BuildKit sans elle. Aujourd'hui, ces deux lignes sont historiques : BuildKit est le constructeur par défaut, et

# 🐋 Docker peut travailler main dans la main avec d'autres outils… 👫🤝

```
# Définition des commandes shell réutilisables pour Docker Compose.

# compose_cmd est une fonction qui exécute la commande 'docker compose' avec le fichier docker-compose.yml du répertoire c
# Elle prend un argument $(1) qui représente les options supplémentaires à passer à la commande 'docker compose'.
# $(CURDIR) est une variable d'environnement dans le Makefile qui représente le répertoire courant dans lequel
# le Makefile est exécuté. C'est une fonctionnalité intégrée de GNU Make. Elle est souvent utilisée pour référencer
# des fichiers ou des répertoires relatifs au répertoire courant.
compose_cmd = docker compose --file=$(CURDIR)/docker-compose.yml $(1)

# compose_up est une fonction qui utilise compose_cmd pour exécuter 'docker compose up'.
# Elle prend un argument $(1) qui représente les options supplémentaires à passer à la commande 'docker compose up'.
# L'option '--build' est toujours incluse, ce qui signifie que Docker construira les images avant de démarrer les conteneu
compose_up = $(call compose_cmd, up --build $(1))

# compose_run est une fonction qui utilise compose_cmd pour exécuter 'docker compose run'.
# Elle prend un argument $(1) qui représente les options supplémentaires à passer à la commande 'docker compose run'.
# L'option '--user=0' est toujours incluse, ce qui signifie que les commandes seront exécutées en tant que root à l'intéri
compose_run = $(call compose_cmd, run --user=0 $(1))
```

# 🐋 Docker peut travailler main dans la main avec d'autres outils… 👫🤝

```
# Construire le projet à l'intérieur d'un conteneur Docker.
# Cette règle Makefile utilise la fonction compose_up définie précédemment pour exécuter 'docker compose up' avec l'option
# L'option '--exit-code-from=build' signifie que la commande 'docker compose up' renverra le code de sortie du service 'bu
# Si le service 'build' se termine avec un code de sortie non nul (ce qui signifie qu'une erreur s'est produite), alors 'd
# Cela permet à Make de savoir si la construction du projet a réussi ou non.
build:
	@$(call compose_up,--exit-code-from=build build)
```

# 🔐 De « ça démarre » à « ça tient en production »

# 📖🔚 Fin du chapitre docker compose 🐋🐙

# Sécurité et Production Docker

# Objectifs de la session

# Projet Fil Rouge : SecureVote

SecureVote : Une application de vote en ligne moderne

**Dépôt GitHub :** https://github.com/gounthar/securevote

# Architecture de l'application

```
services:
  frontend:     # React app
  backend:      # Flask API
  database:     # PostgreSQL
  cache:        # Redis
  proxy:        # Nginx
```

Architecture multi-tiers classique pour DevOps

# Votre mission

# Phase 1 : Découverte (30 min)

```
git clone https://github.com/gounthar/securevote.git
cd securevote/phase1
docker compose up -d

# Vérifier l'état des services
docker compose ps

# Suivre les logs (Ctrl+C pour quitter)
docker compose logs -f
```

# Analyse des vulnérabilités

Explorez les fichiers et identifiez les problèmes :

# Points d’attention

**À vérifier :**

# Sécurité Docker : Les fondamentaux

Les 3 piliers de la sécurité Docker :

# Principe du moindre privilège

*Un conteneur ne devrait avoir QUE les permissions nécessaires à son fonctionnement*

# Scan de vulnérabilités

Les images Docker peuvent contenir des vulnérabilités connues (CVE)

**Outils de scan :**

# Docker Scout en action

```
# Scanner une image locale
docker scout cves python:3.11

# Comparer deux images
docker scout compare python:3.11 --to python:3.11-slim
```

Démonstration live recommandée

# Installation de Trivy

## Méthode rapide (pour le TP) :

```
# Installation directe du binaire
curl -sfL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh \
  | sudo sh -s -- -b /usr/local/bin v0.75.0
trivy --version
```

⚠ `curl | sudo sh` est pratique MAIS dangereux en production !

## Méthode sécurisée (recommandée) :

```
# 1. Télécharger le script
curl -sfL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh -o /tmp/trivy-install.sh

# 2. Inspecter le contenu
less /tmp/trivy-install.sh

# 3. Exécuter après vérification
sudo sh /tmp/trivy-install.sh -b /usr/local/bin v0.75.0
```

⚠ Toujours une version explicite (incident de mars 2026, voir les notes).

# Installation Trivy : Alternatives

### Si vous n'avez pas les droits sudo :

```
# Via Docker (aucune installation nécessaire)
docker container run --rm \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v $HOME/.cache/trivy:/root/.cache/trivy \
  aquasec/trivy:0.75.0 image python:3.11
```

Pin la version (`:0.75.0`, jamais `latest`) et persist le cache (`-v $HOME/.cache/trivy`) pour performances

### Autres systèmes :

# Exemple de résultat Trivy

```
trivy image python:3.11

python:3.11 (debian 13.7)
Total: 3602 (UNKNOWN: 97, LOW: 1592, MEDIUM: 1588, HIGH: 309, CRITICAL: 16)
```

Colonnes : Library, Vulnerability, Severity, Status, Installed Version, Fixed Version, Title.

# Utilisateurs non-root

**Problème :** Par défaut, les processus s'exécutent en tant que root

```dockerfile
FROM python:3.11
COPY app.py /app/
CMD ["python", "/app/app.py"]  # Root !
```

⚠ Si compromis, l'attaquant a les privilèges root !

# Solution : Utilisateur dédié

```dockerfile
FROM python:3.11-slim

RUN groupadd -r appuser && useradd -r -g appuser appuser
WORKDIR /app
COPY app.py /app/
RUN chown -R appuser:appuser /app

USER appuser
CMD ["python", "/app/app.py"]
```

# Bonnes pratiques utilisateur

# Comprendre les UID Linux

Linux numérote les utilisateurs (User ID) :

# UID : Exemple pratique

```
# RISQUÉ : UID d'un compte connu de l'hôte
RUN useradd -u 33 appuser
# Conflit possible si l'hôte a www-data (UID 33) !

# RISQUÉ AUSSI : sans option, useradd donne 1000,
# souvent le premier utilisateur humain de l'hôte
RUN useradd appuser

# MIEUX : un UID explicite, absent de l'hôte
RUN useradd -u 10001 appuser
```

# Gestion des secrets

**Mauvaise pratique :**

```
environment:
  - POSTGRES_PASSWORD=super_secret_123  # NON !
```

⚠ Visible dans `docker container inspect`, logs, historique !

# Bonne pratique : Fichiers .env

## Étape 1 : Créer le fichier .env

```
# .env (à ne JAMAIS commiter)
DB_PASSWORD=super_secret_123
```

## Étape 2 : Protéger avec .gitignore

```
# .gitignore
.env
secrets/
```

## Étape 3 : Utiliser dans docker-compose.yml

```
environment:
  POSTGRES_PASSWORD: ${DB_PASSWORD}
  API_KEY: ${API_KEY:-default_key}      # Valeur par défaut si absent
  DB_HOST: ${DB_HOST?required}           # Erreur si non défini
```

> Docker Compose charge automatiquement le fichier .env

## Syntaxes avancées :

# Alternative 1 : Docker Secrets

**Avec Compose, sans Swarm :**

```yaml
services:
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password
secrets:
  db_password:
    file: ./secrets/db_password.txt    # secrets/ est dans .gitignore
```

# Alternative 2 : HashiCorp Vault

**Solution d’entreprise pour secrets**

```
# Application interroge Vault via API
vault kv get -field=password secret/database/prod
```

# Alternative 3 : Cloud Providers

**Services managés AWS / Azure / GCP**

# Alternative 4 : Variables shell

**Passage direct via environnement**

```
# En une ligne
DB_PASSWORD="secret123" docker compose up -d

# Ou via export
export DB_PASSWORD="secret123"
docker compose up -d
```

# Comparaison des solutions

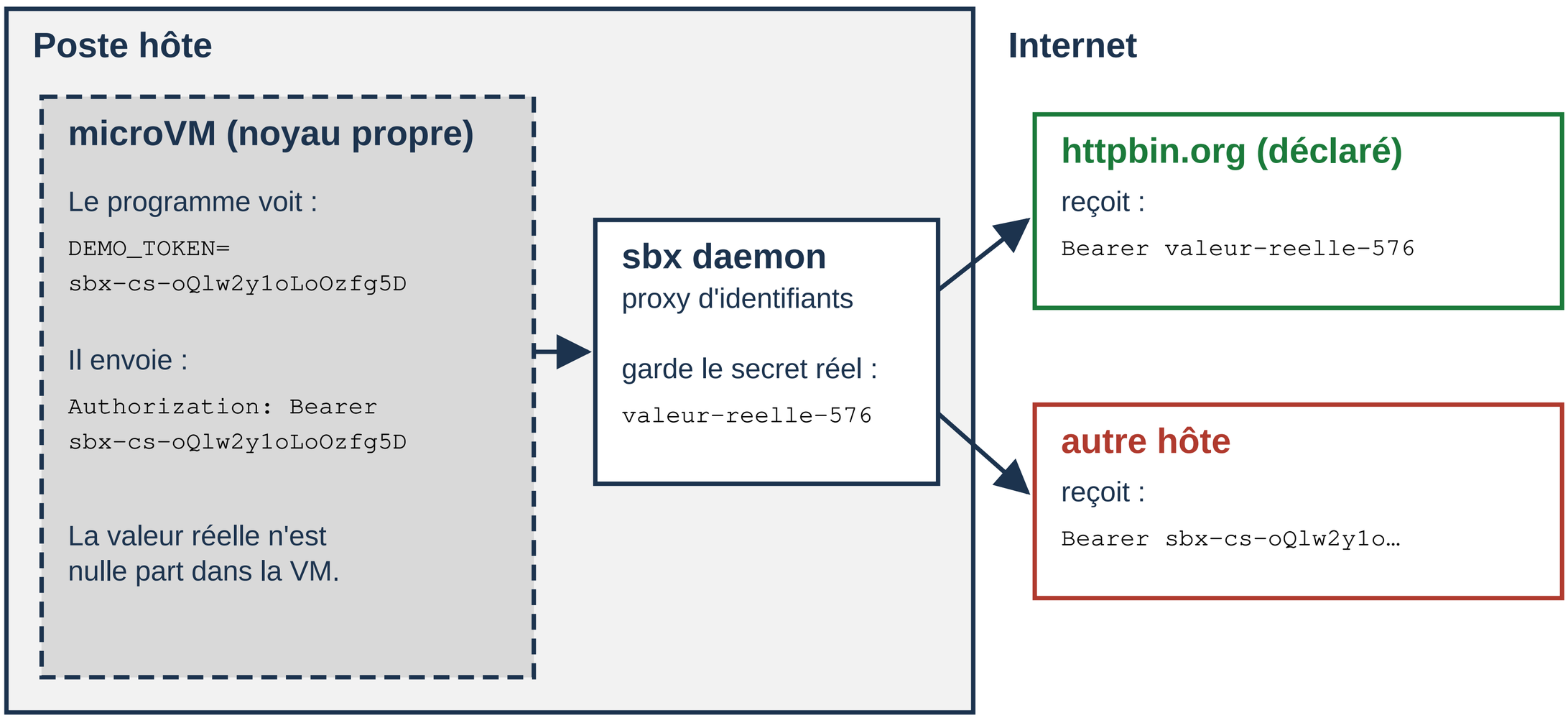
| Solution | Complexité | Coût | Cas d'usage |
|---|---|---|---|
| Fichiers .env | ○ Faible | 💰 Gratuit | Dev, petits projets |
| Docker Secrets | 🟡 Moyenne | Gratuit | Compose (fichier monté), Swarm prod (chiffré) |
| Vault | 🔴 Élevée | Variable | Grandes organisations |
| Cloud Secrets | 🟡 Moyenne | Payant | Déjà sur cloud |
| Variables shell | ○ Faible | 💰 Gratuit | Dev local, CI/CD |

# Le point commun des cinq solutions

À la fin, le secret est **dans** le conteneur :

**Et si le programme ne recevait jamais le secret ?**

# Docker Sandboxes : le proxy d’identifiants




*État au 2026-09-24, `sbx` v0.45.1. Outil récent : les commandes peuvent changer.*

# Démo : ce que voit la sandbox

Sur l'hôte, un secret factice lié à un seul domaine :

```
sbx policy init balanced
sbx secret set-custom --host httpbin.org --env DEMO_TOKEN --value valeur-reelle-576
sbx create shell --name demo .
sbx policy allow network httpbin.org
```

Dans la sandbox :

```
sbx exec demo sh -c 'echo $DEMO_TOKEN'
# sbx-cs-oQlw2y1oLoOzfg5D
sbx exec demo sh -c 'curl -sS -H "Authorization: Bearer $DEMO_TOKEN" https://httpbin.org/headers'
# "Authorization": "Bearer valeur-reelle-576"
```

# Ce que le proxy ne protège pas

> Le proxy garde la valeur hors de la sandbox, tant que l'hôte déclaré ne la renvoie pas. Il n'empêche jamais de **l'utiliser**.

# Phase 2 : Sécurisation (1h15)

Transformer SecureVote en application sécurisée

# Exercice 1 : Images sécurisées (20 min)

**Mission :**

```
# Dans phase1/ : Compose nomme l'image <dossier>-<service>
docker scout cves phase1-backend
# Modifier Dockerfile → python:3.11-slim
docker compose build backend
docker scout cves phase1-backend
```

# Exercice 2 : Utilisateurs non-root (25 min)

**Mission :**

Modifier tous les Dockerfiles de `phase1/` pour des utilisateurs non-root

```
FROM python:3.11-slim
RUN groupadd -r flaskuser && useradd -r -g flaskuser flaskuser
USER flaskuser
```

# Vérification utilisateurs

```
docker compose top backend frontend
# Colonne UID : flaskuser (999) et reactuser (1001), jamais root

docker compose exec backend id
```

> `ps` n'existe pas dans `python:3.11-slim` : `docker compose top` le remplace, depuis l'hôte.

> Si vous voyez UID 0 ou `root` pour le processus principal, c'est à corriger !

# Exercice 3 : Sécuriser les secrets (30 min)

**Mission :**

# Checkpoint Phase 2

Vérification collective :

```
docker compose ps
docker compose top
trivy image phase1-backend --severity HIGH,CRITICAL
```

# Configuration Production

**Développement :**

- Ressources illimitées
- Logs verbeux
- Redémarrages manuels

**Production :**

- Ressources limitées
- Logs structurés
- Redémarrage automatique

# Limites de ressources

## Pourquoi limiter ?

⚠ Sans limites → 100% CPU/RAM !

# Définir les limites

```yaml
services:
  backend:
    deploy:
      resources:
        limits:
          cpus: '0.5'       # Max un demi-cœur
          memory: 512M      # Max 512 Mo RAM
        reservations:
          cpus: '0.25'      # Ignoré hors Swarm
          memory: 256M      # Limite souple, pas une garantie
```

# Choisir les bonnes valeurs

# Politiques de redémarrage

```yaml
services:
  backend:
    restart: unless-stopped
    # Valeurs possibles (une seule clé restart par service) :
    #   no             : jamais
    #   always         : toujours
    #   on-failure     : si le processus sort en erreur
    #   unless-stopped : sauf arrêt manuel
```

> Une seule clé `restart:` par service : quatre clés à la suite, c'est un YAML invalide (`mapping key "restart" already defined`).

# Restart avec limite

```yaml
services:
  backend:
    restart: on-failure:5
```

`on-failure:5` : au plus 5 redémarrages, puis Docker abandonne.

Évite les boucles infinies de redémarrage

# Health checks

```yaml
services:
  backend:
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s
```

Hors Swarm et Kubernetes, un conteneur `unhealthy` n'est **pas** redémarré, même avec `restart: always`. Le healthcheck sert à `depends_on: condition: service_healthy` et à la supervision ; le redémarrage vient de `restart:`, quand le processus meurt.

# Endpoint de santé

```python
@app.route('/health')
def health():
    try:
        db.ping()
        return jsonify({"status": "healthy"}), 200
    except:
        return jsonify({"status": "unhealthy"}), 503
```

# Dépendances entre services

```yaml
services:
  backend:
    depends_on:
      database:
        condition: service_healthy
      cache:
        condition: service_started
```

# Monitoring et logs

**Observabilité :**

Le TP #13 (dans les bonus) monte une stack ELK

# Driver de logs

```yaml
services:
  backend:
    logging:
      driver: "json-file"
      options:
        max-size: "10m"       # Max 10 Mo
        max-file: "3"         # Garder 3 fichiers
```

⚠ Sans limites, les logs remplissent le disque !

# Logs structurés

**Mauvais :** `Server started on port 5000`

**Bon :**

```
{"timestamp":"2025-01-15T10:30:00Z","level":"INFO","service":"backend","message":"Server started","port":5000}
```

# Phase 3 : Production (1h)

Optimiser SecureVote pour la production

# Exercice 4 : Limites de ressources (20 min)

**Mission :**

```
docker container stats --no-stream
./load_test.sh
docker container stats
```

# Exercice 5 : Restart et Health (25 min)

**Mission :**

```
# Simuler un plantage : tuer le processus principal depuis l'extérieur
docker container run --rm --pid=host --privileged alpine:3.22 \
  kill -9 $(docker container inspect -f '{{.State.Pid}}' securevote-backend)
docker compose ps
docker compose logs backend
```

> ⚠ `docker compose kill` ne convient pas : Docker le compte comme un arrêt manuel, et aucune politique `restart:` ne relance le conteneur.

# Exercice 6 : Dépendances (15 min)

**Mission :**

# Checkpoint Phase 3

Application production-ready :

```
docker compose config --services
docker compose ps
docker container stats --no-stream
```

# Registres et Distribution

# Docker Hub

```
docker image pull nginx:latest
# Équivalent à :
docker image pull docker.io/library/nginx:latest
```

# Registres privés

**Pourquoi ?**

# Solutions de registres

# Démarrer un registre local

```
docker container run -d -p 5050:5000 --name registry registry:3
docker image tag securevote-backend:latest localhost:5050/securevote-backend:latest
docker image push localhost:5050/securevote-backend:latest
```

> Port 5050 côté hôte : le 5000 est déjà pris par le backend SecureVote de la phase 1 (et par AirPlay sur macOS récent).

# Tags et versions

```
# Bonne pratique : versionner
docker image tag myapp:latest myregistry.com/myapp:1.2.3
docker image tag myapp:latest myregistry.com/myapp:1.2
docker image tag myapp:latest myregistry.com/myapp:latest
```

Plusieurs tags → même image

# Exercice 7 : Registre local (Bonus)

**Si le temps le permet :**

# Récapitulatif et bonnes pratiques

# Checklist sécurité

# Checklist production

# Ce qu’on a appris

# SecureVote : Avant/Après

**Avant :**

- Images non scannées
- Root partout
- Secrets en clair
- Pas de limites

**Après :**

- Images slim, scannées
- Utilisateurs dédiés
- Secrets protégés
- Production-ready ✅

# 🔧 Troubleshooting Production

## Conteneurs qui ne démarrent pas

### **Étape 1** : Vérifier les logs

```
docker container logs nom-conteneur

# OU pour suivre en temps réel
docker container logs -f nom-conteneur
```

### **Étape 2** : Inspecter le conteneur

```
docker container inspect nom-conteneur

# Voir seulement l'état
docker container inspect --format='{{.State.Status}}' nom-conteneur

# Voir le code de sortie
docker container inspect --format='{{.State.ExitCode}}' nom-conteneur
```

### **Problèmes courants** :

| **Symptôme** | **Cause probable** | **Solution** |
| --- | --- | --- |
| `host not found in upstream` | NGINX : résolution DNS statique | `resolver 127.0.0.11 valid=10s;` (DNS |

# Vérifier la signature d'une image

Cosign (projet Sigstore) vérifie qui a signé une image, pas seulement qu'elle est signée :

```
docker container run --rm ghcr.io/sigstore/cosign/cosign:v3.1.3 verify \
  --certificate-identity=krel-trust@k8s-releng-prod.iam.gserviceaccount.com \
  --certificate-oidc-issuer=https://accounts.google.com \
  registry.k8s.io/pause:3.10
```

```
Verification for registry.k8s.io/pause:3.10 --
The following checks were performed on each of these signatures:
  - The cosign claims were validated
  - Existence of the claims in the transparency log was verified offline
  - The code-signing certificate was verified using trusted certificate authority certificates
```

Avec `--certificate-identity=someone@example.com`, la commande échoue (code de sortie 12) : `no matching signatures`.

# Aller plus loin

# Ressources utiles

- Docker Security: https://docs.docker.com/engine/security/
- CIS Docker Benchmark
- Trivy: https://github.com/aquasecurity/trivy
- OWASP Docker Security Cheat Sheet

# Fin du chapitre sécurité et production

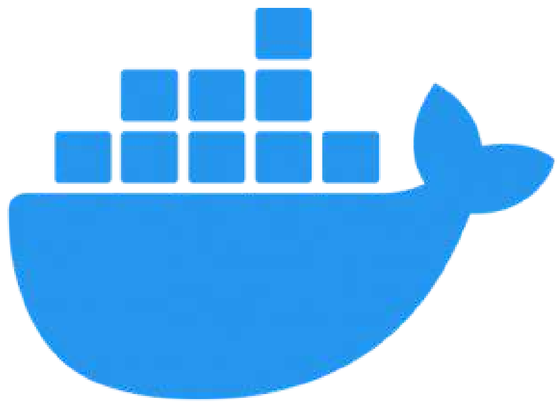
Bravo, vous avez transformé SecureVote en application sécurisée et production-ready !

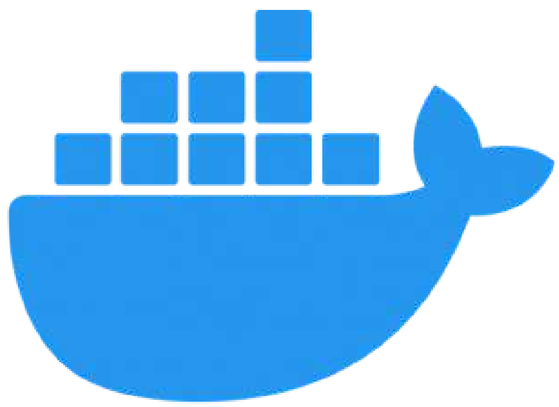
# Questions ?

gounthar@gmail.com

# Bonus

# Les petits plus





# "Il en remet une couche"

```
docker image history nginx
IMAGE          CREATED      CREATED BY                                      SIZE      COMMENT
f9ea18bfa4fa   8 days ago   CMD ["nginx" "-g" "daemon off;"]                0B        buildkit.dockerfile.v0
<missing>      8 days ago   STOPSIGNAL SIGQUIT                              0B        buildkit.dockerfile.v0
<missing>      8 days ago   EXPOSE map[80/tcp:{}]                           0B        buildkit.dockerfile.v0
<missing>      8 days ago   ENTRYPOINT ["/docker-entrypoint.sh"]            0B        buildkit.dockerfile.v0
<missing>      8 days ago   COPY 30-tune-worker-processes.sh /docker-ent…   16.4kB    buildkit.dockerfile.v0
<missing>      8 days ago   COPY 20-envsubst-on-templates.sh /docker-ent…   12.3kB    buildkit.dockerfile.v0
<missing>      8 days ago   COPY 15-local-resolvers.envsh /docker-entryp…   12.3kB    buildkit.dockerfile.v0
<missing>      8 days ago   COPY 10-listen-on-ipv6-by-default.sh /docker…   12.3kB    buildkit.dockerfile.v0
<missing>      8 days ago   COPY docker-entrypoint.sh / # buildkit          8.19kB    buildkit.dockerfile.v0
<missing>      8 days ago   RUN /bin/sh -c set -x     && groupadd --syst…   88.7MB    buildkit.dockerfile.v0
<missing>      8 days ago   ENV DYNPKG_RELEASE=1~trixie                     0B        buildkit.dockerfile.v0
<missing>      8 days ago   ENV PKG_RELEASE=1~trixie                        0B        buildkit.dockerfile.v0
<missing>      8 days ago   ENV ACME_VERSION=0.4.1                          0B        buildkit.dockerfile.v0
<missing>      8 days ago   ENV NJS_RELEASE=1~trixie                        0B        buildkit.dockerfile.v0
<missing>      8 days ago   ENV NJS_VERSION=1.0.1                           0B        buildkit.dockerfile.v0
<missing>      8 days ago   ENV NGINX_VERSION=1.31.6                        0B        buildkit.dockerfile.v0
<missing>      8 days ago   LABEL maintainer=NGINX Docker Maintainers <d…   0B        buildkit.dockerfile.v0
<missing>      9 days ago   # debian.sh --arch 'amd64' out/ 'trixie' '@1…   87.7MB    debuerreotype 0.17
```

# "Il en remet une couche"

```
docker image history nginx
IMAGE          CREATED      CREATED BY                                      SIZE      COMMENT
f9ea18bfa4fa   8 days ago   CMD ["nginx" "-g" "daemon off;"]                0B        buildkit.dockerfile.v0
<missing>      8 days ago   STOPSIGNAL SIGQUIT                              0B        buildkit.dockerfile.v0
<missing>      8 days ago   EXPOSE map[80/tcp:{}]                           0B        buildkit.dockerfile.v0
<missing>      8 days ago   ENTRYPOINT ["/docker-entrypoint.sh"]            0B        buildkit.dockerfile.v0
<missing>      8 days ago   COPY 30-tune-worker-processes.sh /docker-ent…   16.4kB    buildkit.dockerfile.v0
<missing>      8 days ago   COPY 20-envsubst-on-templates.sh /docker-ent…   12.3kB    buildkit.dockerfile.v0
<missing>      8 days ago   COPY 15-local-resolvers.envsh /docker-entryp…   12.3kB    buildkit.dockerfile.v0
<missing>      8 days ago   COPY 10-listen-on-ipv6-by-default.sh /docker…   12.3kB    buildkit.dockerfile.v0
<missing>      8 days ago   COPY docker-entrypoint.sh / # buildkit          8.19kB    buildkit.dockerfile.v0
<missing>      8 days ago   RUN /bin/sh -c set -x     && groupadd --syst…   88.7MB    buildkit.dockerfile.v0
<missing>      8 days ago   ENV DYNPKG_RELEASE=1~trixie                     0B        buildkit.dockerfile.v0
<missing>      8 days ago   ENV PKG_RELEASE=1~trixie                        0B        buildkit.dockerfile.v0
<missing>      8 days ago   ENV ACME_VERSION=0.4.1                          0B        buildkit.dockerfile.v0
<missing>      8 days ago   ENV NJS_RELEASE=1~trixie                        0B        buildkit.dockerfile.v0
<missing>      8 days ago   ENV NJS_VERSION=1.0.1                           0B        buildkit.dockerfile.v0
<missing>      8 days ago   ENV NGINX_VERSION=1.31.6                        0B        buildkit.dockerfile.v0
<missing>      8 days ago   LABEL maintainer=NGINX Docker Maintainers <d…   0B        buildkit.dockerfile.v0
<missing>      9 days ago   # debian.sh --arch 'amd64' out/ 'trixie' '@1…   87.7MB    debuerreotype 0.17
```

Il est préférable de limiter le nombre de couches pour limiter la bande passante.

# "Il en remet une couche"

```
RUN apk add --no-cache \
     patch \
     tar \
     mercurial \
     git \
     ruby \
     ruby-dev \
     ruby-bundler \
     make \
     g++ \
     zlib-dev \
     libxml2-dev \
     docker \
     nodejs \
     npm \
&& mkdir -p /var/lib/docker-builder \
&& mkdir -p /etc/docker-builder
```

# Ne pas charger inutilement !

```
&& apk del make g++ zlib-dev libxml2-dev ruby-dev \
&& rm -f previously-downloaded.tar.gz \
&& rm -rf /tmp/*
```

# Documentez !

Les mots clés `EXPOSE` et `LABEL` peuvent fournir de précieuses informations aux utilisateurs de vos images !

(les ports exposés par défaut, les auteurs de l'image, la version d'un middleware embarqué, …)

# Préparez-vous au read-only

Il est possible de lister toutes les modifications qui ont été apportées au filesystem d'un container.

```
docker container diff 29f1c4
C /run
A /run/nginx.pid
C /var
C /var/lib
C /var/lib/nginx
C /var/lib/nginx/tmp
A /var/lib/nginx/tmp/client_body
A /var/lib/nginx/tmp/fastcgi
A /var/lib/nginx/tmp/proxy
A /var/lib/nginx/tmp/scgi
A /var/lib/nginx/tmp/uwsgi
C /var/log
C /var/log/nginx
A /var/log/nginx/access.log
A /var/log/nginx/error.log
```

# HealthCheck

Savoir que mon PID 1 est toujours en cours d'exécution n'est peut-être pas la meilleure piste pour savoir si mon conteneur est en bonne santé !

```
FROM ghost:6
RUN apt-get update && apt-get install -y --no-install-recommends curl \
        && rm -rf /var/lib/apt/lists/*

HEALTHCHECK --interval=1m --timeout=30s --retries=3 CMD curl --fail http://localhost:2368 || exit 1
```

source : https://www.grottedubarbu.fr/docker-healthcheck/

# Debugging

# Une tonne d'outils !

Les logs

- des containers
- du démon Docker

```
docker container exec
docker container inspect
docker container cp
docker image history
docker container stats
```

# Debugging : Les logs

rappel : `docker container logs` permet de lister les logs des conteneurs.

# Concentrateur de logs

Par défaut, les logs des conteneurs sont stockés dans des fichiers json. Mais comment faire pour les envoyer vers un concentrateur ?

# Travaux pratiques #13

Étapes

Lancer une stack ELK grâce aux fichiers fournis dans le repo https://gitlab.univ-artois.fr/bruno.verachten/devops-docker-tp13

Alimenter en logs avec la commande :

```
docker container run --log-driver=gelf \
  --log-opt gelf-address=udp://localhost:12201 alpine sh -c 'seq 1 100'
```

> Alpine n’a pas `bash`, seulement le `sh` de BusyBox : un script copié d’Internet y échoue souvent avec `executable file not found`.

Créer un index "timestamp" sur Kibana puis aller à l’écran "discover"

Lancer un conteneur nginx et concentrez ses logs dans ELK

# GitLab et Dependabot

# 🤔 Qu’est-ce que Dependabot ?

Dependabot est un outil qui vous aide à maintenir vos dépendances à jour.

# 🛠️ Comment Dependabot fonctionne-t-il avec Docker ?

Dependabot peut analyser vos `Dockerfiles` et vos fichiers `docker-compose` pour trouver les dépendances qui peuvent être mises à jour.

# 🎯 Pourquoi utiliser Dependabot avec Docker ?

1. Garder vos dépendances à jour est crucial pour la sécurité et la stabilité de vos applications.
2. Dependabot automatise ce processus, vous faisant gagner du temps et réduisant le risque d'oublier une mise à jour importante.
3. Dependabot n'est pas magique, il faut que votre CI soit capable de construire votre application avec les nouvelles dépendances.
4. Ce qui n'est pas testé ne fonctionne pas.

# 🚀 Comment configurer Dependabot pour GitLab ? 🦊🛠️

- Malheureusement, il n'y a pas encore de support officiel pour GitLab dans Dependabot (et vice versa).
- Notre forge Gitlab est définie dans un fichier docker-compose.yml, et Dependabot aussi.
- Dependabot doit avoir accès à notre forge GitLab pour pouvoir ouvrir des merge requests.
- Suivons donc la documentation officielle.

Search page

## Personal Access Tokens

You can generate a personal access token for each application you use that needs access to the GitLab API. You can also use personal access tokens to authenticate again
They are the only accepted password when you have Two-Factor Authentication (2FA) enabled.

**Active personal access tokens** 0

| Token name | Scopes | Created | Last Used | Expires | Actio |
|---|---|---|---|---|---|
| This user has no active personal access tokens. | | | | | |

## Feed token

Your feed token authenticates you when your RSS reader loads a personalized RSS feed or when your calendar application loads a personalized calendar. It is visible in thos
cannot be used to access any other data.

**Feed token**

••••••••••••••••••••••••••

Keep this token secret. Anyone who has it can read activity and issue RSS feeds or your calendar feed as if they were you. If that happens, reset this token.

# Personal Access Tokens

## Add a personal access token

**Token name**

dependabot-token

For example, the application using the token or the purpose of the token.

**Expiration date**

2023-12-21

**Select scopes**

Scopes set the permission levels granted to the token. Learn more.

- [x] api
  Grants complete read/write access to the API, including all groups and projects, the container registry, and the package registry.
- [x] read_api
  Grants read access to the API, including all groups and projects, the container registry, and the package registry.
- [ ] read_user
  Grants read-only access to the authenticated user's profile through the /user API endpoint, which includes username, public email, and full name. Also grants access
  API endpoints under /users.
- [ ] create_runner
  Grants create access to the runners.
- [ ] k8s_proxy
  Grants permission to perform Kubernetes API calls using the agent for Kubernetes.
- [x] read_repository
  Grants read-only access to repositories on private projects using Git-over-HTTP or the Repository Files API.
- [x] write_repository
  Grants read-write access to repositories on private projects using Git-over-HTTP (not using the API).
- [x] sudo
  Grants permission to perform API actions as any user in the system, when authenticated as an admin user.
- [x] admin_mode
  Grants permission to perform API actions as an administrator, when Admin Mode is enabled.

Create personal access token | Cancel

| Token name | Scopes | Created | Last Used | Expires | Actio |
|---|---|---|---|---|---|

This user has no active personal access tokens.

# 🚀 Comment configurer Dependabot pour GitLab ? 🦊🛠️

> ⚠️ Notez bien le token, vous ne le reverrez plus jamais.

L'étape suivante, c'est de positionner les valeurs de certaines variables d'environnement. Ça peut être dans un `.env`, ou dans les variables d'environnement de votre machine.

```
export SETTINGS__GITLAB_URL=http://localhost
export SETTINGS__GITLAB_ACCESS_TOKEN=glpat-XXXXXXXXXXXXXXXXXXXX
```

# 🚀 Comment configurer Dependabot pour GitLab ? 🦊🛠️

Démarrez l'application avec `docker compose`:

```
curl -s https://gitlab.com/dependabot-gitlab/dependabot/-/raw/v7.11.0-alpha.1/docker-compose.yml | docker compose -f - up
```

# 🚀 Comment configurer Dependabot pour GitLab ? 🦊🛠️

Vous vous souvenez du chapitre sur les réseaux Docker ?

# 🚀 Comment configurer Dependabot pour GitLab ? 🦊🛠️

- Lorsque vous utilisez Docker Compose, il crée automatiquement un réseau par défaut pour votre application et tout service défini dans le `docker-compose.yml` peut atteindre les autres en utilisant le nom du service comme nom d'hôte.

# 🚀 Comment configurer Dependabot pour GitLab ? 🦊🛠️

- Voici un exemple :

```
services:
  web:
    image: web
    networks:
      - mynetwork
  gitlab-instance-gitlab-1:
    image: gitlab
    networks:
      - mynetwork
networks:
  mynetwork:
```

# 🚀 Comment configurer Dependabot pour GitLab ? 🦊🛠️

- Après avoir mis à jour votre fichier `docker-compose.yml`, vous devez recréer vos conteneurs pour que les modifications prennent effet.
- Vous pouvez le faire avec la commande `docker compose up -d --force-recreate`.
- Après cela, vous devriez pouvoir faire un `ping` sur `gitlab-instance-gitlab-1` depuis le conteneur web.

# 🚀 Comment configurer Dependabot pour GitLab ? 🦊🛠️

```
  web:
    image: *base_image
    networks:
      - mynetwork
[...]
networks:
  mynetwork:
```

# 🚀 Comment configurer Dependabot pour GitLab ? 🦊🛠️

Sauf que…

# 🚀 Comment configurer Dependabot pour GitLab ? 🦊🛠️

Victoire?

# Debugging : docker container exec

rappel : `docker container exec` permet de lancer une commande dans un container

```
docker container exec <containerID> echo "hello"
```

```
docker container exec -it <containerID> bash
```

# Debugging : docker container inspect

rappel : `docker container inspect` et `docker image inspect` listent toutes les caractéristiques d'un conteneur ou d'une image.

On peut filtrer le retour de la commande avec `jq` ou l'option `--format`.

# Debugging : docker container cp

Cette commande permet d'échanger des fichiers entre un conteneur et la machine hôte.

```
docker container cp --help
Usage:  docker container cp [OPTIONS] CONTAINER:SRC_PATH DEST_PATH|-
        docker cp [OPTIONS] SRC_PATH|- CONTAINER:DEST_PATH

Copy files/folders between a container and the local filesystem

Aliases:
  docker container cp, docker cp

Options:
  -a, --archive       Archive mode (copy all uid/gid information)
  -L, --follow-link   Always follow symlinks in SRC_PATH
  -q, --quiet         Suppress progress output during copy
```

# Debugging : docker image history

Cette commande permet d'afficher la concaténation de tous les Dockerfiles qui ont abouti à cette image.

```
docker image history nginx
IMAGE          CREATED      CREATED BY                                      SIZE      COMMENT
f9ea18bfa4fa   8 days ago   CMD ["nginx" "-g" "daemon off;"]                0B        buildkit.dockerfile.v0
<missing>      8 days ago   STOPSIGNAL SIGQUIT                              0B        buildkit.dockerfile.v0
<missing>      8 days ago   EXPOSE map[80/tcp:{}]                           0B        buildkit.dockerfile.v0
<missing>      8 days ago   ENTRYPOINT ["/docker-entrypoint.sh"]            0B        buildkit.dockerfile.v0
<missing>      8 days ago   COPY 30-tune-worker-processes.sh /docker-ent…   16.4kB    buildkit.dockerfile.v0
<missing>      8 days ago   COPY 20-envsubst-on-templates.sh /docker-ent…   12.3kB    buildkit.dockerfile.v0
<missing>      8 days ago   COPY 15-local-resolvers.envsh /docker-entryp…   12.3kB    buildkit.dockerfile.v0
<missing>      8 days ago   COPY 10-listen-on-ipv6-by-default.sh /docker…   12.3kB    buildkit.dockerfile.v0
<missing>      8 days ago   COPY docker-entrypoint.sh / # buildkit          8.19kB    buildkit.dockerfile.v0
<missing>      8 days ago   RUN /bin/sh -c set -x     && groupadd --syst…   88.7MB    buildkit.dockerfile.v0
<missing>      8 days ago   ENV DYNPKG_RELEASE=1~trixie                     0B        buildkit.dockerfile.v0
<missing>      8 days ago   ENV PKG_RELEASE=1~trixie                        0B        buildkit.dockerfile.v0
<missing>      8 days ago   ENV ACME_VERSION=0.4.1                          0B        buildkit.dockerfile.v0
<missing>      8 days ago   ENV NJS_RELEASE=1~trixie                        0B        buildkit.dockerfile.v0
<missing>      8 days ago   ENV NJS_VERSION=1.0.1                           0B        buildkit.dockerfile.v0
<missing>      8 days ago   ENV NGINX_VERSION=1.31.6                        0B        buildkit.dockerfile.v0
<missing>      8 days ago   LABEL maintainer=NGINX Docker Maintainers <d…   0B        buildkit.dockerfile.v0
<missing>      9 days ago   # debian.sh --arch 'amd64' out/ 'trixie' '@1…   87.7MB    debuerreotype 0.17
```

# Debugging : docker container stats

Cette commande permet d'avoir les stats en temps réel d'un conteneur.

```
docker container stats 42f128
CONTAINER           CPU %     MEM USAGE/LIMIT     MEM %     NET I/O
42f128              0.00%     1.454 MB/4.145 GB   0.04%     648 B/648 B
```

# 🎓 Travaux pratiques #14

Notre valeureux collègue Jean-Michel s'initiait à l'art mystérieux de Docker, et généreusement partageait ses créations sous forme d'images Docker pour l'entreprise.

Michel a soudainement décroché le jackpot du Loto, laissant derrière lui son bureau du jour au lendemain, alors qu'il était sur le point de nous offrir une image Nginx.

Désormais, nous n'avons que le binaire de son chef-d'œuvre, accessible à cette adresse :

https://owncloud.univ-artois.fr/index.php/s/9s8XBLXvRwBJsrm

Rassurez-vous, on nous a dit que `docker image load` est la première étape de la formule magique pour percer les secrets de son œuvre.

À vos claviers ! 🧙‍♂️✨

# ✅ Solution travaux pratiques #14 🕵️🔍

`docker image load` est la première étape de la formule magique

# ✅ Solution travaux pratiques #14🕵️🔍

```
docker container ls --format '{{.Names}}' | grep "bad"
```

# ✅ Solution travaux pratiques #14🕵️🔍

Le message accuse le binaire de Nginx.

# ✅ Solution travaux pratiques #14 🕵️ 🔍

```
[...]
"Env": [
    "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
],
```

# ✅ Solution travaux pratiques #14🕵️🔍

```
[...]
"Entrypoint": [
    "/bin/sh",
    "-c",
    "/tmp/entrypoint.sh"
],
```

# ✅ Solution travaux pratiques #14🕵️🔍

```
        [...]
        "Labels": {
            [...]
            "org.label-schema.name": "CentOS Base Image",
            [...]
        }
    },
    "Created": "2020-11-15T15:39:43.438668882Z",
    [...]
    "Architecture": "amd64",
    "Os": "linux",
    [...]
    "Os": "linux",
    [...]
    }
]
```

# ✅ Solution travaux pratiques #14🕵️🔍

Tournons-nous
vers l'histoire !

# ✅ Solution travaux pratiques #14🕵️🔍

```
docker image history bad-nginx:1
IMAGE          CREATED       CREATED BY                                      SIZE      COMMENT
2fd3346e0c2f   5 years ago   /bin/sh -c #(nop)  CMD ["/bin/sh" "-c" "ngin…   0B
<missing>      5 years ago   /bin/sh -c #(nop)  ENTRYPOINT ["/bin/sh" "-c…   0B
<missing>      5 years ago   /bin/sh -c #(nop)  EXPOSE 80                    0B
<missing>      5 years ago   /bin/sh -c #(nop) COPY file:b0fc89bcaf962602…   12.3kB
<missing>      5 years ago   /bin/sh -c yum install -y nginx && yum clean…   63.2MB
<missing>      6 years ago   /bin/sh -c #(nop)  CMD ["/bin/bash"]            0B
<missing>      6 years ago   /bin/sh -c #(nop)  LABEL org.label-schema.sc…   0B
<missing>      6 years ago   /bin/sh -c #(nop) ADD file:538afc0c5c964ce0d…   240MB
```

# ✅ Solution travaux pratiques #14🕵️🔍

Résumons-nous… On a:

# ✅ Solution travaux pratiques #14🕵️🔍💀

```dockerfile
# Utiliser Alpine ou Debian comme image de base
FROM alpine

# Installer nginx sans garder l'index des paquets
RUN apk add --no-cache nginx

# Exposer le port 80
EXPOSE 80

# Ajouter le fichier HTML (à récupérer depuis l'image de Jean-Michel)
COPY html/index.html /usr/share/nginx/html/

# Définir le point d'entrée pour lancer nginx
ENTRYPOINT ["nginx", "-g", "daemon off;"]
```

# ✅ Solution travaux pratiques #14🕵️🔍

Essayons de lancer un conteneur basé sur notre image, et de récupérer le fichier HTML.

# ✅ Solution travaux pratiques #14🕵️🔍

Sans le `sh` de trop, le shell marche :

```
docker container run -it --rm --entrypoint=sh bad-nginx:1
sh-4.4# ls -l /usr/sbin/nginx
-rwxr-xr-x 1 root root 1264168 Oct  7  2019 /usr/sbin/nginx
sh-4.4# nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

# ✅ Solution travaux pratiques #14🕵️🔍

Pour récupérer des fichiers, inutile de lancer quoi que ce soit dans l’image.

# ✅ Solution travaux pratiques #14🕵️🔍

```
head html/index.html
```

# ✅ Solution travaux pratiques #14🕵️🔍

Et ce fameux `/tmp/entrypoint.sh`, celui que lance notre `ENTRYPOINT` ?

# ✅ Solution travaux pratiques #14🕵️🔍

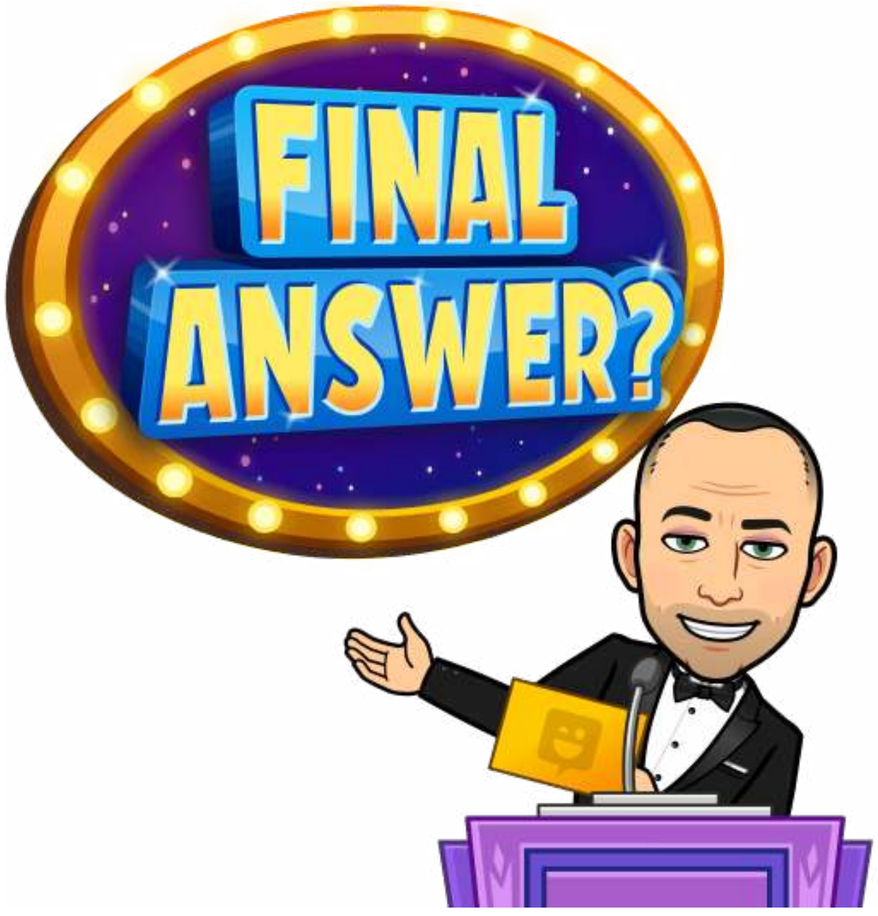
# ✅ Solution travaux pratiques #14🕵️🔍

```dockerfile
# Ce Dockerfile met en place un serveur web simple en utilisant Nginx sur Alpine Linux.

# Utilise Alpine Linux comme image de base. Alpine Linux est une distribution Linux légère et orientée sécurité.
FROM alpine

# Installe Nginx sans conserver l'index des paquets dans la couche.
# Le site par défaut de nginx sous Alpine répond 404 partout : on le remplace.
RUN apk add --no-cache nginx \
 && printf 'server {\n    listen 80;\n    root /usr/share/nginx/html;\n}\n' > /etc/nginx/http.d/default.conf

# Expose le port 80 au monde extérieur. C'est le port standard pour le trafic HTTP.
EXPOSE 80

# Barre finale obligatoire : sans elle, COPY crée un FICHIER nommé html.
COPY html/index.html /usr/share/nginx/html/

# Définit le point d'entrée du conteneur : le serveur Nginx.
# Le flag "-g" passe une directive globale : "daemon off;" garde Nginx au premier plan.
ENTRYPOINT ["nginx", "-g", "daemon off;"]
```

# 🐳😴 Lazy Docker

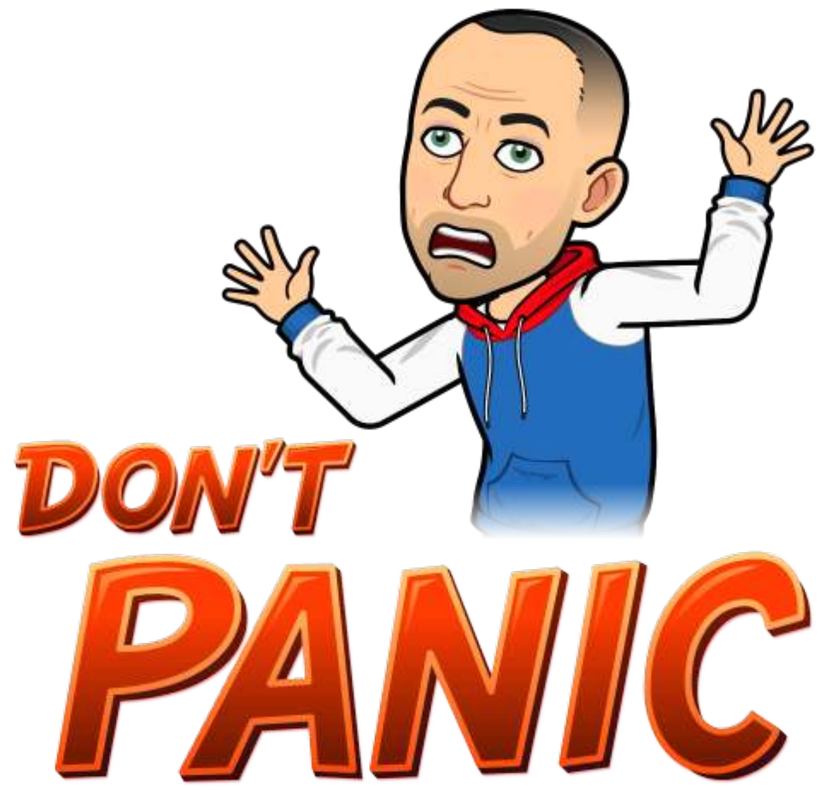
https://blog.stephane-robert.info/docs/conteneurs/outils/lazydocker/

# Conseils pour l’éval'




- https://moodle.univ-artois.fr/mod/resource/view.php?id=39414
- https://moodle.univ-artois.fr/mod/choicegroup/view.php?id=39415
- https://moodle.univ-artois.fr/mod/assign/view.php?id=39416

# Conseils pour l’eval'



# Vous devez avoir les compétences suivantes:

Savoir lancer un container avec toutes les options

- `-v, -p, -w, -e, --rm, etc.`

Écrire un Dockerfile optimisé pour "dockeriser" une application

- pas trop gourmand, facile à modifier, paramétrable.

Étudier une image ou un conteneur pour tout débug éventuel.

Savoir s'outiller pour avoir des images qui servent d'environnement d'exécution à votre CI.

Monter un écosystème applicatif complexe via docker-compose.

# Que revoir ?

Les TPs sont vos meilleurs amis.

La documentation officielle de Docker

https://docs.docker.com/engine/containers/run/

https://docs.docker.com/reference/dockerfile/

https://docs.docker.com/build/building/best-practices/

# 🤔👉 One more thing

# Bibliographie

# Ligne de commande

- https://tldp.org
- https://en.wikipedia.org/wiki/POSIX
- https://en.wikipedia.org/wiki/Read%E2%80%93eval%E2%80%93print_loop
- https://linuxhandbook.com/linux-directory-structure/
- https://blog.stephane-robert.info/docs/admin-serveurs/linux/script-shell/
- https://tldp.org/LDP/abs/html/

# Git / VCS

- https://docs.github.com
- https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control
- https://martinfowler.com/bliki/VersionControlTools.html
- https://martinfowler.com/bliki/FeatureBranch.html
- https://about.gitlab.com/topics/version-control/what-is-gitlab-flow/
- https://www.atlassian.com/git/tutorials/comparing-workflows
- https://nvie.com/posts/a-successful-git-branching-model/
- https://blog.stephane-robert.info/docs/developper/version/git/introduction/

# Intégration Continue

- https://martinfowler.com/articles/continuousIntegration.html
- https://martinfowler.com/bliki/ContinuousDelivery.html
- https://continuousdelivery.com/
- https://blog.arungupta.me/continuous-integration-delivery-deployment-maturity-model
- https://blog.crisp.se/2013/02/05/yassalsundman/continuous-delivery-vs-continuous-deployment

# Docker

- https://labs.play-with-docker.com/
- https://dduportal.github.io/cours/cnam-docker-2018
- https://kodekloud.com/blog/docker-for-beginners/
- https://www.slideshare.net/dotCloud/why-docker
- https://docs.docker.com/engine/reference/builder/
- https://www.r-bloggers.com/2021/05/best-practices-for-r-with-docker/
- https://github.com/wagoodman/dive
- https://docs.docker.com/engine/tutorials/networkingcontainers/
- https://docs.docker.com/engine/network/drivers/
- https://blog.stephane-robert.info/post/docker-buildkit-partie-1/
- https://dev.to/montells/docker-args-1ael

# DevOps

- https://blog.stephane-robert.info/docs/glossaire/introduction/

# Merci !

gounthar@gmail.com

Slides: https://gounthar.github.io/cours-devops-docker/docs/bonus-tp13-overflow

Source on : https://github.com/gounthar/cours-devops-docker